



SunShare

Smart Solar Microgrid Trading System

Client-Server Application (Web, Mobile and Web Service)

SE4040 Enterprise Application Development - Assignment 1

Year 4, Semester 2, 2026 · Submitted 30 September 2026

Part	Member	IT number	Responsible for
A	Gimhan T P K	IT22266996	Accounts & access
B	Ranathunga R A K N	IT22552860	Stations, slots & map; shared foundation, hosting (group lead)
C	Malkith G W L	IT22630834	Reservations
D	Chamara R M L K	IT22076816	Dashboards & QR

Git repository: <https://github.com/kavindu-rakn/sunshare>

Video (≤ 5 min): [video link - added after recording]

Contents

1. Introduction	4
1.1 Scenario	4
2. Requirements and business rules	4
2.1 Roles	4
2.2 Permission matrix	4
2.3 Statuses	5
2.4 Business rules R1-R18	5
2.5 Dashboard and list definitions	6
3. System architecture	7
3.1 Technology stack	7
3.2 How one request flows	8
3.3 Authentication	8
3.4 Error handling	9
3.5 Addresses and ports	9
3.6 Time handling	9
3.7 SQLite on the phone	9
3.8 Security notes	10
4. Use case diagram	10
5. Data flow diagrams	12
5.1 DFD level 1	13
6. Database design (MongoDB)	14
6.1 Relationships	15
6.2 Users	15
6.3 SolarStationInfo	15
6.4 EnergyBookingSlots	16
6.5 EnergyReservations	16
6.6 Sample data	17
6.7 Indexes	18
7. Web API	18
7.1 Controllers and services	21
8. Hosting and deployment	21
8.1 MongoDB	21
8.2 Host the Web API on IIS	21
8.3 Host the web app on IIS	22
8.4 Connecting the emulator and a real phone	22
8.5 Fresh sample data	23
8.6 Troubleshooting	23
9. User interfaces	23
9.1 Web app (Backoffice and Grid Operator)	24

9.2 Android app (Prosumer and Grid Operator).....	33
10. Design decisions.....	37
11. Challenges and reflection	42
12. Individual contributions and AI disclosure	45
12.1 Use of AI tools (shared statement).....	45
12.2 Part A - Gimhan T P K (IT22266996)	45
12.3 Part B - Ranathunga R A K N (IT22552860).....	45
12.4 Part C - Malkith G W L (IT22630834)	46
12.5 Part D - Chamara R M L K (IT22076816)	47
13. References	47
Appendix A. Source code.....	50
A.1 Web API (C#, ASP.NET Core).....	50
A.2 Web app (React + Bootstrap 5)	104
A.3 Android app - Java	151
A.4 Android app - manifest, layouts, resources and map page.....	200
A.5 Deployment scripts (PowerShell)	232

1. Introduction

SunShare is a client-server system for trading solar energy on a neighbourhood microgrid. Homes with rooftop solar panels (**prosumers**) book time slots at microgrid **stations** either to drop off spare energy (**Sell**) or to charge from the station battery (**Buy**). **Backoffice officers** run the system from a web app, and **Grid Operators** manage stations and confirm energy transfers on site by scanning the prosumer's QR code with the mobile app.

The system has three parts, exactly as the assignment asks:

- **Web service (FAT service)** - a C# ASP.NET Core Web API (.NET 10) hosted on Windows IIS with a **MongoDB** database. *All* business rules (R1-R18) live in its Services; the clients only show screens and call the API.
- **Web app** - React + **Bootstrap 5**, hosted on IIS, for Backoffice officers and Grid Operators.
- **Android app** - pure native Java + XML, **SQLite** for the login session and cached stations, Retrofit for the API, ZXing for QR codes and the **Google Maps** JavaScript API for the nearby-stations map. Used by prosumers and Grid Operators.

Git repository: <https://github.com/kavindu-rakn/sunshare> - 120 commits, merged through 22 pull requests (one branch and pull request per build phase).

This report explains the requirements and business rules, the architecture (high-level, use case and data flow diagrams), the database design, the Web API, how the system is hosted, every user interface (screenshots), the design decisions, the challenges we met, each member's contribution with the AI disclosure, and the references. The complete source code is included as text in Appendix A.

1.1 Scenario

Backoffice registers microgrid stations and keeps their operating schedules. Grid Operators (web and mobile) publish battery slot availability and monitor bookings. Prosumers register on the mobile app, wait for a Backoffice officer to activate them, then reserve slots up to 7 days ahead, change or cancel them at least 12 hours before, and see their history. When staff approve a booking, the app shows a secure QR code; at the station the Grid Operator scans it, the server verifies it and the operator finalizes the energy transfer.

2. Requirements and business rules

This chapter is the specification the whole team built against. Every rule is enforced in the Web API (FAT service); the web and Android apps only display the API's answer. In the code, each rule is marked with a `// RULE Rx:` comment.

2.1 Roles

Role	Uses	Can do
Backoffice	Web	Everything admin: web users, prosumers, activations, stations, slots, reservations, approve
GridOperator	Web + Mobile	Operational: slots (availability), reservations (create for prosumer / edit / cancel / approve), view stations. Mobile: scan QR → verify → finalize
Prosumer	Mobile only	Register, login, edit profile, deactivate self, map of nearby stations, reserve / modify / cancel, bookings + history + search, show QR

Login on the "wrong" client is refused with a clear message (e.g. Prosumer on web → "Prosumers use the SunShare mobile app").

2.2 Permission matrix

Feature	Backoffice	GridOperator	Prosumer
Create/edit web users (Backoffice/GridOperator)	✓	✗	✗

Feature	Backoffice	GridOperator	Prosumer
Create/edit/deactivate prosumers	✓	✗ (read list only, for booking on behalf)	✗
Activate / reactivate accounts	✓	✗	✗
Create/edit/deactivate/delete stations	✓	✗ (view)	✗ (view on map)
Create/edit/delete slots	✓	✓	✗
Create/edit/cancel reservations	✓ (any)	✓ (any)	✓ (own only)
Approve reservation	✓	✓	✗
Verify QR + finalize	✗	✓	✗
Own profile edit / self-deactivate	—	—	✓

The API enforces this with `[Authorize(Roles = "...")]`. The clients only hide buttons for nicer UX — they never decide.

2.3 Statuses

User.status: Pending → Active → Deactivated → (Backoffice only) Active

Mobile register → Pending —(Backoffice activates)→ Active
 Staff creates user → Active
 Active —(self or Backoffice deactivates)→ Deactivated —(Backoffice reactivates)→ Active

Reservation.status: Pending → Approved → Completed, or → Cancelled

create → Pending —(staff approves, QR token made)→ Approved —(operator scans QR)→ Completed
 Pending/Approved —(cancel, ≥12h before)→ Cancelled
 Approved —(edited, ≥12h before)→ Pending again (old QR stops working)

2.4 Business rules R1-R18

Every rule has an ID. In code, put `// RULE Rx: ...` on the line where it is enforced so anyone can search it.

ID	Rule
R1	NIC is the primary key (<code>_id</code>) of every user. Must be unique and a valid Sri Lankan NIC: old format 9 digits + v/x (e.g. 981234567V) or new format 12 digits (e.g. 199812345678).
R2	Role access: Backoffice = admin functions, GridOperator = operational tools, Prosumer = own data only (see matrix).
R3	A prosumer who self-registers on mobile starts Pending and cannot log in until a Backoffice user activates them.
R4	A Deactivated user cannot log in. Only Backoffice can reactivate. A Backoffice user cannot deactivate their own account.
R5	A prosumer can deactivate their own account from mobile. They are logged out immediately.
R6	A station cannot be deactivated while it has active reservations (status Pending or Approved with start time in the future).
R7	A station can be deleted only if it has no reservations at all (otherwise deactivate it). Deleting a station also deletes its slots.
R8	Slot rules: end after start; start in the future; station must be active; <code>totalSlots</code> between 1 and the station's <code>batterySlots</code> ; <code>totalSlots</code> can't go below places already booked; a slot with active reservations can't be deleted.
R9	Reservation start time must be in the future and no more than 7 days from now (on create, and when moving to another slot).
R10	A reservation can be updated or cancelled only if its start is at least 12 hours away.
R11	Only an Active prosumer can hold a reservation; the station and slot must be active; the slot must have a free place; a prosumer can't book the same slot twice (unless the earlier one is Cancelled).
R12	Availability counter: create → <code>slot.availableSlots - 1</code> ; cancel → <code>+ 1</code> ; move to another slot → <code>old + 1, new - 1</code> .

ID	Rule
R13	New reservations are Pending . Backoffice/GridOperator approve → Approved + a random QR token is generated. Editing an Approved reservation sends it back to Pending and clears the token.
R14	QR verification: QR text format is SUNSHARE <reservationId> <qrToken>. The API checks the reservation exists, is Approved, and the token matches. Finalize does the same checks, then sets Completed + who/when. Only GridOperator.
R15	Completed and Cancelled reservations cannot be changed .
R16	Prosumers can only see and change their own profile and reservations (NIC is taken from the login token, never from the request body).
R17	energyKwh must be greater than 0 and at most 100 .
R18	Nearby stations: the API calculates distance from the phone's location (Haversine formula) and returns active stations within the radius (default 25 km), nearest first .

Also computed by the API (so clients don't hold logic):

- canModify on each reservation = status is Pending/Approved **and** start is ≥ 12 h away (R10, R15). Clients show Edit/Cancel buttons only when this is true.

2.5 Dashboard and list definitions

Term	Meaning
Pending reservations	status Pending
Approved future reservations (count)	status Approved and start time > now
Current bookings (mobile tab)	Approved, start time > now
Pending bookings (mobile tab)	Pending
History (mobile tab)	Completed, Cancelled, or start time already passed
Search filter	text match on station name / reservation id (staff also: prosumer NIC/name) + optional status, station and date range

3. System architecture

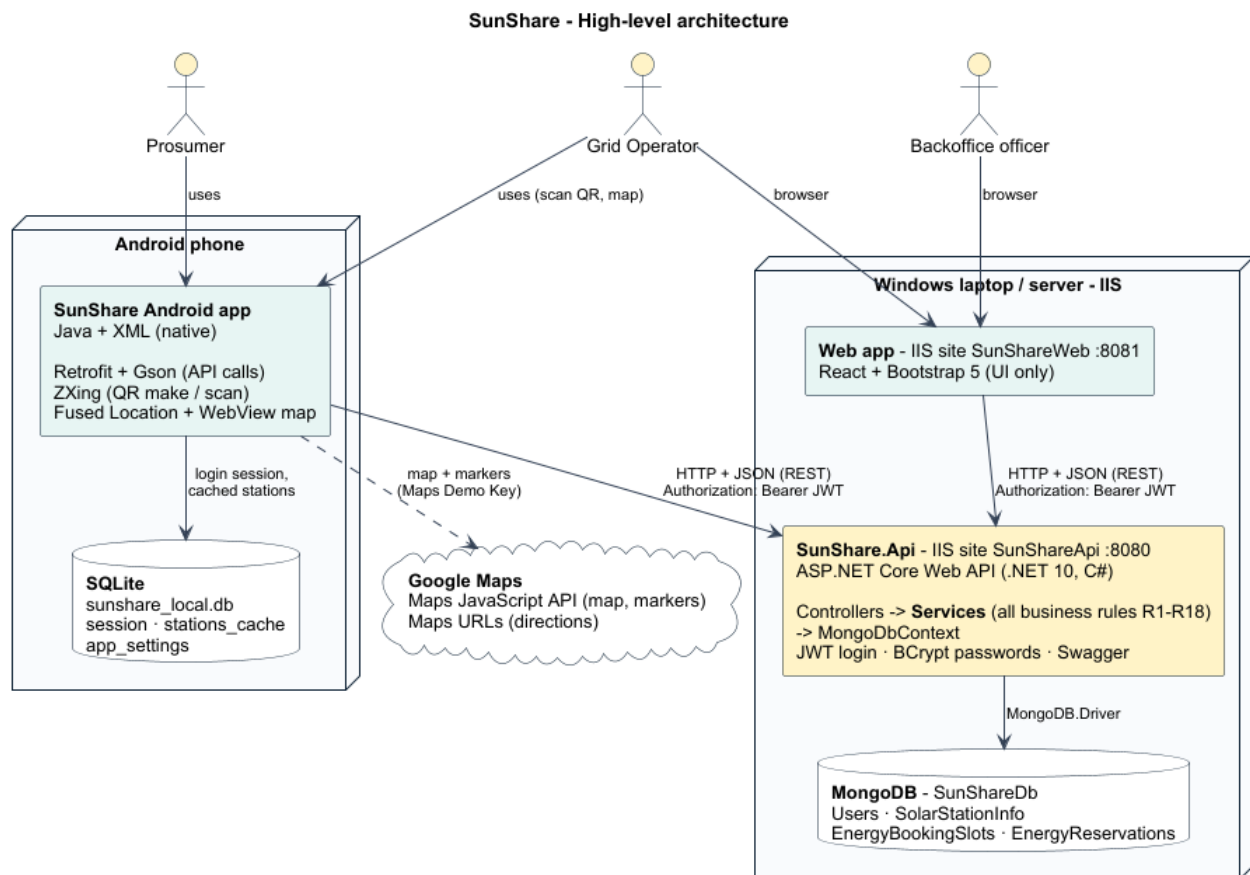


Figure 1: High-level architecture - Android and web clients, the FAT Web API on IIS, MongoDB and Google Maps

- **Clients are "thin"**: they show screens, collect input, call the API, and show what the API says. They never touch MongoDB and never decide business rules.
- **API is "fat"**: it checks roles, validates, applies every rule (R1–R18 in 01-SPEC.md), and reads/writes MongoDB.
- **SQLite** lives only on the phone: saved login session, cached stations, and the server address setting.

3.1 Technology stack

Layer	Choice	Why (one line for the viva)
API	ASP.NET Core Web API, .NET 10 (LTS), C#, Controllers	Brief demands C# Web API; controllers are the classic, easy-to-explain style
Hosting	IIS + ASP.NET Core Hosting Bundle 10	Brief demands IIS (4 marks)
Database	MongoDB Community Server (local) + MongoDB.Driver	Brief demands NoSQL/MongoDB; local = works at viva without internet
Auth	JWT bearer tokens + BCrypt password hashing	JWT = a signed "login ticket" the client sends with every call; BCrypt = passwords are never stored as plain text
API docs/test	Swagger UI (Swashbuckle) at /swagger	Click-to-test every endpoint; great for demo
Web	React (Vite) + JavaScript + Bootstrap 5 + Bootstrap Icons + React Router (HashRouter)	React is named in the brief; Bootstrap 5 is marked in the rubric; HashRouter means IIS needs no extra rewrite module
Mobile	Java + XML layouts (Android Studio "Empty Views Activity"), minSdk 26	Pure native Android; Java looks like C#
Mobile HTTP	Retrofit + Gson (OkHttp underneath)	One short line per API call
Mobile DB	SQLite via SQLiteOpenHelper (no Room)	Rubric says SQLite — raw SQLite is the most literal match

Layer	Choice	Why (one line for the viva)
Maps	Google Maps JavaScript API shown in Android's built-in WebView (Maps Demo Key, D53) + Fused Location	Rubric says Google Maps API (3 + 5 marks); the no-card demo key works with the JavaScript API but not the native Android SDK
QR	ZXing Android Embedded (journeyapps)	One library both makes and scans QR codes

Allowed packages (do not add others without asking Kvn)

- API: MongoDB.Driver, Microsoft.AspNetCore.Authentication.JwtBearer, BCrypt.Net-Next, Swashbuckle.AspNetCore
- Web: react, react-dom, react-router-dom, bootstrap, bootstrap-icons (+ Vite's own dev deps)
- Android: retrofit2:retrofit, retrofit2:converter-gson, com.journeyapps:zxing-android-embedded, play-services-location, androidx.recyclerview:recyclerview, plus the template defaults (appcompat, material, constraintlayout, activity)
- Android map: the **Google Maps JavaScript API inside Android's built-in WebView** with the **Maps Demo Key** (D51, D53) — no extra package; play-services-maps was removed. The key goes `local.properties` → `manifest meta-data com.sunshare.app.MAPS_API_KEY` → filled into `assets/map.html` at runtime
- Android **last-resort fallback only** (D47, if no Google key works at all): `org.osmdroid:osmdroid-android` (OpenStreetMap map, no key)

Banned (keeps it explainable)

TypeScript, Redux/Zustand, Axios (use fetch), react-bootstrap, Tailwind, AutoMapper, MediatR, CQRS, generic repository / unit-of-work, Entity Framework, Kotlin source files, Jetpack Compose, Room, Hilt/Dagger, RxJava, Flutter/React Native/Xamarin/MAUI, any cloud DB.

3.2 How one request flows

1. **Android** `ReservationDetailActivity` → user taps **Cancel** → `apiService.cancelReservation(id)`.
2. **Retrofit** sends PATCH `http://<server>:8080/api/reservations/{id}/cancel` with header `Authorization: Bearer <token from SQLite>`.
3. **IIS** receives it and hands it to the ASP.NET Core app.
4. **JWT middleware** checks the token is valid and reads NIC + role from it.
5. **ReservationsController.Cancel** is allowed for Prosumer/staff → calls `ReservationService.CancelAsync(id, callerNic, callerRole)`.
6. **ReservationService** loads the reservation from MongoDB, checks **R16** (own booking), **R15** (not already done), **R10** (≥ 12 h away), sets `Cancelled`, and does **R12** (`slot availableSlots + 1`).
7. Returns the updated reservation as JSON → Android opens **ReservationSummaryActivity** ("Cancelled ✓").
8. If any rule fails, the service throws `ApiException(400, "...clear message...")` → middleware returns `{ "message": "..."} → Android shows it in a Snackbar/Toast.`

3.3 Authentication

1. POST `/api/auth/login { nic, password }` → API finds user by NIC, checks BCrypt hash, checks status (R3, R4).
2. API returns `{ token, nic, fullName, role }`. Token = JWT signed with the secret in `appsettings.json`, valid 8 hours, contains NIC + role.
3. **Web** saves it in `localStorage`; **Android** saves it in the SQLite session table.
4. Every later call sends `Authorization: Bearer <token>`. `[Authorize(Roles = "...")]` on controllers enforces R2.
5. On 401, clients clear the saved session and go back to Login.

3.4 Error handling

- Services throw **ApiException(statusCode, message)** — e.g. 400 rule broken, 401 bad login, 403 wrong role/not yours, 404 not found, 409 conflict (duplicate NIC, slot full).
- ErrorHandlingMiddleware** turns it into JSON: { "message": "Cancellations need at least 12 hours' notice." }. Unknown errors → 500 { "message": "Something went wrong on the server." } (details only in the server log).
- Web shows message in a Bootstrap alert; Android shows it in a Toast/Snackbar. **Messages must be human-friendly.**
- Ids from the URL or body** (stations, slots, reservations are MongoDB ObjectIds): check `ObjectId.TryParse(id, out _)` first and throw 404 if it fails — otherwise a typo like `/api/stations/abc` crashes the query with a 500. Reuse `StationService.FindAsync` / `SlotService.FindAsync` where possible.
- Times in requests** must be UTC ISO strings (2026-09-30T02:30:00Z). A time with an offset (+05:30) is converted to UTC; a time with no zone at all is taken as UTC. The web converts `datetime-local` values with `new Date(value).toISOString()` before sending.

3.5 Addresses and ports

Thing	Development	Hosted (demo / viva)
API	<code>http://localhost:5080</code> (dotnet run)	IIS site SunShareApi → <code>http://localhost:8080</code> , phone: <code>http://<laptop-LAN-IP>:8080</code>
Swagger	<code>http://localhost:5080/swagger</code>	<code>http://localhost:8080/swagger</code> (kept ON for the demo)
Web	<code>http://localhost:5173</code> (npm run dev)	IIS site SunShareWeb → <code>http://localhost:8081</code>
Android → API	Emulator: <code>http://10.0.2.2:8080/</code> · Phone: <code>http://<laptop-LAN-IP>:8080/</code> — changeable at runtime via the ☞ Server address setting on the Login screen (saved in SQLite)	
MongoDB	<code>mongodb://localhost:27017</code> , database SunShareDb	

CORS on the API allows `http://localhost:5173` and `http://localhost:8081` (list in `appsettings.json` → `Cors:AllowedOrigins`). **Only while developing** (`ASPNETCORE_ENVIRONMENT=Development`, i.e. `dotnet run`) any `http://localhost:<port>` is also allowed; on IIS (Production) only the list counts (D23).

3.6 Time handling

- MongoDB stores **UTC**. API sends ISO strings like `2026-09-29T02:30:00Z`.
- Clients convert to the device's local time (Sri Lanka, UTC+5:30) only for display.
- All rule maths (7 days, 12 hours) happens in the API using `DateTime.UtcNow`.

3.7 SQLite on the phone

Table	Columns	Used for
session	nic TEXT PK, full_name TEXT, role TEXT, token TEXT, logged_in_at TEXT	Stay logged in; attach token to API calls; role-based home on app start (max 1 row)
stations_cache	id TEXT PK, name TEXT, address TEXT, latitude REAL, longitude REAL, capacity_kw REAL, battery_slots INTEGER, open_time TEXT, close_time TEXT, distance_km REAL, cached_at TEXT	Reference data: stations saved after each successful nearby-stations call; shown on the map if the API is unreachable
app_settings	key TEXT PK, value TEXT	<code>api_base_url</code> (server address)

3.8 Security notes

- Passwords hashed with BCrypt; JWT signed with HMAC-SHA256.
- QR "security" = a random 32-char token that only the server and the booking owner know; faking a QR needs that token.
- Demo runs on **HTTP over the local network** (no HTTPS certificate) — acceptable for a lab demo, would be HTTPS in production.
- JWT secret sits in `appsettings.json` for simplicity; production would use environment variables / a secret store.
- A token stays valid until it expires (8 h), even if the account is deactivated meanwhile — the server keeps no session list. We limit the damage: login is refused (R4), profile edits check the status again, and reservations need an Active prosumer (R11). Production could use short-lived tokens + refresh tokens or a revocation list.

4. Use case diagram

Three actors use SunShare. Backoffice officers and Grid Operators are both **Staff**, so they share the staff use cases (dashboard, approving bookings, managing slots, booking on behalf of a prosumer). Only Backoffice manages users, prosumers, activations and stations; only Grid Operators scan QR codes and finalize transfers. The letter after each use case is the Part that built it (A-D, see chapter 12). *Finalize energy transfer* always **includes** *Scan QR and verify*, and *Show booking QR code* **extends** *View bookings* when a booking is Approved.

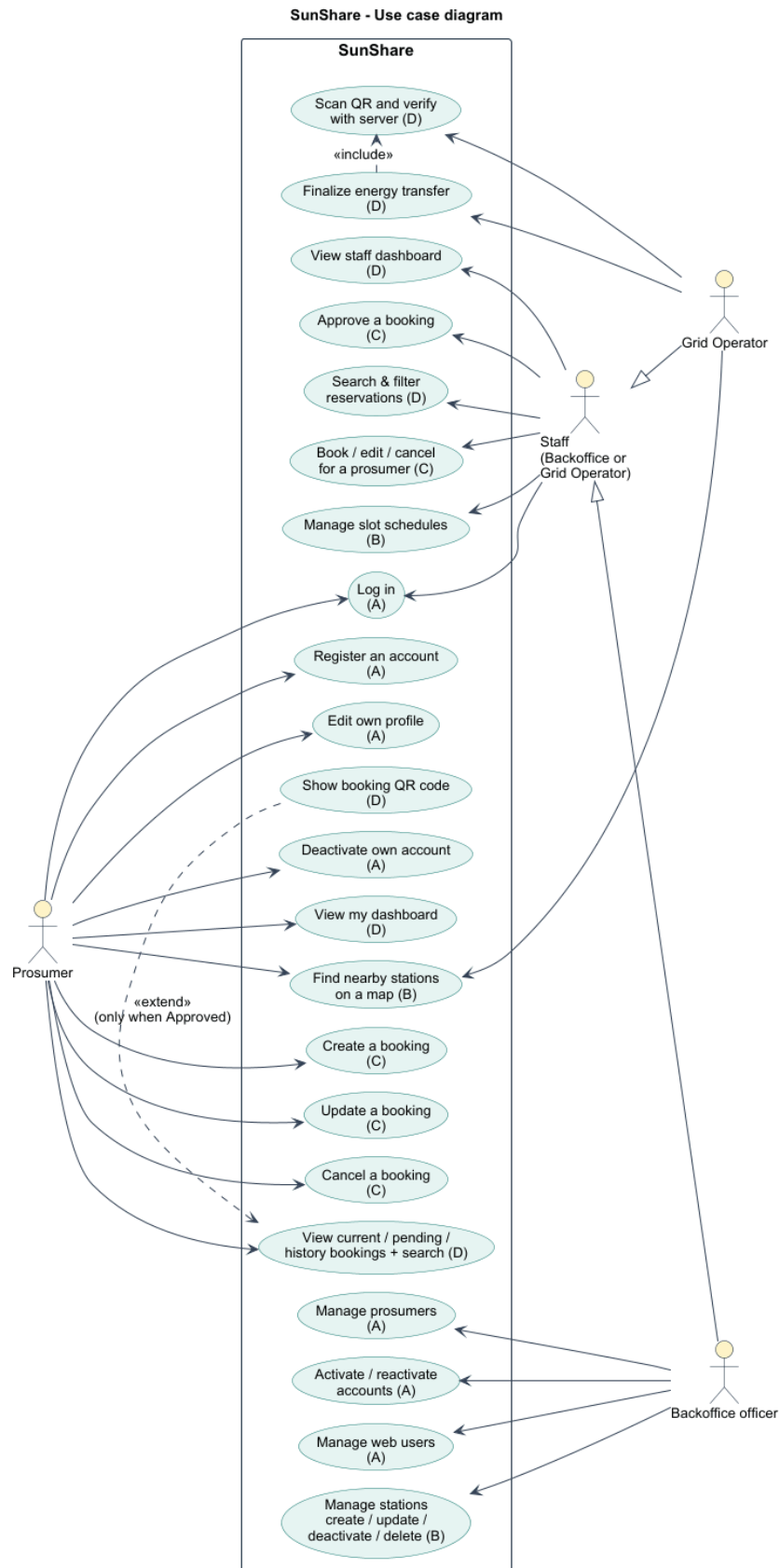


Figure 2: Use case diagram - actors, system boundary SunShare, include and extend relations

5. Data flow diagrams

Notation: rectangle = external entity, circle = process, cylinder = data store. Level 0 shows SunShare as a single process and the data it exchanges with its users and with Google Maps.

SunShare - Data flow diagram, level 0 (context)

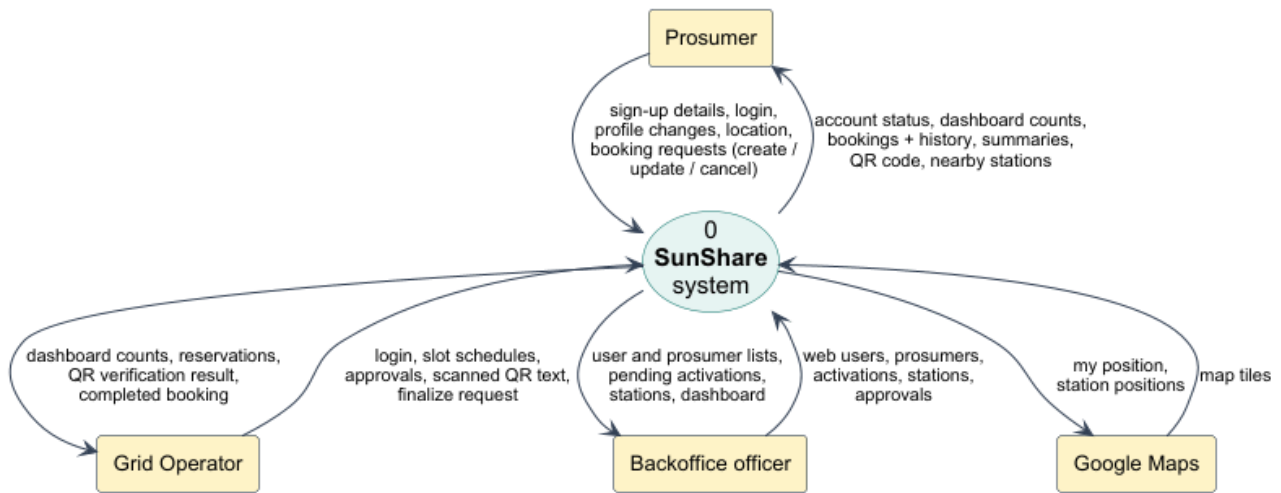


Figure 3: DFD level 0 (context diagram)

5.1 DFD level 1

Level 1 splits the system into its six main processes - they match the API's Services - and shows which data store each reads and writes. D1-D4 are the MongoDB collections; D5 is the SQLite database on the phone.

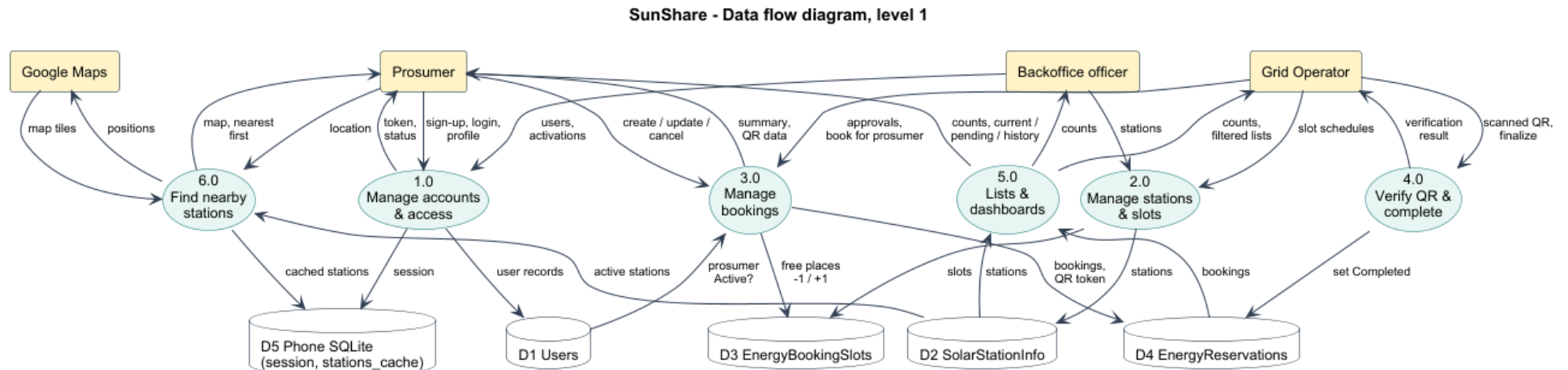


Figure 4: DFD level 1 - processes 1.0-6.0 and data stores D1-D5

6. Database design (MongoDB)

Database: **SunShareDb** on `mongodb://localhost:27017`. **Exactly 4 collections, named to match the rubric** ("User's detail · SolarStationInfo · EnergyBookingSlots · Energy Reservation"):

Rubric line	Collection name	C# model class
User's detail	Users	User
SolarStationInfo	SolarStationInfo	SolarStation
EnergyBookingSlots	EnergyBookingSlots	EnergyBookingSlot
Energy Reservation	EnergyReservations	EnergyReservation

MongoDB stores **documents** (JSON-like records) in **collections** (like tables). There are no JOINS, so we store the *id* of the related document (a "reference") and sometimes a copy of a name for fast display.

SunShare - Database design (MongoDB SunShareDb)

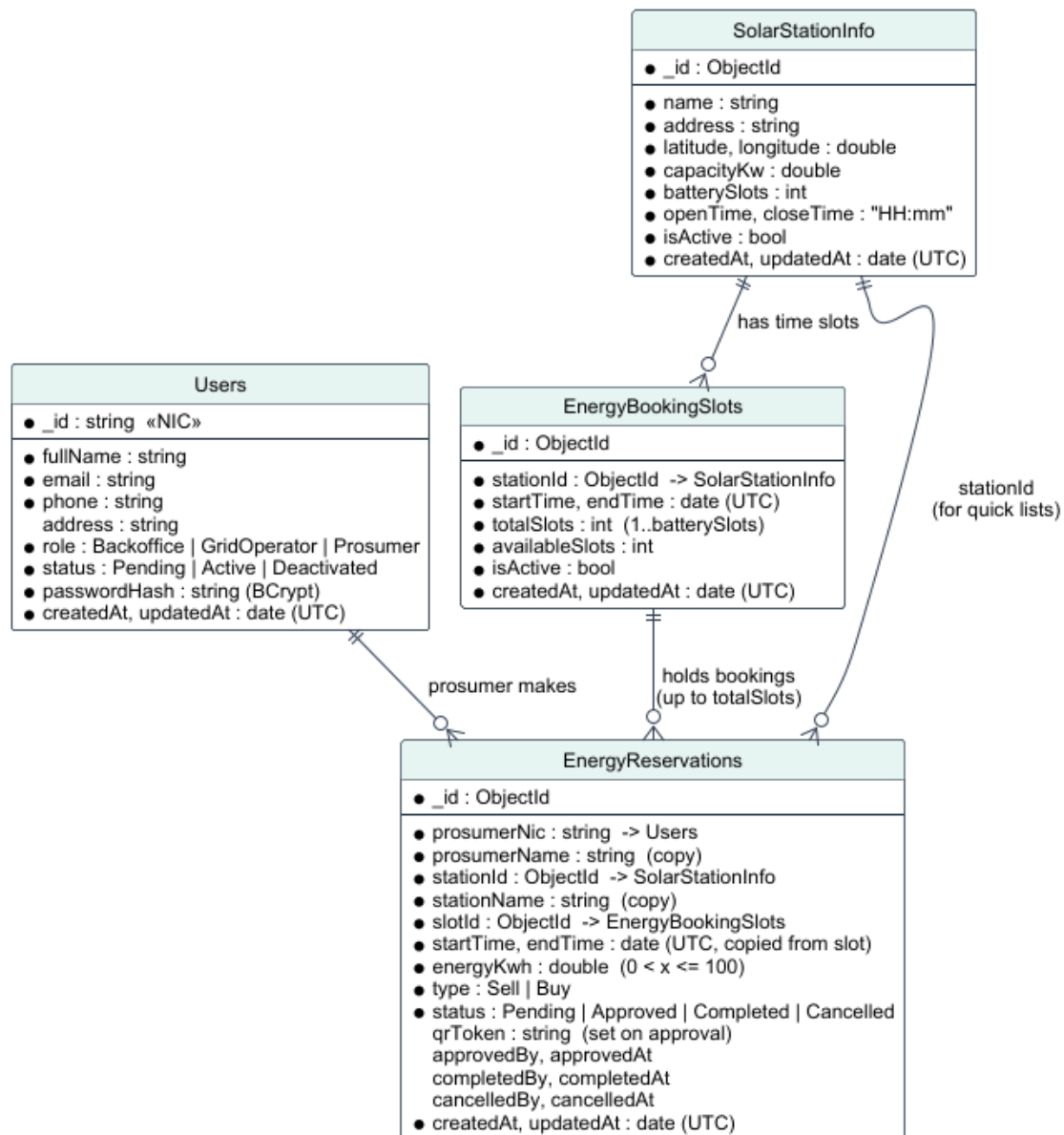


Figure 5: The four collections, their fields and references (one-to-many)

6.1 Relationships

- One **station** has many **slots**. One **slot** has many **reservations** (up to totalSlots). One **prosumer** has many **reservations**.
- prosumerName and stationName inside a reservation are **copies** (snapshots) for fast lists and search.

6.2 Users

All people: Backoffice, GridOperator and Prosumer. **_id is the NIC** (rule R1).

Field	Type	Req	Notes
_id	string	✓	NIC — 981234567V (old) or 199812345678 (new). Unique by nature of _id.
fullName	string	✓	
email	string	✓	
phone	string	✓	e.g. 0771234567
address	string	—	Prosumers' property address
role	string	✓	Backoffice GridOperator Prosumer
status	string	✓	Pending Active Deactivated
passwordHash	string	✓	BCrypt hash — never the plain password; never returned by the API
createdAt	date (UTC)	✓	
updatedAt	date (UTC)	✓	

```
{
  "_id": "199712345678",
  "fullName": "Nimal Perera",
  "email": "nimal.perera@example.com",
  "phone": "0771234567",
  "address": "12 Temple Road, Malabe",
  "role": "Prosumer",
  "status": "Active",
  "passwordHash": "$2a$11$Qm3...",
  "createdAt": { "$date": "2026-09-27T10:00:00Z" },
  "updatedAt": { "$date": "2026-09-27T10:00:00Z" }
}
```

6.3 SolarStationInfo

Field	Type	Req	Notes
_id	ObjectId	✓	C#: string with [BsonRepresentation(BsonType.ObjectId)]
name	string	✓	e.g. "Malabe Solar Hub"
address	string	✓	
latitude	double	✓	GPS, -90..90
longitude	double	✓	GPS, -180..180
capacityKw	double	✓	Solar generation capacity in kW (> 0)
batterySlots	int	✓	Number of battery storage places at the hub (≥ 1) — caps a slot's totalSlots (R8)
openTime	string	✓	Operating schedule start, "HH:mm" e.g. "06:00"
closeTime	string	✓	Operating schedule end, "HH:mm" e.g. "18:00"
isActive	bool	✓	false = deactivated (R6)
createdAt, updatedAt	date (UTC)	✓	

```
{
```

```
{
  "_id": { "$oid": "66f6a1c2e4b0a1b2c3d4e5f1" },
  "name": "Malabe Solar Hub",
  "address": "New Kandy Road, Malabe",
  "latitude": 6.9147, "longitude": 79.9730,
  "capacityKw": 250, "batterySlots": 10,
  "openTime": "06:00", "closeTime": "18:00",
  "isActive": true,
  "createdAt": { "$date": "2026-09-27T10:00:00Z" }, "updatedAt": { "$date": "2026-09-27T10:00:00Z" }
}
```

6.4 EnergyBookingSlots

Field	Type	Req	Notes
_id	ObjectId	✓	
stationId	ObjectId	✓	→ SolarStationInfo._id
startTime	date (UTC)	✓	
endTime	date (UTC)	✓	must be after startTime (R8)
totalSlots	int	✓	places in this window, 1..station.batterySlots (R8)
availableSlots	int	✓	free places left; changed only by reservation create/cancel/move (R12)
isActive	bool	✓	operators can close a window
createdAt, updatedAt	date (UTC)	✓	

```
{
  "_id": { "$oid": "66f6a1c2e4b0a1b2c3d4e6a0" },
  "stationId": { "$oid": "66f6a1c2e4b0a1b2c3d4e5f1" },
  "startTime": { "$date": "2026-09-29T02:30:00Z" },
  "endTime": { "$date": "2026-09-29T04:30:00Z" },
  "totalSlots": 4, "availableSlots": 3, "isActive": true,
  "createdAt": { "$date": "2026-09-27T10:00:00Z" }, "updatedAt": { "$date": "2026-09-27T10:00:00Z" }
}
```

(02:30 UTC = 08:00 Sri Lanka time.)

6.5 EnergyReservations

Field	Type	Req	Notes
_id	ObjectId	✓	
prosumerNic	string	✓	→ Users._id
prosumerName	string	✓	copy for display/search
stationId	ObjectId	✓	→ SolarStationInfo._id
stationName	string	✓	copy for display/search
slotId	ObjectId	✓	→ EnergyBookingSlots._id
startTime / endTime	date (UTC)	✓	copied from the slot; used for R9/R10
energyKwh	double	✓	$0 < x \leq 100$ (R17)
type	string	✓	Sell (drop-off: prosumer stores energy in the grid battery) Buy (charging: prosumer draws energy)
status	string	✓	Pending Approved Completed Cancelled
qrToken	string	—	random 32 hex chars, set on approval (R13), cleared if edited
approvedBy, approvedAt	string, date	—	NIC of staff who approved
completedBy, completedAt	string, date	—	NIC of operator who scanned (R14)
cancelledBy, cancelledAt	string, date	—	
createdAt, updatedAt	date (UTC)	✓	


```
{
  "_id": { "$oid": "66f6a1c2e4b0a1b2c3d4e7b3" },
  "prosumerNic": "199712345678", "prosumerName": "Nimal Perera",
  "stationId": { "$oid": "66f6a1c2e4b0a1b2c3d4e5f1" }, "stationName": "Malabe Solar Hub",
  "slotId": { "$oid": "66f6a1c2e4b0a1b2c3d4e6a0" },
  "startTime": { "$date": "2026-09-29T02:30:00Z" }, "endTime": { "$date": "2026-09-29T04:30:00Z" },
  "energyKwh": 12.5, "type": "Sell",
  "status": "Approved",
  "qrToken": "9f1c2b7e4a5d4c3b8e2f1a0b9c8d7e6f",
  "approvedBy": "199845678912", "approvedAt": { "$date": "2026-09-27T11:00:00Z" },
  "completedBy": null, "completedAt": null, "cancelledBy": null, "cancelledAt": null,
  "createdAt": { "$date": "2026-09-27T10:30:00Z" }, "updatedAt": { "$date": "2026-09-27T11:00:00Z" }
}
```

6.6 Sample data

DataSeeder runs on API start **only if Users is empty** (so it never overwrites real data). The rubric wants sample data "added manually or through the application" — this is "through the application", and more gets added live during the demo.

Test accounts (all fictional)

Role	NIC	Password	Name	Status
Backoffice	200012345678	Admin@123	Dilani Fernando	Active
GridOperator	199845678912	Operator@123	Kasun Silva	Active
Prosumer	199712345678	Prosumer@123	Nimal Perera	Active
Prosumer	200156789123	Prosumer@123	Ayesha Jayasinghe	Active
Prosumer	200198765432	Prosumer@123	Tharindu Wickramasinghe	Pending (shows in Pending Activations)
Prosumer	981234567V	Prosumer@123	Sanduni Rathnayake	Deactivated (shows reactivation)

Stations (around SLIIT Malabe so the map looks right)

Name	Lat, Lng	capacityKw	batterySlots	Active
Malabe Solar Hub	6.9147, 79.9730	250	10	✓
Kaduvela Grid Node	6.9335, 79.9840	180	8	✓
Battaramulla Microgrid	6.9020, 79.9180	300	12	✓
Kottawa Sun Park	6.8410, 79.9650	150	6	✓
Nugegoda Rooftop Node	6.8649, 79.8997	90	4	✗ (shows deactivated state)

Slots

For every active station: **next 7 days × 3 windows** (08:00–10:00, 11:00–13:00, 14:00–16:00 Sri Lanka time), totalSlots = 4. Plus one **past** slot (2 days ago) so history has data.

Reservations

Nimal: 1 **Approved** (tomorrow 11:00, has QR), 1 **Pending** (in 3 days), 1 **Completed** (past slot), 1 **Cancelled**.
Ayesha: 1 **Pending** (in 2 days). Slot availableSlots values must match these bookings.

Before the viva: seeded dates are relative to when seeding ran. Re-seed the day before: in Compass drop SunShareDb → recycle the IIS app pool (or restart the API) → seeder runs again. (Steps in 08-SETUP-AND-HOSTING.md.)

6.7 Indexes

- EnergyBookingSlots: { stationId: 1, startTime: 1 }
- EnergyReservations: { prosumerNic: 1 }, { status: 1, startTime: 1 }

Created once at startup in MongoClientContext (CreateOne is safe to call repeatedly).

7. Web API

The API has **37 endpoints**, all under /api, documented live with Swagger at <http://localhost:8080/swagger>. Every answer is JSON; every error is { "message": "... } with a friendly text. The **Who** column shows the roles allowed (BO = Backoffice, GO = Grid Operator, PR = Prosumer, Any = any logged-in user); the API checks the role from the JWT token on every call.

Health — shared (Kvn)

Method	Path	Who	Response
GET	/api/health	Public	{ "api": "ok", "database": "connected", "time": "... } — pings MongoDB; 503 with "database": "unreachable" if Mongo is down. Used to prove IIS + MongoDB work in the demo.

Auth — Part A

Method	Path	Who	Body → Response	Rules
POST	/api/auth/login	Public	LoginRequest → LoginResponse. Wrong NIC/password → 401 "NIC or password is incorrect." Pending → 403 "Your account is waiting for Backoffice activation." Deactivated → 403 "Your account is deactivated. Please contact the Backoffice."	R3, R4
POST	/api/auth/register	Public	RegisterRequest → 201 UserResponse (always role Prosumer, status Pending). Bad NIC, missing details or password < 6 chars → 400. NIC exists → 409.	R1, R3

Users — Part A

Method	Path	Who	Body → Response	Rules
GET	/api/users?role=&status=&search=	BO, GO (GO: read only, used to pick a prosumer when booking on behalf)	→ UserResponse[]	R2
GET	/api/users/pending-activations	BO	→ UserResponse[] of Prosumers with status Pending or Deactivated (oldest first)	R3, R4
GET	/api/users/{nic}	BO	→ UserResponse	
POST	/api/users	BO	CreateUserRequest → 201 UserResponse (status Active; password ≥ 6 chars)	R1, R2
PUT	/api/users/{nic}	BO	UpdateUserRequest → UserResponse (NIC and role can't change)	R1
PATCH	/api/users/{nic}/activate	BO	→ UserResponse. Pending/Deactivated → Active	R3, R4
PATCH	/api/users/{nic}/deactivate	BO	→ UserResponse. Can't deactivate self → 400	R4

Profile (own account) — Part A

Method	Path	Who	Body → Response	Rules
GET	/api/profile	Any	→ UserResponse of the caller	R16
PUT	/api/profile	PR	UpdateProfileRequest → UserResponse (newPassword optional, min 6 chars)	R16

Method	Path	Who	Body → Response	Rules
PATCH	/api/profile/deactivate	PR	→ { "message": "Account deactivated." }	R5

Stations — Part B

Method	Path	Who	Body → Response	Rules
GET	/api/stations?activeOnly=	Any (PR always gets active only)	→ StationResponse[]	
GET	/api/stations/nearby?lat=&lng=&radiusKm=25	Any	→ StationResponse[] with distanceKm (km, 2 decimals), active only, nearest first. No lat/lng or out-of-range → 400; radiusKm must be > 0	R18
GET	/api/stations/{id}	Any	→ StationResponse	
POST	/api/stations	BO	StationRequest → 201 StationResponse	lat/lng range, capacity > 0, batterySlots ≥ 1, open < close
PUT	/api/stations/{id}	BO	StationRequest → StationResponse (updates schedule too)	
PATCH	/api/stations/{id}/deactivate	BO	→ StationResponse. Active reservations → 409 "This station has N active reservations. Cancel or complete them first."	R6
PATCH	/api/stations/{id}/activate	BO	→ StationResponse	
DELETE	/api/stations/{id}	BO	→ 204. Has any reservations → 409 "Stations with booking history can't be deleted — deactivate it instead." Deletes its slots.	R7

Slots — Part B

Method	Path	Who	Body → Response	Rules
GET	/api/stations/{stationId}/slots?from=&to=	BO, GO	→ SlotResponse[] soonest first (default: from the start of today UTC, for 14 days; from/to are UTC, to exclusive)	
GET	/api/slots/available?stationId=	Any	→ SlotResponse[] that are active, availableSlots > 0, start in the future and ≤ 7 days away, station active. stationId optional (omit = all active stations)	R9, R11
GET	/api/slots/{id}	Any	→ SlotResponse	
POST	/api/slots	BO, GO	SlotRequest → 201 SlotResponse (availableSlots = totalSlots)	R8
PUT	/api/slots/{id}	BO, GO	SlotUpdateRequest → SlotResponse (availableSlots recalculated = total – booked). Changing the time of a slot that has bookings → 409	R8
DELETE	/api/slots/{id}	BO, GO	→ 204. Active reservations → 409	R8

Reservations: actions — Part C

Method	Path	Who	Body → Response	Rules
POST	/api/reservations	PR (own NIC from token), BO/GO (must	CreateReservationRequest → 201 ReservationResponse (status Pending). Errors: no slot / bad energy or type / slot closed / station	R9, R11, R12, R13, R16, R17

Method	Path	Who	Body → Response	Rules
		send prosumerNic)	inactive / too soon or > 7 days / prosumer not Active → 400; same prosumer + slot twice or slot full → 409; staff NIC not a prosumer → 404	
PUT	/api/reservations/{id}	PR (own), BO, GO	UpdateReservationRequest → ReservationResponse (Approved → back to Pending; nothing changed → returned as it is). Not yours → 403; Completed/Cancelled or < 12 h → 400	R9, R10, R12, R13, R15, R16, R17
PATCH	/api/reservations/{id}/cancel	PR (own), BO, GO	→ ReservationResponse (Cancelled, QR token cleared, place given back)	R10, R12, R15, R16
PATCH	/api/reservations/{id}/approve	BO, GO	→ ReservationResponse (Approved, qrData filled). Only from Pending and not in the past → else 400	R13

Reservations: views & QR — Part D

Method	Path	Who	Body → Response	Rules
GET	/api/reservations?view=&status=&stationId=&from=&to=&search=	PR (own only), BO, GO	view = current pending history all (default). search matches station name (not case-sensitive) or a full 24-character reservation id; staff also prosumer NIC/name. from/to filter the start time (UTC, to exclusive). → ReservationResponse[] (all/current/pending: soonest first; history: newest first). Unknown view or bad stationId → 400	R16, dashboard definitions
GET	/api/reservations/{id}	PR (own), BO, GO	→ ReservationResponse	R16
POST	/api/reservations/verify-qr	GO	VerifyQrRequest → ReservationResponse. Bad format → 400 "This is not a SunShare QR code." Unknown booking / already completed / cancelled / not approved / wrong token → 400, each with its own reason	R14
PATCH	/api/reservations/{id}/complete	GO	VerifyQrRequest → ReservationResponse (Completed, completedBy/At). Same checks as verify, and the QR must belong to {id} → else 400	R14

Dashboards — Part D

Method	Path	Who	Response
GET	/api/dashboard/summary	BO, GO	StaffDashboardResponse (web dashboard + mobile operator home). todayReservations = not Cancelled, starting today in Sri Lanka time; pendingList = next 5 upcoming Pending
GET	/api/dashboard/prosumer	PR	ProsumerDashboardResponse (mobile prosumer home)

7.1 Controllers and services

Controller	Service	Part
AuthController, UsersController, ProfileController	AuthService, UserService	A
StationsController, SlotsController	StationService, SlotService	B
ReservationsController — create / update / cancel / approve	ReservationService	C
ReservationViewsController — list / get by id	ReservationQueryService	D
QrController — verify-qr / complete	QrService	D
DashboardController	DashboardService	D

The three reservation controllers all use [Route("api/reservations")]; splitting them by file keeps each member's code in their own files.

8. Hosting and deployment

The Web API and the web app are both hosted on Windows **IIS** on the same laptop (API site **SunShareApi** on port 8080, web site **SunShareWeb** on port 8081), with **MongoDB Community Server** running as a Windows service. Two PowerShell scripts in `scripts/` make the deployment reproducible: run them in an administrator PowerShell from the repository folder, as often as needed.

Needed once: Windows IIS feature, the **ASP.NET Core Hosting Bundle 10**, **MongoDB Community Server** (service MongoDB), the **.NET 10 SDK** and **Node.js** (to build), and Android Studio for the mobile app.

8.1 MongoDB

1. Get-Service MongoDB → Running (if Stopped: Start-Service MongoDB in an admin PowerShell).
2. Compass → mongod://localhost:27017 → after the API's first run you'll see database **SunShareDb** with the 4 collections.

8.2 Host the Web API on IIS

3.1 Turn on IIS

1. Press Win+R → type `optionalfeatures` → Enter.
2. Tick **Internet Information Services** (defaults are fine). Expand it → *Web Management Tools* → make sure **IIS Management Console** is ticked. OK → wait → restart if asked.
3. Browse `http://localhost` → blue IIS welcome page = working.

3.2 Install the Hosting Bundle → then in an **admin** terminal: `net stop was /y` then `net start w3svc`.

3.3 Deploy with the script (admin PowerShell, from sunshare/) — first time AND after every code change:

```
powershell -ExecutionPolicy Bypass -File .\scripts\deploy-api.ps1
```

It is safe to run again and again. What it does (this is also how to explain IIS hosting at the viva):

1. Checks it runs as **admin** and finds a dotnet that has an **SDK** (see the PATH note in §1).
2. Creates the **application pool** **SunShareApi** with **No Managed Code** (only the first time). *Why:* IIS must not load the old .NET Framework — ASP.NET Core brings its own runtime; IIS only forwards requests to it through the **ASP.NET Core Module** (from the Hosting Bundle).
3. **Stops** the app pool (a running app locks its own files), then `dotnet publish -c Release -o C:\inetpub\sunshare\api`.
4. Creates the **site** **SunShareApi** → port **8080** → that folder → that app pool (only the first time), and starts it.
5. Adds the **firewall rule** "SunShare API 8080" so the phone can connect (only the first time).

6. Calls `http://localhost:8080/api/health` and prints the phone address (`http://<laptop IPv4>:8080/api/health`).

The same by hand (fallback, or to show in IIS Manager at the viva): Win+R → `inetmgr` → *Application Pools* → Add → SunShareApi, .NET CLR version **No Managed Code** · *Sites* → Add Website → SunShareApi, physical path `C:\inetpub\sunshare\api`, port **8080**, app pool SunShareApi.

3.4 Check it: `http://localhost:8080/swagger` and `http://localhost:8080/api/health` → "database": "connected".

3.5 Let the phone reach it: the script adds the firewall rule. Find the laptop's IP: `ipconfig` → *Wireless LAN adapter Wi-Fi* → **IPv4 Address** (e.g. 192.168.1.23) — the script prints it too. Phone browser → `http://192.168.1.23:8080/api/health`. Tip: set the Wi-Fi network to **Private** in Windows settings (Settings → Network & internet → Wi-Fi → your network → Network profile type).

8.3 Host the web app on IIS

Deploy (admin PowerShell, from sunshare/) — first time and after every web change:

```
powershell -ExecutionPolicy Bypass -File .\scripts\deploy-web.ps1
```

What it does:

1. `npm run build` in `web/` — Vite reads `web/.env.production` (`VITE_API_BASE_URL=http://localhost:8080`), so the built app calls the **IIS API**.
2. Creates the app pool SunShareWeb (No Managed Code — the site is only static files) if missing.
3. Mirrors `web/dist/` into `C:\inetpub\sunshare\web` with `robocopy /MIR` (old files removed).
4. Creates the site SunShareWeb on port **8081** if missing, starts it, and opens the home page to check.

`web/public/web.config` (copied into `dist/` on every build) sets `index.html` as the start page and declares the `.woff2` icon font type. Because we use **HashRouter** (`#/login`), IIS needs no URL Rewrite module.

Check: `http://localhost:8081` → log in as Backoffice → the Dashboard loads (web on IIS → API on IIS → MongoDB). The API allows the `http://localhost:8081` origin (CORS list in `appsettings.json`).

8.4 Connecting the emulator and a real phone

- Everything points to the **IIS API on port 8080**.
- **Emulator** → server address `http://10.0.2.2:8080/` (10.0.2.2 = "the laptop" as seen from the emulator).
- **Phone, over Wi-Fi** → `http://<laptop IPv4>:8080/` — laptop and phone must get addresses in the **same range** (e.g. both 192.168.1.x). The same Wi-Fi *name* is not enough: a router with two bands or a repeater can hand out different ranges (C12). Check: laptop `ipconfig` → IPv4; phone → Settings → Wi-Fi → the network → IP address. **Most reliable at the viva:** turn on the **Android phone's own hotspot** and connect the laptop to it (campus Wi-Fi often blocks device-to-device traffic).
- **Phone, over the USB cable** (no Wi-Fi needed): with the phone plugged in and USB debugging on, run once per connection in a terminal:

```
& "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe" -d reverse tcp:8080 tcp:8080
```

then in the app: ⚙ Server → `http://localhost:8080/`. The phone's port 8080 is carried through the cable to the laptop's IIS API. Unplugging the cable ends it.

- Change it at runtime: Login screen ⚙ → Server address (saved in SQLite). **At the viva the Wi-Fi/IP will be different — just update it there, no rebuild.**
- Phone: Settings → About phone → tap **Build number** 7 times → Developer options → **USB debugging** ON → plug in → Allow.
- QR demo: prosumer booking open on the **emulator** (QR on the laptop screen) → operator logged in on the **phone** → Scan.

- **Map on the emulator:** the emulator has no real GPS. Give it a position near SLIIT before opening *Nearby stations*: emulator ... **(Extended controls)** → **Location** → search "SLIIT Malabe" → **Set location** (or `adb -s emulator-5554 emu geo fix 79.9729 6.9147` — longitude first). Without a position the app waits 10 s and then uses SLIIT Malabe anyway.
- **Emulator memory:** the Medium_Phone AVD has only **2 GB** — with Google Play + Chrome's WebView it can freeze ("isn't responding") or restart (C13). For demos give it more: Device Manager →  Edit → Show Advanced Settings → **RAM 3072 MB** (or start it with `-memory 3072`), and close apps you don't need on the laptop.

8.5 Fresh sample data

Seeded bookings are dated relative to the seeding day, so refresh the day before:

1. Compass → SunShareDb →  **Drop database.**
2. IIS Manager → Application Pools → SunShareApi → **Recycle.**
3. Open `http://localhost:8080/api/health` → the API starts and re-seeds.

8.6 Troubleshooting

Symptom	Likely cause → fix
IIS shows HTTP 500.19	Hosting Bundle missing or installed before IIS → (re)install Hosting Bundle, <code>net stop was /y, net start w3svc</code>
HTTP 500.30 / 500.31	App crashed on start (e.g. Mongo not running, wrong .NET) → check Get-Service MongoDB; temporarily set <code>stdoutLogEnabled="true"</code> in <code>web.config</code> and read <code>logs\</code>
PUT/DELETE/PATCH return 405 on IIS only	WebDAV module intercepting → our <code>web.config</code> removes it; make sure it was published
Web shows "Failed to fetch" / CORS error	API not running, wrong <code>VITE_API_BASE_URL</code> , or origin missing in CORS list
Phone can't reach the API	Not same Wi-Fi · firewall rule missing · wrong IP · Wi-Fi network set to Public
Android "CLEARTEXT communication not permitted"	<code>network_security_config.xml</code> not applied in the manifest
Map is grey/blank	Key not in <code>local.properties</code> , wrong SHA-1/package restriction, billing not linked, or emulator image without Google Play
<code>dotnet build / dotnet run</code> : " No .NET SDKs were found "	The runtime-only <code>C:\Program Files\dotnet</code> (from the Hosting Bundle) comes first on the PATH → install the SDK there: <code>winget install --id Microsoft.DotNet.SDK.10 -e (admin)</code> , then open a new terminal. The deploy script works either way
Login works on web, 401 on mobile later	Token expired (8 h) → app should clear the session and return to Login

9. User interfaces

All screenshots were taken from the finished system with the seeded sample data: the web app from the IIS-hosted site (port 8081) at 1440 px wide, and the Android app on the emulator connected to the IIS-hosted API. Every screen is shown once.

9.1 Web app (Backoffice and Grid Operator)

W1 Home

Landing: hero ("Trade your sunshine"), 3-step "how it works" (Register → Reserve → Scan), role cards (Backoffice / Grid Operator / Prosumer), CTA to Login, footer with group info.

SunShare How it works [Staff login](#)

Smart solar microgrid trading

Trade your sunshine.

SunShare connects homes with rooftop solar panels to neighbourhood microgrid stations. Book a time slot, drop off your spare energy or charge up - and check in with a QR code.

[Staff login](#) [How it works](#)

Prosumers use the SunShare Android app.

Your next booking **Approved**
Malabe Solar Hub
 Tue 11:00 - 13:00 · 12.5 kWh · Sell

Show this QR code at the station. The operator scans it and finishes your energy transfer.

4 stations nearby
 Nearest: 0.4 km away

How it works

Three simple steps from rooftop to grid.

- 1 Register**
Prosumers sign up in the SunShare Android app with their NIC. A Backoffice officer checks and activates the account.
- 2 Reserve**
Pick a nearby station on the map and a time slot up to 7 days ahead - to drop off spare energy or to charge up.
- 3 Scan**
Once staff approve the booking, the app shows a QR code. At the station a Grid Operator scans it to finish the transfer.

One system, three roles

Everyone sees exactly the tools they need.

Backoffice

Web

Manages staff accounts and prosumers, activates new sign-ups, registers solar stations and their schedules, and approves bookings.

Grid Operator

Web + mobile

Opens time slots, books and approves on behalf of prosumers, and scans QR codes at the station to complete energy transfers.

Prosumer

Mobile

Finds the nearest stations on a map, reserves and manages bookings, and shows a QR code to check in at the station.

Keep the microgrid running smoothly

Stations, slots, prosumers and bookings - all in one place for staff. [Staff login](#)

SunShare
 Smart Solar Microgrid Trading System.
 SE4040 Enterprise Application Development · Assignment 1 · SLIIT 2026

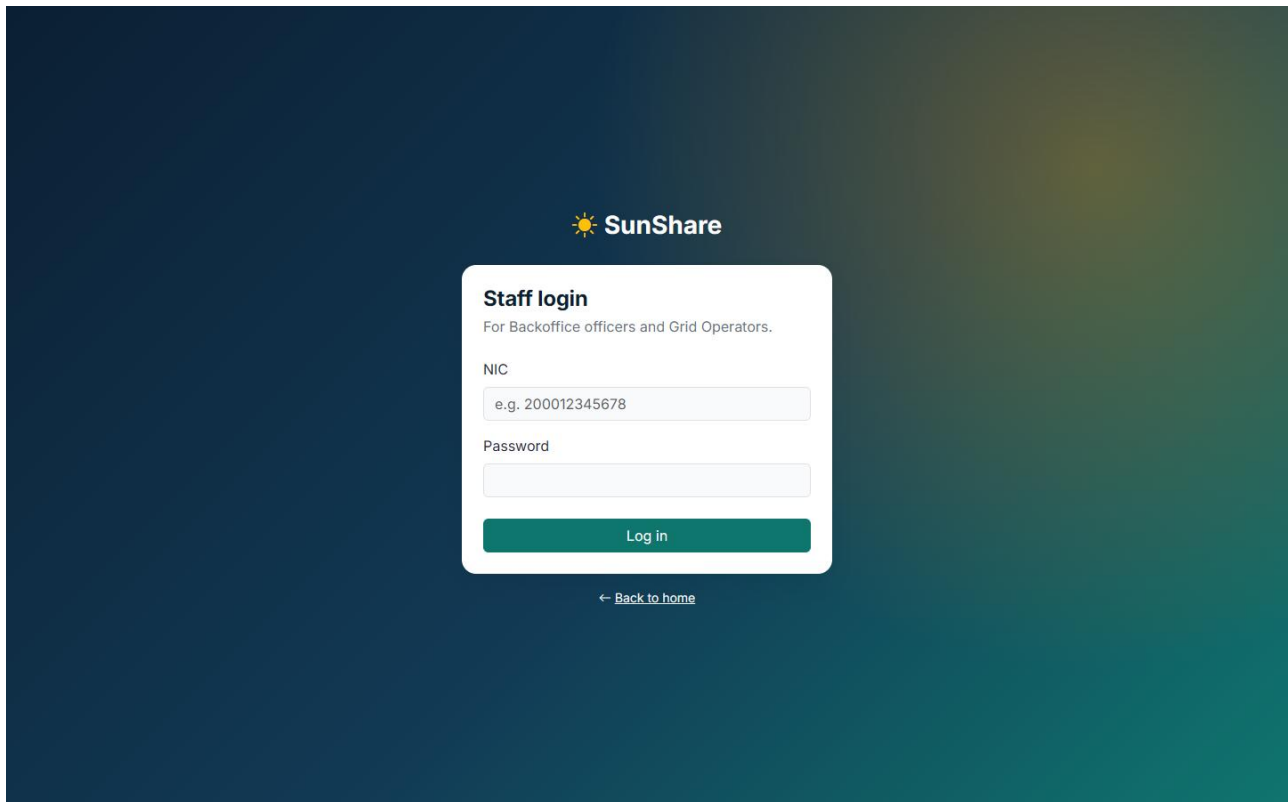
Team
 Ranathunga R A K N · IT22552860
 Gimhan T P K · IT22266996
 Malkith G W L · IT22630834
 Chamara R M L K · IT22076816

System status
 Server and database online

Figure 6: W1 Home - Public

W2 Login

NIC + password → POST /auth/login. Backoffice/GridOperator → #/dashboard. Prosumer → message "Prosumers use the SunShare mobile app". Shows API messages for Pending/Deactivated.

The image shows a web page for SunShare staff login. The background is a dark blue gradient. At the top center is the SunShare logo, which consists of a yellow sun icon followed by the text "SunShare". Below the logo is a white rectangular login form. Inside the form, the title "Staff login" is followed by the subtitle "For Backoffice officers and Grid Operators.". There are two input fields: the first is labeled "NIC" and contains the placeholder text "e.g. 200012345678"; the second is labeled "Password" and is empty. Below these fields is a green "Log in" button. At the bottom of the form, there is a link that says "← Back to home".

 **SunShare**

Staff login
For Backoffice officers and Grid Operators.

NIC

Password

[Log in](#)

[← Back to home](#)

Figure 7: W2 Login - Public

W3 Dashboard

Count cards: Pending reservations, **Approved future reservations**, Today's reservations, Active stations, Pending activations (BO). Table: next pending reservations with **Approve** button. GET /dashboard/summary

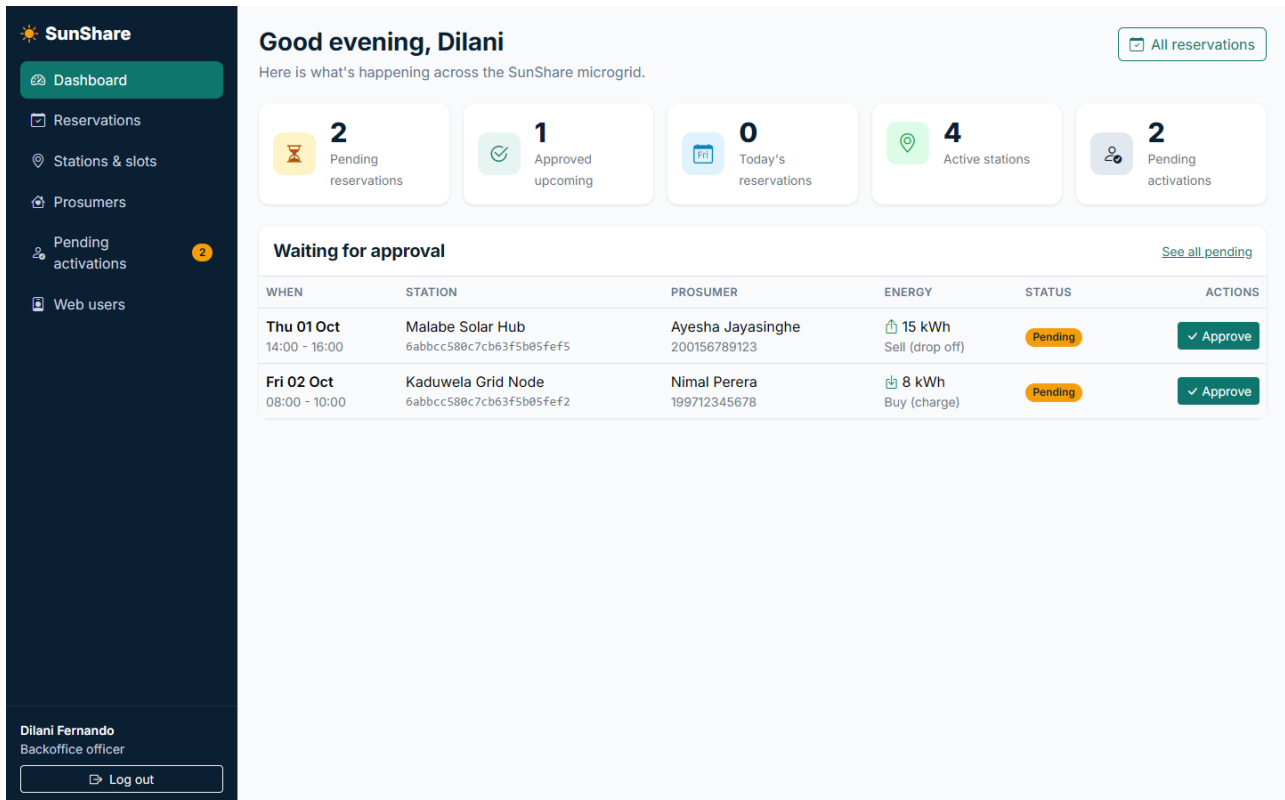


Figure 8: W3 Dashboard - BO, GO

W4 Web Users

List Backoffice + GridOperator users, filter by role/status, Activate/Deactivate. GET /users?role=

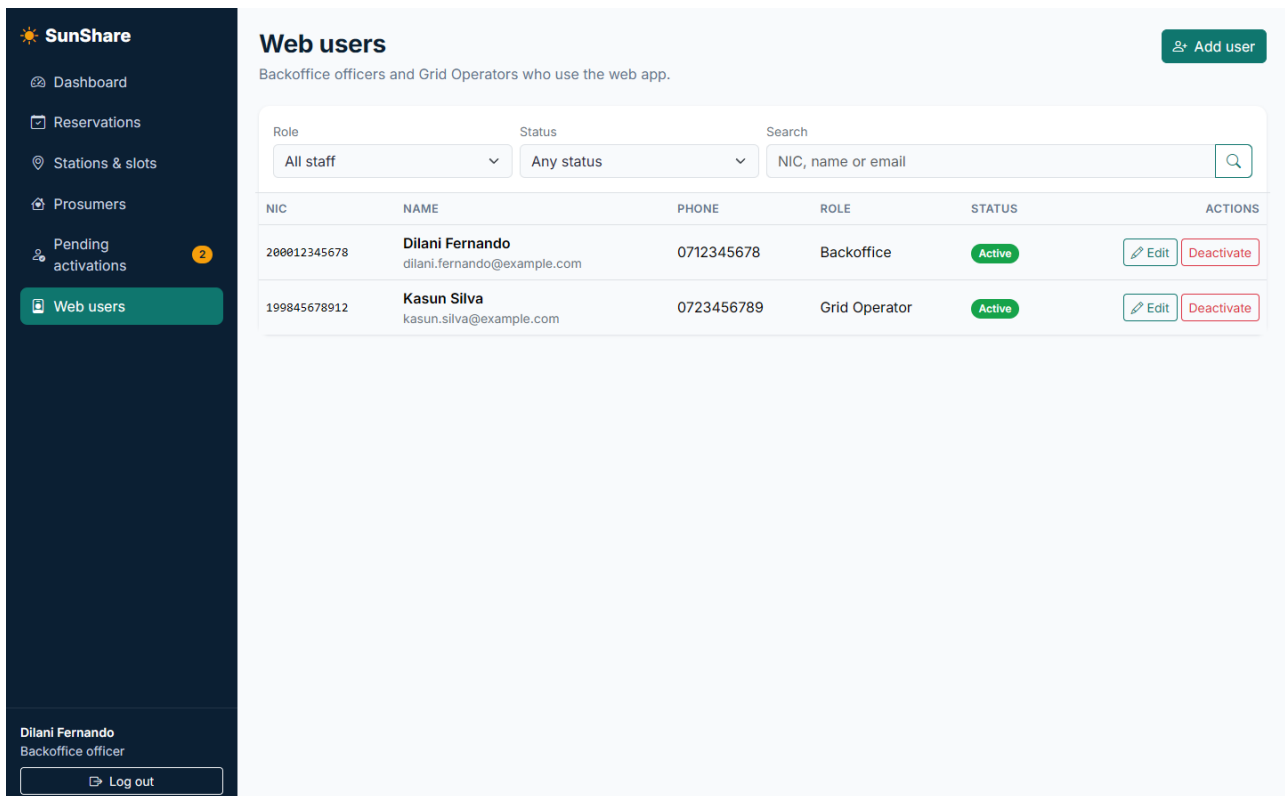


Figure 9: W4 Web Users - BO

W5 User form

Create (NIC, name, email, phone, role, password) / edit. POST/PUT /users

- Dashboard
- Reservations
- Stations & slots
- Prosumers
- Pending activations 2
- Web users**

Dilani Fernando
Backoffice officer
Log out

Edit web user

The NIC is the account key and cannot be changed.

NIC

199845678912

Role

Grid Operator

Full name

Kasun Silva

Email

kasun.silva@example.com

Phone

0723456789

Save changes

Cancel

Active

Figure 10: W5 User form - BO

W6 Prosumers

List prosumers, search, status filter, Deactivate/Reactivate, Edit.

- Dashboard
- Reservations
- Stations & slots
- Prosumers**
- Pending activations 2
- Web users

Dilani Fernando
Backoffice officer
Log out

Prosumers

Homes with solar panels that trade energy through SunShare.

Status
Any status

Search
NIC, name or email

NIC	NAME	PHONE	ADDRESS	STATUS	ACTIONS
200156789123	Ayesha Jayasinghe ayesha.jayasinghe@example.com	0762345678	45 Lake Drive, Kaduwela	Active	Edit Deactivate
199712345678	Nimal Perera nimal.perera@example.com	0771234567	12 Temple Road, Malabe	Active	Edit Deactivate
981234567V	Sanduni Rathnayake sanduni.r@example.com	0704567890	22 Flower Lane, Kottawa	Deactivated	Edit Reactivate
200198765432	Tharindu Wickramasinghe tharindu.w@example.com	0753456789	8 Station Road, Battaramulla	Pending	Edit Activate

Add prosumer

Figure 11: W6 Prosumers - BO

W7 Prosumer form

Create/edit prosumer profile (NIC is the key, read-only on edit).

SunShare

- Dashboard
- Reservations
- Stations & slots
- Prosumers**
- Pending activations 2
- Web users

Dilani Fernando
Backoffice officer
[Log out](#)

Edit prosumer

The NIC is the account key and cannot be changed.

Active

NIC	Role
199712345678	Prosumer
Full name	Email
Nimal Perera	nimal.perera@example.com
Phone	Address (optional)
0771234567	12 Temple Road, Malabe

[Save changes](#) [Cancel](#)

Figure 12: W7 Prosumer form - BO

W8 Pending Activations

Prosumers waiting: **Pending** (new sign-ups) and **Deactivated** (need reactivation). One-click Activate. Sidebar shows a count badge. GET /users/pending-activations, PATCH /users/{nic}/activate

SunShare

- Dashboard
- Reservations
- Stations & slots
- Prosumers
- Pending activations** 2
- Web users

Dilani Fernando
Backoffice officer
[Log out](#)

Pending activations

New prosumer sign-ups and deactivated accounts waiting for a Backoffice officer.

NIC	PROSUMER	PHONE	REASON	WAITING SINCE	ACTION
200198765432	Tharindu Wickramasinghe tharindu.w@example.com	0753456789	Pending New sign-up	Tue 29 Sept, 20:03	✓ Activate
981234567V	Sanduni Rathnayake sanduni.r@example.com	0704567890	Deactivated Needs reactivation	Tue 29 Sept, 20:03	✓ Reactivate

Figure 13: W8 Pending Activations - BO

W9 Stations

Table: name, location, capacity, battery slots, schedule, status, active bookings. Actions: Edit, Slots, Deactivate/Activate, Delete. Shows API's R6/R7 messages.

SunShare

Dashboard
Reservations
Stations & slots
Prosumers
Pending activations
Web users

Dilani Fernando
Backoffice officer
Log out

Stations

Solar microgrid stations, their battery places and opening hours.

+ Add station

STATION	LOCATION	CAPACITY	BATTERY SLOTS	HOURS	ACTIVE BOOKINGS	STATUS	ACTIONS
Battaramulla Microgrid Pannipitiya Road, Battaramulla	6.902, 79.918 Map	300 kW	12	07:00 - 19:00	0	Active	Slots Edit Deactivate
Kaduwela Grid Node Kaduwela Road, Kaduwela	6.9335, 79.984 Map	180 kW	8	06:00 - 18:00	1	Active	Slots Edit Deactivate
Kottawa Sun Park High Level Road, Kottawa	6.841, 79.965 Map	150 kW	6	06:00 - 18:00	0	Active	Slots Edit Deactivate
Malabe Solar Hub New Kandy Road, Malabe	6.9147, 79.973 Map	250 kW	10	06:00 - 18:00	2	Active	Slots Edit Deactivate
Nugegoda Rooftop Node Stanley Thilakaratne Mawatha, Nugegoda	6.8649, 79.8997 Map	90 kW	4	08:00 - 17:00	0	Deactivated	Slots Edit Activate

Figure 14: W9 Stations - BO (edit), GO (view)

W10 Station form

Name, address, latitude, longitude (+ "Open in Google Maps" link to check the pin), capacity kW, battery slots, open/close time.

 **SunShare**

 Dashboard

 Reservations

 **Stations & slots**

 Prosumers

 Pending activations 2

 Web users

Dilani Fernando

Backoffice officer

 Log out

Edit station

Tip: find the place in Google Maps, right-click it and copy the two numbers (latitude, longitude).

Station name

Address

Malabe Solar Hub

New Kandy Road, Malabe

Latitude

Longitude

6.9147

79.973

 Open in Google Maps

Capacity (kW)

Battery slots

Opens at

Closes at

250

10

06:00 

18:00 

Save changes

Cancel

Figure 15: W10 Station form - BO

Page 30 of 235

W11 Slots

Slot table for the station (date, time, total, booked, available, active). Add / Edit / Delete.

SunShare

Dashboard
Reservations
Stations & slots
Prosumers
Pending activations 2
Web users

Dilani Fernando
Backoffice officer
Log out

Slots - Malabe Solar Hub
← Stations + Add slot

Booking windows for the next 14 days. This station has 10 battery slots and opens 06:00 - 18:00.

DATE	TIME	TOTAL	BOOKED	FREE	BOOKING	ACTIONS
Wed 30 Sept	08:00 - 10:00	4	0	4	Open	Edit
Wed 30 Sept	11:00 - 13:00	4	1	3	Open	Edit
Wed 30 Sept	14:00 - 16:00	4	0	4	Open	Edit
Thu 01 Oct	08:00 - 10:00	4	0	4	Open	Edit
Thu 01 Oct	11:00 - 13:00	4	0	4	Open	Edit
Thu 01 Oct	14:00 - 16:00	4	1	3	Open	Edit
Fri 02 Oct	08:00 - 10:00	4	0	4	Open	Edit
Fri 02 Oct	11:00 - 13:00	4	0	4	Open	Edit
Fri 02 Oct	14:00 - 16:00	4	0	4	Open	Edit
Sat 03 Oct	08:00 - 10:00	4	0	4	Open	Edit
Sat 03 Oct	11:00 - 13:00	4	0	4	Open	Edit
Sat 03 Oct	14:00 - 16:00	4	0	4	Open	Edit
Sun 04 Oct	08:00 - 10:00	4	0	4	Open	Edit
Sun 04 Oct	11:00 - 13:00	4	0	4	Open	Edit
Sun 04 Oct	14:00 - 16:00	4	0	4	Open	Edit
Mon 05 Oct	08:00 - 10:00	4	0	4	Open	Edit
Mon 05 Oct	11:00 - 13:00	4	0	4	Open	Edit
Mon 05 Oct	14:00 - 16:00	4	0	4	Open	Edit
Tue 06 Oct	08:00 - 10:00	4	0	4	Open	Edit
Tue 06 Oct	11:00 - 13:00	4	0	4	Open	Edit
Tue 06 Oct	14:00 - 16:00	4	0	4	Open	Edit

Figure 16: W11 Slots - BO, GO

W12 Slot form

Start, end (datetime-local), total places, active toggle.

Figure 17: W12 Slot form - BO, GO

W13 Reservations

Filter bar (view: all/current/pending/history, status, station, date range, search). Table with status badges. Row actions shown only if API says canModify: Edit, Cancel; plus **Approve** for Pending.

WHEN	STATION	PROSUMER	ENERGY	STATUS	ACTIONS
Sun 27 Sept 11:00 - 13:00	Malabe Solar Hub 6abbcc580c7cb63f5b05fef3	Nimal Perera 199712345678	20 kWh Sell (drop off)	Completed	
Wed 30 Sept 11:00 - 13:00	Malabe Solar Hub 6abbcc580c7cb63f5b05fef1	Nimal Perera 199712345678	12.5 kWh Sell (drop off)	Approved	Edit Cancel
Thu 01 Oct 14:00 - 16:00	Malabe Solar Hub 6abbcc580c7cb63f5b05fef5	Ayesha Jayasinghe 200156789123	15 kWh Sell (drop off)	Pending	Approve Edit Cancel
Fri 02 Oct 08:00 - 10:00	Kaduwela Grid Node 6abbcc580c7cb63f5b05fef2	Nimal Perera 199712345678	8 kWh Buy (charge)	Pending	Approve Edit Cancel
Sat 03 Oct 14:00 - 16:00	Battaramulla Microgrid 6abbcc580c7cb63f5b05fef4	Nimal Perera 199712345678	5 kWh Buy (charge)	Cancelled	

Figure 18: W13 Reservations - BO, GO

W14 Reservation form

Pick prosumer (active) → station → available slot → kWh → type (Sell/Buy). Shows 7-day / 12-hour errors from API.



Dashboard

Reservations

Stations & slots

Prosumers

Pending activations2

Web users

Dilani Fernando

Backoffice officer

Log out

Edit reservation

Change the slot, energy amount or type.

Pending

Prosumer

Ayesha Jayasinghe (200156789123)

Station

Malabe Solar Hub

Time slot

Thu 01 Oct, 14:00 - 16:00 · 3 free

Energy (kWh)

15

More than 0 and up to 100 kWh.

Type

Sell - drop off spare energy

Buy - charge from the station

Save changes

Cancel

Figure 19: W14 Reservation form - BO, GO

9.2 Android app (Prosumer and Grid Operator)

Three screenshots per row; what each screen does is listed after the pictures.

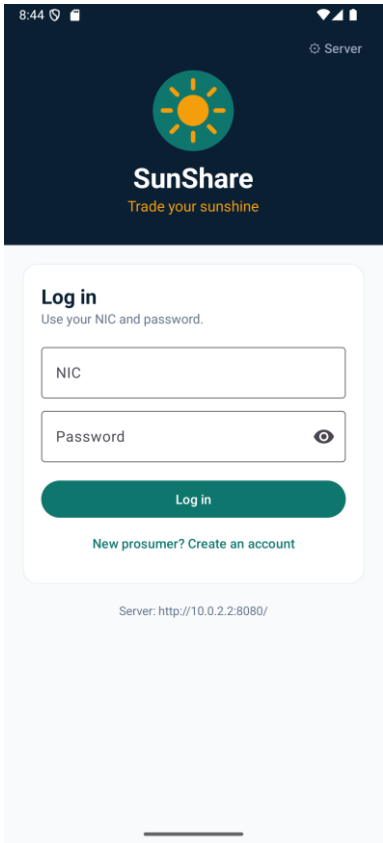


Figure 20: M1 Login

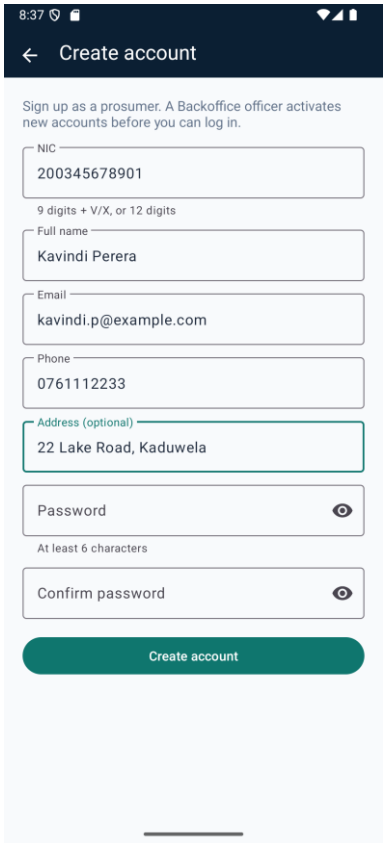


Figure 21: M2 Register

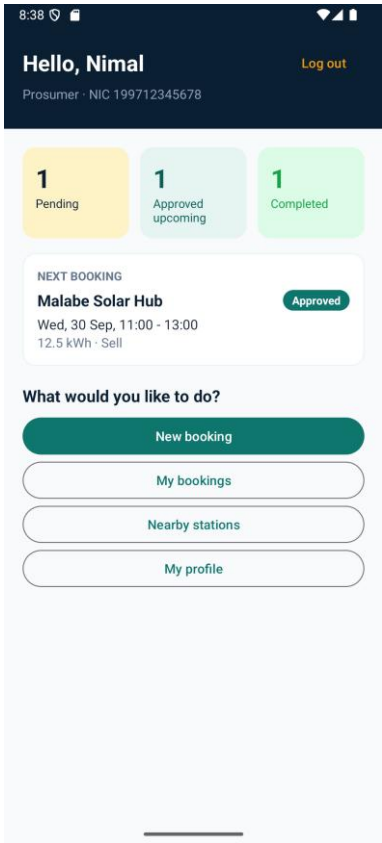


Figure 22: M3 Prosumer Home

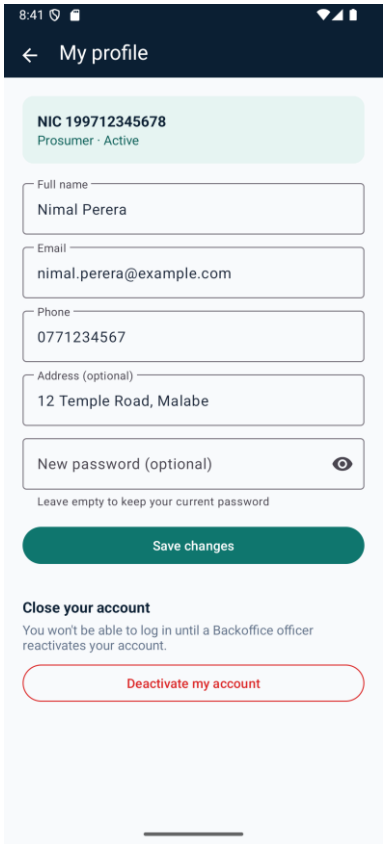


Figure 23: M4 Profile

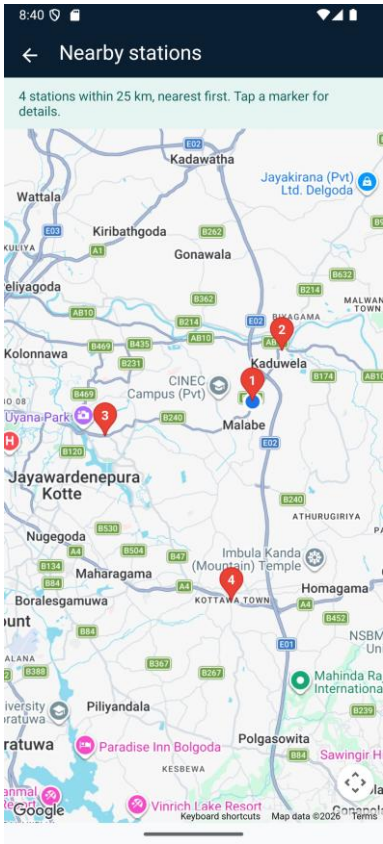


Figure 24: M5 Station Map - markers, nearest first

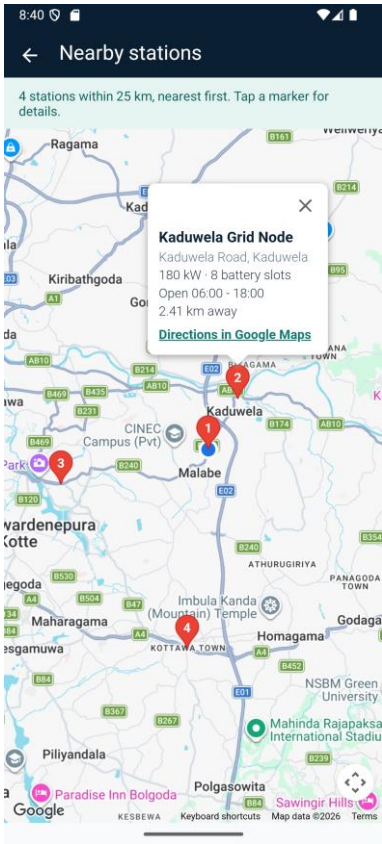


Figure 25: M5 Station Map - station details on tap

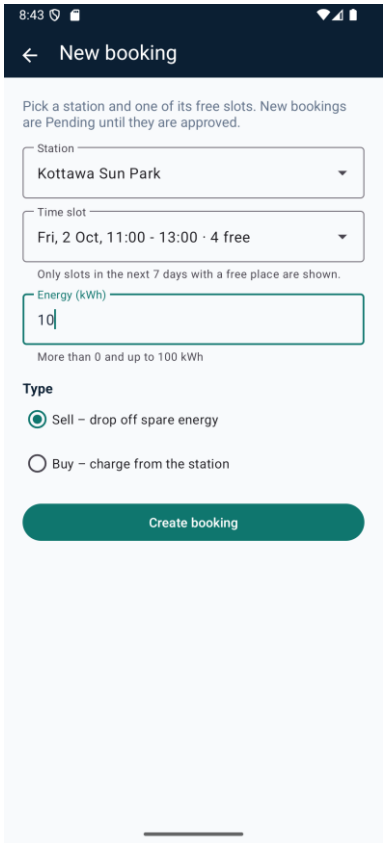


Figure 26: M6 Reservation Form

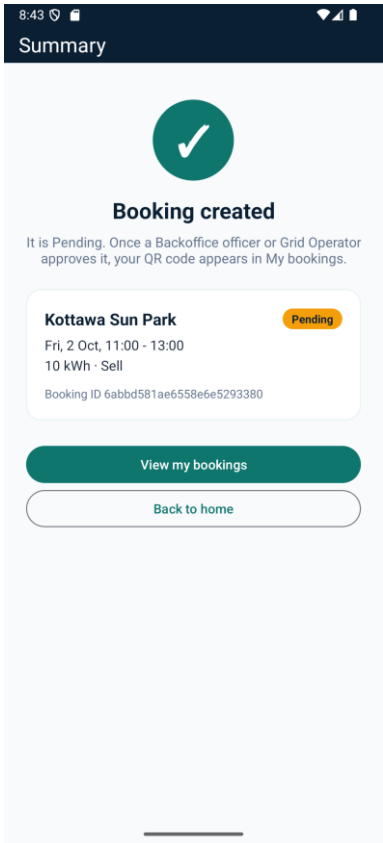


Figure 27: M7 Reservation Summary - after creating a booking

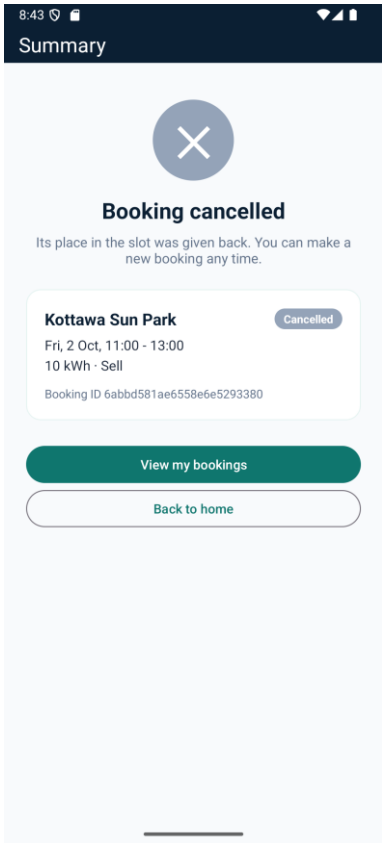


Figure 28: M7 Reservation Summary - after cancelling

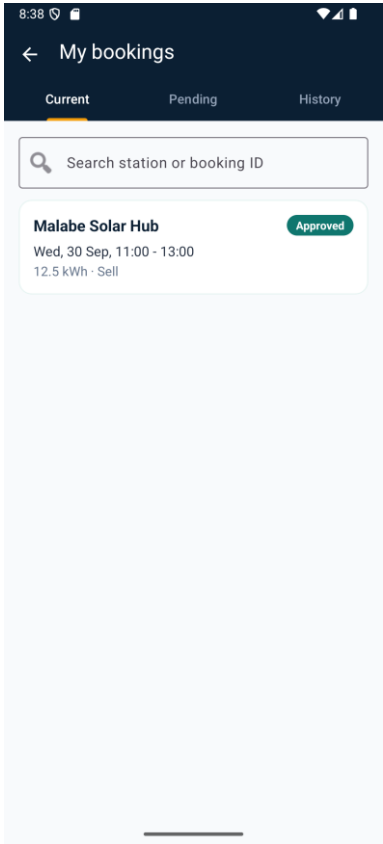


Figure 29: M8 Bookings - Current tab

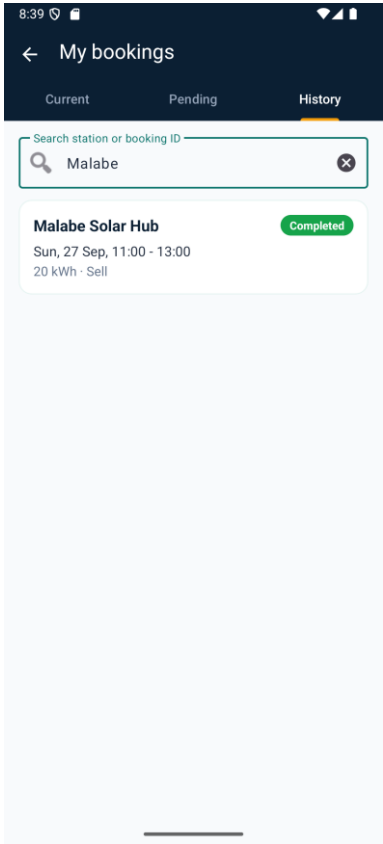


Figure 30: M8 Bookings - History tab with a search

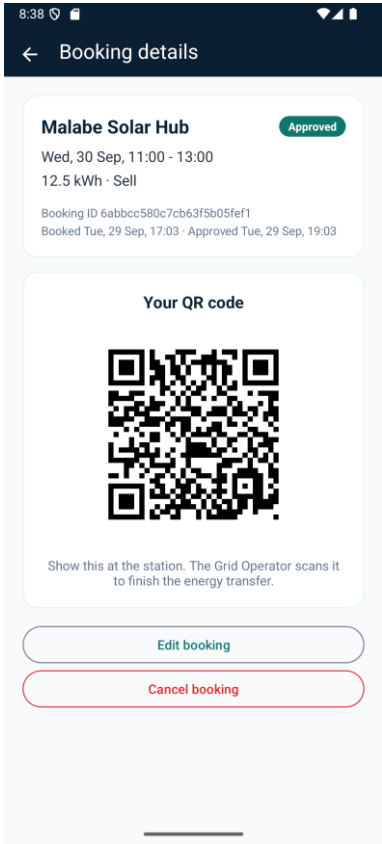


Figure 31: M9 Reservation Detail

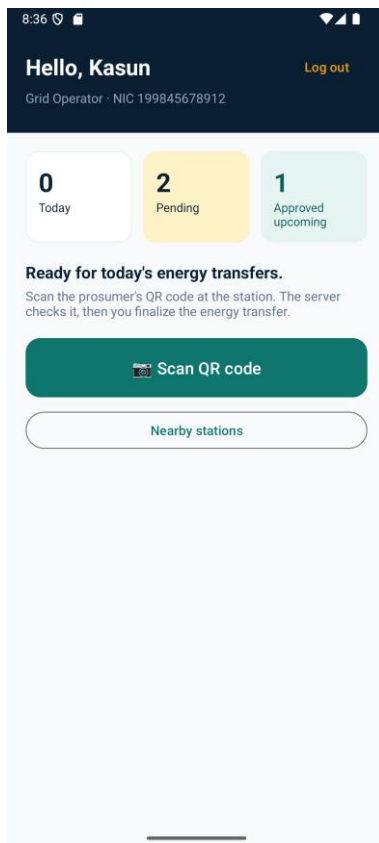


Figure 32: M10 Operator Home

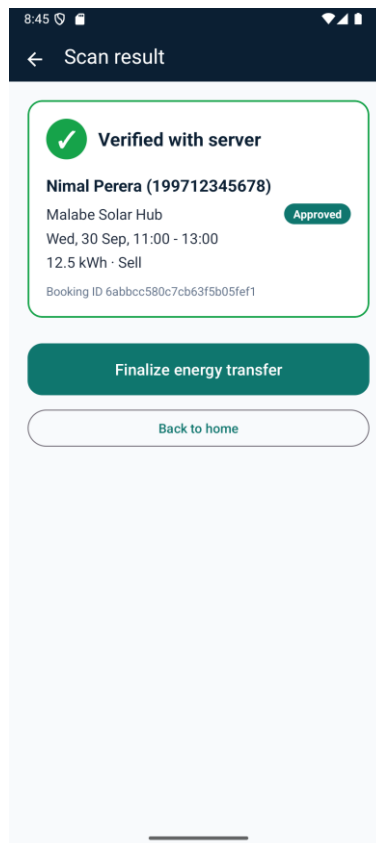


Figure 33: M11 Scan Result

What each Android screen does

M1 Login (LoginActivity, All) - If a session exists in SQLite → jump to the right home. Else NIC + password → POST /auth/login → save session in SQLite → **Prosumer** → **M3**, **GridOperator** → **M10**, Backoffice → "Please use the web app". ☺ icon → dialog to change **Server address** (saved in SQLite). Link to Register.

M2 Register (RegisterActivity, Public) - NIC, full name, email, phone, address, password, confirm password → POST /auth/register → message "Registered! A Backoffice officer will activate your account."

M3 Prosumer Home (ProsumerHomeActivity, PR) - Greeting + **count cards: Pending, Approved upcoming, Completed** + "Next booking" card. Buttons: New Reservation, My Bookings, Nearby Stations (map), Profile, Logout. GET /dashboard/prosumer (refresh on resume).

M4 Profile (ProfileActivity, PR) - View/edit name, email, phone, address, optional new password → PUT /profile. **Deactivate my account** (confirm dialog) → PATCH /profile/deactivate → clear SQLite session → Login.

M5 Station Map (StationMapActivity, PR, GO) - Google Map. Gets phone location (permission prompt) → GET /stations/nearby?lat&lng → markers for each station; **tap marker → info window with name, address, capacity, battery slots, schedule, distance**. Blue "my location" dot. Saves stations to SQLite stations_cache; if offline, shows cached stations + "offline" note.

M6 Reservation Form (ReservationFormActivity, PR) - Station dropdown (GET /stations?activeOnly=true) → available slot dropdown (GET /slots/available?stationId=) → kWh → Sell/Buy → POST or PUT /reservations. On success → M7.

M7 Reservation Summary (ReservationSummaryActivity, PR, GO) - **Shown after every action** (Created / Updated / Cancelled / Completed): big status icon, action title, booking details (station, time, kWh, type, status), "what happens next" line, buttons: View bookings / Home.

M8 Bookings (BookingsActivity, PR) - Tabs **Current • Pending • History** + **search box** (station name / id). GET /reservations?view=&search=. Tap row → M9.

M9 Reservation Detail (ReservationDetailActivity, PR) - Full details + status badge. If Approved → **QR code** made on the phone from qrData (ZXing). If canModify → **Edit** (→ M6) and **Cancel** (confirm → PATCH /cancel → M7).

M10 Operator Home (OperatorHomeActivity, GO) - Greeting + counts (today, pending, approved upcoming) from GET /dashboard/summary. Big **Scan QR** button (ZXing scanner), Nearby Stations (M5), Logout.

M11 Scan Result (ScanResultActivity, GO) - After scanning: POST /reservations/verify-qr → shows prosumer, station, time, kWh, type ✓ "Verified with server". **Finalize energy transfer** → PATCH /reservations/{id}/complete → M7 (Completed). Invalid QR → red card with the API's reason.

10. Design decisions

Every design decision was written down when it was made (59 decisions), with the alternatives we rejected and why. Later decisions sometimes replace earlier ones - for example the map plan changed from the native Google Maps SDK to the Maps JavaScript API after a test (D47, D51, D53).

#	Date	Decision	Alternatives considered	Why
D1	2026-09-27	Brand name SunShare , repo sunshare	HelioGrid, VoltSwap, solar-micro	Friendly and prosumer-focused: neighbours "sharing sunshine"
D2	2026-09-27	API: ASP.NET Core Web API on .NET 10 (LTS) , controller style	.NET 8 (installed), minimal APIs	Brief requires C# Web API; .NET 10 is the current long-term-support version; controllers map cleanly to "one controller per resource" and are easy to explain
D3	2026-09-27	Layers: Controller → Service → MongoDBContext (no repository layer)	Repository + Unit of Work	Services hold all business rules (FAT service); an extra repository layer adds files without adding understanding
D4	2026-09-27	MongoDB Community on the laptop	MongoDB Atlas (cloud)	Demo/viva works without internet; campus Wi-Fi may block Atlas
D5	2026-09-27	4 collections named exactly like the rubric; NIC as _id for all users with a role field	Separate Prosumers collection	Rubric lists exactly four collections; brief makes NIC the primary key
D6	2026-09-27	JWT login tokens + BCrypt password hashes	Sessions/cookies, plain passwords	Stateless tokens work the same for web and Android; never store plain passwords
D7	2026-09-27	Web: React (Vite) + JavaScript + Bootstrap 5 , HashRouter, fetch	React + Tailwind, Razor Pages, plain HTML	React is named in the brief and familiar; Bootstrap gives admin tables/forms fast and is marked in the rubric; HashRouter needs no IIS rewrite module
D8	2026-09-27	Android: Java + XML views , minSdk 26	Kotlin, Jetpack Compose	Java is closest to C#, so one coding style to explain; XML views are unmistakably "pure native"
D9	2026-09-27	Android HTTP: Retrofit + Gson	Volley, HttpURLConnection	Industry standard, one line per endpoint, still native Android
D10	2026-09-27	SQLite via SQLiteOpenHelper (no Room): tables session, stations_cache, app_settings	Room	Most literal match to "SQLite"; stores login details + reference data as the rubric asks
D11	2026-09-27	Reservations need staff approval (Pending → Approved) before a QR exists	Auto-approve	Gives real "pending" data for the dashboards the rubric marks, and matches "once approved, the app generates a QR"
D12	2026-09-27	QR contains <code>SUNSHARE <reservationId> <randomToken></code> ; token lives on the server	Signed JWT inside the QR	Simple and secure enough: nobody can fake a QR without the random token; easy to explain
D13	2026-09-27	"Nearby" distance is calculated in the API (Haversine)	Calculate on the phone	FAT service: even this logic stays on the server; the phone only plots results
D14	2026-09-27	API computes canModify for each reservation	Clients check the 12-hour rule themselves	Keeps the 12-hour rule in one place (API); clients just hide/show buttons
D15	2026-09-27	Demo on HTTP over LAN ; Android server address editable at runtime	HTTPS with a certificate; hard-coded IP	Certificates on IIS + Android add setup risk; editable address avoids rebuilding at the viva when the IP changes
D16	2026-09-27	Work split by feature (Parts A–D), each end-to-end (API + web + mobile)	Split by layer	Matches how individual marks are given; each member explains one complete story
D17	2026-09-27	AI coding assistants used (Claude Code, possibly Codex/Antigravity) under strict docs (CLAUDE.md), with rule tags and mandatory comments	No AI / unguided AI	Speed under a 3.5-day deadline while keeping code simple and explainable; disclosed in the report as the brief requires

#	Date	Decision	Alternatives considered	Why
D18	2026-09-27	API docs page: Swashbuckle Swagger UI , replacing the .NET 10 template's Microsoft.AspNetCore.OpenApi package; Swagger stays on in every environment	Built-in OpenAPI (JSON only, no UI) + a separate UI package	The .NET 10 template makes only the JSON description; Swashbuckle adds the click-to-test page with an Authorize button for JWT in one package, and it is on our allowed list
D19	2026-09-27	API runs on plain HTTP only (template's HTTPS profile and UseHttpsRedirection removed)	Keep HTTPS with a dev certificate	Matches D15: IIS and the phone use HTTP on the LAN; the redirect would only cause warnings and certificate problems
D20	2026-09-28	One branch + one pull request per phase , reviewed by Kvn, merged with a merge commit; nothing committed straight to main	Commit straight to main; squash-merge PRs	The brief asks for meaningful commits and critically checked AI output — a PR per phase is a visible review step and a tidy page to link in the report; merge commits keep every commit and co-author trailer
D21	2026-09-28	Android Gradle build files in Kotlin DSL (*.gradle.kts, Android Studio's default), while all app code stays Java	Groovy DSL (build.gradle)	It is Android Studio's default and recommended option; these files only configure the build (SDK versions, libraries) — they are not app source, so "no Kotlin source files" still holds
D22	2026-09-28	Removed the Android template's example unit/UI tests and their test libraries (JUnit, Espresso)	Keep them	The rubric doesn't mark automated tests; we test through Swagger, the browser and the emulator (golden rule 8). Unused files and libraries would only need comments and explaining
D23	2026-09-28	CORS: allowed list (5173, 8081) from appsettings.json; in Development only , any localhost port is allowed too	Fixed list everywhere; allow any origin	A preview browser or a second dev server may get a random port when 5173 is busy; production (IIS) stays strict, and outside sites are always blocked
D24	2026-09-28	The API starts even if MongoDB is down (start-up DB work is logged, not fatal) and gives up on MongoDB after 5 seconds (driver default is 30)	Crash on start; default 30 s timeout	/api/health can then honestly answer 503 "unreachable" instead of IIS showing a 500.30 error page; a 5 s wait is much friendlier in the demo. Requests that hit the DB while it is down get 503 { message }
D25	2026-09-28	Every error uses { "message" }: rule errors (ApiException), missing/expired token (401), wrong role (403), bad request bodies (400) and unexpected bugs (500, details only in the server log)	ASP.NET defaults (empty 401/403, ProblemDetails for 400)	Web and Android need one simple way to show errors: read message, show it
D26	2026-09-28	JWT carries short claim names nic, name, role (MapInboundClaims = false, RoleClaimType = "role")	.NET default long claim URLs (http://schemas.microsoft.com/.../role)	Easy to read on jwt.io at the viva and in code (User.FindFirstValue("nic")); [Authorize(Roles = ...)] still works
D27	2026-09-28	Required/format checks for request fields live in the services (plain if + friendly message); ASP.NET's automatic "required" check is switched off, and unreadable JSON gets a friendly 400	Data-annotation attributes ([Required], [EmailAddress]) on DTOs	FAT service: all validation is in one place and every message is human-friendly ("Please enter the full name." instead of "The FullName field is required.")
D28	2026-09-28	A slot's time can't be changed once it has bookings (409 "Add a new slot instead"); places and open/closed can still change	Move the bookings along with the slot	Booked prosumers chose that exact time, and each booking keeps a copy of it for the 7-day/12-hour rules (R9, R10); silently moving them would be unfair and would make those copies wrong
D29	2026-09-28	Renaming a station updates the stationName copies in its bookings	Keep the old name in old bookings (pure snapshot)	Booking lists and the search box (Part D) match on stationName; stale names would look like a bug in the demo
D30	2026-09-28	Malformed ids in URLs → 404 , and request times without a zone are taken as UTC	Let the driver throw (500); guess the laptop's time zone	Friendly, predictable answers for every client; the API never depends on the server's clock settings (IIS could run anywhere)
D31	2026-09-28	Taking a slot place is one conditional update (availableSlots > 0 in the filter + \$inc -1); 0 matches → 409 "slot is full"	Read the slot, check in C#, then save	Check-then-save leaves a gap where two bookings both see the last place; MongoDB does the check and the subtraction in one atomic step. Tested: two parallel bookings for 1 place → one 201, one 409, never below 0
D32	2026-09-28	Saving an edit with nothing changed leaves the booking as it is (an Approved one stays Approved)	Any PUT resets Approved → Pending	Only a real change should need re-approval (R13); pressing Save twice shouldn't cancel the prosumer's QR
D33	2026-09-28	Booking search matches the station name, or a reservation id only when the full 24-character id is typed	Partial id search	Ids are stored as MongoDB ObjectIds (not text), so "contains" doesn't work on them without slow conversions; people copy/scan whole ids anyway

#	Date	Decision	Alternatives considered	Why
D34	2026-09-28	"Today's reservations" = not Cancelled and starting on today's Sri Lanka date ; the dashboard's pending table only lists upcoming Pending bookings	UTC day; include past Pending	Staff think in local days; a past Pending booking can't be approved (approve needs a future start), so listing it next to an Approve button would only produce errors
D35	2026-09-29	One idempotent deploy script (scripts/deploy-api.ps1, uses IIS's appcmd) creates the app pool/site/firewall rule only when missing and republishes every time	Click through IIS Manager for each deploy; Visual Studio publish profile	Repeatable in one command before the viva (and after every fix); appcmd ships with IIS, so no extra PowerShell modules are needed; each step is commented so it doubles as the explanation of IIS hosting
D36	2026-09-29	Web talks to the dev API (:5080) during npm run dev and to the IIS API (:8080) in the built site (.env.development / .env.production → VITE_API_BASE_URL)	One address for both	Dev work can change and restart the API freely; the hosted demo always uses IIS. No address is hard-coded in the code
D37	2026-09-29	Login session kept in localStorage and in a React context (AuthContext); the fetch wrapper adds the token and logs out automatically on 401 ; expired sessions are dropped on load	Cookies; Redux	Survives a page refresh, simple to explain, no extra packages (Redux is banned); matches the Android app keeping its session in SQLite
D38	2026-09-29	With HashRouter , in-page jumps (skip link, "How it works") scroll/focus by code instead of href="#id"	Normal #anchor links	HashRouter uses the # part of the address for pages, so #how would open a page called "/how" (404)
D39	2026-09-29	One UserForm page for web users and prosumers (kind="staff" / kind="prosumer"), and one shared UserTable for both lists	Separate ProsumerForm / table copies	Both call the same /api/users endpoints with almost the same fields; one file is less to maintain and explain (prosumer = role fixed to Prosumer + address field)
D40	2026-09-29	Web lists load inside useEffect and set state only when the API answers, with a cancelled flag; actions ask for a reload with a reloadKey	A load() function that sets loading first and is called from the effect	Follows React's data-loading pattern (no linter warnings) and stops an older, slower answer from overwriting a newer one when filters change quickly
D41	2026-09-29	Web station pages check a pin with a plain Google Maps URLs link (maps/search/?api=1&query=lat,lng, opens in a new tab); the real map with markers is on Android (M5)	Embed a Google map on the web	The web only needs to <i>verify</i> coordinates; a link needs no API key or billing and nothing extra to explain
D42	2026-09-29	Slot times are typed in datetime-local boxes (local Sri Lanka time) and converted to UTC with new Date(value).toISOString() before sending; shown back in local time	Ask staff to type UTC	Staff think in local time; the API still receives and checks UTC only (D30), so no rule maths happens in the browser
D43	2026-09-29	One shared ReservationTable for the dashboard and the Reservations page; pages pass their own row buttons with renderActions (dashboard: Approve; Phase 11 adds Edit / Cancel / Approve)	Two separate tables	Same columns everywhere; Part C adds its actions without touching Part D's table code
D44	2026-09-29	Dashboard cards are links to the matching filtered list (/reservations?view=pending etc.)	Plain numbers	One click from "2 pending" to the 2 bookings; the list reads the view from the address
D45	2026-09-29	Reservation form only offers bookable slots (from GET /api/slots/available), and Edit/Cancel buttons appear only when the API says canModify	Show every slot and let the API refuse	Fewer dead ends for staff; the rules still live only in the API — the page just uses the API's own lists and flags (and the API re-checks everything on save)
D46	2026-09-29	Web app hosted on IIS as static files (site SunShareWeb :8081) deployed by scripts/deploy-web.ps1; web.config only sets the start page and the .woff2 type	Serve the React build from the API site; Node server	A React build is just HTML/CSS/JS — IIS serves it directly; separate sites keep the API (8080) and web (8081) independent, matching the architecture
D47	2026-09-29	Map screen built last (Android order 13 → 14 → 16 → 17 → 18 → 15). If no Google Maps key arrives in time: an OpenStreetMap in-app map (osmdroid, no key/card) with station markers, details on tap and "my location", plus an "Open in Google Maps" button per station (opens the Google Maps app, no key)	Wait for the key before any Android work; a list without a map; a Gemini/AI Studio key (does not work for Maps)	Google Maps Platform requires a billing account (card) even though Android map loads are free. Building the map last buys time to get a key from someone with a card; the fallback still earns "nearby stations on map" (5) and keeps Google Maps in the flow, at the risk of the 3 "Google Maps API" marks

#	Date	Decision	Alternatives considered	Why
D48	2026-09-29	Android ApiService lists only the 20 endpoints the phone screens call (M1–M11 in 05-SCREENS.md + health); Backoffice-only endpoints (users, activations, station/slot editing, approve) stay web-only. One shared ApiClient adds the token from the SQLite session table to every call	Every one of the 37 endpoints in the interface	Unused methods are dead code a member could be asked to explain; the phone has no Backoffice screens (Backoffice users are sent to the web app)
D49	2026-09-29	Phase 14 builds basic M3 Prosumer Home and M10 Operator Home (greeting, NIC, Log out; Prosumer also <i>My profile</i>) as Part D files , so Login has a home to open for each role. Later phases add their own buttons/cards to these screens (like the web Sidebar badge) and note it in the file header	Wait for Phase 16/18 before Login can be tested; put the homes in Part B	M3/M10 are Part D screens in 05-SCREENS.md, so Chamara owns them end to end; login routing (Part A) can be tested now
D50	2026-09-29	Every Android screen calls UiUtils.setupEdgeToEdge(activity, root, topBar) : the navy top bar extends under the status bar (white icons) and the root gets bottom padding for the navigation bar or the keyboard . Form errors show under the box (TextInputLayout.setError)	The template's per-screen insets code; adjustResize alone	Android 15+ forces edge-to-edge and adjustResize no longer shrinks the screen for the keyboard; one shared helper keeps every screen consistent and short. The old error popup covered the password eye icon
D51	2026-09-29	Get the map key as a Google Maps Demo Key (no credit card, daily limit, for prototyping). Phase 15 tries it in the native Maps SDK for Android first; if refused, the map screen shows the Maps JavaScript API in a WebView (pins + tap-for-details) with Java still doing location, the API call and the SQLite cache. OpenStreetMap (D47) only if neither works	Gemini "Map grounding" (AI text answers, no map, can't see our own stations); Maps Grounding Lite (turns place names into Place IDs, no map); Lite Mode (same SDK, same billing rule); a borrowed card key; OSM only	Every Maps Platform key needs a billing-enabled project (card) — except the Demo Key. Google's pages disagree on whether it covers Android, so we test the native SDK first; the JavaScript map is officially supported and is still the real Google Maps API, protecting the 3 "Google Maps" marks
D52	2026-09-29	Mobile lists: when a new tab or search starts, the previous Retrofit call is cancelled (call.cancel(), cancelled calls ignored in onFailure). Home count cards open the matching My bookings tab; the QR is drawn on the phone from the API's qrData with ZXing BarcodeEncoder	Let every answer update the list; a request counter	Same problem as the web (D40): a slow old answer could overwrite a newer tab. Cancelling is Retrofit's own tool and also saves data. Cards → tabs mirror the web dashboard links (D44)
D53	2026-09-29	Phase 15 map = Google Maps JavaScript API inside a WebView with the Maps Demo Key. Java does location, the /stations/nearby call and the SQLite cache; the small HTML page only draws markers and info windows	Native Maps SDK for Android; OpenStreetMap	Tested with a throwaway screen: the native SDK answered "Authorization failure" (Google's token service returned INVALID_ARGUMENT, blank map), while the JavaScript API drew tiles, a marker and an info window with the same key. It is still the real Google Maps API, and it needs no card
D54	2026-09-29	Android bookings: Edit / Cancel live in ReservationActions (Part C file) and the M9 detail screen (Part D) calls it with one line; the Summary (M7) receives the saved booking as JSON in the Intent (Gson) instead of loading it again; the form closes itself after saving and <i>Back to home</i> clears the screens above Home	Put Part C's cancel code inside Chamara's detail screen; reload the booking on the summary	Each member explains only their own file at the viva; the API's answer is already the truth, so a second call adds nothing; closing the form stops a double submit with Back
D55	2026-09-29	Operator mode: the scanner is ZXing's CaptureActivity through a tiny subclass QrScanActivity (only adds padding for the phone's bars); M11 never judges a QR — it sends the text to verify-qr and shows the server's answer; Finalize asks first and sends the same text again to complete	Build our own camera screen; check the QR format on the phone; finalize without a confirm	ZXing already handles camera, permission and decoding; one small override fixes the Android 15 hint problem (C11). The server is the only judge of a QR (R14) and re-checks on finalize because the booking may change in between; completing can't be undone (R15)
D56	2026-09-29	M5 map details: Java does permission, location (Fused, high accuracy, 10 s limit , a fix up to 1 min old is fine; otherwise SLIIT Malabe with a note), the /nearby call and the SQLite cache; assets/map.html only draws (markers numbered 1 = nearest, blue dot, info window built with textContent, list fallback if Google can't load). The WebView is created when the map is ready , not in the layout. <i>Directions</i> hands off to the Google Maps app via a Maps URL	Location/API calls in JavaScript; waiting for a location with no time limit; the WebView in the XML layout	Keeps rules and data in Java/API (FAT service, explainable), the page tiny and safe; the first WebView start (Chrome's engine) and slow GPS must never freeze the screen (C13)
D57	2026-09-29	The phone's SQLite file sunshare_local.db is excluded from Android backups and device transfer (backup_rules.xml, data_extraction_rules.xml)	Keep the template (backs up everything); turn backups off completely	The file holds the JWT login token; a token copied to the cloud or to another phone could be reused. After a restore the user just logs in again. Everything else in the app has nothing private to back up

#	Date	Decision	Alternatives considered	Why
D58	2026-09-29	Diagrams as PlantUML text (diagrams/*.puml) rendered to PNG with plantuml.jar (Maven Central, checksum checked) on Android Studio's Java, using PlantUML's built-in smetana layout. DFD level 1 is drawn in layers (entities → processes → stores)	draw.io / mermaid.live by hand; Mermaid CLI (needs a ~150 MB browser download); online renderers (send our diagrams to a third party)	Text diagrams can be reviewed in Git and re-drawn in seconds after a change; no Graphviz install; nothing leaves the laptop
D59	2026-09-29	The report is generated : report/generate-report.js (Node + the docx npm package) builds SunShare-Report.docx from the docs, the diagrams, the screenshots, each member's own report/contributions/*.md and every source file (git ls-files); report/update-toc.ps1 then lets Word fill in the contents page and save a PDF copy (C14). Member reflections are never written for them — an empty one shows a yellow "to be written" box	Write the report by hand in Word; pandoc (not installed); paste the code file by file	The report can't drift from the docs and the code: after a last fix, or when a member adds their reflection, npm run build rebuilds it in seconds. Pasting 160+ files by hand would miss some. docx is only a report tool — the apps don't use it

11. Challenges and reflection

These are the real problems we met while building SunShare (14 entries), what we did about them and what we learned. They were logged on the day they happened.

#	Date	Part	What went wrong	How we found / fixed it	What we learned
C1	2026-09-27	B (setup)	The laptop only had the .NET 8 SDK, but the API targets .NET 10. Microsoft's dotnet-install.ps1 downloaded .NET 10 fine but failed while unzipping: <i>"cannot access `dotnet.exe` because it is being used by another process"</i> .	Listed the running dotnet.exe processes (Get-CimInstance Win32_Process): two dotnet run servers from another project had been open since the night before and were locking the file. Rather than kill someone else's servers, we unzipped the downloaded SDK ourselves and skipped files that already existed (only the dotnet.exe launcher). dotnet --list-sdks then showed 8.0.425 and 10.0.401.	dotnet.exe is only a small launcher that hands over to the newest .NET version installed next to it, so several SDKs live side by side. Check tool versions on day one, and read the actual error before retrying.
C2	2026-09-28	Shared (API)	While testing login with a broken request body, the API answered with raw .NET parser text: <i>"The JSON value could not be converted to System.String. Path: \$.nic..."</i> — not the human-friendly message our rules promise.	Found by deliberately sending wrong/missing JSON during Phase 2 testing (not just the happy path). Fixed the 400 handler in Program.cs to name the bad field in plain words ("The value of 'nic' is not in the right format.").	Test the error paths, not only the success paths — framework defaults can leak technical text to users.
C3	2026-09-28	B (API)	A build during Phase 3 reported "2 errors", yet the commit went ahead: the check only looked for the word "Error(s)", not for 0 errors. The errors were MSB3027: file is locked by SunShare.Api — Kvn's own dotnet run (from trying Swagger) was still running and holding bin\SunShare.Api.exe.	Read the full build output; found the running process with Get-CimInstance Win32_Process (it was Kvn's, so it was left alone). Built into a separate output folder (-p:OutDir=..) to prove the code itself compiled (0 errors, 0 warnings), and tested on a second port (5081). Replaced the loose check with a strict one that stops unless both counts are 0.	A running app locks its own files on Windows — stop it (Ctrl+C) before building. And a check must test for success, not just for the presence of a summary line.
C4	2026-09-29	B (hosting)	First IIS deploy: every request gave HTTP 500.19 (0x80070021) "This configuration section cannot be used at this path... locked at a parent level", pointing at <modules><remove name="WebDAVModule" /> in our web.config.	Read IIS's detailed error page (curl on the laptop shows full details). IIS's server-wide applicationHost.config locks the <modules> section, so a site may not change it. WebDAV isn't even installed on the laptop, so we removed the <modules> block and kept only <handlers><remove name="WebDAV" />. Redeployed; proved PUT/PATCH/DELETE reach the API (our own 400/403/404, no 405).	Copy-pasted "best practice" config can clash with the server's locks. Read the exact error code, fix the smallest thing, then prove the original goal (no 405) still holds.
C5	2026-09-29	B (setup)	After installing the Hosting Bundle, new terminals would say "No .NET SDKs were found": it put a runtime-only C:\Program Files\dotnet on the system PATH, which Windows searches before the user PATH where our SDK lives.	Checked both PATH lists and the installed runtimes/SDKs with PowerShell. The deploy script now looks for a dotnet that actually has an SDK; the permanent fix is installing the SDK next to that runtime (winget install --id Microsoft.DotNet.SDK.10 -e).	Installers change PATH; the first match wins. When a command suddenly "forgets" something, check which copy of the program is running.
C6	2026-09-29	A (web)	Two redirect bugs found while clicking through the new Login page: (1) after logging in from a deep link (e.g. /#/prosumers) the app still went to the Dashboard; (2) after Log out from a Backoffice-only page, the next user (a Grid Operator) landed on that page and saw "no access".	Scripted login/logout runs in the browser showed the wrong final address. Cause: React Router changes pages inside a low-priority React <i>transition</i> , so a state change made at the same time (the user being saved / forgotten) re-rendered first and triggered a different redirect. Fixes: Login keeps a single redirect ("logged in → page asked for, else dashboard"); logout forgets the user and navigates inside the same startTransition, so both happen in one step. Re-tested all four paths.	When two things change "at the same time" in React, their order can differ from the code order. Test the unhappy paths (deep links, switching users), not only a clean login.
C7	2026-09-29	B (setup)	Could not create the Google Maps API key: Google Cloud needs a billing account, and the card authorization hold was more than Kvn could cover. A Google AI Studio key was suggested but only works for the Gemini AI service, not Maps.	Checked the rubric: 5 marks need "nearby stations on a map with details" (any map), 3 need a real Google map. Decided (D47): build the map screen last, try to get a key from someone with a card (key locked to our package + SHA-1, free Android map loads, \$1 budget alert), otherwise use an OpenStreetMap map plus an	External services can have non-technical blockers (money, accounts). Find out what is really marked, reorder work so the blocker isn't on the critical path, and prepare a fallback early.

#	Date	Part	What went wrong	How we found / fixed it	What we learned
				"Open in Google Maps" hand-off. Update (D51): checked the suggested AI Studio "Map grounding", Maps Grounding Lite and Lite Mode — none shows our stations on a map without a billing project — and found Google's no-card Maps Demo Key .	
C8	2026-09-29	B (Android shell)	On the first emulator run the server-check card touched both screen edges although the layout had 24dp padding. The Android Studio template's edge-to-edge code (<code>setOnApplyWindowInsetsListener → v.setPadding(...)</code>) replaces the root layout's padding with the size of the status/navigation bars.	Kept the template code (it stops content hiding under the phone's bars) and gave the card its own side margin. Rule for every new screen: put spacing on an inner layout or as margins, never as padding on the root view with id <code>main</code> .	Template code can silently undo your XML. Always look at the real screen (screenshot) — the build succeeding proves nothing about layout.
C9	2026-09-29	A (Android auth)	Testing Register on the emulator: the "passwords don't match" error appeared as Android's black popup with a red icon that sat on top of the show-password eye , and with the keyboard open the Create account button was pushed out of sight.	Errors now go under the box (<code>TextInputLayout.setError</code> , <code>sharedUtils.showFieldError</code>), and the edge-to-edge helper pads the screen by the keyboard height so the form scrolls above it (D50). Re-tested: error under the box, button reachable with the keyboard open.	Test forms with the real on-screen keyboard and with every error path, not just the happy path — layout problems only show up there.
C10	2026-09-29	B (map)	Google's pages disagreed on whether the no-card Maps Demo Key works in Android apps.	Before building the map screen, a 10-minute throwaway test: a plain native map showed only a beige background and logcat said "Authorization failure"; the same key in a WebView with the Maps JavaScript API showed a full Google map with a marker and info window. The test code was deleted, not committed. Decision D53.	When documentation is unclear, a tiny experiment answers faster and more reliably than reading more — and do it before building the real feature.
C11	2026-09-29	D (operator scan)	(1) The ZXing scanner's hint text ("Point the camera at the prosumer's QR code") was hidden under the navigation bar — Android 15's edge-to-edge also applies to library screens. (2) The emulator's camera shows a pretend room, so it can't scan a real QR code.	(1) <code>QrScanActivity</code> extends <code>ZXing's CaptureActivity</code> and pads its view by the bars' size (D55). (2) Tested M11 by making it openable from adb for the test only and passing the exact QR text (invalid text, wrong token, the real code, the same code twice); the change was reverted before committing. The real camera scan is done on a physical phone (Phase 18 check).	Separate <i>what can be tested automatically</i> (the server logic and screens) from <i>what needs real hardware</i> (the camera), and say honestly which is which.
C12	2026-09-29	B (demo networking)	The real phone said "Can't reach the SunShare server at <code>http://172.20.10.4:8080/</code> " although Kvn said both devices were on the same Wi-Fi. Using adb on the plugged-in phone: phone <code>192.168.1.8</code> , laptop <code>172.20.10.4</code> — two different address ranges under the same Wi-Fi name ("SLT-FIBER"), and a ping from the phone to the laptop got 0 replies. The firewall rule was fine (it applies to all network types).	For testing now: <code>adb reverse tcp:8080 tcp:8080</code> carries the phone's port 8080 through the USB cable, and the app's server address becomes <code>http://localhost:8080/</code> . For the viva: put both on one range — easiest is the Android phone's own hotspot for the laptop. Both ways written in the setup doc §7.	"Same Wi-Fi" means the same <i>address range</i> , not the same name. Check the IP on both devices first, then the firewall.
C13	2026-09-29	B (map)	Opening <i>Nearby stations</i> on the emulator showed "SunShare isn't responding", later the emulator's system restarted, and once the map sat on "Finding your location..." for ever.	Measured instead of guessing: timing logs showed <code>setContentView</code> took 2.8 s — the first <code>WebView</code> starts Chrome's engine on the main thread; the emulator (2 GB RAM, laptop 1.7 GB free) was swapping, and its low-memory killer killed apps. The location request used <i>balanced accuracy</i> (Wi-Fi/cell), which the emulator doesn't have. Fixes: <code>WebView</code> created only when the map is ready; high-accuracy location with a 10 s limit and a SLIIT Malabe fallback; emulator restarted with 3 GB. Result: the screen opens at once, no freeze, map in ~10 s. Temporary timing logs removed before committing.	A freeze is a symptom — time each step to find the slow one. Never wait for a sensor without a time limit. Test on a device with realistic memory.
C14	2026-09-29	B (report)	Saving the ~235-page report as PDF with Word (automated from PowerShell, <code>update-toc.ps1</code>) ran for over 15 minutes and never	Timed it instead of waiting: 20-page chunks took ~2 s each, and a small test script exported all pages in 33 s — every time — while the real script stayed slow. Changing one thing at a time ruled	Measure, then change one thing at a time — our first guesses were wrong. Between two programs (PowerShell → Word), pass plain values (text, numbers), not objects.

#	Date	Part	What went wrong	How we found / fixed it	What we learned
			finished, several times. Word used little CPU while the PowerShell process driving it stayed busy.	out the printer, the output folder, the working folder, functions and separate processes. The real difference: the test passed the PDF path as a plain string, the real script as the result of <code>Join-Path</code> , which PowerShell hands to Word as a <i>wrapped object</i> ; Word then kept calling back into PowerShell for it while writing. Fix: <code>[string]\$pdf = Join-Path ...</code> → the whole script (contents + PDF) takes about 1 minute. Also: page-range export without accessibility tags, and the code appendix marked "no proofing" so Word doesn't spell-check 14,000 lines of code.	

12. Individual contributions and AI disclosure

The work was split into four Parts so that each member owns complete features end to end (API, web and Android), matching the individual marking scheme. Kvn (group lead) made every commit on GitHub; each commit that belongs to another member's Part carries a Co-authored-by: line naming that member.

12.1 Use of AI tools (shared statement)

The group used **Claude Code** (Anthropic) as an AI coding assistant, driven by the group lead Ranathunga R A K N. It was used to turn the brief into a phased build plan and the project documents in docs/, to write the code phase by phase (API, web app, Android app), to test the running system (Swagger calls, browser, Android emulator), to draw the diagrams, to take the screenshots and to generate this report.

How the AI output was controlled: the rules, names and screens were fixed in the documents first (01-SPEC, 02-ARCHITECTURE, 03-DATABASE, 04-API, 05-SCREENS) and the AI had to follow them; every phase was built on its own branch and merged through a pull request only after the group lead reviewed and tested it on the IIS-hosted build, the emulator and a real phone; every decision and problem was logged (chapter 10 and 11). Several AI suggestions were changed or rejected after testing - for example the map approach (D47, D51, D53) and the fixes in C3, C8, C9, C12 and C13. Each member's own statement follows in their section.

12.2 Part A - Gimhan T P K (IT22266996)

Area: Accounts & access.

Features: Web login + role redirect; web users (Backoffice/Grid Operator); prosumer management; **Pending Activations** page; mobile login with role-based home, register, edit profile, deactivate; session saved in SQLite

Business rules: R1–R5, R16 · **Marking-scheme lines:** Web: Login & role-based access (4), User Management (4) · Mobile Auth & Account Mgmt (9) · SQLite login persistence (part of 3)

Summary: Gimhan T P K owns **who can get in and what they can do:** login with role-based routing (web + mobile), creating staff users, managing prosumers, the **Pending Activations** page, and mobile register / edit profile / deactivate.

Files

- API (Phase 2): Controllers/AuthController.cs, UsersController.cs, ProfileController.cs · Services/AuthService.cs, UserService.cs · Helpers/JwtTokenHelper.cs, NicValidator.cs · Dtos/LoginRequest.cs, LoginResponse.cs, RegisterRequest.cs, CreateUserRequest.cs, UpdateUserRequest.cs, UpdateProfileRequest.cs, UserResponse.cs
- Web: pages/Login.jsx, Users.jsx, UserForm.jsx, Prosumers.jsx, ProsumerForm.jsx, PendingActivations.jsx · api/authApi.js, usersApi.js
- Android (Phase 14): ui/auth/LoginActivity.java, RegisterActivity.java · ui/prosumer/ProfileActivity.java · layouts/activity_login.xml, activity_register.xml, activity_profile.xml, dialog_server_address.xml

Commits: 16 commits carry "Co-authored-by: Gimhan T P K" (visible on GitHub under each commit).

AI tools used and personal reflection

[To be written by Gimhan T P K in their own words: the AI tools they used, what for, what they checked, changed or rejected, and what they learned - edit report/contributions/A-Gimhan.md and rebuild.]

12.3 Part B - Ranathunga R A K N (IT22552860)

Area: Stations, slots & map; shared foundation, hosting (group lead).

Features: Stations create/update/deactivate/delete; slot schedules; nearby-stations API; **Google Map** with station details; stations cached in SQLite; shared foundation (Mongo, JWT, errors, seeding), IIS hosting, web shell + Home page, Android shell

Business rules: R6–R8, R18 · **Marking-scheme lines:** Web: Microgrid Node Management (5) · Nearby stations on map (5) · Google Maps (3) · SQLite reference data (part of 3) · IIS hosting & MongoDB connection (Table 1)

Summary: Ranathunga R A K N owns **where and when energy can be traded**: solar stations (create, edit schedule, deactivate, delete), their time **slots**, the **nearby-stations Google Map** on the phone with details on tap — plus the shared foundation (MongoDB connection, errors, seeding), **IIS hosting**, and the web/Android app shells.

Files

- API (Part B) — **built in Phase 3**: Controllers/StationsController.cs, SlotsController.cs · Services/StationService.cs, SlotService.cs · Helpers/GeoHelper.cs · Dtos/StationRequest.cs, StationResponse.cs, SlotRequest.cs, SlotUpdateRequest.cs, SlotResponse.cs
- API (shared foundation) — **built in Phase 1** (all in api/SunShare.Api/):
 - Program.cs (start-up), appsettings.json (Mongo address, JWT settings, CORS list), web.config (IIS)
 - Data/MongoDbSettings.cs, Data/MongoDbContext.cs, Data/DataSeeder.cs
 - Models/User.cs, SolarStation.cs, EnergyBookingSlot.cs, EnergyReservation.cs, Roles.cs, UserStatuses.cs, ReservationStatuses.cs, ReservationTypes.cs
 - Helpers/ApiException.cs, Middleware/ErrorHandlerMiddleware.cs, Controllers/HealthController.cs
 - scripts/deploy-api.ps1 — built in Phase 6 (IIS deploy)
- Web — Home (Phase 7), Stations / Slots (Phase 9): pages/Home.jsx, Stations.jsx, StationForm.jsx, Slots.jsx, SlotForm.jsx, NotFound.jsx · api/client.js, stationsApi.js, slotsApi.js · components/* · context/AuthContext.jsx · styles/theme.css · App.jsx, main.jsx
- Android shell — **built in Phase 13** (in android/app/src/main/java/com/sunshare/app/): api/ApiConfig.java, ApiClient.java, ApiService.java, ApiErrorParser.java, api/models/* (15 classes) · db/SunShareDbHelper.java, db/Session.java · util/DateUtils.java, UiUtils.java, SessionGuard.java · MainActivity.java (temporary server check) · res/values/*, res/layout/activity_main.xml, AndroidManifest.xml, res/xml/network_security_config.xml, app/build.gradle.kts
- Android map (Phase 15): ui/map/StationMapActivity.java · assets/map.html · res/layout/activity_station_map.xml · uses db/SunShareDbHelper.java (stations_cache) · Nearby stations buttons on both home screens · api/ApiClient.java, ApiService.java, ApiConfig.java, ApiErrorParser.java, api/models/* · util/* · res/values/*, AndroidManifest.xml, res/xml/network_security_config.xml

Commits: all commits were made from Ranathunga R A K N's GitHub account (see <https://github.com/kavindurakn/sunshare/commits/main>); Part B commits have no co-author line.

AI tools used and personal reflection

[To be written by Ranathunga R A K N in their own words: the AI tools they used, what for, what they checked, changed or rejected, and what they learned - edit report/contributions/B-Ranathunga.md and rebuild.]

12.4 Part C - Malkith G W L (IT22630834)

Area: Reservations.

Features: Reservation create / update / cancel (web + mobile), approve (web), 7-day & 12-hour rules, slot availability counter, **summary page after each action**

Business rules: R9–R13, R15, R17 · **Marking-scheme lines:** Web: Slot Booking Management (5) · Reservation Workflow & Booking Mgmt (9)

Summary: Malkith G W L owns **booking itself**: create, update and cancel reservations (web + mobile), staff **approval**, the **7-day** and **12-hour** rules, keeping slot places correct, and the **summary page** shown after every action.

Files

- API: Controllers/ReservationsController.cs · Services/ReservationService.cs · Helpers/ReservationMapper.cs (builds ReservationResponse incl. canModify and qrData, shared with Part D) · Dtos/CreateReservationRequest.cs, UpdateReservationRequest.cs, ReservationResponse.cs
- Web: pages/ReservationForm.jsx · action functions in api/reservationsApi.js · Edit/Cancel/Approve buttons in pages/Reservations.jsx (shared with D)
- Android (Phase 17): ui/prosumer/ReservationFormActivity.java, ReservationSummaryActivity.java, ReservationActions.java (Edit / Cancel on Chamara's detail screen, D54) · layouts activity_reservation_form.xml, activity_reservation_summary.xml · the *New booking* button on Prosumer Home

Commits: 10 commits carry "Co-authored-by: Malkith G W L" (visible on GitHub under each commit).

AI tools used and personal reflection

[To be written by Malkith G W L in their own words: the AI tools they used, what for, what they checked, changed or rejected, and what they learned - edit report/contributions/C-Malkith.md and rebuild.]

12.5 Part D - Chamara R M L K (IT22076816)

Area: Dashboards & QR.

Features: Reservation lists (current / pending / history) + **search filter**; web dashboard + mobile dashboards (**pending, approved future count**); QR shown on mobile; operator **scan → verify → finalize**

Business rules: R14 + dashboard definitions · **Marking-scheme lines:** Booking Views & Dashboards (10) · Read QR & mark job done (2) · QR scanning (2)

Summary: Chamara R M L K owns **seeing and finishing bookings**: booking lists (current / pending / history) with a **search filter**, the **dashboards** (pending reservations + count of approved future reservations), the **QR code** on the prosumer's phone, and the operator's **scan → verify with server → finalize** flow.

Files

- API: Controllers/ReservationViewsController.cs, QrController.cs, DashboardController.cs · Services/ReservationQueryService.cs, QrService.cs, DashboardService.cs · Dtos/VerifyQrRequest.cs, StaffDashboardResponse.cs, ProsumerDashboardResponse.cs
- Web: pages/Dashboard.jsx, Reservations.jsx (list + filter bar; C adds row actions) · api/dashboardApi.js + list functions in api/reservationsApi.js
- Android prosumer views (Phase 16; basic homes from Phase 14, D49): ui/prosumer/ProsumerHomeActivity.java, BookingsActivity.java, BookingAdapter.java, ReservationDetailActivity.java · layouts activity_prosumer_home.xml, activity_bookings.xml, item_booking.xml, activity_reservation_detail.xml · ui/operator/OperatorHomeActivity.java + activity_operator_home.xml (basic)
- Android operator mode (Phase 18): ui/operator/OperatorHomeActivity.java (counts + **Scan QR**), QrScanActivity.java (ZXing scanner screen), ScanResultActivity.java · layouts activity_operator_home.xml, activity_scan_result.xml

Commits: 15 commits carry "Co-authored-by: Chamara R M L K" (visible on GitHub under each commit).

AI tools used and personal reflection

[To be written by Chamara R M L K in their own words: the AI tools they used, what for, what they checked, changed or rejected, and what they learned - edit report/contributions/D-Chamara.md and rebuild.]

13. References

References are in IEEE style. Where code follows one of these sources, the code has a // Reference: comment next to it.

- [1] Microsoft, "Create a web API with ASP.NET Core and MongoDB," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mongo-app> [Accessed: 27-Sep-2026].
- [2] Microsoft, "Host ASP.NET Core on Windows with IIS," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/> [Accessed: 27-Sep-2026].
- [3] Microsoft, "Configure JWT bearer authentication in ASP.NET Core," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication> [Accessed: 27-Sep-2026].
- [4] MongoDB Inc., "MongoDB C# Driver documentation." [Online]. Available: <https://www.mongodb.com/docs/drivers/csharp/current/> [Accessed: 27-Sep-2026].
- [5] Swashbuckle, "Swashbuckle.AspNetCore," *GitHub*. [Online]. Available: <https://github.com/domaindrivendev/Swashbuckle.AspNetCore> [Accessed: 27-Sep-2026].
- [6] BCrypt.Net, "BCrypt.Net-Next," *GitHub*. [Online]. Available: <https://github.com/BcryptNet/bcrypt.net> [Accessed: 27-Sep-2026].
- [7] Bootstrap Team, "Bootstrap 5 documentation." [Online]. Available: <https://getbootstrap.com/docs/5.3/> [Accessed: 27-Sep-2026].
- [8] Meta Platforms, "React documentation." [Online]. Available: <https://react.dev/> [Accessed: 27-Sep-2026].
- [9] Vite, "Getting Started." [Online]. Available: <https://vite.dev/guide/> [Accessed: 27-Sep-2026].
- [10] React Router, "React Router documentation." [Online]. Available: <https://reactrouter.com/> [Accessed: 27-Sep-2026].
- [11] Google, "Save data using SQLite," *Android Developers*. [Online]. Available: <https://developer.android.com/training/data-storage/sqlite> [Accessed: 27-Sep-2026].
- [12] Square, "Retrofit: A type-safe HTTP client for Android and Java." [Online]. Available: <https://square.github.io/retrofit/> [Accessed: 27-Sep-2026].
- [13] Google, "Gson," *GitHub*. [Online]. Available: <https://github.com/google/gson> [Accessed: 27-Sep-2026].
- [14] JourneyApps, "ZXing Android Embedded," *GitHub*. [Online]. Available: <https://github.com/journeyapps/zxing-android-embedded> [Accessed: 27-Sep-2026].
- [15] Google, "Maps SDK for Android overview," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk/overview> [Accessed: 27-Sep-2026].
- [16] Google, "Get the last known location," *Android Developers*. [Online]. Available: <https://developer.android.com/develop/sensors-and-location/location/retrieve-current> [Accessed: 27-Sep-2026].
- [17] C. Veness, "Calculate distance, bearing and more between Latitude/Longitude points," *Movable Type Scripts*. [Online]. Available: <https://www.movable-type.co.uk/scripts/latlong.html> [Accessed: 27-Sep-2026].
- [18] Anthropic, "Claude Code overview." [Online]. Available: <https://docs.claude.com/en/docs/claude-code/overview> [Accessed: 27-Sep-2026]. (*AI tool disclosure*)
- [19] Vite, "Configuring Vite." [Online]. Available: <https://vite.dev/config/> [Accessed: 27-Sep-2026].
- [20] R. Rasmussen, "Inter typeface," *Google Fonts*. [Online]. Available: <https://fonts.google.com/specimen/Inter> [Accessed: 29-Sep-2026].
- [21] Bootstrap Team, "Bootstrap Icons." [Online]. Available: <https://icons.getbootstrap.com/> [Accessed: 29-Sep-2026].
- [22] Google, "Get started with Maps URLs," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/documentation/urls/get-started> [Accessed: 29-Sep-2026].
- [23] Square, "Interceptors," *OkHttp*. [Online]. Available: <https://square.github.io/okhttp/features/interceptors/> [Accessed: 29-Sep-2026].
- [24] Google, "Get a Maps Demo Key and use it with Google Maps Platform," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/demo-key> [Accessed: 29-Sep-2026].
- [25] Google, "Maps SDK for Android Usage and Billing," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk/usage-and-billing> [Accessed: 29-Sep-2026].
- [26] Google, "Load the Maps JavaScript API," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/documentation/javascript/load-maps-js-api> [Accessed: 29-Sep-2026].

- [27] Google, "Build web apps in WebView," *Android Developers*. [Online]. Available: <https://developer.android.com/develop/ui/views/layout/webapps/webview> [Accessed: 29-Sep-2026].
- [28] PlantUML, "PlantUML - Open-source tool that uses simple textual descriptions to draw UML diagrams." [Online]. Available: <https://plantuml.com/> [Accessed: 29-Sep-2026].

Appendix A. Source code

The complete source code of SunShare, as text, in the same folder structure as the Git repository (<https://github.com/kavindu-rakn/sunshare>). Build output, downloaded packages and images are left out.

A.1 Web API (C#, ASP.NET Core)

api/SunShare.Api/Controllers/AuthController.cs

```
/*
 * =====
 * File      : AuthController.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : Public endpoints POST /api/auth/login and
 *               POST /api/auth/register (no token needed).
 * =====
 */
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// [AllowAnonymous] = anyone may call these; you can't have a token before you log in.
[ApiController]
[Route("api/auth")]
[AllowAnonymous]
public class AuthController : ControllerBase
{
    private readonly AuthService _auth;

    // Receives the auth service (registered in Program.cs).
    public AuthController(AuthService auth)
    {
        _auth = auth;
    }

    // POST /api/auth/login - NIC + password -> token, NIC, name, role, expiry.
    // Errors: 401 wrong NIC/password, 403 Pending (R3) or Deactivated (R4).
    [HttpPost("login")]
    public async Task<ActionResult<LoginResponse>> Login(LoginRequest request)
    {
        return Ok(await _auth.LoginAsync(request));
    }

    // POST /api/auth/register - mobile sign-up; the new prosumer starts Pending (R3). Returns 201.
    // Errors: 400 bad NIC or missing details (R1), 409 NIC already registered.
    [HttpPost("register")]
    public async Task<ActionResult<UserResponse>> Register(RegisterRequest request)
    {
        UserResponse user = await _auth.RegisterAsync(request);
        return Created($"api/users/{user.Nic}", user);
    }
}
```

api/SunShare.Api/Controllers/DashboardController.cs

```
/*
 * =====
 * File      : DashboardController.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : GET /api/dashboard/summary (staff) and
 *               GET /api/dashboard/prosumer (prosumer home).
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
```

```

using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Dashboard numbers. Staff and prosumers each get their own endpoint (RULE R2).
[ApiController]
[Route("api/dashboard")]
[Authorize]
public class DashboardController : ControllerBase
{
    private readonly DashboardService _dashboard;

    // Receives the dashboard service (registered in Program.cs).
    public DashboardController(DashboardService dashboard)
    {
        _dashboard = dashboard;
    }

    // GET /api/dashboard/summary - Backoffice and Grid Operator: count cards + next pending bookings.
    [HttpGet("summary")]
    [Authorize(Roles = Roles.Staff)]
    public async Task<ActionResult<StaffDashboardResponse>> Summary()
    {
        return Ok(await _dashboard.GetStaffSummaryAsync());
    }

    // GET /api/dashboard/prosumer - Prosumer: counts of their own bookings and their next one (R16).
    [HttpGet("prosumer")]
    [Authorize(Roles = Roles.Prosumer)]
    public async Task<ActionResult<ProsumerDashboardResponse>> Prosumer()
    {
        string callerNic = User.FindFirstValue(JwtTokenHelper.NicClaim!);
        return Ok(await _dashboard.GetProsumerSummaryAsync.callerNic));
    }
}

```

api/SunShare.Api/Controllers/HealthController.cs

```

/*
 * =====
 * File      : HealthController.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : GET /api/health - a public check that the API is running and
 *              can reach MongoDB. Used in the demo to prove IIS + MongoDB work.
 * =====
 */
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Data;

namespace SunShare.Api.Controllers;

[ApiController]
[Route("api/health")]
public class HealthController : ControllerBase
{
    private readonly MongoDBContext _db;

    // Receives the shared database context (registered once in Program.cs).
    public HealthController(MongoDbContext db)
    {
        _db = db;
    }

    // GET /api/health (no login needed). Pings MongoDB and answers:
    // 200 { "api": "ok", "database": "connected", "time": ... } or
    // 503 { "api": "ok", "database": "unreachable", "time": ... } when MongoDB is down.
    [HttpGet]
    public async Task<IActionResult> Get()
    {
        bool connected = await _db.PingAsync();
        var result = new
        {
            api = "ok",
            database = connected ? "connected" : "unreachable",

```

```

        time = DateTime.UtcNow
    };

    if (!connected)
    {
        return StatusCode(StatusCodes.Status503ServiceUnavailable, result);
    }
    return Ok(result);
}
}

```

api/SunShare.Api/Controllers/ProfileController.cs

```

/*
 * =====
 * File       : ProfileController.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : A - Accounts & Access
 * Author     : Gimhan T P K (IT22266996)
 * Created    : 2026-09-28
 * Description : /api/profile endpoints: the logged-in user's own account -
 *              view, edit and self-deactivate (rules R5, R16).
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Every action works on the caller's OWN account; the NIC is read from the login token (RULE R16).
[ApiController]
[Route("api/profile")]
[Authorize]
public class ProfileController : ControllerBase
{
    private readonly UserService _users;

    // Receives the user service (registered in Program.cs).
    public ProfileController(UserService users)
    {
        _users = users;
    }

    // GET /api/profile - any logged-in user: their own account details.
    [HttpGet]
    public async Task<ActionResult<UserResponse>> Get()
    {
        return Ok(await _users.GetAsync(CallerNic()));
    }

    // PUT /api/profile - Prosumer: edit own details, optional new password.
    [HttpPut]
    [Authorize(Roles = Roles.Prosumer)]
    public async Task<ActionResult<UserResponse>> Update(UpdateProfileRequest request)
    {
        return Ok(await _users.UpdateProfileAsync(CallerNic(), request));
    }

    // PATCH /api/profile/deactivate - Prosumer: deactivate own account (R5). The app then clears its saved login.
    [HttpPatch("deactivate")]
    [Authorize(Roles = Roles.Prosumer)]
    public async Task<ActionResult> Deactivate()
    {
        await _users.DeactivateSelfAsync(CallerNic());
        return Ok(new { message = "Account deactivated." });
    }

    // Reads the caller's NIC from their login token (never from the request body - RULE R16).
    private string CallerNic()
    {
        return User.FindFirstValue(JwtTokenHelper.NicClaim)!;
    }
}

```

api/SunShare.Api/Controllers/QrController.cs

```

/*
 * =====
 * File       : QrController.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : D - Dashboards & QR
 * Author     : Chamara R M L K (IT22076816)
 * Created    : 2026-09-28
 * Description : POST /api/reservations/verify-qr and
 *              PATCH /api/reservations/{id}/complete - Grid Operator only (R14).
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Only Grid Operators scan QR codes at the station (RULE R14 / R2).
[ApiController]
[Route("api/reservations")]
[Authorize(Roles = Roles.GridOperator)]
public class QrController : ControllerBase
{
    private readonly QrService _qr;

    // Receives the QR service (registered in Program.cs).
    public QrController(QrService qr)
    {
        _qr = qr;
    }

    // POST /api/reservations/verify-qr - checks the scanned text and returns the booking's details.
    [HttpPost("verify-qr")]
    public async Task<ActionResult<ReservationResponse>> Verify(VerifyQrRequest request)
    {
        return Ok(await _qr.VerifyAsync(request.QrData));
    }

    // PATCH /api/reservations/{id}/complete - checks the QR again and marks the booking Completed.
    [HttpPatch("{id}/complete")]
    public async Task<ActionResult<ReservationResponse>> Complete(string id, VerifyQrRequest request)
    {
        string callerNic = User.FindFirstValue(JwtTokenHelper.NicClaim)!;
        return Ok(await _qr.CompleteAsync(id, request.QrData, callerNic));
    }
}

```

api/SunShare.Api/Controllers/ReservationViewsController.cs

```

/*
 * =====
 * File       : ReservationViewsController.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : D - Dashboards & QR
 * Author     : Chamara R M L K (IT22076816)
 * Created    : 2026-09-28
 * Description : GET /api/reservations (list with view, filters and search)
 *              and GET /api/reservations/{id}. Queries are in
 *              ReservationQueryService.
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Same /api/reservations route as Part C's ReservationsController; this file holds the read-only endpoints.
[ApiController]
[Route("api/reservations")]
[Authorize]

```

```

public class ReservationViewsController : ControllerBase
{
    private readonly ReservationQueryService _queries;

    // Receives the query service (registered in Program.cs).
    public ReservationViewsController(ReservationQueryService queries)
    {
        _queries = queries;
    }

    // GET /api/reservations?view=&status=&stationId=&from=&to=&search= - any logged-in user.
    // Prosumers only get their own bookings (R16).
    [HttpGet]
    public async Task<ActionResult<List<ReservationResponse>>> List([FromQuery] string? view, [FromQuery] string? status,
        [FromQuery] string? stationId, [FromQuery] DateTime? from, [FromQuery] DateTime? to, [FromQuery] string? search)
    {
        string callerNic = User.FindFirstValue(JwtTokenHelper.NicClaim!);
        string callerRole = User.FindFirstValue(JwtTokenHelper.RoleClaim!);
        return Ok(await _queries.ListAsync(view, status, stationId, from, to, search, callerNic, callerRole));
    }

    // GET /api/reservations/{id} - owner Prosumer or staff: one booking (incl. qrData when Approved).
    [HttpGet("{id}")]
    public async Task<ActionResult<ReservationResponse>> Get(string id)
    {
        string callerNic = User.FindFirstValue(JwtTokenHelper.NicClaim!);
        string callerRole = User.FindFirstValue(JwtTokenHelper.RoleClaim!);
        return Ok(await _queries.GetAsync(id, callerNic, callerRole));
    }
}

```

api/SunShare.Api/Controllers/ReservationsController.cs

```

/*
 * =====
 * File      : ReservationsController.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author    : Malkith G W L (IT22630834)
 * Created   : 2026-09-28
 * Description : Reservation actions: create, update, cancel and approve.
 *              (Lists, QR and dashboards are Part D's controllers on the same
 *              /api/reservations route.) Rules are in ReservationService.
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Every action needs a login. Who is calling (NIC + role) always comes from the token (RULE R16).
[ApiController]
[Route("api/reservations")]
[Authorize]
public class ReservationsController : ControllerBase
{
    private readonly ReservationService _reservations;

    // Receives the reservation service (registered in Program.cs).
    public ReservationsController(ReservationService reservations)
    {
        _reservations = reservations;
    }

    // POST /api/reservations - Prosumer (for themselves) or staff (for the prosumerNic they send). Returns 201, status
    Pending.
    [HttpPost]
    public async Task<ActionResult<ReservationResponse>> Create(CreateReservationRequest request)
    {
        ReservationResponse reservation = await _reservations.CreateAsync(request, CallerNic(), CallerRole());
        return Created($"/api/reservations/{reservation.Id}", reservation);
    }

    // PUT /api/reservations/{id} - owner Prosumer or staff: move slot / change energy or type (Approved -> Pending).
    [HttpPut("{id}")]
    public async Task<ActionResult<ReservationResponse>> Update(string id, UpdateReservationRequest request)

```

```

{
    return Ok(await _reservations.UpdateAsync(id, request, CallerNic(), CallerRole()));
}

// PATCH /api/reservations/{id}/cancel - owner Prosumer or staff; at least 12 hours before the start (R10).
[HttpPatch("{id}/cancel")]
public async Task<ActionResult<ReservationResponse>> Cancel(string id)
{
    return Ok(await _reservations.CancelAsync(id, CallerNic(), CallerRole()));
}

// PATCH /api/reservations/{id}/approve - Backoffice or Grid Operator: Pending -> Approved + QR (R13).
[HttpPatch("{id}/approve")]
[Authorize(Roles = Roles.Staff)]
public async Task<ActionResult<ReservationResponse>> Approve(string id)
{
    return Ok(await _reservations.ApproveAsync(id, CallerNic()));
}

// The caller's NIC, from their login token (never from the request body - RULE R16).
private string CallerNic()
{
    return User.FindFirstValue(JwtTokenHelper.NicClaim!);
}

// The caller's role (Backoffice, GridOperator or Prosumer), from their login token.
private string CallerRole()
{
    return User.FindFirstValue(JwtTokenHelper.RoleClaim!);
}
}

```

api/SunShare.Api/Controllers/SlotsController.cs

```

/*
 * =====
 * File      : SlotsController.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Slot endpoints: a station's slots, bookable slots (R9, R11),
 *               get, create, edit and delete (R8).
 *               Role checks (R2) are the [Authorize] lines; rules are in SlotService.
 * =====
 */
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Routes start at /api so one controller can serve both /api/stations/{id}/slots and /api/slots/...
// Backoffice and Grid Operators manage slots; anyone logged in can read them (RULE R2).
[ApiController]
[Route("api")]
[Authorize]
public class SlotsController : ControllerBase
{
    private readonly SlotService _slots;

    // Receives the slot service (registered in Program.cs).
    public SlotsController(SlotService slots)
    {
        _slots = slots;
    }

    // GET /api/stations/{stationId}/slots?from=&to= - staff: one station's slots (default: today + 14 days).
    [HttpGet("stations/{stationId}/slots")]
    [Authorize(Roles = Roles.Staff)]
    public async Task<ActionResult<List<SlotResponse>>> ListForStation(string stationId, [FromQuery] DateTime? from,
    [FromQuery] DateTime? to)
    {
        return Ok(await _slots.ListForStationAsync(stationId, from, to));
    }

    // GET /api/slots/available?stationId= - any logged-in user: slots that can be booked now (R9, R11).
    [HttpGet("slots/available")]

```

```

public async Task<ActionResult<List<SlotResponse>>> Available([FromQuery] string? stationId)
{
    return Ok(await _slots.GetAvailableAsync(stationId));
}

// GET /api/slots/{id} - any logged-in user: one slot.
[HttpGet("slots/{id}")]
public async Task<ActionResult<SlotResponse>> Get(string id)
{
    return Ok(await _slots.GetAsync(id));
}

// POST /api/slots - staff add a slot (R8). Returns 201.
[HttpPost("slots")]
[Authorize(Roles = Roles.Staff)]
public async Task<ActionResult<SlotResponse>> Create(SlotRequest request)
{
    SlotResponse slot = await _slots.CreateAsync(request);
    return Created($"api/slots/{slot.Id}", slot);
}

// PUT /api/slots/{id} - staff edit times, places or open/closed (R8).
[HttpPut("slots/{id}")]
[Authorize(Roles = Roles.Staff)]
public async Task<ActionResult<SlotResponse>> Update(string id, SlotUpdateRequest request)
{
    return Ok(await _slots.UpdateAsync(id, request));
}

// DELETE /api/slots/{id} - staff; 409 while it has active reservations (R8). Returns 204.
[HttpDelete("slots/{id}")]
[Authorize(Roles = Roles.Staff)]
public async Task<ActionResult> Delete(string id)
{
    await _slots.DeleteAsync(id);
    return NoContent();
}
}

```

api/SunShare.Api/Controllers/StationsController.cs

```

/*
 * =====
 * File       : StationsController.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Nodes, Slots & Map
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-28
 * Description : /api/stations endpoints: list, nearby (R18), get, create,
 *              edit, activate, deactivate (R6) and delete (R7).
 *              Role checks (R2) are the [Authorize] lines; rules are in StationService.
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Any logged-in user can read stations; only Backoffice can change them (RULE R2).
[ApiController]
[Route("api/stations")]
[Authorize]
public class StationsController : ControllerBase
{
    private readonly StationService _stations;

    // Receives the station service (registered in Program.cs).
    public StationsController(StationService stations)
    {
        _stations = stations;
    }

    // GET /api/stations?activeOnly= - any logged-in user; prosumers always get active stations only.
    [HttpGet]
    public async Task<ActionResult<List<StationResponse>>> List([FromQuery] bool activeOnly = false)
    {

```



```

        string callerRole = User.FindFirstValue(JwtTokenHelper.RoleClaim!);
        return Ok(await _stations.ListAsync(activeOnly, callerRole));
    }

    // GET /api/stations/nearby?lat=&lng=&radiusKm=25 - any logged-in user (R18): active stations, nearest first.
    [HttpGet("nearby")]
    public async Task<ActionResult<List<StationResponse>>> Nearby([FromQuery] double? lat, [FromQuery] double? lng,
[FromQuery] double radiusKm = 25)
    {
        return Ok(await _stations.GetNearbyAsync(lat, lng, radiusKm));
    }

    // GET /api/stations/{id} - any logged-in user: one station.
    [HttpGet("{id}")]
    public async Task<ActionResult<StationResponse>> Get(string id)
    {
        return Ok(await _stations.GetAsync(id));
    }

    // POST /api/stations - Backoffice registers a station. Returns 201.
    [HttpPost]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<StationResponse>> Create(StationRequest request)
    {
        StationResponse station = await _stations.CreateAsync(request);
        return Created($"api/stations/{station.Id}", station);
    }

    // PUT /api/stations/{id} - Backoffice edits details and opening hours.
    [HttpPut("{id}")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<StationResponse>> Update(string id, StationRequest request)
    {
        return Ok(await _stations.UpdateAsync(id, request));
    }

    // PATCH /api/stations/{id}/deactivate - Backoffice; 409 while it has active reservations (R6).
    [HttpPatch("{id}/deactivate")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<StationResponse>> Deactivate(string id)
    {
        return Ok(await _stations.DeactivateAsync(id));
    }

    // PATCH /api/stations/{id}/activate - Backoffice switches it back on.
    [HttpPatch("{id}/activate")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<StationResponse>> Activate(string id)
    {
        return Ok(await _stations.ActivateAsync(id));
    }

    // DELETE /api/stations/{id} - Backoffice; only without booking history (R7). Returns 204.
    [HttpDelete("{id}")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult> Delete(string id)
    {
        await _stations.DeleteAsync(id);
        return NoContent();
    }
}

```

api/SunShare.Api/Controllers/UsersController.cs

```

/*
 * =====
 * File      : UsersController.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : /api/users endpoints for the web app: list, pending
 *              activations, get, create, edit, activate and deactivate.
 *              Role checks (R2) are the [Authorize] lines; rules are in UserService.
 * =====
 */
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;

```

```

using SunShare.Api.Models;
using SunShare.Api.Services;

namespace SunShare.Api.Controllers;

// Every action needs a login; most are Backoffice only (RULE R2).
[ApiController]
[Route("api/users")]
[Authorize]
public class UsersController : ControllerBase
{
    private readonly UserService _users;

    // Receives the user service (registered in Program.cs).
    public UsersController(UserService users)
    {
        _users = users;
    }

    // GET /api/users?role=&status=&search= - Backoffice and GridOperator.
    // Grid Operators only read it, to pick a prosumer when booking for them.
    [HttpGet]
    [Authorize(Roles = Roles.Staff)]
    public async Task<ActionResult<List<UserResponse>>> List([FromQuery] string? role, [FromQuery] string? status,
    [FromQuery] string? search)
    {
        return Ok(await _users.ListAsync(role, status, search));
    }

    // GET /api/users/pending-activations - Backoffice: prosumers that are Pending or Deactivated.
    [HttpGet("pending-activations")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<List<UserResponse>>> PendingActivations()
    {
        return Ok(await _users.GetPendingActivationsAsync());
    }

    // GET /api/users/{nic} - Backoffice: one user.
    [HttpGet("{nic}")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<UserResponse>> Get(string nic)
    {
        return Ok(await _users.GetAsync(nic));
    }

    // POST /api/users - Backoffice creates an account of any role (it starts Active). Returns 201.
    [HttpPost]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<UserResponse>> Create(CreateUserRequest request)
    {
        UserResponse user = await _users.CreateAsync(request);
        return Created($"api/users/{user.Nic}", user);
    }

    // PUT /api/users/{nic} - Backoffice edits name, email, phone and address.
    [HttpPut("{nic}")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<UserResponse>> Update(string nic, UpdateUserRequest request)
    {
        return Ok(await _users.UpdateAsync(nic, request));
    }

    // PATCH /api/users/{nic}/activate - Backoffice: Pending or Deactivated -> Active (R3, R4).
    [HttpPatch("{nic}/activate")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<UserResponse>> Activate(string nic)
    {
        return Ok(await _users.ActivateAsync(nic));
    }

    // PATCH /api/users/{nic}/deactivate - Backoffice: -> Deactivated (R4). The caller's NIC comes from the token.
    [HttpPatch("{nic}/deactivate")]
    [Authorize(Roles = Roles.Backoffice)]
    public async Task<ActionResult<UserResponse>> Deactivate(string nic)
    {
        string callerNic = User.FindFirstValue(JwtTokenHelper.NicClaim)!;
        return Ok(await _users.DeactivateAsync(nic, callerNic));
    }
}

```

api/SunShare.Api/Data/DataSeeder.cs

```

/*
 * =====
 * File      : DataSeeder.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Fills an EMPTY database with the sample data from
 *              docs\03-DATABASE.md §5: 6 test accounts, 5 stations, a week
 *              of slots and 5 reservations. Runs once at API start-up.
 * =====
 */
using System.Security.Cryptography;
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Models;

namespace SunShare.Api.Data;

// Builds the sample documents in memory first (so reservations can point at slot ids),
// then saves them. Dates are relative to "now", so re-seed the day before the viva.
public class DataSeeder
{
    // Sri Lanka time is UTC+5:30. Slot times are chosen in local time and stored in UTC.
    private static readonly TimeSpan SriLankaOffset = TimeSpan.FromHours(5.5);

    // The seeded Grid Operator (Kasun Silva), used as "approved by" / "completed by".
    private const string OperatorNic = "199845678912";

    private readonly MongoDBContext _db;
    private readonly DateTime _now = DateTime.UtcNow;

    // Receives the database context to write the sample data into.
    public DataSeeder(MongoDbContext db)
    {
        _db = db;
    }

    // Adds all sample data, but only when the Users collection is empty,
    // so it never overwrites real data.
    public async Task SeedAsync()
    {
        long userCount = await _db.Users.CountDocumentsAsync(FilterDefinition<User>.Empty);
        if (userCount > 0)
        {
            return;
        }

        List<User> users = BuildUsers();
        List<SolarStation> stations = BuildStations();
        List<EnergyBookingSlot> slots = BuildSlots(stations);
        List<EnergyReservation> reservations = BuildReservations(users, stations, slots);

        await _db.Users.InsertManyAsync(users);
        await _db.SolarStationInfo.InsertManyAsync(stations);
        await _db.EnergyBookingSlots.InsertManyAsync(slots);
        await _db.EnergyReservations.InsertManyAsync(reservations);
    }

    // The 6 fictional test accounts: one per role, plus a Pending and a Deactivated prosumer.
    private List<User> BuildUsers()
    {
        return
        [
            MakeUser("200012345678", "Dilani Fernando", "dilani.fernando@example.com", "0712345678", null, Roles.Backoffice,
            UserStatuses.Active, "Admin@123"),
            MakeUser("199845678912", "Kasun Silva", "kasun.silva@example.com", "0723456789", null, Roles.GridOperator,
            UserStatuses.Active, "Operator@123"),
            MakeUser("199712345678", "Nimal Perera", "nimal.perera@example.com", "0771234567", "12 Temple Road, Malabe",
            Roles.Prosumer, UserStatuses.Active, "Prosumer@123"),
            MakeUser("200156789123", "Ayesha Jayasinghe", "ayasha.jayasinghe@example.com", "0762345678", "45 Lake Drive,
            Kaduwela", Roles.Prosumer, UserStatuses.Active, "Prosumer@123"),
            MakeUser("200198765432", "Tharindu Wickramasinghe", "tharindu.w@example.com", "0753456789", "8 Station Road,
            Battaramulla", Roles.Prosumer, UserStatuses.Pending, "Prosumer@123"),
            MakeUser("981234567V", "Sanduni Rathnayake", "sanduni.r@example.com", "0704567890", "22 Flower Lane, Kottawa",
            Roles.Prosumer, UserStatuses.Deactivated, "Prosumer@123")
        ];
    }

    // Builds one user. The plain password is turned into a BCrypt hash here and is never stored.

```

```

private User MakeUser(string nic, string fullName, string email, string phone, string? address,
    string role, string status, string password)
{
    return new User
    {
        Nic = nic,
        FullName = fullName,
        Email = email,
        Phone = phone,
        Address = address,
        Role = role,
        Status = status,
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(password),
        CreatedAt = _now,
        UpdatedAt = _now
    };
}

// The 5 stations around SLIIT Malabe, so the map looks right. The last one is deactivated.
private List<SolarStation> BuildStations()
{
    return
    [
        MakeStation("Malabe Solar Hub", "New Kandy Road, Malabe", 6.9147, 79.9730, 250, 10, "06:00", "18:00", true),
        MakeStation("Kaduwela Grid Node", "Kaduwela Road, Kaduwela", 6.9335, 79.9840, 180, 8, "06:00", "18:00", true),
        MakeStation("Battaramulla Microgrid", "Pannipitiya Road, Battaramulla", 6.9020, 79.9180, 300, 12, "07:00",
"19:00", true),
        MakeStation("Kottawa Sun Park", "High Level Road, Kottawa", 6.8410, 79.9650, 150, 6, "06:00", "18:00", true),
        MakeStation("Nugegoda Rooftop Node", "Stanley Thilakaratne Mawatha, Nugegoda", 6.8649, 79.8997, 90, 4, "08:00",
"17:00", false)
    ];
}

// Builds one station with a new MongoDB id.
private SolarStation MakeStation(string name, string address, double latitude, double longitude,
    double capacityKw, int batterySlots, string openTime, string closeTime, bool isActive)
{
    return new SolarStation
    {
        Id = ObjectId.GenerateNewId().ToString(),
        Name = name,
        Address = address,
        Latitude = latitude,
        Longitude = longitude,
        CapacityKw = capacityKw,
        BatterySlots = batterySlots,
        OpenTime = openTime,
        CloseTime = closeTime,
        IsActive = isActive,
        CreatedAt = _now,
        UpdatedAt = _now
    };
}

// For every active station: 3 windows a day (08:00, 11:00 and 14:00 Sri Lanka time) for the
// next 7 days. Plus one slot 2 days ago at Malabe Solar Hub, so History has data.
private List<EnergyBookingSlot> BuildSlots(List<SolarStation> stations)
{
    var slots = new List<EnergyBookingSlot>();
    int[] startHours = {8, 11, 14};

    foreach (SolarStation station in stations.Where(s => s.IsActive))
    {
        for (int day = 1; day <= 7; day++)
        {
            foreach (int hour in startHours)
            {
                slots.Add(MakeSlot(station.Id, day, hour));
            }
        }
    }

    slots.Add(MakeSlot(stations[0].Id, -2, 11));
    return slots;
}

// Builds one 2-hour slot with 4 free places, starting "today + dayOffset" at localHour Sri Lanka time.
private EnergyBookingSlot MakeSlot(string stationId, int dayOffset, int localHour)
{
    DateTime start = SriLankaTimeToUtc(dayOffset, localHour);
    return new EnergyBookingSlot
    {
        Id = ObjectId.GenerateNewId().ToString(),
        StationId = stationId,
        StartTime = start,

```

```

        EndTime = start.AddHours(2),
        TotalSlots = 4,
        AvailableSlots = 4,
        IsActive = true,
        CreatedAt = _now,
        UpdatedAt = _now
    };
}

// Turns "today + dayOffset days, at localHour o'clock in Sri Lanka" into the matching UTC time.
// Example: tomorrow 08:00 in Sri Lanka = tomorrow 02:30 UTC.
private DateTime SriLankaTimeToUtc(int dayOffset, int localHour)
{
    DateTime todayInSriLanka = (_now + SriLankaOffset).Date;
    DateTime localTime = todayInSriLanka.AddDays(dayOffset).AddHours(localHour);
    return DateTime.SpecifyKind(localTime - SriLankaOffset, DateTimeKind.Utc);
}

// The 5 sample bookings: Nimal has one Approved (tomorrow 11:00, with a QR token), one Pending,
// one Completed (the past slot) and one Cancelled; Ayesha has one Pending.
// Each one goes through the same steps as a real booking, so every slot's availableSlots is right.
private List<EnergyReservation> BuildReservations(List<User> users, List<SolarStation> stations,
    List<EnergyBookingSlot> slots)
{
    User nimal = users.First(u => u.Nic == "199712345678");
    User ayesha = users.First(u => u.Nic == "200156789123");
    SolarStation malabe = stations[0];
    SolarStation kaduwela = stations[1];
    SolarStation battaramulla = stations[2];

    EnergyReservation approved = MakeReservation(nimal, malabe, FindSlot(slots, malabe, 1, 11), 12.5,
ReservationTypes.Sell, _now.AddHours(-3));
    MarkApproved(approved, _now.AddHours(-1));

    EnergyReservation pending = MakeReservation(nimal, kaduwela, FindSlot(slots, kaduwela, 3, 8), 8,
ReservationTypes.Buy, _now.AddHours(-2));

    EnergyBookingSlot pastSlot = FindSlot(slots, malabe, -2, 11);
    EnergyReservation completed = MakeReservation(nimal, malabe, pastSlot, 20, ReservationTypes.Sell,
pastSlot.StartTime.AddDays(-2));
    MarkApproved(completed, pastSlot.StartTime.AddDays(-1));
    MarkCompleted(completed, pastSlot.StartTime.AddMinutes(30));

    EnergyBookingSlot cancelledSlot = FindSlot(slots, battaramulla, 4, 14);
    EnergyReservation cancelled = MakeReservation(nimal, battaramulla, cancelledSlot, 5, ReservationTypes.Buy,
_now.AddHours(-5));
    MarkCancelled(cancelled, cancelledSlot, nimal.Nic, _now.AddHours(-4));

    EnergyReservation ayeshaPending = MakeReservation(ayesha, malabe, FindSlot(slots, malabe, 2, 14), 15,
ReservationTypes.Sell, _now.AddHours(-1));

    return [approved, pending, completed, cancelled, ayeshaPending];
}

// Finds the seeded slot at a station that starts "today + dayOffset" at localHour Sri Lanka time.
private EnergyBookingSlot FindSlot(List<EnergyBookingSlot> slots, SolarStation station, int dayOffset, int localHour)
{
    DateTime start = SriLankaTimeToUtc(dayOffset, localHour);
    return slots.First(s => s.StationId == station.Id && s.StartTime == start);
}

// Builds a new Pending booking and takes one free place from its slot (like R12 does).
private static EnergyReservation MakeReservation(User prosumer, SolarStation station, EnergyBookingSlot slot,
    double energyKwh, string type, DateTime createdAt)
{
    slot.AvailableSlots -= 1;
    return new EnergyReservation
    {
        Id = ObjectId.GenerateNewId().ToString(),
        ProsumerNic = prosumer.Nic,
        ProsumerName = prosumer.FullName,
        StationId = station.Id,
        StationName = station.Name,
        SlotId = slot.Id,
        StartTime = slot.StartTime,
        EndTime = slot.EndTime,
        EnergyKwh = energyKwh,
        Type = type,
        Status = ReservationStatuses.Pending,
        CreatedAt = createdAt,
        UpdatedAt = createdAt
    };
}

```

```

// Approves a sample booking as the Grid Operator and gives it a random 32-character QR token (like R13).
private static void MarkApproved(EnergyReservation reservation, DateTime when)
{
    reservation.Status = ReservationStatuses.Approved;
    reservation.QrToken = Convert.ToHexString(RandomNumberGenerator.GetBytes(16)).ToLowerInvariant();
    reservation.ApprovedBy = OperatorNic;
    reservation.ApprovedAt = when;
    reservation.UpdatedAt = when;
}

// Marks a sample booking as finished: the operator scanned the QR code (like R14).
private static void MarkCompleted(EnergyReservation reservation, DateTime when)
{
    reservation.Status = ReservationStatuses.Completed;
    reservation.CompletedBy = OperatorNic;
    reservation.CompletedAt = when;
    reservation.UpdatedAt = when;
}

// Marks a sample booking as cancelled and gives its place back to the slot (like R12).
private static void MarkCancelled(EnergyReservation reservation, EnergyBookingSlot slot, string cancelledBy, DateTime
when)
{
    reservation.Status = ReservationStatuses.Cancelled;
    reservation.CancelledBy = cancelledBy;
    reservation.CancelledAt = when;
    reservation.UpdatedAt = when;
    slot.AvailableSlots += 1;
}
}

```

api/SunShare.Api/Data/MongoDbContext.cs

```

/*
 * =====
 * File      : MongoDBContext.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : The one place that talks to MongoDB directly. Opens the
 *               SunShareDb database, exposes its 4 collections (exact names
 *               from docs/03-DATABASE.md), creates indexes and pings the DB.
 * =====
 */
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Models;

namespace SunShare.Api.Data;

// Reference: Microsoft Learn, "Create a web API with ASP.NET Core and MongoDB"
// https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mongo-app (adapted: settings + collection setup)
public class MongoDBContext
{
    private readonly IMongoDatabase _database;

    // The 4 collections. Property names = collection names, so they match the rubric exactly.
    public IMongoCollection<User> Users { get; }

    public IMongoCollection<SolarStation> SolarStationInfo { get; }

    public IMongoCollection<EnergyBookingSlot> EnergyBookingSlots { get; }

    public IMongoCollection<EnergyReservation> EnergyReservations { get; }

    // Creates the MongoDB client from the settings and picks the database and its 4 collections.
    // Nothing is sent over the network here; the driver connects on the first real query.
    public MongoDBContext(MongoDbSettings settings)
    {
        var clientSettings = MongoClientSettings.FromConnectionString(settings.ConnectionString);
        // Give up after 5 seconds (instead of the default 30) if MongoDB is not running.
        clientSettings.ServerSelectionTimeout = TimeSpan.FromSeconds(5);
        var client = new MongoClient(clientSettings);

        _database = client.GetDatabase(settings.DatabaseName);
        Users = _database.GetCollection<User>("Users");
        SolarStationInfo = _database.GetCollection<SolarStation>("SolarStationInfo");
    }
}

```

```

        EnergyBookingSlots = _database.GetCollection<EnergyBookingSlot>("EnergyBookingSlots");
        EnergyReservations = _database.GetCollection<EnergyReservation>("EnergyReservations");
    }

    // Sends MongoDB a "ping" command. Returns true if it answered, false if it can't be reached.
    // Used by GET /api/health to prove the API and the database are connected.
    public async Task<bool> PingAsync()
    {
        try
        {
            await _database.RunCommandAsync((Command<BsonDocument>){ ping: 1 });
            return true;
        }
        catch (Exception)
        {
            return false;
        }
    }

    // Creates the small indexes from docs/03-DATABASE.md §6 so common searches stay fast.
    // Safe to run on every start: MongoDB keeps an index that already exists.
    public async Task CreateIndexesAsync()
    {
        var slotsByStationAndTime = Builders<EnergyBookingSlot>.IndexKeys
            .Ascending(s => s.StationId)
            .Ascending(s => s.StartTime);
        await EnergyBookingSlots.Indexes.CreateOneAsync(new CreateIndexModel<EnergyBookingSlot>(slotsByStationAndTime));

        var reservationsByProsumer = Builders<EnergyReservation>.IndexKeys.Ascending(r => r.ProsumerNic);
        await EnergyReservations.Indexes.CreateOneAsync(new CreateIndexModel<EnergyReservation>(reservationsByProsumer));

        var reservationsByStatusAndTime = Builders<EnergyReservation>.IndexKeys
            .Ascending(r => r.Status)
            .Ascending(r => r.StartTime);
        await EnergyReservations.Indexes.CreateOneAsync(new
            CreateIndexModel<EnergyReservation>(reservationsByStatusAndTime));
    }
}

```

api/SunShare.Api/Data/MongoDbSettings.cs

```

/*
 * =====
 * File       : MongoDbSettings.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Shared foundation
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-28
 * Description : Holds the "MongoDb" section of appsettings.json: where the
 *               MongoDB server is and which database to use (SunShareDb).
 * =====
 */

namespace SunShare.Api.Data;

// Plain settings class; Program.cs fills it from appsettings.json at start-up.
public class MongoDbSettings
{
    public string ConnectionString { get; set; } = "";

    public string DatabaseName { get; set; } = "";
}

```

api/SunShare.Api/Dtos/CreateReservationRequest.cs

```

/*
 * =====
 * File       : CreateReservationRequest.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : C - Reservations
 * Author     : Malkith G W L (IT22630834)
 * Created    : 2026-09-28
 * Description : JSON body for POST /api/reservations (web W14 / mobile M6).
 * =====
 */

namespace SunShare.Api.Dtos;

```

```
// A new booking. Prosumers leave prosumerNic empty (their NIC comes from the login token, R16);
// Backoffice / Grid Operators booking on behalf of someone must send the prosumer's NIC.
public class CreateReservationRequest
{
    public string? ProsumerNic { get; set; }

    public string SlotId { get; set; } = "";

    // More than 0 and at most 100 kWh (R17).
    public double EnergyKwh { get; set; }

    // "Sell" (drop off energy) or "Buy" (charge).
    public string Type { get; set; } = "";
}
```

api/SunShare.Api/Dtos/CreateUserRequest.cs

```
/*
 * =====
 * File      : CreateUserRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : JSON body for POST /api/users, used by Backoffice to create
 *              any account (Backoffice, GridOperator or Prosumer).
 * =====
 */

namespace SunShare.Api.Dtos;

// Details for a new account. UserService checks every field and sends friendly messages.
public class CreateUserRequest
{
    public string Nic { get; set; } = "";

    public string FullName { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";

    public string? Address { get; set; }

    // Backoffice, GridOperator or Prosumer.
    public string Role { get; set; } = "";

    public string Password { get; set; } = "";
}
```

api/SunShare.Api/Dtos/LoginRequest.cs

```
/*
 * =====
 * File      : LoginRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : JSON body for POST /api/auth/login.
 * =====
 */

namespace SunShare.Api.Dtos;

// What the user types on the Login screen (web W2 / mobile M1).
public class LoginRequest
{
    public string Nic { get; set; } = "";

    public string Password { get; set; } = "";
}
```


api/SunShare.Api/Dtos/LoginResponse.cs

```

/*
 * =====
 * File      : LoginResponse.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : JSON answer of a successful login: the token plus who the
 *              user is, so the client can pick the right home screen.
 * =====
 */

namespace SunShare.Api.Dtos;

// The web saves this in localStorage; Android saves it in the SQLite "session" table.
public class LoginResponse
{
    // The signed JWT to send as "Authorization: Bearer <token>" on every later request.
    public string Token { get; set; } = "";

    public string Nic { get; set; } = "";

    public string FullName { get; set; } = "";

    // Backoffice, GridOperator or Prosumer - decides which menu / home screen to show.
    public string Role { get; set; } = "";

    // When the token stops working (8 hours after login, UTC).
    public DateTime ExpiresAt { get; set; }
}

```

api/SunShare.Api/Dtos/ProsumerDashboardResponse.cs

```

/*
 * =====
 * File      : ProsumerDashboardResponse.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : JSON answer of GET /api/dashboard/prosumer (the prosumer's
 *              mobile home M3). Only the caller's own bookings are counted.
 * =====
 */

namespace SunShare.Api.Dtos;

// The prosumer home screen's count cards and "Next booking" card.
public class ProsumerDashboardResponse
{
    // My bookings with status Pending.
    public long PendingCount { get; set; }

    // My Approved bookings that haven't started yet.
    public long ApprovedFutureCount { get; set; }

    // My Completed bookings.
    public long CompletedCount { get; set; }

    // My soonest upcoming Pending or Approved booking, or null if I have none.
    public ReservationResponse? NextReservation { get; set; }
}

```

api/SunShare.Api/Dtos/RegisterRequest.cs

```

/*
 * =====
 * File      : RegisterRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access

```

```

* Author      : Gimhan T P K (IT22266996)
* Created     : 2026-09-28
* Description : JSON body for POST /api/auth/register (mobile self-sign-up
*              by a prosumer). There is no role field: it is always Prosumer.
* =====
*/

```

```
namespace SunShare.Api.Dtos;
```

```
// What a prosumer fills in on the mobile Register screen (M2).
```

```
public class RegisterRequest
{
    public string Nic { get; set; } = "";

    public string FullName { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";

    // The prosumer's property address (optional).
    public string? Address { get; set; }

    public string Password { get; set; } = "";
}

```

api/SunShare.Api/Dtos/ReservationResponse.cs

```

/*
* =====
* File       : ReservationResponse.cs
* Project    : SunShare - Smart Solar Microgrid Trading System
* Module     : SE4040 Enterprise Application Development - Assignment 1
* Part       : C - Reservations
* Author     : Malkith G W L (IT22630834)
* Created    : 2026-09-28
* Description : The JSON shape of a booking sent to the clients, including
*              the API-calculated canModify and qrData. Built by
*              ReservationMapper (also used by Part D).
* =====
*/

```

```
namespace SunShare.Api.Dtos;
```

```
// What clients see about a booking (docs/04-API.md "ReservationResponse").
```

```
public class ReservationResponse
{
    public string Id { get; set; } = "";

    public string ProsumerNic { get; set; } = "";

    public string ProsumerName { get; set; } = "";

    public string StationId { get; set; } = "";

    public string StationName { get; set; } = "";

    public string SlotId { get; set; } = "";

    public DateTime StartTime { get; set; }

    public DateTime EndTime { get; set; }

    public double EnergyKwh { get; set; }

    public string Type { get; set; } = "";

    public string Status { get; set; } = "";

    // true = the apps may show Edit / Cancel (Pending or Approved, and at least 12 hours before the start).
    // Calculated by the API so the apps hold no rule logic (R10, R15).
    public bool CanModify { get; set; }

    // The text to put in the QR code, "SUNSHARE|<id>|<token>" - only for Approved bookings, otherwise null.
    public string? QrData { get; set; }
}

```

```

    public string? ApprovedBy { get; set; }

    public DateTime? ApprovedAt { get; set; }

    public string? CompletedBy { get; set; }

    public DateTime? CompletedAt { get; set; }

    public string? CancelledBy { get; set; }

    public DateTime? CancelledAt { get; set; }

    public DateTime CreatedAt { get; set; }

    public DateTime UpdatedAt { get; set; }
}

```

api/SunShare.Api/Dtos/SlotRequest.cs

```

/*
 * =====
 * File      : SlotRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : JSON body for POST /api/slots (the Slot form, web W12).
 * =====
 */

namespace SunShare.Api.Dtos;

// A new time window at a station. Times should be sent in UTC, e.g. "2026-09-30T02:30:00Z".
public class SlotRequest
{
    public string StationId { get; set; } = "";

    public DateTime StartTime { get; set; }

    public DateTime EndTime { get; set; }

    // Places in this window: 1 up to the station's battery slots (R8).
    public int TotalSlots { get; set; }
}

```

api/SunShare.Api/Dtos/SlotResponse.cs

```

/*
 * =====
 * File      : SlotResponse.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : The JSON shape of a slot sent to the clients, with the
 *              station's name and the number of booked places.
 * =====
 */

using SunShare.Api.Models;

namespace SunShare.Api.Dtos;

// What clients see about a slot (docs/04-API.md "SlotResponse").
public class SlotResponse
{
    public string Id { get; set; } = "";

    public string StationId { get; set; } = "";

    public string StationName { get; set; } = "";
}

```

```

public DateTime StartTime { get; set; }

public DateTime EndTime { get; set; }

public int TotalSlots { get; set; }

public int AvailableSlots { get; set; }

// Places already taken = total - available.
public int BookedSlots { get; set; }

public bool IsActive { get; set; }

// Builds the response from a stored slot and the name of its station.
public static SlotResponse FromSlot(EnergyBookingSlot slot, string stationName)
{
    return new SlotResponse
    {
        Id = slot.Id,
        StationId = slot.StationId,
        StationName = stationName,
        StartTime = slot.StartTime,
        EndTime = slot.EndTime,
        TotalSlots = slot.TotalSlots,
        AvailableSlots = slot.AvailableSlots,
        BookedSlots = slot.TotalSlots - slot.AvailableSlots,
        IsActive = slot.IsActive
    };
}
}

```

api/SunShare.Api/Dtos/SlotUpdateRequest.cs

```

/*
 * =====
 * File      : SlotUpdateRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : JSON body for PUT /api/slots/{id}: new times, number of
 *              places and whether the window is open for booking.
 * =====
 */

namespace SunShare.Api.Dtos;

// The editable parts of a slot. The station can't change (make a new slot instead).
public class SlotUpdateRequest
{
    public DateTime StartTime { get; set; }

    public DateTime EndTime { get; set; }

    public int TotalSlots { get; set; }

    // false = closed: prosumers can't book it any more.
    public bool IsActive { get; set; }
}

```

api/SunShare.Api/Dtos/StaffDashboardResponse.cs

```

/*
 * =====
 * File      : StaffDashboardResponse.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : JSON answer of GET /api/dashboard/summary (web W3 and the
 *              operator's mobile home M10).
 * =====
 */

namespace SunShare.Api.Dtos;

```

```
// Count cards + the next pending bookings (definitions in docs/01-SPEC.md §5).
public class StaffDashboardResponse
{
    // Bookings with status Pending.
    public long PendingReservations { get; set; }

    // Bookings with status Approved that haven't started yet.
    public long ApprovedFutureReservations { get; set; }

    // Bookings (not cancelled) that start today, Sri Lanka time.
    public long TodayReservations { get; set; }

    public long ActiveStations { get; set; }

    // Prosumers that are Pending or Deactivated (the Pending Activations page).
    public long PendingActivations { get; set; }

    // Up to 5 upcoming Pending bookings, soonest first (the dashboard table with Approve buttons).
    public List<ReservationResponse> PendingList { get; set; } = [];
}

```

api/SunShare.Api/Dtos/StationRequest.cs

```
/*
 * =====
 * File      : StationRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : JSON body for POST /api/stations and PUT /api/stations/{id}
 *              (the Station form, web W10).
 * =====
 */

namespace SunShare.Api.Dtos;

// The station form. StationService checks every field (FAT service).
public class StationRequest
{
    public string Name { get; set; } = "";

    public string Address { get; set; } = "";

    // GPS position: latitude -90..90, longitude -180..180.
    public double Latitude { get; set; }

    public double Longitude { get; set; }

    // Solar generation capacity in kW (more than 0).
    public double CapacityKw { get; set; }

    // Battery places at the hub (at least 1).
    public int BatterySlots { get; set; }

    // Operating schedule, 24-hour "HH:mm", e.g. "06:00" and "18:00".
    public string OpenTime { get; set; } = "";

    public string CloseTime { get; set; } = "";
}

```

api/SunShare.Api/Dtos/StationResponse.cs

```
/*
 * =====
 * File      : StationResponse.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : The JSON shape of a station sent to the clients, with the
 *              API-calculated active booking count and (for /nearby) distance.
 * =====
 */

```

```

*/
using SunShare.Api.Models;

namespace SunShare.Api.Dtos;

// What clients see about a station (docs/04-API.md "StationResponse").
public class StationResponse
{
    public string Id { get; set; } = "";

    public string Name { get; set; } = "";

    public string Address { get; set; } = "";

    public double Latitude { get; set; }

    public double Longitude { get; set; }

    public double CapacityKw { get; set; }

    public int BatterySlots { get; set; }

    public string OpenTime { get; set; } = "";

    public string CloseTime { get; set; } = "";

    public bool IsActive { get; set; }

    // Pending/Approved reservations that haven't started yet (the R6 definition).
    public long ActiveReservationCount { get; set; }

    // Kilometres from the phone; only filled by GET /api/stations/nearby (R18), otherwise null.
    public double? DistanceKm { get; set; }

    public DateTime CreatedAt { get; set; }

    public DateTime UpdatedAt { get; set; }

    // Builds the response from a stored station plus the numbers the service calculated.
    public static StationResponse FromStation(SolarStation station, long activeReservationCount, double? distanceKm = null)
    {
        return new StationResponse
        {
            Id = station.Id,
            Name = station.Name,
            Address = station.Address,
            Latitude = station.Latitude,
            Longitude = station.Longitude,
            CapacityKw = station.CapacityKw,
            BatterySlots = station.BatterySlots,
            OpenTime = station.OpenTime,
            CloseTime = station.CloseTime,
            IsActive = station.IsActive,
            ActiveReservationCount = activeReservationCount,
            DistanceKm = distanceKm,
            CreatedAt = station.CreatedAt,
            UpdatedAt = station.UpdatedAt
        };
    }
}

```

api/SunShare.Api.Dtos/UpdateProfileRequest.cs

```

/*
=====
* File      : UpdateProfileRequest.cs
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : A - Accounts & Access
* Author    : Gimhan T P K (IT22266996)
* Created   : 2026-09-28
* Description : JSON body for PUT /api/profile: a prosumer edits their own
               details and can optionally set a new password.
=====
*/

namespace SunShare.Api.Dtos;

```

```
// What the prosumer can change on the mobile Profile screen (M4). There is no NIC here on purpose:
// the NIC comes from the login token (R16).
public class UpdateProfileRequest
{
    public string FullName { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";

    public string? Address { get; set; }

    // Leave empty/null to keep the current password; otherwise at least 6 characters.
    public string? NewPassword { get; set; }
}
```

api/SunShare.Api/Dtos/UpdateReservationRequest.cs

```
/*
 * =====
 * File      : UpdateReservationRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author     : Malkith G W L (IT22630834)
 * Created    : 2026-09-28
 * Description : JSON body for PUT /api/reservations/{id}: move to another
 *              slot and/or change the energy amount or type.
 * =====
 */

namespace SunShare.Api.Dtos;

// The editable parts of a booking. The prosumer can't change (cancel and book again instead).
public class UpdateReservationRequest
{
    // The same slot id to stay, or another one to move the booking (R12).
    public string SlotId { get; set; } = "";

    public double EnergyKwh { get; set; }

    public string Type { get; set; } = "";
}
```

api/SunShare.Api/Dtos/UpdateUserRequest.cs

```
/*
 * =====
 * File      : UpdateUserRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author     : Gimhan T P K (IT22266996)
 * Created    : 2026-09-28
 * Description : JSON body for PUT /api/users/{nic}, used by Backoffice to
 *              edit an account. The NIC and role can't be changed.
 * =====
 */

namespace SunShare.Api.Dtos;

// The editable details of an account.
public class UpdateUserRequest
{
    public string FullName { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";

    public string? Address { get; set; }
}
```

api/SunShare.Api/Dtos/UserResponse.cs

```

/*
 * =====
 * File      : UserResponse.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : The JSON shape of a user sent back to the clients.
 *             It never contains the password hash.
 * =====
 */
using SunShare.Api.Models;

namespace SunShare.Api.Dtos;

// What clients see about a user (docs/04-API.md "UserResponse").
public class UserResponse
{
    public string Nic { get; set; } = "";

    public string FullName { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";

    public string? Address { get; set; }

    public string Role { get; set; } = "";

    public string Status { get; set; } = "";

    public DateTime CreatedAt { get; set; }

    public DateTime UpdatedAt { get; set; }

    // Copies the safe fields of a stored User into a response (the password hash is left out on purpose).
    public static UserResponse FromUser(User user)
    {
        return new UserResponse
        {
            Nic = user.Nic,
            FullName = user.FullName,
            Email = user.Email,
            Phone = user.Phone,
            Address = user.Address,
            Role = user.Role,
            Status = user.Status,
            CreatedAt = user.CreatedAt,
            UpdatedAt = user.UpdatedAt
        };
    }
}

```

api/SunShare.Api/Dtos/VerifyQrRequest.cs

```

/*
 * =====
 * File      : VerifyQrRequest.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : JSON body for POST /api/reservations/verify-qr and
 *             PATCH /api/reservations/{id}/complete: the scanned QR text.
 * =====
 */
namespace SunShare.Api.Dtos;

// The text the operator's phone read from the QR code, e.g. "SUNSHARE|66f6...|9f1c..." (R14).
public class VerifyQrRequest
{
    public string QrData { get; set; } = "";
}

```


api/SunShare.Api/Helpers/ApiException.cs

```

/*
 * =====
 * File      : ApiException.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : The error that services throw on purpose when a request
 *               breaks a rule. Carries an HTTP status code and a friendly
 *               message; ErrorHandlingMiddleware sends both to the client.
 * =====
 */

namespace SunShare.Api.Helpers;

// Example: throw new ApiException(409, "This slot is already full.");
// Status codes we use: 400 rule broken, 401 bad login, 403 not allowed / not yours, 404 not found, 409 conflict.
public class ApiException : Exception
{
    public int StatusCode { get; }

    // Creates the error with the HTTP status code to send and the message the user will see.
    public ApiException(int statusCode, string message) : base(message)
    {
        StatusCode = statusCode;
    }
}

```

api/SunShare.Api/Helpers/GeoHelper.cs

```

/*
 * =====
 * File      : GeoHelper.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Distance between two GPS points (Haversine formula), used
 *               by the "nearby stations" search (rule R18).
 * =====
 */

namespace SunShare.Api.Helpers;

// The Earth is (almost) a ball, so the distance between two map points is measured along its
// curved surface, not with a straight ruler line. The Haversine formula does exactly that.
public static class GeoHelper
{
    // Average radius of the Earth in kilometres.
    private const double EarthRadiusKm = 6371.0;

    // Reference: C. Veness, "Calculate distance, bearing and more between Latitude/Longitude points"
    // https://www.movable-type.co.uk/scripts/latlong.html (Haversine formula)
    // Returns the distance in kilometres between point 1 and point 2 (latitudes/longitudes in degrees).
    public static double DistanceKm(double lat1, double lng1, double lat2, double lng2)
    {
        double dLat = ToRadians(lat2 - lat1);
        double dLng = ToRadians(lng2 - lng1);

        // a = the square of half the straight-line (chord) distance between the points, on a ball of radius 1.
        double a = Math.Sin(dLat / 2) * Math.Sin(dLat / 2)
            + Math.Cos(ToRadians(lat1)) * Math.Cos(ToRadians(lat2)) * Math.Sin(dLng / 2) * Math.Sin(dLng / 2);
        // c = the angle between the points, seen from the centre of the Earth (in radians).
        double c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));

        return EarthRadiusKm * c;
    }

    // Converts degrees to radians (C#'s Math.Sin and Math.Cos work in radians).
    private static double ToRadians(double degrees)
    {
        return degrees * Math.PI / 180.0;
    }
}

```

api/SunShare.Api/Helpers/JwtTokenHelper.cs

```

/*
 * =====
 * File      : JwtTokenHelper.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : Makes the signed login token (JWT) given to a user after a
 *              successful login. The token holds the NIC, name and role and
 *              is valid for 8 hours (settings in appsettings.json -> "Jwt").
 * =====
 */
using System.Text;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;
using SunShare.Api.Models;

namespace SunShare.Api.Helpers;

// A JWT is a signed "login ticket": anyone can read what is inside, but only our API
// (which knows the secret key) can make or change one without breaking the signature.
public class JwtTokenHelper
{
    // Names of the values ("claims") stored inside the token. Controllers read the caller's NIC
    // with User.FindFirstValue(JwtTokenHelper.NicClaim) - never from the request body (R16).
    public const string NicClaim = "nic";
    public const string NameClaim = "name";
    public const string RoleClaim = "role";

    private readonly IConfiguration _config;

    // Receives the app settings, to read the "Jwt" section (key, issuer, audience, expiry hours).
    public JwtTokenHelper(IConfiguration config)
    {
        _config = config;
    }

    // Makes a token for this user, signed with HMAC-SHA256 and our secret key.
    // Returns the token text and the moment it expires.
    public (string Token, DateTime ExpiresAt) CreateToken(User user)
    {
        DateTime expiresAt = DateTime.UtcNow.AddHours(_config.GetValue<int>("Jwt:ExpiryHours"));
        var signingKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));

        var descriptor = new SecurityTokenDescriptor
        {
            Issuer = _config["Jwt:Issuer"],
            Audience = _config["Jwt:Audience"],
            Expires = expiresAt,
            Claims = new Dictionary<string, object>
            {
                { NicClaim, user.Nic },
                { NameClaim, user.FullName },
                { RoleClaim, user.Role }
            },
            SigningCredentials = new SigningCredentials(signingKey, SecurityAlgorithms.HmacSha256)
        };

        string token = new JwtSecurityTokenHandler().CreateToken(descriptor);
        return (token, expiresAt);
    }
}

```

api/SunShare.Api/Helpers/NicValidator.cs

```

/*
 * =====
 * File      : NicValidator.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : Checks that a Sri Lankan NIC has a valid format (rule R1)
 *              and cleans typed NICs so the same person always matches.
 * =====

```

```

*/
using System.Text.RegularExpressions;

namespace SunShare.Api.Helpers;

// The NIC is every user's primary key (_id), so it must be in one of the two official formats.
public static class NicValidator
{
    // Old NIC: 9 digits + V or X (e.g. 981234567V). New NIC: 12 digits (e.g. 199812345678).
    private static readonly Regex NicPattern = new(@"^(?(\d{9}[VX])|(\d{12}))$");

    // Returns true when the (already cleaned) text is a valid old-style or new-style NIC.
    public static bool IsValid(string nic)
    {
        return NicPattern.IsMatch(nic);
    }

    // Cleans a typed NIC: removes spaces around it and makes the V/X letter uppercase,
    // so "981234567v" and "981234567V " are treated as the same person.
    public static string Normalize(string? nic)
    {
        return (nic ?? "").Trim().ToUpperInvariant();
    }
}

```

api/SunShare.Api/Helpers/ReservationMapper.cs

```

/*
 * =====
 * File      : ReservationMapper.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author     : Malkith G W L (IT22630834)
 * Created    : 2026-09-28
 * Description : Turns a stored reservation into the ReservationResponse sent
 *               to clients, and works out canModify (R10, R15) and the QR
 *               text (R13, R14). Shared with Part D's list and QR code.
 * =====
 */
using SunShare.Api.Dtos;
using SunShare.Api.Models;

namespace SunShare.Api.Helpers;

// One place that builds every reservation answer, so all endpoints show the same fields.
public static class ReservationMapper
{
    // Minimum notice before the start for any change or cancel (R10). ReservationService uses it too.
    public static readonly TimeSpan MinChangeNotice = TimeSpan.FromHours(12);

    // Every SunShare QR code starts with this word (R14).
    public const string QrPrefix = "SUNSHARE";

    // Copies a stored reservation into a response and adds canModify and qrData.
    public static ReservationResponse ToResponse(EnergyReservation reservation)
    {
        return new ReservationResponse
        {
            Id = reservation.Id,
            ProsumerNic = reservation.ProsumerNic,
            ProsumerName = reservation.ProsumerName,
            StationId = reservation.StationId,
            StationName = reservation.StationName,
            SlotId = reservation.SlotId,
            StartTime = reservation.StartTime,
            EndTime = reservation.EndTime,
            EnergyKwh = reservation.EnergyKwh,
            Type = reservation.Type,
            Status = reservation.Status,
            CanModify = CanModify(reservation, DateTime.UtcNow),
            QrData = BuildQrData(reservation),
            ApprovedBy = reservation.ApprovedBy,
            ApprovedAt = reservation.ApprovedAt,
            CompletedBy = reservation.CompletedBy,
            CompletedAt = reservation.CompletedAt,
            CancelledBy = reservation.CancelledBy,
            CancelledAt = reservation.CancelledAt,
            CreatedAt = reservation.CreatedAt,
            UpdatedAt = reservation.UpdatedAt
        };
    }
}

```

```
// RULE R15 + R10: a booking can still be edited or cancelled only while it is Pending or Approved
// AND it starts at least 12 hours from now.
public static bool CanModify(EnergyReservation reservation, DateTime now)
{
    bool stillOpen = reservation.Status == ReservationStatuses.Pending
        || reservation.Status == ReservationStatuses.Approved;
    return stillOpen && reservation.StartTime - now >= MinChangeNotice;
}

// RULE R13 + R14: only an Approved booking has a QR code. Its text is "SUNSHARE|<reservationId>|<qrToken>".
// The phone turns this text into the QR picture; the token is the secret part.
public static string? BuildQrData(EnergyReservation reservation)
{
    if (reservation.Status != ReservationStatuses.Approved || string.IsNullOrEmpty(reservation.QrToken))
    {
        return null;
    }
    return $"{QrPrefix}|{reservation.Id}|{reservation.QrToken}";
}
}
```

api/SunShare.Api/Helpers/TimeHelper.cs

```
/*
 * =====
 * File      : TimeHelper.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Small time helpers shared by the services. The API stores
 *              and compares all times in UTC (docs/02-ARCHITECTURE.md §8).
 * =====
 */

namespace SunShare.Api.Helpers;

// One place for time conversions, so every service treats times the same way.
public static class TimeHelper
{
    // Sri Lanka time is UTC + 5 hours 30 minutes (no daylight saving).
    public static readonly TimeSpan SriLankaOffset = TimeSpan.FromHours(5.5);

    // Makes sure a time from a client is in UTC. A time sent with a zone ("...Z" or "...+05:30")
    // is converted; a time sent without one ("2026-09-30T08:00") is taken as UTC already.
    public static DateTime AsUtc(DateTime value)
    {
        if (value.Kind == DateTimeKind.Unspecified)
        {
            return DateTime.SpecifyKind(value, DateTimeKind.Utc);
        }
        return value.ToUniversalTime();
    }

    // The moment "today" began in Sri Lanka (local midnight), written in UTC.
    // Example: at 23:00 on 28 Sep in Sri Lanka this returns 27 Sep 18:30 UTC.
    public static DateTime StartOfTodayInSriLankaUtc()
    {
        DateTime todayInSriLanka = (DateTime.UtcNow + SriLankaOffset).Date;
        return DateTime.SpecifyKind(todayInSriLanka - SriLankaOffset, DateTimeKind.Utc);
    }
}
```

api/SunShare.Api/Middleware/ErrorHandlingMiddleware.cs

```
/*
 * =====
 * File      : ErrorHandlingMiddleware.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Catches every error thrown while handling a request and
 *              answers with the same JSON shape: { "message": "..." }
 *              (docs/02-ARCHITECTURE.md §6).
 * =====
 */
using SunShare.Api.Helpers;
```

```

namespace SunShare.Api.Middleware;

// Middleware = a step every request passes through. This one is first in the pipeline,
// so it can catch errors from everything after it (controllers, services, database).
public class ErrorHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ErrorHandlingMiddleware> _logger;

    // Receives the next step of the pipeline and a logger (ASP.NET Core passes both in).
    public ErrorHandlingMiddleware(RequestDelegate next, ILogger<ErrorHandlingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    // Runs for every request: passes it on, and if anything throws, sends a clean JSON error instead.
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (ApiException ex)
        {
            // A rule was broken (thrown on purpose by a service): send its status code and message as they are.
            await WriteErrorAsync(context, ex.StatusCode, ex.Message);
        }
        catch (TimeoutException ex)
        {
            // The MongoDB driver gave up waiting: the database is not running or not reachable.
            _logger.LogError(ex, "Database timeout for {Method} {Path}", context.Request.Method, context.Request.Path);
            await WriteErrorAsync(context, StatusCodes.Status503ServiceUnavailable,
                "The database can't be reached right now. Please try again shortly.");
        }
        catch (Exception ex)
        {
            // An unexpected bug: keep the details in the server log, show only a safe message to the client.
            _logger.LogError(ex, "Unhandled error for {Method} {Path}", context.Request.Method, context.Request.Path);
            await WriteErrorAsync(context, StatusCodes.Status500InternalServerError, "Something went wrong on the server.");
        }
    }

    // Writes { "message": "..."} with the given HTTP status code.
    private static async Task WriteErrorAsync(HttpContext context, int statusCode, string message)
    {
        context.Response.StatusCode = statusCode;
        await context.Response.WriteAsJsonAsync(new { message });
    }
}

```

api/SunShare.Api/Models/EnergyBookingSlot.cs

```

/*
 * =====
 * File      : EnergyBookingSlot.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : One document in the "EnergyBookingSlots" collection: a time
 *              window at a station with a number of bookable places.
 * =====
 */
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SunShare.Api.Models;

// A bookable time window at one station, e.g. tomorrow 08:00-10:00 with 4 places.
[BsonIgnoreExtraElements]
public class EnergyBookingSlot
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; } = "";

    // Reference to the station (SolarStationInfo._id).
    [BsonElement("stationId")]
    [BsonRepresentation(BsonType.ObjectId)]

```

```

public string StationId { get; set; } = "";

// Stored in UTC (docs/02-ARCHITECTURE.md §8). End must be after start (R8).
[BsonElement("startTime")]
public DateTime StartTime { get; set; }

[BsonElement("endTime")]
public DateTime EndTime { get; set; }

// Places in this window: 1 up to the station's batterySlots (R8).
[BsonElement("totalSlots")]
public int TotalSlots { get; set; }

// Free places left. Only reservation create / cancel / move change it (R12).
[BsonElement("availableSlots")]
public int AvailableSlots { get; set; }

// Operators can close a window by setting this to false.
[BsonElement("isActive")]
public bool IsActive { get; set; }

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; }

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; }
}

```

api/SunShare.Api/Models/EnergyReservation.cs

```

/*
 * =====
 * File      : EnergyReservation.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : One document in the "EnergyReservations" collection: a
 *              prosumer's booking of a slot to sell or buy energy.
 * =====
 */
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SunShare.Api.Models;

// A booking. MongoDB has no JOINS, so it keeps the ids of the prosumer, station and slot,
// plus copies of the prosumer and station names for fast lists and search.
[BsonIgnoreExtraElements]
public class EnergyReservation
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; } = "";

    // Reference to Users._id (the NIC) and a copy of the name.
    [BsonElement("prosumerNic")]
    public string ProsumerNic { get; set; } = "";

    [BsonElement("prosumerName")]
    public string ProsumerName { get; set; } = "";

    // Reference to SolarStationInfo._id and a copy of the name.
    [BsonElement("stationId")]
    [BsonRepresentation(BsonType.ObjectId)]
    public string StationId { get; set; } = "";

    [BsonElement("stationName")]
    public string StationName { get; set; } = "";

    // Reference to EnergyBookingSlots._id.
    [BsonElement("slotId")]
    [BsonRepresentation(BsonType.ObjectId)]
    public string SlotId { get; set; } = "";

    // Copied from the slot (UTC); used by the 7-day and 12-hour rules (R9, R10).
    [BsonElement("startTime")]

```

```

public DateTime StartTime { get; set; }

[BsonElement("endTime")]
public DateTime EndTime { get; set; }

// Amount of energy, more than 0 and at most 100 kWh (R17).
[BsonElement("energyKwh")]
public double EnergyKwh { get; set; }

// One of ReservationTypes: Sell (drop off energy) or Buy (charge).
[BsonElement("type")]
public string Type { get; set; } = "";

// One of ReservationStatuses: Pending, Approved, Completed, Cancelled.
[BsonElement("status")]
public string Status { get; set; } = "";

// Random 32-character token set on approval (R13); part of the QR code text.
[BsonElement("qrToken")]
public string? QrToken { get; set; }

// NIC of the staff member who approved, and when.
[BsonElement("approvedBy")]
public string? ApprovedBy { get; set; }

[BsonElement("approvedAt")]
public DateTime? ApprovedAt { get; set; }

// NIC of the operator who scanned the QR and finished the transfer (R14), and when.
[BsonElement("completedBy")]
public string? CompletedBy { get; set; }

[BsonElement("completedAt")]
public DateTime? CompletedAt { get; set; }

// NIC of whoever cancelled, and when.
[BsonElement("cancelledBy")]
public string? CancelledBy { get; set; }

[BsonElement("cancelledAt")]
public DateTime? CancelledAt { get; set; }

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; }

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; }
}

```

api/SunShare.Api/Models/ReservationStatuses.cs

```

/*
 * =====
 * File      : ReservationStatuses.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : The reservation statuses from docs/01-SPEC.md §3
 *              (Pending -> Approved -> Completed, or -> Cancelled).
 * =====
 */

namespace SunShare.Api.Models;

// Values of EnergyReservations.status.
public static class ReservationStatuses
{
    // New booking waiting for staff approval (R13).
    public const string Pending = "Pending";

    // Approved by staff; has a QR token (R13).
    public const string Approved = "Approved";

    // QR scanned and energy transfer finished by an operator (R14). Can't be changed (R15).
    public const string Completed = "Completed";
}

```

```

    // Cancelled at least 12 hours before the start (R10). Can't be changed (R15).
    public const string Cancelled = "Cancelled";
}

```

api/SunShare.Api/Models/ReservationTypes.cs

```

/*
 * =====
 * File      : ReservationTypes.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-28
 * Description : The two kinds of energy booking: Sell and Buy.
 * =====
 */

namespace SunShare.Api.Models;

// Values of EnergyReservations.type.
public static class ReservationTypes
{
    // Drop-off: the prosumer stores spare solar energy in the station's battery.
    public const string Sell = "Sell";

    // Charging: the prosumer takes energy from the station.
    public const string Buy = "Buy";
}

```

api/SunShare.Api/Models/Roles.cs

```

/*
 * =====
 * File      : Roles.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-28
 * Description : The three account roles (rule R2), written once so every
 *              file uses exactly the same spelling.
 * =====
 */

namespace SunShare.Api.Models;

// Role names stored in Users.role, put inside the login token, and used in [Authorize(Roles = ...)].
public static class Roles
{
    public const string Backoffice = "Backoffice";

    public const string GridOperator = "GridOperator";

    public const string Prosumer = "Prosumer";

    // Both staff roles, for endpoints Backoffice and GridOperator can both use: [Authorize(Roles = Roles.Staff)].
    public const string Staff = Backoffice + "," + GridOperator;
}

```

api/SunShare.Api/Models/SolarStation.cs

```

/*
 * =====
 * File      : SolarStation.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-28
 * Description : One document in the "SolarStationInfo" collection: a solar
 *              microgrid station (node) with battery places and opening hours.
 * =====
 */

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

```



```

namespace SunShare.Api.Models;

// A solar station where prosumers drop off (Sell) or collect (Buy) energy.
[BsonIgnoreExtraElements]
public class SolarStation
{
    // MongoDB ObjectId, handled as a string in C# (e.g. "66f6a1c2e4b0a1b2c3d4e5f1").
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; } = "";

    [BsonElement("name")]
    public string Name { get; set; } = "";

    [BsonElement("address")]
    public string Address { get; set; } = "";

    // GPS position, used for the map and the "nearby stations" distance (R18).
    [BsonElement("latitude")]
    public double Latitude { get; set; }

    [BsonElement("longitude")]
    public double Longitude { get; set; }

    // Solar generation capacity in kilowatts.
    [BsonElement("capacityKw")]
    public double CapacityKw { get; set; }

    // Number of battery places at the hub; a slot can't offer more places than this (R8).
    [BsonElement("batterySlots")]
    public int BatterySlots { get; set; }

    // Operating schedule as "HH:mm" text, e.g. "06:00" to "18:00".
    [BsonElement("openTime")]
    public string OpenTime { get; set; } = "";

    [BsonElement("closeTime")]
    public string CloseTime { get; set; } = "";

    // false = deactivated (R6).
    [BsonElement("isActive")]
    public bool IsActive { get; set; }

    [BsonElement("createdAt")]
    public DateTime CreatedAt { get; set; }

    [BsonElement("updatedAt")]
    public DateTime UpdatedAt { get; set; }
}

```

api/SunShare.Api/Models/User.cs

```

/*
 * =====
 * File      : User.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : One document in the "Users" collection: a Backoffice,
 *              GridOperator or Prosumer account. The NIC is the _id.
 * =====
 */
using MongoDB.Bson.Serialization.Attributes;

namespace SunShare.Api.Models;

// A user account. [BsonElement] gives each field its camelCase name in MongoDB.
// [BsonIgnoreExtraElements] = skip unknown fields instead of crashing when reading old documents.
[BsonIgnoreExtraElements]
public class User
{
    // The NIC is the primary key (_id), as the brief asks. Its format is checked by rule R1.
    [BsonId]
    public string Nic { get; set; } = "";
}

```

```

[BsonElement("fullName")]
public string FullName { get; set; } = "";

[BsonElement("email")]
public string Email { get; set; } = "";

[BsonElement("phone")]
public string Phone { get; set; } = "";

// Optional: the prosumer's property address.
[BsonElement("address")]
public string? Address { get; set; }

// One of Roles: Backoffice, GridOperator, Prosumer.
[BsonElement("role")]
public string Role { get; set; } = "";

// One of UserStatuses: Pending, Active, Deactivated.
[BsonElement("status")]
public string Status { get; set; } = "";

// BCrypt hash of the password. The plain password is never stored and this is never sent to clients.
[BsonElement("passwordHash")]
public string PasswordHash { get; set; } = "";

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; }

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; }
}

```

api/SunShare.Api/Models/UserStatuses.cs

```

/*
 * =====
 * File       : UserStatuses.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Shared foundation
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-28
 * Description : The account statuses from docs/01-SPEC.md §3
 *              (Pending -> Active -> Deactivated -> Active).
 * =====
 */

namespace SunShare.Api.Models;

// Values of Users.status. Only Active users can log in (R3, R4).
public static class UserStatuses
{
    // Self-registered prosumer waiting for Backoffice activation (R3).
    public const string Pending = "Pending";

    public const string Active = "Active";

    // Can't log in; only Backoffice can reactivate (R4).
    public const string Deactivated = "Deactivated";
}

```

api/SunShare.Api/Program.cs

```

/*
 * =====
 * File       : Program.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Shared foundation
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-27
 * Description : Start-up of the SunShare Web API. Registers the services
 *              (MongoDB, controllers + JSON, JWT login tokens, CORS, Swagger),
 *              prepares the database, sets up the request pipeline and starts
 *              listening for requests.
 * =====
 */

```

```

*/
using System.Text;
using System.Text.Json;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi;
using SunShare.Api.Data;
using SunShare.Api.Helpers;
using SunShare.Api.Middleware;
using SunShare.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// ----- MongoDB -----
// Read the "MongoDB" section of appsettings.json and register ONE shared MongoDBContext
// (a singleton) that every controller and service receives.
var mongoSettings = builder.Configuration.GetSection("MongoDB").Get<MongoDbSettings>(!);
builder.Services.AddSingleton(new MongoDBContext(mongoSettings));

// ----- Our helpers and services -----
// Registered here so ASP.NET Core can hand them to the controllers that ask for them ("dependency injection").
// Singleton = one shared object for the whole app; Scoped = a fresh object for each request.
builder.Services.AddSingleton<JwtTokenHelper>();
builder.Services.AddScoped<UserService>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<StationService>();
builder.Services.AddScoped<SlotService>();
builder.Services.AddScoped<ReservationService>();
builder.Services.AddScoped<ReservationQueryService>();
builder.Services.AddScoped<QrService>();
builder.Services.AddScoped<DashboardService>();

// ----- Controllers + JSON -----
// JSON uses camelCase names (e.g. availableSlots). If a request body is broken or badly typed,
// answer 400 with { "message": "..." } like every other error, instead of ASP.NET's default shape.
// Empty text fields are NOT rejected automatically: the services check them and give friendly
// messages ("Please enter the full name.") - FAT service.
builder.Services.AddControllers(options => options.SuppressImplicitRequiredAttributeForNonNullableReferenceTypes = true)
    .AddJsonOptions(options => options.JsonSerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase)
    .ConfigureApiBehaviorOptions(options =>
        options.InvalidModelStateResponseFactory = context =>
        {
            // Find the first field that could not be read, e.g. "$.energyKwh" -> "energyKwh"
            // ("request" is just the name of the whole body, so it is skipped).
            string? field = context.ModelState
                .Where(entry => entry.Value!.Errors.Count > 0)
                .Select(entry => entry.Key.TrimStart('$', '.'))
                .FirstOrDefault(key => key != "" && key != "request");
            string message = field == null
                ? "The request data is missing or not in the right format."
                : $"The value of '{field}' is not in the right format.";
            return new BadRequestObjectResult(new { message });
        });

// ----- Login tokens (JWT) -----
// Protected requests must send "Authorization: Bearer <token>". A token is accepted only if it was
// signed with our secret key, made by this API, meant for our apps, and has not expired.
// Reference: Microsoft Learn, "Configure JWT bearer authentication in ASP.NET Core"
// https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication
var jwtSettings = builder.Configuration.GetSection("Jwt");
var signingKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["Key"]!));
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        // Keep the short claim names from the token ("nic", "name", "role") instead of renaming them
        // to long Microsoft URLs, and tell ASP.NET which claim holds the role for [Authorize(Roles = ...)].
        options.MapInboundClaims = false;
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = jwtSettings["Issuer"],
            ValidateAudience = true,
            ValidAudience = jwtSettings["Audience"],
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = signingKey,
            ClockSkew = TimeSpan.FromMinutes(1),
            NameClaimType = JwtTokenHelper.NameClaim,
            RoleClaimType = JwtTokenHelper.RoleClaim
        };
        // Missing/expired token (401) and wrong role (403) also answer with { "message": "..." }.
        options.Events = new JwtBearerEvents
        {
            OnChallenge = async context =>
            {

```

```

        context.HandleResponse();
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.Response.WriteAsJsonAsync(new { message = "Please log in to continue." });
    },
    OnForbidden = async context =>
    {
        context.Response.StatusCode = StatusCodes.Status403Forbidden;
        await context.Response.WriteAsJsonAsync(new { message = "You don't have permission to do that." });
    }
    });
builder.Services.AddAuthorization();

// ----- CORS -----
// Browsers only let a web page call an API on another address if the API allows that page's address.
// Allowed: the React dev server (5173) and the IIS-hosted web app (8081), from appsettings.json.
string[] allowedOrigins = builder.Configuration.GetSection("Cors:AllowedOrigins").Get<string[]>() ?? [];
bool isDevelopment = builder.Environment.IsDevelopment();
builder.Services.AddCors(options => options.AddPolicy("WebApp", policy => policy
    .SetIsOriginAllowed(IsAllowedOrigin)
    .AllowAnyHeader()
    .AllowAnyMethod()));

// ----- Swagger -----
// The click-to-test page at /swagger, with an "Authorize" button to paste a login token.
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "SunShare API",
        Version = "v1",
        Description = "Smart Solar Microgrid Trading System - SE4040 Assignment 1"
    });
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        Description = "Paste the token from POST /api/auth/login (just the token, without the word Bearer).",
    });
    options.AddSecurityRequirement(document => new OpenApiSecurityRequirement
    {
        [new OpenApiSecuritySchemeReference("Bearer", document)] = []
    });
});

var app = builder.Build();

// ----- Prepare the database -----
// Once at start-up: create the indexes, then add the sample data if the database is empty.
// If MongoDB is down we only log it, so the API still starts and GET /api/health can report "unreachable".
var db = app.Services.GetRequiredService<MongoDbContext>();
try
{
    await db.CreateIndexesAsync();
    await new DataSeeder(db).SeedAsync();
}
catch (TimeoutException ex)
{
    app.Logger.LogError(ex, "MongoDB could not be reached at start-up, so indexes and sample data were skipped.");
}

// ----- Request pipeline (the order matters) -----
app.UseMiddleware<ErrorHandlingMiddleware>(); // 1. catch every error -> { "message" }
app.UseSwagger(); // 2. Swagger stays ON in every environment (also on IIS for the demo)
app.UseSwaggerUI();
app.UseCors("WebApp"); // 3. CORS headers for the web app
app.UseAuthentication(); // 4. read the token: who is calling?
app.UseAuthorization(); // 5. check [Authorize(Roles = ...)]: may they do this?
app.MapControllers(); // 6. run the matching controller action

app.Run();

// CORS check: may a web page from this address (its "origin") call the API?
// Allowed if it is in the list; while developing, any localhost port is also fine
// (e.g. a preview browser that had to use a different port).
bool IsAllowedOrigin(string origin)
{
    if (allowedOrigins.Contains(origin))
    {
        return true;
    }
    return isDevelopment && Uri.TryCreate(origin, UriKind.Absolute, out Uri? uri) && uri.Host == "localhost";
}

```

```
}
```

api/SunShare.Api/Services/AuthService.cs

```
/*
 * =====
 * File      : AuthService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-28
 * Description : Login (checks the password and the account status, then
 *                  gives a JWT) and mobile self-registration (rules R1, R3, R4).
 * =====
 */
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// Everything about getting into the system.
public class AuthService
{
    private readonly MongoDBContext _db;
    private readonly JwtTokenHelper _jwt;
    private readonly UserService _users;

    // Receives the database, the token maker and the user service (for registration).
    public AuthService(MongoDbContext db, JwtTokenHelper jwt, UserService users)
    {
        _db = db;
        _jwt = jwt;
        _users = users;
    }

    // Logs a user in: finds them by NIC, checks the password against the BCrypt hash,
    // checks the account is Active (R3, R4), then returns a signed token and who they are.
    public async Task<LoginResponse> LoginAsync(LoginRequest request)
    {
        if (string.IsNullOrEmpty(request.Nic) || string.IsNullOrEmpty(request.Password))
        {
            throw new ApiException(400, "Please enter your NIC and password.");
        }

        string nic = NicValidator.Normalize(request.Nic);
        User? user = await _db.Users.Find(u => u.Nic == nic).FirstOrDefaultAsync();

        // Same message for "no such NIC" and "wrong password", so nobody can guess which NICs exist.
        if (user == null || !BCrypt.Net.BCrypt.Verify(request.Password, user.PasswordHash))
        {
            throw new ApiException(401, "NIC or password is incorrect.");
        }

        // RULE R3: a self-registered prosumer can't log in until Backoffice activates them.
        if (user.Status == UserStatuses.Pending)
        {
            throw new ApiException(403, "Your account is waiting for Backoffice activation.");
        }
        // RULE R4: a deactivated user can't log in.
        if (user.Status == UserStatuses.Deactivated)
        {
            throw new ApiException(403, "Your account is deactivated. Please contact the Backoffice.");
        }

        var (token, expiresAt) = _jwt.CreateToken(user);
        return new LoginResponse
        {
            Token = token,
            Nic = user.Nic,
            FullName = user.FullName,
            Role = user.Role,
            ExpiresAt = expiresAt
        };
    }

    // Mobile self-registration. The NIC, details and password are checked by UserService (R1).
    public async Task<UserResponse> RegisterAsync(RegisterRequest request)
```

```

    {
        // RULE R3: a self-registered account is always a Prosumer and starts Pending.
        return await _users.AddUserAsync(request.Nic, request.FullName, request.Email, request.Phone,
            request.Address, Roles.Prosumer, UserStatuses.Pending, request.Password);
    }
}

```

api/SunShare.Api/Services/DashboardService.cs

```

/*
 * =====
 * File      : DashboardService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : Counts for the staff dashboard and the prosumer home screen,
 *              using the definitions in docs/01-SPEC.md §5.
 * =====
 */
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// Every number on a dashboard is counted here, on the server, so web and mobile always agree.
public class DashboardService
{
    // How many pending bookings the staff dashboard table shows.
    private const int PendingListSize = 5;

    private readonly MongoDBContext _db;

    // Receives the shared database context.
    public DashboardService(MongoDbContext db)
    {
        _db = db;
    }

    // Staff dashboard (web W3, operator mobile home M10): the count cards and the next pending bookings.
    public async Task<StaffDashboardResponse> GetStaffSummaryAsync()
    {
        DateTime now = DateTime.UtcNow;
        DateTime todayStart = TimeHelper.StartOfTodayInSriLankaUtc();
        DateTime tomorrowStart = todayStart.AddDays(1);
        var reservations = _db.EnergyReservations;

        List<EnergyReservation> nextPending = await reservations
            .Find(r => r.Status == ReservationStatuses.Pending && r.StartTime > now)
            .SortBy(r => r.StartTime)
            .Limit(PendingListSize)
            .ToListAsync();

        return new StaffDashboardResponse
        {
            PendingReservations = await reservations.CountDocumentsAsync(r => r.Status == ReservationStatuses.Pending),
            ApprovedFutureReservations = await reservations.CountDocumentsAsync(r =>
                r.Status == ReservationStatuses.Approved && r.StartTime > now),
            TodayReservations = await reservations.CountDocumentsAsync(r =>
                r.Status != ReservationStatuses.Cancelled && r.StartTime >= todayStart && r.StartTime < tomorrowStart),
            ActiveStations = await _db.SolarStationInfo.CountDocumentsAsync(s => s.IsActive),
            PendingActivations = await _db.Users.CountDocumentsAsync(u => u.Role == Roles.Prosumer
                && (u.Status == UserStatuses.Pending || u.Status == UserStatuses.Deactivated)),
            PendingList = nextPending.Select(ReservationMapper.ToResponse).ToList()
        };
    }

    // Prosumer home (M3): counts of MY bookings and my next upcoming one.
    // RULE R16: only the caller's own bookings (NIC from the token) are counted.
    public async Task<ProsumerDashboardResponse> GetProsumerSummaryAsync(string callerNic)
    {
        DateTime now = DateTime.UtcNow;
        var reservations = _db.EnergyReservations;

        EnergyReservation? next = await reservations
            .Find(r => r.ProsumerNic == callerNic && r.StartTime > now
                && (r.Status == ReservationStatuses.Pending || r.Status == ReservationStatuses.Approved))

```

```

        .SortBy(r => r.StartTime)
        .FirstOrDefaultAsync();

return new ProsumerDashboardResponse
{
    PendingCount = await reservations.CountDocumentsAsync(r =>
        r.ProsumerNic == callerNic && r.Status == ReservationStatuses.Pending),
    ApprovedFutureCount = await reservations.CountDocumentsAsync(r =>
        r.ProsumerNic == callerNic && r.Status == ReservationStatuses.Approved && r.StartTime > now),
    CompletedCount = await reservations.CountDocumentsAsync(r =>
        r.ProsumerNic == callerNic && r.Status == ReservationStatuses.Completed),
    NextReservation = next == null ? null : ReservationMapper.ToResponse(next)
};
}
}
}

```

api/SunShare.Api/Services/QrService.cs

```

/*
 * =====
 * File      : QrService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : Checks a QR code scanned by a Grid Operator and finishes the
 *              energy transfer (rule R14).
 * =====
 */
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// The QR only holds "SUNSHARE|<reservationId>|<qrToken>". The server decides if it is valid,
// so a screenshot of an old or edited booking's QR is useless (its token no longer matches).
public class QrService
{
    private readonly MongoDBContext _db;

    // Receives the shared database context.
    public QrService(MongoDbContext db)
    {
        _db = db;
    }

    // Verify (M11 "Verified with server"): returns the booking's details if the QR is valid right now.
    public async Task<ReservationResponse> VerifyAsync(string qrData)
    {
        EnergyReservation reservation = await CheckQrAsync(qrData);
        return ReservationMapper.ToResponse(reservation);
    }

    // Finalize: the same checks again, then Completed + who/when. After this the QR can't be used again.
    public async Task<ReservationResponse> CompleteAsync(string id, string qrData, string callerNic)
    {
        EnergyReservation reservation = await CheckQrAsync(qrData);
        // The scanned QR must belong to the booking being completed (the id in the URL).
        if (reservation.Id != id)
        {
            throw new ApiException(400, "This QR code belongs to a different booking.");
        }

        DateTime now = DateTime.UtcNow;
        reservation.Status = ReservationStatuses.Completed;
        reservation.CompletedBy = callerNic;
        reservation.CompletedAt = now;
        reservation.UpdatedAt = now;

        await _db.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id, reservation);
        return ReservationMapper.ToResponse(reservation);
    }

    // RULE R14: the QR text must be "SUNSHARE|<reservationId>|<qrToken>", the booking must exist and be
    // Approved, and the token must match the one saved when it was approved. Each failure has its own message.
    private async Task<EnergyReservation> CheckQrAsync(string qrData)

```

```

{
    string[] parts = (qrData ?? "").Trim().Split('|');
    if (parts.Length != 3 || parts[0] != ReservationMapper.QrPrefix || !ObjectId.TryParse(parts[1], out _) || parts[2] ==
    "")
    {
        throw new ApiException(400, "This is not a SunShare QR code.");
    }
    string reservationId = parts[1];
    string token = parts[2];

    EnergyReservation? reservation = await _db.EnergyReservations.Find(r => r.Id == reservationId).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new ApiException(400, "This QR code's booking doesn't exist.");
    }
    if (reservation.Status == ReservationStatuses.Completed)
    {
        throw new ApiException(400, "This booking is already completed - a QR code can only be used once.");
    }
    if (reservation.Status == ReservationStatuses.Cancelled)
    {
        throw new ApiException(400, "This booking was cancelled.");
    }
    if (reservation.Status != ReservationStatuses.Approved)
    {
        throw new ApiException(400, "This booking hasn't been approved yet.");
    }
    if (reservation.QrToken != token)
    {
        throw new ApiException(400, "This QR code is out of date (the booking was changed). Ask the prosumer to open the
    booking again.");
    }
    return reservation;
}
}

```

api/SunShare.Api/Services/ReservationQueryService.cs

```

/*
 * =====
 * File      : ReservationQueryService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-28
 * Description : Reads bookings for the lists: views current / pending /
 *               history / all, filters (status, station, dates), search,
 *               and one booking by id (rule R16 + list definitions).
 * =====
 */
using System.Text.RegularExpressions;
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// Read-only booking queries used by the web Reservations page (W13) and the mobile Bookings tabs (M8).
public class ReservationQueryService
{
    private readonly MongoDBContext _db;
    private readonly ReservationService _reservations;

    // Receives the database and Part C's ReservationService (for find-by-id and the owner check).
    public ReservationQueryService(MongoDbContext db, ReservationService reservations)
    {
        _db = db;
        _reservations = reservations;
    }

    // Lists bookings for the caller. view = all (default) | current | pending | history.
    // Optional filters: status, stationId, from/to (start time, UTC), and a search text.
    // current/pending/all are soonest first; history is newest first.
    public async Task<List<ReservationResponse>> ListAsync(string? view, string? status, string? stationId,
        DateTime? from, DateTime? to, string? search, string callerNic, string callerRole)
    {
        var filter = Builders<EnergyReservation>.Filter.Empty;
        // RULE R16: a prosumer only ever sees their own bookings.
        if (callerRole == Roles.Prosumer)
        {

```



```

        filter &= Builders<EnergyReservation>.Filter.Eq(r => r.ProsumerNic, callerNic);
    }

    string chosenView = string.IsNullOrEmpty(view) ? "all" : view.Trim().ToLowerInvariant();
    filter &= ViewFilter(chosenView);
    filter &= OptionalFilters(status, stationId, from, to);
    if (!string.IsNullOrEmpty(search))
    {
        filter &= SearchFilter(search.Trim(), callerRole);
    }

    var query = _db.EnergyReservations.Find(filter);
    List<EnergyReservation> reservations = chosenView == "history"
        ? await query.SortByDescending(r => r.StartTime).ToListAsync()
        : await query.SortBy(r => r.StartTime).ToListAsync();
    return reservations.Select(ReservationMapper.ToResponse).ToList();
}

// Gets one booking. RULE R16: a prosumer can only open their own (403 otherwise).
public async Task<ReservationResponse> GetAsync(string id, string callerNic, string callerRole)
{
    EnergyReservation reservation = await _reservations.FindOrThrowAsync(id);
    ReservationService.CheckOwner(reservation, callerNic, callerRole);
    return ReservationMapper.ToResponse(reservation);
}

// The list definitions from docs/01-SPEC.md §5:
// current = Approved and in the future; pending = Pending; history = Completed, Cancelled or already started.
private static FilterDefinition<EnergyReservation> ViewFilter(string view)
{
    var f = Builders<EnergyReservation>.Filter;
    DateTime now = DateTime.UtcNow;
    switch (view)
    {
        case "all":
            return f.Empty;
        case "current":
            return f.Eq(r => r.Status, ReservationStatuses.Approved) & f.Gt(r => r.StartTime, now);
        case "pending":
            return f.Eq(r => r.Status, ReservationStatuses.Pending);
        case "history":
            return f.In(r => r.Status, [ReservationStatuses.Completed, ReservationStatuses.Cancelled])
                | f.Lte(r => r.StartTime, now);
        default:
            throw new ApiException(400, "View must be all, current, pending or history.");
    }
}

// Adds the optional filter-bar choices: one status, one station, and a start-time range (to is exclusive).
private static FilterDefinition<EnergyReservation> OptionalFilters(string? status, string? stationId, DateTime? from,
DateTime? to)
{
    var f = Builders<EnergyReservation>.Filter;
    var filter = f.Empty;
    if (!string.IsNullOrEmpty(status))
    {
        filter &= f.Eq(r => r.Status, status);
    }
    if (!string.IsNullOrEmpty(stationId))
    {
        if (!ObjectId.TryParse(stationId, out _))
        {
            throw new ApiException(400, "The station filter is not a valid station id.");
        }
        filter &= f.Eq(r => r.StationId, stationId);
    }
    if (from.HasValue)
    {
        filter &= f.Gte(r => r.StartTime, TimeHelper.AsUtc(from.Value));
    }
    if (to.HasValue)
    {
        filter &= f.Lt(r => r.StartTime, TimeHelper.AsUtc(to.Value));
    }
    return filter;
}

// Search box: matches the station name (not case-sensitive), or a full 24-character reservation id.
// Staff can also search by prosumer NIC or name. Regex.Escape makes symbols count as plain text.
private static FilterDefinition<EnergyReservation> SearchFilter(string search, string callerRole)
{
    var f = Builders<EnergyReservation>.Filter;
    var pattern = new BsonRegularExpression(Regex.Escape(search), "i");

```

```

        var matches = new List<FilterDefinition<EnergyReservation>> { f.Regex(r => r.StationName, pattern) };
        if (ObjectId.TryParse(search, out _))
        {
            matches.Add(f.Eq(r => r.Id, search));
        }
        if (callerRole != Roles.Prosumer)
        {
            matches.Add(f.Regex(r => r.ProsumerNic, pattern));
            matches.Add(f.Regex(r => r.ProsumerName, pattern));
        }
        return f.Or(matches);
    }
}

```

api/SunShare.Api/Services/ReservationService.cs

```

/*
 * =====
 * File      : ReservationService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author    : Malkith G W L (IT22630834)
 * Created   : 2026-09-28
 * Description : Business rules for creating, updating, cancelling and
 *              approving energy reservations (rules R9-R13, R15-R17).
 * =====
 */
using System.Security.Cryptography;
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// All booking rules live here (FAT service). Controllers only pass in the request and who is calling.
public class ReservationService
{
    // Bookings can be made at most this many days ahead (R9).
    private const int MaxDaysAhead = 7;

    // Largest energy amount for one booking, in kWh (R17).
    private const double MaxEnergyKwh = 100;

    private readonly MongoDBContext _db;
    private readonly SlotService _slots;
    private readonly StationService _stations;

    // Receives the database and the slot/station services (to look up and check the chosen slot).
    public ReservationService(MongoDbContext db, SlotService slots, StationService stations)
    {
        _db = db;
        _slots = slots;
        _stations = stations;
    }

    // Creates a booking. It starts Pending and takes one place from its slot.
    // Checks R16 (who it is for), R11 (active prosumer, bookable slot, no double booking), R9 (time), R17 (energy).
    public async Task<ReservationResponse> CreateAsync(CreateReservationRequest request, string callerNic, string callerRole)
    {
        // RULE R16: a prosumer always books for themselves (NIC from the token); staff must say which prosumer.
        string prosumerNic = callerRole == Roles.Prosumer ? callerNic : NicValidator.Normalize(request.ProsumerNic);
        if (prosumerNic == "")
        {
            throw new ApiException(400, "Please choose the prosumer this booking is for.");
        }
        User prosumer = await FindActiveProsumerAsync(prosumerNic);
        ValidateEnergyAndType(request.EnergyKwh, request.Type);
        var (slot, station) = await LoadBookableSlotAsync(request.SlotId);
        await CheckNotAlreadyBookedAsync(prosumer.Nic, slot.Id, null);

        // RULE R12: take one place from the slot (fails with 409 if someone just took the last one).
        await TakePlaceAsync(slot.Id);

        DateTime now = DateTime.UtcNow;
        var reservation = new EnergyReservation
        {
            Id = ObjectId.GenerateNewId().ToString(),
            ProsumerNic = prosumer.Nic,

```

```

        ProsumerName = prosumer.FullName,
        EnergyKwh = request.EnergyKwh,
        Type = request.Type,
        // RULE R13: every new booking starts Pending and needs staff approval.
        Status = ReservationStatuses.Pending,
        CreatedAt = now,
        UpdatedAt = now
    };
    CopySlotDetails(reservation, slot, station);

    await _db.EnergyReservations.InsertOneAsync(reservation);
    return ReservationMapper.ToResponse(reservation);
}

// Changes a booking's slot, energy amount or type.
// Checks R16 (own booking), R15 + R10 (still changeable), R11 + R9 for a new slot, R17 (energy).
public async Task<ReservationResponse> UpdateAsync(string id, UpdateReservationRequest request, string callerNic, string callerRole)
{
    EnergyReservation reservation = await FindOrThrowAsync(id);
    CheckOwner(reservation, callerNic, callerRole);
    CheckCanChange(reservation);
    await FindActiveProsumerAsync(reservation.ProsumerNic);
    ValidateEnergyAndType(request.EnergyKwh, request.Type);

    bool moving = !string.IsNullOrEmpty(request.SlotId) && request.SlotId != reservation.SlotId;
    bool changed = moving || request.EnergyKwh != reservation.EnergyKwh || request.Type != reservation.Type;
    if (!changed)
    {
        return ReservationMapper.ToResponse(reservation);
    }

    if (moving)
    {
        var (newSlot, newStation) = await LoadBookableSlotAsync(request.SlotId);
        await CheckNotAlreadyBookedAsync(reservation.ProsumerNic, newSlot.Id, reservation.Id);
        // RULE R12: moving = the new slot loses a place, the old slot gets one back.
        await TakePlaceAsync(newSlot.Id);
        await GivePlaceBackAsync(reservation.SlotId);
        CopySlotDetails(reservation, newSlot, newStation);
    }
    reservation.EnergyKwh = request.EnergyKwh;
    reservation.Type = request.Type;

    // RULE R13: an edited Approved booking must be approved again, and its old QR code stops working.
    if (reservation.Status == ReservationStatuses.Approved)
    {
        reservation.Status = ReservationStatuses.Pending;
        reservation.QrToken = null;
        reservation.ApprovedBy = null;
        reservation.ApprovedAt = null;
    }
    reservation.UpdatedAt = DateTime.UtcNow;

    await _db.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id, reservation);
    return ReservationMapper.ToResponse(reservation);
}

// Cancels a booking and gives its place back to the slot.
// Checks R16 (own booking) and R15 + R10 (still changeable), then R12 (+1 place).
public async Task<ReservationResponse> CancelAsync(string id, string callerNic, string callerRole)
{
    EnergyReservation reservation = await FindOrThrowAsync(id);
    CheckOwner(reservation, callerNic, callerRole);
    CheckCanChange(reservation);

    DateTime now = DateTime.UtcNow;
    reservation.Status = ReservationStatuses.Cancelled;
    reservation.QrToken = null;
    reservation.CancelledBy = callerNic;
    reservation.CancelledAt = now;
    reservation.UpdatedAt = now;
    await _db.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id, reservation);

    // RULE R12: a cancelled booking frees its place.
    await GivePlaceBackAsync(reservation.SlotId);
    return ReservationMapper.ToResponse(reservation);
}

// Staff approve a Pending booking: it becomes Approved and gets a random QR token (R13).
public async Task<ReservationResponse> ApproveAsync(string id, string callerNic)
{
    EnergyReservation reservation = await FindOrThrowAsync(id);

```

```

// RULE R13: only Pending bookings can be approved.
if (reservation.Status != ReservationStatuses.Pending)
{
    throw new ApiException(400, $"Only pending bookings can be approved. This one is {reservation.Status}.");
}
if (reservation.StartTime <= DateTime.UtcNow)
{
    throw new ApiException(400, "This booking's time has already passed, so it can't be approved.");
}

DateTime now = DateTime.UtcNow;
reservation.Status = ReservationStatuses.Approved;
// RULE R13: a random 32-character token; the QR code is only valid with it (R14).
reservation.QrToken = Convert.ToHexString(RandomNumberGenerator.GetBytes(16)).ToLowerInvariant();
reservation.ApprovedBy = callerNic;
reservation.ApprovedAt = now;
reservation.UpdatedAt = now;

await _db.EnergyReservations.ReplaceOneAsync(r => r.Id == reservation.Id, reservation);
return ReservationMapper.ToResponse(reservation);
}

// Loads a booking by id, or throws 404 (also for ids that aren't valid MongoDB ObjectIds).
public async Task<EnergyReservation> FindOrThrowAsync(string id)
{
    if (!ObjectId.TryParse(id, out _))
    {
        throw new ApiException(404, "Reservation not found.");
    }
    EnergyReservation? reservation = await _db.EnergyReservations.Find(r => r.Id == id).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new ApiException(404, "Reservation not found.");
    }
    return reservation;
}

// RULE R16: a prosumer may only touch their own bookings; staff may touch any.
public static void CheckOwner(EnergyReservation reservation, string callerNic, string callerRole)
{
    if (callerRole == Roles.Prosumer && reservation.ProsumerNic != callerNic)
    {
        throw new ApiException(403, "This booking belongs to someone else.");
    }
}

// RULE R15: Completed and Cancelled bookings can't be changed.
// RULE R10: changes and cancellations need at least 12 hours' notice before the start.
private static void CheckCanChange(EnergyReservation reservation)
{
    if (reservation.Status == ReservationStatuses.Completed || reservation.Status == ReservationStatuses.Cancelled)
    {
        throw new ApiException(400, $"This booking is already {reservation.Status.ToLowerInvariant()} and can't be changed.");
    }
    if (reservation.StartTime - DateTime.UtcNow < ReservationMapper.MinChangeNotice)
    {
        throw new ApiException(400, "Changes and cancellations need at least 12 hours' notice before the booking starts.");
    }
}

// RULE R11: only an existing, Active prosumer can hold a booking.
private async Task<User> FindActiveProsumerAsync(string nic)
{
    User? user = await _db.Users.Find(u => u.Nic == nic).FirstOrDefaultAsync();
    if (user == null || user.Role != Roles.Prosumer)
    {
        throw new ApiException(404, "Prosumer not found.");
    }
    if (user.Status != UserStatuses.Active)
    {
        throw new ApiException(400, "Only active prosumers can hold bookings.");
    }
    return user;
}

// Loads the chosen slot and its station, and checks the slot can take a booking now.
// RULE R11: the slot is open, its station is active and it has a free place.
// RULE R9: it starts in the future and no more than 7 days from now.
private async Task<EnergyBookingSlot Slot, SolarStation Station> LoadBookableSlotAsync(string slotId)
{
    if (string.IsNullOrWhiteSpace(slotId))
    {

```

```

        throw new ApiException(400, "Please choose a slot.");
    }
    EnergyBookingSlot slot = await _slots.FindOrThrowAsync(slotId);
    SolarStation station = await _stations.FindOrThrowAsync(slot.StationId);

    if (!station.IsActive)
    {
        throw new ApiException(400, "This station is not taking bookings right now.");
    }
    if (!slot.IsActive)
    {
        throw new ApiException(400, "This slot is closed for booking.");
    }
    DateTime now = DateTime.UtcNow;
    if (slot.StartTime <= now)
    {
        throw new ApiException(400, "This slot has already started. Please pick a future slot.");
    }
    if (slot.StartTime > now.AddDays(MaxDaysAhead))
    {
        throw new ApiException(400, $"Bookings can be made at most {MaxDaysAhead} days ahead.");
    }
    if (slot.AvailableSlots <= 0)
    {
        throw new ApiException(409, "This slot is full. Please pick another one.");
    }
    return (slot, station);
}

// RULE R11: a prosumer can't have two live bookings in the same slot (a Cancelled one doesn't count).
// exceptId = the booking being edited, so it doesn't count against itself.
private async Task CheckNotAlreadyBookedAsync(string prosumerNic, string slotId, string? exceptId)
{
    bool alreadyBooked = await _db.EnergyReservations
        .Find(r => r.ProsumerNic == prosumerNic && r.SlotId == slotId
            && r.Status != ReservationStatuses.Cancelled && r.Id != exceptId)
        .AnyAsync();
    if (alreadyBooked)
    {
        throw new ApiException(409, "This prosumer already has a booking in this slot.");
    }
}

// RULE R17: energy must be more than 0 and at most 100 kWh; the type must be Sell or Buy.
private static void ValidateEnergyAndType(double energyKwh, string type)
{
    if (energyKwh <= 0 || energyKwh > MaxEnergyKwh)
    {
        throw new ApiException(400, $"Energy must be more than 0 and at most {MaxEnergyKwh} kWh.");
    }
    if (type != ReservationTypes.Sell && type != ReservationTypes.Buy)
    {
        throw new ApiException(400, "Type must be Sell or Buy.");
    }
}

// RULE R12: takes one free place from a slot. "Is a place free?" and "subtract 1" happen in ONE
// database step (the filter only matches while availableSlots > 0), so two people can never both
// get the last place.
private async Task TakePlaceAsync(string slotId)
{
    var takeOne = Builders<EnergyBookingSlot>.Update.Inc(s => s.AvailableSlots, -1);
    UpdateResult result = await _db.EnergyBookingSlots.UpdateOneAsync(
        s => s.Id == slotId && s.AvailableSlots > 0, takeOne);
    if (result.ModifiedCount == 0)
    {
        throw new ApiException(409, "This slot is full. Please pick another one.");
    }
}

// RULE R12: gives one place back to a slot (after a cancel, or when a booking moves to another slot).
private async Task GivePlaceBackAsync(string slotId)
{
    var giveOne = Builders<EnergyBookingSlot>.Update.Inc(s => s.AvailableSlots, 1);
    await _db.EnergyBookingSlots.UpdateOneAsync(s => s.Id == slotId, giveOne);
}

// Copies the slot's times and the station's id/name into the booking (MongoDB has no JOINS,
// so bookings keep these copies for fast lists and the time rules).
private static void CopySlotDetails(EnergyReservation reservation, EnergyBookingSlot slot, SolarStation station)
{
    reservation.SlotId = slot.Id;
    reservation.StartTime = slot.StartTime;
    reservation.EndTime = slot.EndTime;
    reservation.StationId = station.Id;
}

```

```

        reservation.StationName = station.Name;
    }
}

```

api/SunShare.Api/Services/SlotService.cs

```

/*
 * =====
 * File      : SlotService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Business rules for booking slots (time windows at a
 *              station): list, bookable slots (R9, R11), create, edit and
 *              delete (R8).
 * =====
 */
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// All slot rules live here (FAT service).
public class SlotService
{
    // A booking can be made at most this many days ahead (R9).
    private const int MaxDaysAhead = 7;

    private readonly MongoDBContext _db;
    private readonly StationService _stations;

    // Receives the database and the station service (to look up a slot's station).
    public SlotService(MongoDbContext db, StationService stations)
    {
        _db = db;
        _stations = stations;
    }

    // Lists one station's slots that start between "from" and "to", soonest first.
    // Default: from the start of today (UTC) for 14 days.
    public async Task<List<SlotResponse>> ListForStationAsync(string stationId, DateTime? from, DateTime? to)
    {
        SolarStation station = await _stations.FindOrThrowAsync(stationId);
        DateTime start = from.HasValue ? TimeHelper.AsUtc(from.Value) : DateTime.UtcNow.Date;
        DateTime end = to.HasValue ? TimeHelper.AsUtc(to.Value) : start.AddDays(14);

        List<EnergyBookingSlot> slots = await _db.EnergyBookingSlots
            .Find(s => s.StationId == station.Id && s.StartTime >= start && s.StartTime < end)
            .SortBy(s => s.StartTime)
            .ToListAsync();
        return slots.Select(s => SlotResponse.FromSlot(s, station.Name)).ToList();
    }

    // Slots a prosumer can book right now (optionally only at one station), soonest first.
    // RULE R11: the slot is active, has at least one free place, and its station is active.
    // RULE R9: it starts in the future and no more than 7 days from now.
    public async Task<List<SlotResponse>> GetAvailableAsync(string? stationId)
    {
        var stationFilter = Builders<SolarStation>.Filter.Eq(s => s.IsActive, true);
        if (!string.IsNullOrEmpty(stationId))
        {
            SolarStation chosen = await _stations.FindOrThrowAsync(stationId);
            stationFilter &= Builders<SolarStation>.Filter.Eq(s => s.Id, chosen.Id);
        }
        List<SolarStation> activeStations = await _db.SolarStationInfo.Find(stationFilter).ToListAsync();
        Dictionary<string, string> stationNames = activeStations.ToDictionary(s => s.Id, s => s.Name);

        DateTime now = DateTime.UtcNow;
        DateTime latest = now.AddDays(MaxDaysAhead);
        var slotFilter = Builders<EnergyBookingSlot>.Filter.In(s => s.StationId, stationNames.Keys)
            & Builders<EnergyBookingSlot>.Filter.Where(s =>
                s.IsActive && s.AvailableSlots > 0 && s.StartTime > now && s.StartTime <= latest);

        List<EnergyBookingSlot> slots = await _db.EnergyBookingSlots.Find(slotFilter).SortBy(s => s.StartTime).ToListAsync();
        return slots.Select(s => SlotResponse.FromSlot(s, stationNames[s.StationId])).ToList();
    }
}

```

```

}

// Gets one slot by id, with its station's name, or 404.
public async Task<SlotResponse> GetAsync(string id)
{
    EnergyBookingSlot slot = await FindOrThrowAsync(id);
    SolarStation station = await _stations.FindOrThrowAsync(slot.StationId);
    return SlotResponse.FromSlot(slot, station.Name);
}

// Staff add a time window to a station. All its places start free (availableSlots = totalSlots).
public async Task<SlotResponse> CreateAsync(SlotRequest request)
{
    if (string.IsNullOrEmpty(request.StationId))
    {
        throw new ApiException(400, "Please choose a station.");
    }
    SolarStation station = await _stations.FindOrThrowAsync(request.StationId);
    // RULE R8: slots can only be added to an active station.
    if (!station.IsActive)
    {
        throw new ApiException(400, "This station is deactivated, so it can't get new slots.");
    }

    DateTime start = TimeHelper.AsUtc(request.StartTime);
    DateTime end = TimeHelper.AsUtc(request.EndTime);
    ValidateTimes(start, end);
    ValidateTotal(request.TotalSlots, station, 0);

    DateTime now = DateTime.UtcNow;
    var slot = new EnergyBookingSlot
    {
        Id = ObjectId.GenerateNewId().ToString(),
        StationId = station.Id,
        StartTime = start,
        EndTime = end,
        TotalSlots = request.TotalSlots,
        AvailableSlots = request.TotalSlots,
        IsActive = true,
        CreatedAt = now,
        UpdatedAt = now
    };
    await _db.EnergyBookingSlots.InsertOneAsync(slot);
    return SlotResponse.FromSlot(slot, station.Name);
}

// Staff edit a slot's times, number of places, or open/closed state.
// availableSlots is recalculated as the new total minus the places already booked.
public async Task<SlotResponse> UpdateAsync(string id, SlotUpdateRequest request)
{
    EnergyBookingSlot slot = await FindOrThrowAsync(id);
    SolarStation station = await _stations.FindOrThrowAsync(slot.StationId);
    int booked = slot.TotalSlots - slot.AvailableSlots;

    DateTime start = TimeHelper.AsUtc(request.StartTime);
    DateTime end = TimeHelper.AsUtc(request.EndTime);
    bool timeChanged = start != slot.StartTime || end != slot.EndTime;
    if (timeChanged)
    {
        // Booked prosumers chose this exact time, and their bookings keep a copy of it (used by R9/R10).
        if (booked > 0)
        {
            throw new ApiException(409, $"This slot already has {booked} booking(s), so its time can't be changed. Add a new slot instead.");
        }
        ValidateTimes(start, end);
    }
    ValidateTotal(request.TotalSlots, station, booked);

    slot.StartTime = start;
    slot.EndTime = end;
    slot.TotalSlots = request.TotalSlots;
    slot.AvailableSlots = request.TotalSlots - booked;
    slot.IsActive = request.IsActive;
    slot.UpdatedAt = DateTime.UtcNow;

    await _db.EnergyBookingSlots.ReplaceOneAsync(s => s.Id == slot.Id, slot);
    return SlotResponse.FromSlot(slot, station.Name);
}

// Staff delete a slot.
public async Task DeleteAsync(string id)
{

```

```

        EnergyBookingSlot slot = await FindOrThrowAsync(id);

        // RULE R8: a slot with active reservations (Pending/Approved, not started) can't be deleted.
        DateTime now = DateTime.UtcNow;
        long activeCount = await _db.EnergyReservations.CountDocumentsAsync(r =>
            r.SlotId == slot.Id
            && (r.Status == ReservationStatuses.Pending || r.Status == ReservationStatuses.Approved)
            && r.StartTime > now);
        if (activeCount > 0)
        {
            throw new ApiException(409, $"This slot has {activeCount} active reservation(s). Cancel them first, or close the
slot instead.");
        }

        await _db.EnergyBookingSlots.DeleteOneAsync(s => s.Id == slot.Id);
    }

    // Loads a slot by id, or throws 404 (also for ids that aren't valid MongoDB ObjectIds).
    public async Task<EnergyBookingSlot> FindOrThrowAsync(string id)
    {
        if (!ObjectId.TryParse(id, out _))
        {
            throw new ApiException(404, "Slot not found.");
        }
        EnergyBookingSlot? slot = await _db.EnergyBookingSlots.Find(s => s.Id == id).FirstOrDefaultAsync();
        if (slot == null)
        {
            throw new ApiException(404, "Slot not found.");
        }
        return slot;
    }

    // RULE R8: a slot must start in the future and end after it starts.
    private static void ValidateTimes(DateTime start, DateTime end)
    {
        if (start <= DateTime.UtcNow)
        {
            throw new ApiException(400, "The slot must start in the future.");
        }
        if (end <= start)
        {
            throw new ApiException(400, "The end time must be after the start time.");
        }
    }

    // RULE R8: places must be between 1 and the station's battery slots,
    // and can never be less than the places already booked.
    private static void ValidateTotal(int totalSlots, SolarStation station, int booked)
    {
        if (totalSlots < 1 || totalSlots > station.BatterySlots)
        {
            throw new ApiException(400, $"Total places must be between 1 and {station.BatterySlots} (the station's battery
slots).");
        }
        if (totalSlots < booked)
        {
            throw new ApiException(400, $"{{booked}} places are already booked, so the total can't be less than {{booked}}.");
        }
    }
}

```

api/SunShare.Api/Services/StationService.cs

```

/*
 * =====
 * File      : StationService.cs
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-28
 * Description : Business rules for solar stations: list, nearby search (R18),
 *              create, edit, activate, deactivate (R6) and delete (R7).
 * =====
 */
using System.Globalization;
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

```



```

namespace SunShare.Api.Services;

// All station rules live here (FAT service). SlotService also uses FindOrThrowAsync from here.
public class StationService
{
    private readonly MongoDBContext _db;

    // Receives the shared database context.
    public StationService(MongoDbContext db)
    {
        _db = db;
    }

    // Lists stations by name, each with its number of active reservations.
    // RULE R2: prosumers only ever see active stations; staff can ask for active ones only.
    public async Task<List<StationResponse>> ListAsync(bool activeOnly, string callerRole)
    {
        bool onlyActive = activeOnly || callerRole == Roles.Prosumer;
        var filter = onlyActive
            ? Builders<SolarStation>.Filter.Eq(s => s.IsActive, true)
            : Builders<SolarStation>.Filter.Empty;

        List<SolarStation> stations = await _db.SolarStationInfo.Find(filter).SortBy(s => s.Name).ToListAsync();
        List<string> bookedStationIds = await GetActiveBookingStationIdsAsync();
        return stations
            .Select(s => StationResponse.FromStation(s, bookedStationIds.Count(id => id == s.Id)))
            .ToList();
    }

    // RULE R18: active stations within radiusKm of the phone's location, nearest first,
    // with distanceKm filled in. The distance maths happens here, not on the phone.
    public async Task<List<StationResponse>> GetNearbyAsync(double? lat, double? lng, double radiusKm)
    {
        if (lat == null || lng == null)
        {
            throw new ApiException(400, "Please send your location (lat and lng).");
        }
        ValidateCoordinates(lat.Value, lng.Value);
        if (radiusKm <= 0)
        {
            throw new ApiException(400, "The search radius must be more than 0 km.");
        }

        List<SolarStation> stations = await _db.SolarStationInfo.Find(s => s.IsActive).ToListAsync();
        List<string> bookedStationIds = await GetActiveBookingStationIdsAsync();

        var nearby = new List<StationResponse>();
        foreach (SolarStation station in stations)
        {
            double distance = GeoHelper.DistanceKm(lat.Value, lng.Value, station.Latitude, station.Longitude);
            if (distance <= radiusKm)
            {
                long activeCount = bookedStationIds.Count(id => id == station.Id);
                nearby.Add(StationResponse.FromStation(station, activeCount, Math.Round(distance, 2)));
            }
        }
        return nearby.OrderBy(s => s.DistanceKm).ToList();
    }

    // Gets one station by id, or 404.
    public async Task<StationResponse> GetAsync(string id)
    {
        SolarStation station = await FindOrThrowAsync(id);
        return StationResponse.FromStation(station, await CountActiveReservationsAsync(station.Id));
    }

    // Backoffice registers a new station. It is active straight away.
    public async Task<StationResponse> CreateAsync(StationRequest request)
    {
        ValidateRequest(request);

        DateTime now = DateTime.UtcNow;
        var station = new SolarStation
        {
            Id = ObjectId.GenerateNewId().ToString(),
            IsActive = true,
            CreatedAt = now,
            UpdatedAt = now
        };
        ApplyRequest(station, request);

        await _db.SolarStationInfo.InsertOneAsync(station);
    }
}

```

```

    return StationResponse.FromStation(station, 0);
}

// Backoffice edits a station's details and opening hours. Bookings keep a copy of the
// station's name, so if the name changes those copies are updated too (search keeps working).
public async Task<StationResponse> UpdateAsync(string id, StationRequest request)
{
    SolarStation station = await FindOrThrowAsync(id);
    ValidateRequest(request);

    bool nameChanged = station.Name != request.Name.Trim();
    ApplyRequest(station, request);
    station.UpdatedAt = DateTime.UtcNow;
    await _db.SolarStationInfo.ReplaceOneAsync(s => s.Id == station.Id, station);

    if (nameChanged)
    {
        var renameCopies = Builders<EnergyReservation>.Update.Set(r => r.StationName, station.Name);
        await _db.EnergyReservations.UpdateManyAsync(r => r.StationId == station.Id, renameCopies);
    }
    return StationResponse.FromStation(station, await CountActiveReservationsAsync(station.Id));
}

// Backoffice switches a station off so it can't be booked or shown on the map.
public async Task<StationResponse> DeactivateAsync(string id)
{
    SolarStation station = await FindOrThrowAsync(id);
    if (!station.IsActive)
    {
        throw new ApiException(400, "This station is already deactivated.");
    }

    // RULE R6: a station with active reservations can't be deactivated (don't strand booked prosumers).
    long activeCount = await CountActiveReservationsAsync(station.Id);
    if (activeCount > 0)
    {
        string plural = activeCount == 1 ? "" : "s";
        throw new ApiException(409, $"This station has {activeCount} active reservation{plural}. Cancel or complete them
first.");
    }

    return await SetActiveAsync(station, false);
}

// Backoffice switches a deactivated station back on.
public async Task<StationResponse> ActivateAsync(string id)
{
    SolarStation station = await FindOrThrowAsync(id);
    if (station.IsActive)
    {
        throw new ApiException(400, "This station is already active.");
    }
    return await SetActiveAsync(station, true);
}

// Backoffice deletes a station together with its slots.
public async Task DeleteAsync(string id)
{
    SolarStation station = await FindOrThrowAsync(id);

    // RULE R7: only a station that never had any reservation can be deleted; otherwise deactivate it,
    // so no booking history points at a station that no longer exists.
    bool hasHistory = await _db.EnergyReservations.Find(r => r.StationId == station.Id).AnyAsync();
    if (hasHistory)
    {
        throw new ApiException(409, "Stations with booking history can't be deleted – deactivate it instead.");
    }

    await _db.EnergyBookingSlots.DeleteManyAsync(s => s.StationId == station.Id);
    await _db.SolarStationInfo.DeleteOneAsync(s => s.Id == station.Id);
}

// Loads a station by id, or throws 404. An id that isn't a valid MongoDB ObjectId
// (e.g. "abc") also gives 404 instead of crashing the query.
public async Task<SolarStation> FindOrThrowAsync(string id)
{
    if (!ObjectId.TryParse(id, out _))
    {
        throw new ApiException(404, "Station not found.");
    }
    SolarStation? station = await _db.SolarStationInfo.Find(s => s.Id == id).FirstOrDefaultAsync();
    if (station == null)

```

```

    {
        throw new ApiException(404, "Station not found.");
    }
    return station;
}

// Counts a station's active reservations: Pending or Approved and not started yet (the R6 definition).
public async Task<long> CountActiveReservationsAsync(string stationId)
{
    DateTime now = DateTime.UtcNow;
    return await _db.EnergyReservations.CountDocumentsAsync(r =>
        r.StationId == stationId
        && (r.Status == ReservationStatuses.Pending || r.Status == ReservationStatuses.Approved)
        && r.StartTime > now);
}

// The station ids of ALL active reservations in one query, so a whole list of stations
// can be counted without asking the database once per station.
private async Task<List<string>> GetActiveBookingStationIdsAsync()
{
    DateTime now = DateTime.UtcNow;
    return await _db.EnergyReservations
        .Find(r => (r.Status == ReservationStatuses.Pending || r.Status == ReservationStatuses.Approved)
            && r.StartTime > now)
        .Project(r => r.StationId)
        .ToListAsync();
}

// Saves the station's new on/off state.
private async Task<StationResponse> SetActiveAsync(SolarStation station, bool isActive)
{
    station.IsActive = isActive;
    station.UpdatedAt = DateTime.UtcNow;
    await _db.SolarStationInfo.ReplaceOneAsync(s => s.Id == station.Id, station);
    return StationResponse.FromStation(station, await CountActiveReservationsAsync(station.Id));
}

// Checks the station form: name and address filled in, GPS in range, capacity above 0,
// at least 1 battery place, and opening hours as "HH:mm" with opening before closing.
private static void ValidateRequest(StationRequest request)
{
    if (string.IsNullOrEmpty(request.Name))
    {
        throw new ApiException(400, "Please enter the station name.");
    }
    if (string.IsNullOrEmpty(request.Address))
    {
        throw new ApiException(400, "Please enter the station address.");
    }
    ValidateCoordinates(request.Latitude, request.Longitude);
    if (request.CapacityKw <= 0)
    {
        throw new ApiException(400, "Capacity must be more than 0 kW.");
    }
    if (request.BatterySlots < 1)
    {
        throw new ApiException(400, "A station needs at least 1 battery slot.");
    }
    if (!TryParseTime(request.OpenTime, out TimeOnly open) || !TryParseTime(request.CloseTime, out TimeOnly close))
    {
        throw new ApiException(400, "Opening and closing times must be in 24-hour HH:mm format, e.g. 06:00.");
    }
    if (open >= close)
    {
        throw new ApiException(400, "The opening time must be before the closing time.");
    }
}

// Latitude must be between -90 and 90 and longitude between -180 and 180 (the valid GPS ranges).
private static void ValidateCoordinates(double lat, double lng)
{
    if (lat < -90 || lat > 90 || lng < -180 || lng > 180)
    {
        throw new ApiException(400, "Latitude must be between -90 and 90, and longitude between -180 and 180.");
    }
}

// Reads "HH:mm" text such as "06:00" as a time of day. Returns false if the text is not in that format.
private static bool TryParseTime(string? text, out TimeOnly time)
{
    return TimeOnly.TryParseExact(text ?? "", "HH:mm", CultureInfo.InvariantCulture, DateTimeStyles.None, out time);
}

// Copies the form fields onto a station (text trimmed).

```

```

private static void ApplyRequest(SolarStation station, StationRequest request)
{
    station.Name = request.Name.Trim();
    station.Address = request.Address.Trim();
    station.Latitude = request.Latitude;
    station.Longitude = request.Longitude;
    station.CapacityKw = request.CapacityKw;
    station.BatterySlots = request.BatterySlots;
    station.OpenTime = request.OpenTime.Trim();
    station.CloseTime = request.CloseTime.Trim();
}
}

```

api/SunShare.Api/Services/UserService.cs

```

/*
 * =====
 * File       : UserService.cs
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : A - Accounts & Access
 * Author     : Gimhan T P K (IT22266996)
 * Created    : 2026-09-28
 * Description : Business rules for user accounts: list/search, create,
 *               edit, activate and deactivate (rules R1, R3, R4), plus a
 *               prosumer's own profile edit and self-deactivation (R5, R16).
 * =====
 */
using System.Text.RegularExpressions;
using MongoDB.Bson;
using MongoDB.Driver;
using SunShare.Api.Data;
using SunShare.Api.Dtos;
using SunShare.Api.Helpers;
using SunShare.Api.Models;

namespace SunShare.Api.Services;

// All account rules live here (FAT service). Controllers only pass the request in and the result out.
public class UserService
{
    private const int MinPasswordLength = 6;

    private readonly MongoDBContext _db;

    // Receives the shared database context.
    public UserService(MongoDbContext db)
    {
        _db = db;
    }

    // Lists users for the web tables. Optional filters: role, status, and a search text that is
    // matched against NIC, name or email (not case-sensitive). Sorted by name.
    public async Task<List<UserResponse>> ListAsync(string? role, string? status, string? search)
    {
        var filter = Builders<User>.Filter.Empty;
        if (!string.IsNullOrEmpty(role))
        {
            filter &= Builders<User>.Filter.Eq(u => u.Role, role);
        }
        if (!string.IsNullOrEmpty(status))
        {
            filter &= Builders<User>.Filter.Eq(u => u.Status, status);
        }
        if (!string.IsNullOrEmpty(search))
        {
            // Regex.Escape makes characters like "." or "*" in the search text count as plain text.
            var pattern = new BsonRegularExpression(Regex.Escape(search.Trim()), "i");
            filter &= Builders<User>.Filter.Or(
                Builders<User>.Filter.Regex(u => u.Nic, pattern),
                Builders<User>.Filter.Regex(u => u.FullName, pattern),
                Builders<User>.Filter.Regex(u => u.Email, pattern));
        }

        List<User> users = await _db.Users.Find(filter).SortBy(u => u.FullName).ToListAsync();
        return users.Select(UserResponse.FromUser).ToList();
    }

    // Prosumers waiting for Backoffice: new sign-ups (Pending, R3) and deactivated accounts
    // that need reactivation (R4). The one waiting longest comes first.
    public async Task<List<UserResponse>> GetPendingActivationsAsync()
    {

```

```

var filter = Builders<User>.Filter.Eq(u => u.Role, Roles.Prosumer)
    & Builders<User>.Filter.In(u => u.Status, [UserStatuses.Pending, UserStatuses.Deactivated]);

List<User> users = await _db.Users.Find(filter).SortBy(u => u.UpdatedAt).ToListAsync();
return users.Select(UserResponse.FromUser).ToList();
}

// Gets one user by NIC, or 404 if there is none.
public async Task<UserResponse> GetAsync(string nic)
{
    User user = await FindOrThrowAsync(nic);
    return UserResponse.FromUser(user);
}

// Backoffice creates an account of any role. Accounts made by staff are Active straight away.
public async Task<UserResponse> CreateAsync(CreateUserRequest request)
{
    // RULE R2: only the three known roles exist.
    if (request.Role != Roles.Backoffice && request.Role != Roles.GridOperator && request.Role != Roles.Prosumer)
    {
        throw new ApiException(400, "Role must be Backoffice, GridOperator or Prosumer.");
    }

    return await AddUserAsync(request.Nic, request.FullName, request.Email, request.Phone,
        request.Address, request.Role, UserStatuses.Active, request.Password);
}

// Checks and saves a brand-new account. Used by Backoffice "create user" (Active)
// and by mobile self-registration in AuthService (Pending, R3).
public async Task<UserResponse> AddUserAsync(string nic, string fullName, string email, string phone,
    string? address, string role, string status, string password)
{
    nic = NicValidator.Normalize(nic);
    // RULE R1: the NIC must be a valid Sri Lankan NIC.
    if (!NicValidator.IsValid(nic))
    {
        throw new ApiException(400, "Please enter a valid NIC: 9 digits followed by V or X (e.g. 981234567V), or 12
digits (e.g. 199812345678).");
    }
    ValidateDetails(fullName, email, phone);
    ValidatePassword(password);

    // RULE R1: the NIC is the primary key (_id), so two accounts can't share it.
    bool nicTaken = await _db.Users.Find(u => u.Nic == nic).AnyAsync();
    if (nicTaken)
    {
        throw new ApiException(409, "An account with this NIC already exists.");
    }

    DateTime now = DateTime.UtcNow;
    var user = new User
    {
        Nic = nic,
        Role = role,
        Status = status,
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(password),
        CreatedAt = now,
        UpdatedAt = now
    };
    ApplyDetails(user, fullName, email, phone, address);

    await _db.Users.InsertOneAsync(user);
    return UserResponse.FromUser(user);
}

// Backoffice edits an account's details. The NIC and the role never change.
public async Task<UserResponse> UpdateAsync(string nic, UpdateUserRequest request)
{
    User user = await FindOrThrowAsync(nic);
    ValidateDetails(request.FullName, request.Email, request.Phone);

    ApplyDetails(user, request.FullName, request.Email, request.Phone, request.Address);
    user.UpdatedAt = DateTime.UtcNow;

    await _db.Users.ReplaceOneAsync(u => u.Nic == user.Nic, user);
    return UserResponse.FromUser(user);
}

// Backoffice activates a Pending prosumer (R3) or reactivates a Deactivated account (R4).
public async Task<UserResponse> ActivateAsync(string nic)
{
    User user = await FindOrThrowAsync(nic);

```

```

        if (user.Status == UserStatuses.Active)
        {
            throw new ApiException(400, "This account is already active.");
        }

        // RULE R3 + R4: only Backoffice (checked by UsersController) can make an account Active.
        return await SetStatusAsync(user, UserStatuses.Active);
    }

    // Backoffice deactivates an account so it can no longer log in (R4).
    public async Task<UserResponse> DeactivateAsync(string nic, string callerNic)
    {
        User user = await FindOrThrowAsync(nic);
        // RULE R4: a Backoffice user can't deactivate their own account.
        if (user.Nic == callerNic)
        {
            throw new ApiException(400, "You can't deactivate your own account.");
        }
        if (user.Status == UserStatuses.Deactivated)
        {
            throw new ApiException(400, "This account is already deactivated.");
        }

        return await SetStatusAsync(user, UserStatuses.Deactivated);
    }

    // A prosumer edits their own details and can set a new password (at least 6 characters).
    // RULE R16: callerNic comes from the login token, so nobody can edit someone else's profile.
    public async Task<UserResponse> UpdateProfileAsync(string callerNic, UpdateProfileRequest request)
    {
        User user = await FindOrThrowAsync(callerNic);
        // RULE R4: a deactivated account can't be changed, even if an old token is still valid.
        if (user.Status == UserStatuses.Deactivated)
        {
            throw new ApiException(403, "Your account is deactivated. Please contact the Backoffice.");
        }
        ValidateDetails(request.FullName, request.Email, request.Phone);

        ApplyDetails(user, request.FullName, request.Email, request.Phone, request.Address);
        if (!string.IsNullOrEmpty(request.NewPassword))
        {
            ValidatePassword(request.NewPassword);
            user.PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.NewPassword);
        }
        user.UpdatedAt = DateTime.UtcNow;

        await _db.Users.ReplaceOneAsync(u => u.Nic == user.Nic, user);
        return UserResponse.FromUser(user);
    }

    // RULE R5: a prosumer deactivates their own account from the mobile app.
    // Afterwards they can't log in (R4) and they appear on Backoffice's Pending Activations list.
    public async Task DeactivateSelfAsync(string callerNic)
    {
        User user = await FindOrThrowAsync(callerNic);
        if (user.Status == UserStatuses.Deactivated)
        {
            throw new ApiException(400, "Your account is already deactivated.");
        }
        await SetStatusAsync(user, UserStatuses.Deactivated);
    }

    // Saves a new status for a user and returns the updated user.
    private async Task<UserResponse> SetStatusAsync(User user, string status)
    {
        user.Status = status;
        user.UpdatedAt = DateTime.UtcNow;
        await _db.Users.ReplaceOneAsync(u => u.Nic == user.Nic, user);
        return UserResponse.FromUser(user);
    }

    // Loads a user by NIC (cleaned first), or throws 404 if there is none.
    private async Task<User> FindOrThrowAsync(string nic)
    {
        string cleanNic = NicValidator.Normalize(nic);
        User? user = await _db.Users.Find(u => u.Nic == cleanNic).FirstOrDefaultAsync();
        if (user == null)
        {
            throw new ApiException(404, "User not found.");
        }
        return user;
    }

```

```
// Checks the details every account needs: a full name, an email with an "@" and a phone number.
private static void ValidateDetails(string fullName, string email, string phone)
{
    if (string.IsNullOrEmpty(fullName))
    {
        throw new ApiException(400, "Please enter the full name.");
    }
    if (string.IsNullOrEmpty(email) || !email.Contains('@'))
    {
        throw new ApiException(400, "Please enter a valid email address.");
    }
    if (string.IsNullOrEmpty(phone))
    {
        throw new ApiException(400, "Please enter a phone number.");
    }
}

// Passwords must be at least 6 characters long.
private static void ValidatePassword(string password)
{
    if (string.IsNullOrEmpty(password) || password.Length < MinPasswordLength)
    {
        throw new ApiException(400, $"The password must be at least {MinPasswordLength} characters long.");
    }
}

// Copies the editable details onto a user, trimmed. An empty address is saved as "no address".
private static void ApplyDetails(User user, string fullName, string email, string phone, string? address)
{
    user.FullName = fullName.Trim();
    user.Email = email.Trim();
    user.Phone = phone.Trim();
    user.Address = string.IsNullOrEmpty(address) ? null : address.Trim();
}
}
```

api/SunShare.Api/SunShare.Api.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="BCrypt.Net-Next" Version="4.2.0" />
    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.12" />
    <PackageReference Include="MongoDB.Driver" Version="3.12.0" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="10.2.3" />
  </ItemGroup>

</Project>
```

api/SunShare.Api/appsettings.Development.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

api/SunShare.Api/appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "MongoDb": {
    "ConnectionString": "mongodb://localhost:27017",
    "DatabaseName": "SunShareDb"
  },
}
```

```

"Jwt": {
  "Key": "9720ff0b74df180e43de831e63191b2e325d047a9c640fb0dd0a3f57d4dee4a4",
  "Issuer": "SunShare.Api",
  "Audience": "SunShare.Clients",
  "ExpiryHours": 8
},
"Cors": {
  "AllowedOrigins": [ "http://localhost:5173", "http://localhost:8081" ]
}
}

```

api/SunShare.Api/web.config

```

<?xml version="1.0" encoding="utf-8"?>
<!--
  File      : web.config
  Project   : SunShare - Smart Solar Microgrid Trading System
  Part      : B - Shared foundation (hosting)
  Author    : Ranathunga R A K N (IT22552860)
  Created   : 2026-09-28
  Description: Tells IIS how to run the API (through the ASP.NET Core Module
                from the Hosting Bundle) and removes the WebDAV handler, which
                would otherwise answer PUT/DELETE/PATCH with "405 Method Not Allowed".
  Reference : Microsoft Learn, "Host ASP.NET Core on Windows with IIS"
                https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/
  Note      : dotnet publish fills in processPath/arguments for the published app.
                IIS locks the <modules> section by default (a site may not change it:
                error 500.19 / 0x80070021), so only the WebDAV *handler* is removed here.
                The WebDAV feature is not installed on the demo laptop anyway.
-->
<configuration>
  <location path="." inheritInChildApplications="false">
    <system.webServer>
      <handlers>
        <remove name="WebDAV" />
        <add name="aspNetCore" path="*" verb="*" modules="AspNetCoreModuleV2" resourceType="Unspecified" />
      </handlers>
      <aspNetCore processPath="dotnet" arguments=".\SunShare.Api.dll" stdoutLogEnabled="false" stdoutLogFile=".\logs\stdout"
        hostingModel="inprocess" />
    </system.webServer>
  </location>
</configuration>

```

A.2 Web app (React + Bootstrap 5)

web/.env.development

Address of the SunShare API used by "npm run dev" (the API started with dotnet run).
 VITE_API_BASE_URL=http://localhost:5080

web/.env.production

Address of the SunShare API used by "npm run build" (the API hosted on IIS).
 VITE_API_BASE_URL=http://localhost:8080

web/index.html

```

<!doctype html>
<!-- File: index.html | Part B (shared) | The single HTML page; React draws every screen inside #root -->
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="SunShare - book solar microgrid slots, trade spare energy and manage stations." />
    <title>SunShare - Trade your sunshine</title>
    <!-- Inter font (docs/05-SCREENS.md). If there is no internet, the system font is used instead. -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```


web/package.json

```
{
  "name": "web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "bootstrap": "^5.3.8",
    "bootstrap-icons": "^1.13.1",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.4"
  },
  "devDependencies": {
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "oxlint": "^1.81.0",
    "vite": "^8.3.0"
  }
}
```

web/public/web.config

```
<?xml version="1.0" encoding="utf-8"?>
<!--
  File      : web.config
  Project   : SunShare - Smart Solar Microgrid Trading System
  Part      : B - Hosting (web app on IIS)
  Author    : Ranathunga R A K N (IT22552860)
  Created   : 2026-09-29
  Description : IIS settings for the built React app (site SunShareWeb, port
                8081). The app is plain static files: index.html is the start
                page, and the icon font type is declared so IIS serves it.
                HashRouter keeps page addresses after "#", so no URL rewrite
                rules are needed (decision D7).
  Note      : Vite copies this file from web/public into dist/ on every build.
-->
<configuration>
  <system.webServer>
    <defaultDocument>
      <files>
        <clear />
        <add value="index.html" />
      </files>
    </defaultDocument>
    <staticContent>
      <!-- remove first, then add: avoids a "duplicate" error on IIS versions that already know .woff2 -->
      <remove fileExtension=".woff2" />
      <mimeMap fileExtension=".woff2" mimeType="font/woff2" />
    </staticContent>
  </system.webServer>
</configuration>
```

web/src/App.jsx

```
/*
 * =====
 * File      : App.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-27
 * Description : Root component: every page address (route) of the web app
 *              and who may open it. Pages not built yet show a placeholder.
 * =====
 */
import { HashRouter, Route, Routes } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext.jsx';
import AppLayout from './components/AppLayout.jsx';
import ProtectedRoute from './components/ProtectedRoute.jsx';
import Dashboard from './pages/Dashboard.jsx';
import Home from './pages/Home.jsx';
import Login from './pages/Login.jsx';
import Prosumers from './pages/Prosumers.jsx';
import UserForm from './pages/UserForm.jsx';
```

```

import Users from './pages/Users.jsx';
import NotFound from './pages/NotFound.jsx';
import PendingActivations from './pages/PendingActivations.jsx';
import ReservationForm from './pages/ReservationForm.jsx';
import Reservations from './pages/Reservations.jsx';
import SlotForm from './pages/SlotForm.jsx';
import Slots from './pages/Slots.jsx';
import StationForm from './pages/StationForm.jsx';
import Stations from './pages/Stations.jsx';

const STAFF = ['Backoffice', 'GridOperator'];
const BACKOFFICE = ['Backoffice'];

// HashRouter keeps the page in the part of the address after "#" (e.g. /#/stations), so IIS
// always serves the same index.html and needs no URL-rewrite setup (docs/11-DECISIONS.md D7).
export default function App() {
  return (
    <HashRouter>
      <AuthProvider>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />

          { /* Staff pages (Backoffice + Grid Operator), inside the sidebar layout. */ }
          <Route element={<ProtectedRoute roles={STAFF} />>
            <Route element={<AppLayout />>
              <Route path="/dashboard" element={<Dashboard />} />
              <Route path="/reservations" element={<Reservations />} />
              <Route path="/reservations/new" element={<ReservationForm />} />
              <Route path="/reservations/:id/edit" element={<ReservationForm />} />
              <Route path="/stations" element={<Stations />} />
              <Route path="/stations/:id/slots" element={<Slots />} />
              <Route path="/slots/new" element={<SlotForm />} />
              <Route path="/slots/:id/edit" element={<SlotForm />} />

              { /* Backoffice-only pages. */ }
              <Route element={<ProtectedRoute roles={BACKOFFICE} />>
                <Route path="/stations/new" element={<StationForm />} />
                <Route path="/stations/:id/edit" element={<StationForm />} />
                <Route path="/users" element={<Users />} />
                <Route path="/users/new" element={<UserForm kind="staff" />} />
                <Route path="/users/:nic/edit" element={<UserForm kind="staff" />} />
                <Route path="/prosumers" element={<Prosumers />} />
                <Route path="/prosumers/new" element={<UserForm kind="prosumer" />} />
                <Route path="/prosumers/:nic/edit" element={<UserForm kind="prosumer" />} />
                <Route path="/activations" element={<PendingActivations />} />
              </Route>
            </Route>
          </Route>

          <Route path="*" element={<NotFound />} />
        </Routes>
      </AuthProvider>
    </HashRouter>
  );
}

```

web/src/api/authApi.js

```

/*
 * =====
 * File       : authApi.js
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : A - Accounts & Access
 * Author     : Gimhan T P K (IT22266996)
 * Created    : 2026-09-29
 * Description : Calls POST /api/auth/login for the web Login page.
 * =====
 */
import { api } from './client.js';

// Sends NIC + password; the API answers { token, nic, fullName, role, expiresAt } or an error message.
export function login(nic, password) {
  return api.post('/api/auth/login', { nic, password });
}

```

web/src/api/client.js

```

/*
 * =====

```

```

* File      : client.js
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (web shell)
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : The one place the web app talks to the API: builds the URL,
*               adds the login token, turns { "message" } errors into
*               JavaScript errors, and logs out on 401. Also saves/loads the
*               login session in localStorage.
* =====
*/

// API address: .env.development (npm run dev) or .env.production (npm run build).
const BASE_URL = import.meta.env.VITE_API_BASE_URL;

// Key under which the login session is kept in the browser's localStorage.
const SESSION_KEY = 'sunshare.session';

// Function to call when the API says 401 (token missing/expired). AuthContext sets it.
let unauthorizedHandler = null;

// An error from the API: keeps the HTTP status and the API's human-friendly message.
export class ApiError extends Error {
  // Creates the error with the HTTP status (0 = server not reachable) and the message to show.
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}

// Saves the login session ({ token, nic, fullName, role, expiresAt }) in localStorage.
export function saveSession(session) {
  localStorage.setItem(SESSION_KEY, JSON.stringify(session));
}

// Loads the saved session, or null if there is none or its token has expired.
export function loadSession() {
  const text = localStorage.getItem(SESSION_KEY);
  if (!text) {
    return null;
  }
  const session = JSON.parse(text);
  if (new Date(session.expiresAt) <= new Date()) {
    clearSession();
    return null;
  }
  return session;
}

// Removes the saved session (logout).
export function clearSession() {
  localStorage.removeItem(SESSION_KEY);
}

// Lets AuthContext say what to do when the API answers 401 (log out and go to the login page).
export function setUnauthorizedHandler(handler) {
  unauthorizedHandler = handler;
}

// Sends one request to the API and returns the JSON answer (null for "204 No Content").
// Throws an ApiError with the API's { "message" } when the answer is not OK.
async function request(method, path, body) {
  const session = loadSession();
  const headers = { Accept: 'application/json' };
  if (body !== undefined) {
    headers['Content-Type'] = 'application/json';
  }
  if (session) {
    headers.Authorization = `Bearer ${session.token}`;
  }

  let response;
  try {
    response = await fetch(BASE_URL + path, {
      method,
      headers,
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    throw new ApiError(0, 'Cannot reach the SunShare server. Please check that the API is running.');
```

```
// Our token was refused (expired or invalid): log out so the user can sign in again.
if (response.status === 401 && session && unauthorizedHandler) {
  unauthorizedHandler();
}
if (response.status === 204) {
  return null;
}

const data = await response.json().catch(() => null);
if (!response.ok) {
  const message = data && data.message ? data.message : `Something went wrong (HTTP ${response.status}).`;
  throw new ApiError(response.status, message);
}
return data;
}

// Short helpers used by the resource files (usersApi.js, stationsApi.js ...).
export const api = {
  get: (path) => request('GET', path),
  post: (path, body) => request('POST', path, body ?? {}),
  put: (path, body) => request('PUT', path, body ?? {}),
  patch: (path, body) => request('PATCH', path, body),
  delete: (path) => request('DELETE', path),
};
```

web/src/api/dashboardApi.js

```
/*
 * =====
 * File      : dashboardApi.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : Calls GET /api/dashboard/summary for the staff dashboard (W3).
 * =====
 */
import { api } from './client.js';

// The count cards + next pending bookings, all counted by the API (docs/01-SPEC.md §5).
export function getStaffSummary() {
  return api.get('/api/dashboard/summary');
}
```

web/src/api/healthApi.js

```
/*
 * =====
 * File      : healthApi.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Calls GET /api/health (public) - used by the Home page to
 *              show whether the server and database are online.
 * =====
 */
import { api } from './client.js';

// Asks the API if it is running and connected to MongoDB: { api, database, time }.
export function getHealth() {
  return api.get('/api/health');
}
```

web/src/api/reservationsApi.js

```
/*
 * =====
 * File      : reservationsApi.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR (lists) / C - Reservations (actions)
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : Calls the /api/reservations endpoints: the list with its
 *              filters, one booking, and approve (used by the dashboard).
 *              Create / update / cancel were added by Part C (Malkith G W L,
```

```

*          IT22630834) for the reservation form and the row actions.
* =====
*/
import { api } from './client.js';

// GET /api/reservations?view=&status=&stationId=&from=&to=&search= - empty filters are left out.
export function listReservations(filters = {}) {
  const params = new URLSearchParams();
  for (const [name, value] of Object.entries(filters)) {
    if (value) params.set(name, value);
  }
  const query = params.toString();
  return api.get(`/api/reservations${query ? `?${query}` : ''}`);
}

// GET /api/reservations/{id} - one booking (with canModify and qrData from the API).
export function getReservation(id) {
  return api.get(`/api/reservations/${id}`);
}

// PATCH /api/reservations/{id}/approve - staff approve a Pending booking; the API makes the QR token (R13).
export function approveReservation(id) {
  return api.patch(`/api/reservations/${id}/approve`);
}

// POST /api/reservations - staff book for a prosumer: { prosumerNic, slotId, energyKwh, type }.
// The API checks R9, R11, R12, R13, R17 and answers the new Pending booking.
export function createReservation(data) {
  return api.post('/api/reservations', data);
}

// PUT /api/reservations/{id} - { slotId, energyKwh, type }; an Approved booking goes back to Pending (R13).
export function updateReservation(id, data) {
  return api.put(`/api/reservations/${id}`, data);
}

// PATCH /api/reservations/{id}/cancel - at least 12 hours before the start (R10); frees the place (R12).
export function cancelReservation(id) {
  return api.patch(`/api/reservations/${id}/cancel`);
}

```

web/src/api/slotsApi.js

```

/*
* =====
* File      : slotsApi.js
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Nodes, Slots & Map
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : Calls the slot endpoints. Times are sent and received in UTC;
*              the rules (R8, R9, R11) are checked by the API.
* =====
*/
import { api } from './client.js';

// GET /api/stations/{stationId}/slots - a station's slots (the API's default: from today for 14 days).
export function listStationSlots(stationId) {
  return api.get(`/api/stations/${stationId}/slots`);
}

// GET /api/slots/available?stationId= - slots that can be booked now (used by the reservation form).
export function listAvailableSlots(stationId) {
  return api.get(`/api/slots/available${stationId ? `?stationId=${stationId}` : ''}`);
}

// GET /api/slots/{id} - one slot, with its station's name and booked places.
export function getSlot(id) {
  return api.get(`/api/slots/${id}`);
}

// POST /api/slots - { stationId, startTime, endTime, totalSlots } (times in UTC).
export function createSlot(data) {
  return api.post('/api/slots', data);
}

// PUT /api/slots/{id} - { startTime, endTime, totalSlots, isActive }.
export function updateSlot(id, data) {
  return api.put(`/api/slots/${id}`, data);
}

```

```

}

// DELETE /api/slots/{id} - refused (409) while it has active reservations (R8).
export function deleteSlot(id) {
  return api.delete(`/api/slots/${id}`);
}

```

web/src/api/stationsApi.js

```

/*
 * =====
 * File      : stationsApi.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Nodes, Slots & Map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Calls the /api/stations endpoints. One small function per
 *              endpoint; the rules (R6, R7) are checked by the API.
 * =====
 */
import { api } from './client.js';

// GET /api/stations?activeOnly= - all stations (or only active ones), each with activeReservationCount.
export function listStations(activeOnly = false) {
  return api.get(`/api/stations${activeOnly ? '?activeOnly=true' : ''}`);
}

// GET /api/stations/{id} - one station.
export function getStation(id) {
  return api.get(`/api/stations/${id}`);
}

// POST /api/stations - { name, address, latitude, longitude, capacityKw, batterySlots, openTime, closeTime }.
export function createStation(data) {
  return api.post('/api/stations', data);
}

// PUT /api/stations/{id} - same fields as create.
export function updateStation(id, data) {
  return api.put(`/api/stations/${id}`, data);
}

// PATCH /api/stations/{id}/deactivate - refused (409) while it has active reservations (R6).
export function deactivateStation(id) {
  return api.patch(`/api/stations/${id}/deactivate`);
}

// PATCH /api/stations/{id}/activate - switch a deactivated station back on.
export function activateStation(id) {
  return api.patch(`/api/stations/${id}/activate`);
}

// DELETE /api/stations/{id} - refused (409) if it has any booking history (R7).
export function deleteStation(id) {
  return api.delete(`/api/stations/${id}`);
}

```

web/src/api/usersApi.js

```

/*
 * =====
 * File      : usersApi.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-29
 * Description : Calls the /api/users endpoints (web users, prosumers,
 *              pending activations). One small function per endpoint.
 * =====
 */
import { api } from './client.js';

// Name of the browser event fired when an account's status changes, so the sidebar
// can refresh its "Pending activations" count badge.
export const ACTIVATIONS_CHANGED = 'sunshare:activations-changed';

// Tells the rest of the page (the sidebar badge) that activations may have changed.

```

```

export function notifyActivationsChanged() {
  window.dispatchEvent(new Event(ACTIVATIONS_CHANGED));
}

// GET /api/users?role=&status=&search= - any filter can be left empty.
export function listUsers({ role = '', status = '', search = '' } = {}) {
  const params = new URLSearchParams();
  if (role) params.set('role', role);
  if (status) params.set('status', status);
  if (search) params.set('search', search);
  const query = params.toString();
  return api.get(`/api/users${query ? `?` + query : ''}`);
}

// GET /api/users/pending-activations - prosumers that are Pending or Deactivated.
export function getPendingActivations() {
  return api.get('/api/users/pending-activations');
}

// GET /api/users/{nic} - one account.
export function getUser(nic) {
  return api.get(`/api/users/${encodeURIComponent(nic)}`);
}

// POST /api/users - Backoffice creates an account { nic, fullName, email, phone, address, role, password }.
export function createUser(data) {
  return api.post('/api/users', data);
}

// PUT /api/users/{nic} - edit { fullName, email, phone, address }.
export function updateUser(nic, data) {
  return api.put(`/api/users/${encodeURIComponent(nic)}`, data);
}

// PATCH /api/users/{nic}/activate - Pending or Deactivated -> Active.
export function activateUser(nic) {
  return api.patch(`/api/users/${encodeURIComponent(nic)}/activate`);
}

// PATCH /api/users/{nic}/deactivate - Active -> Deactivated.
export function deactivateUser(nic) {
  return api.patch(`/api/users/${encodeURIComponent(nic)}/deactivate`);
}

```

web/src/components/AlertMessage.jsx

```

/*
 * =====
 * File      : AlertMessage.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Bootstrap alert that shows the API's message: red for errors,
 *              green for success. Screen readers announce it automatically.
 * =====
 */

const ICONS = { danger: 'bi-exclamation-triangle-fill', success: 'bi-check-circle-fill', info: 'bi-info-circle-fill' };

// Example: <AlertMessage type="danger" message={error} onClose={() => setError('')} />. Shows nothing if message is empty.
export default function AlertMessage({ type = 'danger', message, onClose }) {
  if (!message) {
    return null;
  }
  return (
    <div
      className={`alert alert-${type} d-flex align-items-start gap-2`}
      role={type === 'danger' ? 'alert' : 'status'}
    >
      <i className={`bi ${ICONS[type]} mt-1`} aria-hidden="true"></i>
      <div className="flex-grow-1">{message}</div>
      {onClose && <button type="button" className="btn-close" aria-label="Close message" onClick={onClose}></button>}
    </div>
  );
}

```

web/src/components/AppLayout.jsx

```

/*
 * =====
 * File      : AppLayout.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : The frame around every staff page: sidebar menu + the page
 *              itself (<Outlet />). On phones the sidebar becomes a top bar
 *              with a menu button.
 * =====
 */
import { useState } from 'react';
import { Outlet } from 'react-router-dom';
import Sidebar from './Sidebar.jsx';
import Skiplink from './Skiplink.jsx';

// Staff page frame. menuOpen only matters on small screens (the menu button toggles it).
export default function AppLayout() {
  const [menuOpen, setMenuOpen] = useState(false);

  return (
    <div className="app-shell">
      <Skiplink />
      <Sidebar
        open={menuOpen}
        onToggle={() => setMenuOpen(!menuOpen)}
        onNavigate={() => setMenuOpen(false)}
      />
      <div className="app-content">
        { /* tabIndex -1 lets the skip link move focus here. */ }
        <main id="content" tabIndex=-1 className="container-fluid p-3 p-lg-4">
          <Outlet />
        </main>
      </div>
    </div>
  );
}

```

web/src/components/ConfirmButton.jsx

```

/*
 * =====
 * File      : ConfirmButton.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : A button that asks "Are you sure?" (the browser's
 *              window.confirm) before deactivate, delete or cancel.
 * =====
 */

// Example: <ConfirmButton message="Delete this slot?" onConfirm={deleteSlot} className="btn btn-sm btn-outline-danger">Delete</ConfirmButton>
export default function ConfirmButton({ message, onConfirm, className = 'btn btn-outline-danger', disabled = false, children, ...rest }) {
  // Runs onConfirm only if the user clicks OK in the confirmation box.
  function handleClick() {
    if (window.confirm(message)) {
      onConfirm();
    }
  }

  return (
    <button type="button" className={className} disabled={disabled} onClick={handleClick} {...rest}>
      {children}
    </button>
  );
}

```

web/src/components/EmptyState.jsx

```

/*
 * =====
 * File      : EmptyState.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1

```



```

* Part      : B - Shared foundation (web shell)
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : Friendly message shown when a list has nothing to show,
*             with an optional button (e.g. "Add station").
* =====
*/

// Example: <EmptyState icon="bi-geo-alt" title="No stations yet" text="Add the first one." />
export default function EmptyState({ icon = 'bi-inbox', title, text, children }) {
  return (
    <div className="text-center py-5 px-3">
      <i className={`bi ${icon} display-5 text-secondary`} aria-hidden="true"></i>
      <h2 className="h5 mt-3">{title}</h2>
      {text} && <p className="text-secondary mb-3">{text}</p>
      {children}
    </div>
  );
}

```

web/src/components>LoadingSpinner.jsx

```

/*
* =====
* File      : LoadingSpinner.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (web shell)
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : Spinning circle shown while waiting for the API.
* =====
*/

// Example: {loading} && <LoadingSpinner text="Loading stations..." />
export default function LoadingSpinner({ text = 'Loading...' }) {
  return (
    <div className="d-flex align-items-center gap-2 text-secondary py-4" role="status">
      <span className="spinner-border spinner-border-sm text-primary" aria-hidden="true"></span>
      <span>{text}</span>
    </div>
  );
}

```

web/src/components/PageHeader.jsx

```

/*
* =====
* File      : PageHeader.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (web shell)
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : Page title + short subtitle + optional buttons on the right,
*             so every staff page starts the same way.
* =====
*/

// Example: <PageHeader title="Stations" subtitle="Solar hubs and their schedules"><button ...>Add</button></PageHeader>
export default function PageHeader({ title, subtitle, children }) {
  return (
    <div className="d-flex flex-wrap align-items-start justify-content-between gap-3 mb-4">
      <div>
        <h1 className="h3 mb-1">{title}</h1>
        {subtitle} && <p className="page-subtitle mb-0">{subtitle}</p>
      </div>
      {children} && <div className="d-flex flex-wrap gap-2">{children}</div>
    </div>
  );
}

```

web/src/components/ProtectedRoute.jsx

```

/*
* =====
* File      : ProtectedRoute.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (web shell)
* Author    : Ranathunga R A K N (IT22552860)

```

```

* Created      : 2026-09-29
* Description  : Guards pages: not logged in -> Login page; wrong role ->
*               "no access" message. Only for a nicer UI - the API still
*               checks the role on every call (rule R2).
* =====
*/
import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext.jsx';
import AlertMessage from './AlertMessage.jsx';

// Shows the pages inside it (<Outlet />) only to logged-in users whose role is in "roles".
export default function ProtectedRoute({ roles }) {
  const { user } = useAuth();
  const location = useLocation();

  if (!user) {
    return <Navigate to="/login" replace state={{ from: location.pathname }} />;
  }
  if (roles && !roles.includes(user.role)) {
    return <AlertMessage message="You don't have access to this page." />;
  }
  return <Outlet />;
}

```

web/src/components/ReservationTable.jsx

```

/*
* =====
* File       : ReservationTable.jsx
* Project    : SunShare - Smart Solar Microgrid Trading System
* Module     : SE4040 Enterprise Application Development - Assignment 1
* Part      : D - Dashboards & QR
* Author     : Chamara R M L K (IT22076816)
* Created    : 2026-09-29
* Description : Table of bookings shared by the Dashboard (W3) and the
*               Reservations page (W13). Pages can add their own buttons per
*               row with renderActions (e.g. Approve, Edit, Cancel).
* =====
*/
import StatusBadge from './StatusBadge.jsx';
import { formatDate, formatTime } from '../utils/format.js';

// reservations = rows from the API; renderActions(reservation) = optional buttons for the last column.
export default function ReservationTable({ caption, reservations, renderActions }) {
  return (
    <div className="table-responsive">
      <table className="table table-hover align-middle mb-0">
        <caption className="visually-hidden">{caption}</caption>
        <thead>
          <tr>
            <th scope="col">When</th>
            <th scope="col">Station</th>
            <th scope="col">Prosumer</th>
            <th scope="col">Energy</th>
            <th scope="col">Status</th>
            {renderActions && <th scope="col" className="text-end">Actions</th>}
          </tr>
        </thead>
        <tbody>
          {reservations.map((reservation) => (
            <tr key={reservation.id}>
              <td className="text-nowrap">
                <div className="fw-semibold">{formatDate(reservation.startTime)}</div>
                <div className="small text-secondary">{formatTime(reservation.startTime)} -
                {formatTime(reservation.endTime)}</div>
              </td>
              <td>
                <div>{reservation.stationName}</div>
                <div className="small text-secondary font-monospace" title="Reservation id">{reservation.id}</div>
              </td>
              <td>
                <div>{reservation.prosumerName}</div>
                <div className="small text-secondary">{reservation.prosumerNic}</div>
              </td>
              <td className="text-nowrap">
                <i
                  className={`bi ${reservation.type === 'Sell' ? 'bi-box-arrow-up text-success' : 'bi-box-arrow-in-down text-primary'} me-1`}
                  aria-hidden="true"
                ></i>
                {reservation.energyKwh} kWh
                <div className="small text-secondary">{reservation.type === 'Sell' ? 'Sell (drop off)' : 'Buy
                (charge)'}</div>
              </td>
              <td><StatusBadge status={reservation.status} /></td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```

```

        {renderActions && <td className="text-end text-nowrap">{renderActions(reservation)}</td>}
      </tr>
    </tbody>
  </table>
</div>
);
}

```

web/src/components/Sidebar.jsx

```

/*
 * =====
 * File       : Sidebar.jsx
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Shared foundation (web shell)
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-29
 * Description : Dark navy side menu. Shows only the links for the logged-in
 *               user's role (Backoffice sees everything, Grid Operators see
 *               the operational pages), plus the user's name and Log out.
 *               Backoffice also sees how many prosumers wait for activation
 *               (count badge added with the Pending activations page, Part A).
 * =====
 */
import { useEffect, useState } from 'react';
import { Link, NavLink } from 'react-router-dom';
import { ACTIVATIONS_CHANGED, getPendingActivations } from '../api/usersApi.js';
import { useAuth } from '../context/AuthContext.jsx';

const BO = 'Backoffice';
const GO = 'GridOperator';

// Menu items and which roles see them (docs/01-SPEC.md §2 permission matrix).
const MENU = [
  { to: '/dashboard', label: 'Dashboard', icon: 'bi-speedometer2', roles: [BO, GO] },
  { to: '/reservations', label: 'Reservations', icon: 'bi-calendar-check', roles: [BO, GO] },
  { to: '/stations', label: 'Stations & slots', icon: 'bi-geo-alt', roles: [BO, GO] },
  { to: '/prosumers', label: 'Prosumers', icon: 'bi-house-heart', roles: [BO] },
  { to: '/activations', label: 'Pending activations', icon: 'bi-person-check', roles: [BO] },
  { to: '/users', label: 'Web users', icon: 'bi-person-badge', roles: [BO] },
];

// Friendly names for the roles.
const ROLE_NAMES = { Backoffice: 'Backoffice officer', GridOperator: 'Grid operator', Prosumer: 'Prosumer' };

// The menu. "open" shows the links on small screens; onToggle = menu button; onNavigate = a link was clicked.
export default function Sidebar({ open, onToggle, onNavigate }) {
  const { user, logout } = useAuth();
  const links = MENU.filter((item) => item.roles.includes(user.role));
  const [pendingCount, setPendingCount] = useState(0);

  // Backoffice only: load how many prosumers are waiting for activation, and load it again
  // whenever a page says an account's status changed (the ACTIVATIONS_CHANGED browser event).
  useEffect(() => {
    if (user.role !== BO) {
      return undefined;
    }
    let cancelled = false;

    // Asks the API for the waiting list and keeps only its length. If it fails, the badge just stays as it was.
    function refreshCount() {
      getPendingActivations()
        .then((list) => {
          if (!cancelled) setPendingCount(list.length);
        })
        .catch(() => {});
    }

    refreshCount();
    window.addEventListener(ACTIVATIONS_CHANGED, refreshCount);
    return () => {
      cancelled = true;
      window.removeEventListener(ACTIVATIONS_CHANGED, refreshCount);
    };
  }, [user.role]);

  return (
    <aside className={`app-sidebar ${open ? 'open' : ''}`>
      <div className="d-flex align-items-center justify-content-between p-3">
        <Link to="/dashboard" className="brand" onClick={onNavigate}>

```

```

        <i className="bi bi-sun-fill me-2" aria-hidden="true"></i>SunShare
      </Link>
      <button
        type="button"
        className="btn btn-sm btn-outline-light d-lg-none"
        aria-expanded={open}
        aria-controls="sidebar-menu"
        onClick={onToggle}
      >
        <i className={`bi ${open ? 'bi-x-lg' : 'bi-list'}`} aria-hidden="true"></i>
        <span className="visually-hidden">Menu</span>
      </button>
    </div>

    <div id="sidebar-menu" className="sidebar-body">
      <nav aria-label="Main" className="px-3">
        <ul className="nav flex-column gap-1">
          {links.map((item) => (
            <li key={item.to} className="nav-item">
              <NavLink to={item.to} className="nav-link" onClick={onNavigate}>
                <i className={`bi ${item.icon}`} aria-hidden="true"></i>
                {item.label}
                {item.to === '/activations' && pendingCount > 0 && (
                  <span className="badge rounded-pill status-pending ms-auto">
                    {pendingCount}<span className="visually-hidden"> waiting</span>
                  </span>
                )}
              </NavLink>
            </li>
          ))}
        </ul>
      </nav>

      <div className="user-box mt-auto p-3">
        <div className="text-white fw-semibold">{user.fullName}</div>
        <div>{ROLE_NAMES[user.role] ?? user.role}</div>
        <button type="button" className="btn btn-sm btn-outline-light w-100 mt-2" onClick={logout}>
          <i className="bi bi-box-arrow-right me-2" aria-hidden="true"></i>Log out
        </button>
      </div>
    </div>
  </aside>
);
}

```

web/src/components/SkipLink.jsx

```

/*
 * =====
 * File      : SkipLink.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : "Skip to main content" link for keyboard users. It is hidden
 *              until it gets keyboard focus (first Tab on the page).
 * =====
 */

// HashRouter uses the "#" part of the address for pages, so a normal href="#content" would change
// the page. Instead the click moves keyboard focus to <main id="content"> by code.
export default function SkipLink() {
  // Moves keyboard focus to the main content.
  function handleClick(event) {
    event.preventDefault();
    const main = document.getElementById('content');
    if (main) {
      main.focus();
    }
  }

  return (
    <a href="#content" className="skip-link visually-hidden-focusable" onClick={handleClick}>
      Skip to main content
    </a>
  );
}

```

web/src/components/StatusBadge.jsx

```

/*

```

```

* =====
* File      : StatusBadge.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (web shell)
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : Coloured pill for a status word: Pending (amber), Approved
*             (teal), Completed/Active (green), Cancelled (grey),
*             Deactivated (red). Colours are in styles/theme.css.
* =====
*/

// Example: <StatusBadge status="Approved" /> - the status text comes straight from the API.
export default function StatusBadge({ status }) {
  const cssName = `status-${String(status).toLowerCase()}`;
  return <span className={`badge rounded-pill status-badge ${cssName}`}>{status}</span>;
}

```

web/src/components/UserTable.jsx

```

/*
* =====
* File      : UserTable.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : A - Accounts & Access
* Author    : Gimhan T P K (IT22266996)
* Created   : 2026-09-29
* Description : Table of accounts shared by the Web users (W4) and
*             Prosumers (W6) pages, with Edit and Activate / Deactivate.
* =====
*/
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { activateUser, deactivateUser, notifyActivationsChanged } from '../api/usersApi.js';
import ConfirmButton from './ConfirmButton.jsx';
import StatusBadge from './StatusBadge.jsx';

const ROLE_NAMES = { Backoffice: 'Backoffice', GridOperator: 'Grid Operator', Prosumer: 'Prosumer' };

// users = rows from the API; showRole / showAddress = extra columns; editBase = "/users" or "/prosumers";
// onChanged(message) / onError(message) tell the page what happened.
export default function UserTable({ caption, users, showRole = false, showAddress = false, editBase, onChanged, onError }) {
  const [busyNic, setBusyNic] = useState('');

  // Activates or deactivates one account. The API decides if it's allowed (e.g. R4: not your own account)
  // and this table just shows its answer.
  async function changeStatus(user, activate) {
    setBusyNic(user.nic);
    try {
      const updated = activate ? await activateUser(user.nic) : await deactivateUser(user.nic);
      notifyActivationsChanged();
      onChanged(activate ? `${updated.fullName} is now active.` : `${updated.fullName} was deactivated.`);
    } catch (err) {
      onError(err.message);
    } finally {
      setBusyNic('');
    }
  }

  return (
    <div className="table-responsive">
      <table className="table table-hover align-middle mb-0">
        <caption className="visually-hidden">{caption}</caption>
        <thead>
          <tr>
            <th scope="col">NIC</th>
            <th scope="col">Name</th>
            <th scope="col">Phone</th>
            {showRole && <th scope="col">Role</th>}
            {showAddress && <th scope="col">Address</th>}
            <th scope="col">Status</th>
            <th scope="col" className="text-end">Actions</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.nic}>
              <td className="font-monospace small">{user.nic}</td>
              <td>
                <div className="fw-semibold">{user.fullName}</div>
                <div className="small text-secondary">{user.email}</div>
              </td>

```

```

        <td>{user.phone}</td>
        {showRole && <td>{ROLE_NAMES[user.role] ?? user.role}</td>}
        {showAddress && <td className="small">{user.address || <span className="text-secondary">-</span></td>}
        <td><StatusBadge status={user.status} /></td>
        <td className="text-end text-nowrap">
          <Link to={` ${editBase}/${encodeURIComponent(user.nic)}/edit`} className="btn btn-sm btn-outline-primary me-
1">
            <i className="bi bi-pencil me-1" aria-hidden="true"></i>Edit<span className="visually-hidden">
{user.fullName}</span>
          </Link>
          {user.status === 'Active' ? (
            <ConfirmButton
              message={`Deactivate ${user.fullName}? They won't be able to log in.`}
              onConfirm={() => changeStatus(user, false)}
              className="btn btn-sm btn-outline-danger"
              disabled={busyNic === user.nic}
            >
              Deactivate<span className="visually-hidden"> {user.fullName}</span>
            </ConfirmButton>
          ) : (
            <button
              type="button"
              className="btn btn-sm btn-outline-success"
              disabled={busyNic === user.nic}
              onClick={() => changeStatus(user, true)}
            >
              {user.status === 'Pending' ? 'Activate' : 'Reactivate'}<span className="visually-hidden">
{user.fullName}</span>
            </button>
          )}
        </td>
      </tr>
    </tbody>
  </table>
</div>
);
}

```

web/src/context/AuthContext.jsx

```

/*
 * =====
 * File      : AuthContext.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Keeps "who is logged in" for the whole web app (React
 *               context), saved in localStorage so a page refresh keeps you
 *               logged in. Any component can read it with useAuth().
 * =====
 */
import { createContext, startTransition, useCallback, useContext, useEffect, useMemo, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { clearSession, loadSession, saveSession, setUnauthorizedHandler } from '../api/client.js';

const AuthContext = createContext(null);

// Wraps the app and shares the logged-in user ({ token, nic, fullName, role, expiresAt } or null).
export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => loadSession());
  const navigate = useNavigate();

  // Called after a successful POST /api/auth/login: remember the session.
  const login = useCallback((session) => {
    saveSession(session);
    setUser(session);
  }, []);

  // Forget the session and go back to the login page. React Router changes pages inside a React
  // "transition", so we forget the user in the same transition: both happen in one step. Otherwise
  // the protected page would redirect first and remember itself as the page to come back to.
  const logout = useCallback(() => {
    clearSession();
    startTransition(() => {
      setUser(null);
      navigate('/login');
    });
  }, [navigate]);

  // If the API answers 401 later (token expired), log out automatically.
  useEffect(() => {

```

```

    setUnauthorizedHandler(logout);
    return () => setUnauthorizedHandler(null);
  }, [logout]);

const value = useMemo(() => ({ user, login, logout }), [user, login, logout]);
return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// Hook for components: const { user, login, logout } = useAuth();
export function useAuth() {
  return useContext(AuthContext);
}

```

web/src/main.jsx

```

/*
 * =====
 * File      : main.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-27
 * Description : Entry point of the React web app. Loads the Bootstrap 5 and
 *               Bootstrap Icons stylesheets plus our SunShare theme on top,
 *               then draws <App /> on the page.
 * =====
 */
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap-icons/font/bootstrap-icons.css'
import './styles/theme.css'
import App from './App.jsx'

// Start-up: find <div id="root"> in index.html and draw the App inside it.
// StrictMode only adds extra warnings while developing; it does nothing in the built site.
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

web/src/pages/Dashboard.jsx

```

/*
 * =====
 * File      : Dashboard.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : W3 Dashboard - count cards (pending, approved upcoming,
 *               today, active stations, pending activations) and the next
 *               pending bookings with an Approve button. All numbers come
 *               from GET /api/dashboard/summary.
 * =====
 */
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { getStaffSummary } from '../api/dashboardApi.js';
import { approveReservation } from '../api/reservationsApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import ReservationTable from '../components/ReservationTable.jsx';
import { useAuth } from '../context/AuthContext.jsx';

// The count cards: which number from the API, its label, icon, colour and where clicking it goes.
const CARDS = [
  { key: 'pendingReservations', label: 'Pending reservations', icon: 'bi-hourglass-split', tone: 'amber', link:
    '/reservations?view=pending' },
  { key: 'approvedFutureReservations', label: 'Approved upcoming', icon: 'bi-check2-circle', tone: 'teal', link:
    '/reservations?view=current' },
  { key: 'todayReservations', label: "Today's reservations", icon: 'bi-calendar-day', tone: 'blue', link: '/reservations' },
  { key: 'activeStations', label: 'Active stations', icon: 'bi-geo-alt', tone: 'green', link: '/stations' },
  { key: 'pendingActivations', label: 'Pending activations', icon: 'bi-person-check', tone: 'navy', link: '/activations',
    backofficeOnly: true },
];

```

```

// "Good morning" / "Good afternoon" / "Good evening" from the local hour (display only).
function greeting() {
  const hour = new Date().getHours();
  if (hour < 12) return 'Good morning';
  if (hour < 17) return 'Good afternoon';
  return 'Good evening';
}

// The staff home page (Backoffice and Grid Operator).
export default function Dashboard() {
  const { user } = useAuth();
  const [summary, setSummary] = useState(null);
  const [reloadKey, setReloadKey] = useState(0);
  const [busyId, setBusyId] = useState('');
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');
  const cards = CARDS.filter((card) => !card.backofficeOnly || user.role === 'Backoffice');

  // Loads the summary (again after every approval). "cancelled" ignores late answers.
  useEffect(() => {
    let cancelled = false;
    getStaffSummary()
      .then((data) => {
        if (!cancelled) setSummary(data);
      })
      .catch((err) => {
        if (!cancelled) setError(err.message);
      });
    return () => {
      cancelled = true;
    };
  }, [reloadKey]);

  // Approves one pending booking; the API sets it Approved and creates its QR token (R13).
  async function handleApprove(reservation) {
    setBusyId(reservation.id);
    setError('');
    try {
      await approveReservation(reservation.id);
      setSuccess('Approved ${reservation.prosumerName}'s booking at ${reservation.stationName}. Their QR code is ready in the app.`);
      setReloadKey((key) => key + 1);
    } catch (err) {
      setSuccess('');
      setError(err.message);
    } finally {
      setBusyId('');
    }
  }

  return (
    <>
      <PageHeader
        title={` ${greeting()}, ${user.fullName.split(' ')[0]} `}
        subtitle="Here is what's happening across the SunShare microgrid."
      >
        <Link to="/reservations" className="btn btn-outline-primary">
          <i className="bi bi-calendar-check me-2" aria-hidden="true"></i>All reservations
        </Link>
      </PageHeader>

      <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
      <AlertMessage message={error} onClose={() => setError('')} />

      {!summary && !error && <LoadingSpinner text="Loading dashboard..." />}

      {summary && (
        <>
          <div className="row g-3 mb-4">
            {cards.map((card) => (
              <div key={card.key} className="col-sm-6 col-xl">
                <div className="card stat-card h-100 p-3 position-relative">
                  <div className="d-flex align-items-center gap-3">
                    <span className={ `stat-icon tone-${card.tone}` }><i className={ `bi ${card.icon}` }` } aria-
hidden="true"></i></span>
                  <div>
                    <div className="stat-number">{summary[card.key]}</div>
                    <Link to={card.link} className="stretched-link text-decoration-none text-secondary
small">{card.label}</Link>
                  </div>
                </div>
              </div>
            ))}
          </div>
        </>
      )}
    </>
  );
}

```



```

</div>

<div className="card">
  <div className="d-flex justify-content-between align-items-center p-3 border-bottom">
    <h2 className="h5 mb-0">Waiting for approval</h2>
    <Link to="/reservations?view=pending" className="small">See all pending</Link>
  </div>
  {summary.pendingList.length === 0 ? (
    <EmptyState icon="bi-check2-all" title="Nothing to approve" text="New bookings will appear here." />
  ) : (
    <ReservationTable
      caption="Next pending reservations"
      reservations={summary.pendingList}
      renderActions={(reservation) => (
        <button
          type="button"
          className="btn btn-sm btn-primary"
          disabled={busyId === reservation.id}
          onClick={() => handleApprove(reservation)}
        >
          <i className="bi bi-check-lg me-1" aria-hidden="true"></i>Approve
          <span className="visually-hidden"> {reservation.prosumerName} at {reservation.stationName}</span>
        </button>
      )}
    </div>
  )}
</div>
</>
</>
);
}

```

web/src/pages/Home.jsx

```

/*
 * =====
 * File       : Home.jsx
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Shared foundation (web shell)
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-29
 * Description : W1 Home (index) page - the public landing page: hero,
 *               "how it works" in 3 steps, the three roles, a call to action
 *               and a footer with the team and a live server status.
 * =====
 */
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { getHealth } from '../api/healthApi.js';
import StatusBadge from '../components/StatusBadge.jsx';
import SkipLink from '../components/SkipLink.jsx';
import { useAuth } from '../context/AuthContext.jsx';

const STEPS = [
  {
    icon: 'bi-person-plus',
    title: 'Register',
    text: 'Prosumers sign up in the SunShare Android app with their NIC. A Backoffice officer checks and activates the account.',
  },
  {
    icon: 'bi-calendar-check',
    title: 'Reserve',
    text: 'Pick a nearby station on the map and a time slot up to 7 days ahead - to drop off spare energy or to charge up.',
  },
  {
    icon: 'bi-qr-code-scan',
    title: 'Scan',
    text: 'Once staff approve the booking, the app shows a QR code. At the station a Grid Operator scans it to finish the transfer.',
  },
];

const ROLES = [
  {
    icon: 'bi-building-gear',
    title: 'Backoffice',
    where: 'Web',
    text: 'Manages staff accounts and prosumers, activates new sign-ups, registers solar stations and their schedules, and approves bookings.',
  },
  {
    icon: 'bi-lightning-charge',

```

```

    title: 'Grid Operator',
    where: 'Web + mobile',
    text: 'Opens time slots, books and approves on behalf of prosumers, and scans QR codes at the station to complete energy
transfers.',
  },
  {
    icon: 'bi-house-heart',
    title: 'Prosumer',
    where: 'Mobile',
    text: 'Finds the nearest stations on a map, reserves and manages bookings, and shows a QR code to check in at the
station.',
  },
];

const TEAM = [
  { name: 'Ranathunga R A K N', it: 'IT22552860' },
  { name: 'Gimhan T P K', it: 'IT22266996' },
  { name: 'Malkith G W L', it: 'IT22630834' },
  { name: 'Chamara R M L K', it: 'IT22076816' },
];

// Scrolls to a section of this page. (HashRouter uses "#" for page addresses, so we can't use #anchors.)
function scrollToSection(id) {
  const reduceMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
  const section = document.getElementById(id);
  if (section) {
    section.scrollIntoView({ behavior: reduceMotion ? 'auto' : 'smooth' });
  }
}

// The landing page. Logged-in staff see "Open dashboard" instead of "Staff login".
export default function Home() {
  const { user } = useAuth();
  const [serverStatus, setServerStatus] = useState('checking');
  const loginLink = user ? '/dashboard' : '/login';
  const loginText = user ? 'Open dashboard' : 'Staff login';

  // Once, when the page opens: ask GET /api/health whether the server and database are online.
  useEffect(() => {
    getHealth()
      .then((health) => setServerStatus(health.database === 'connected' ? 'online' : 'offline'))
      .catch(() => setServerStatus('offline'));
  }, []);

  return (
    <>
      <Skiplink />
      <header className="home-hero">
        <nav className="container d-flex align-items-center justify-content-between py-3" aria-label="Site">
          <span className="fs-4 fw-bold text-white">
            <i className="bi bi-sun-fill text-warning me-2" aria-hidden="true"></i>SunShare
          </span>
          <div className="d-flex align-items-center gap-2">
            <button type="button" className="btn btn-link text-white text-decoration-none d-none d-sm-inline" onClick={() =>
scrollToSection('how')}>
              How it works
            </button>
            <Link to={loginLink} className="btn btn-accent">{loginText}</Link>
          </div>
        </nav>

        <div className="container py-5">
          <div className="row align-items-center g-5 py-lg-4">
            <div className="col-lg-6">
              <span className="badge rounded-pill text-bg-light mb-3 px-3 py-2">
                <i className="bi bi-lightning-charge-fill text-warning me-1" aria-hidden="true"></i>
                Smart solar microgrid trading
              </span>
              <h1 className="mb-3">Trade your sunshine.</h1>
              <p className="lead mb-4">
                SunShare connects homes with rooftop solar panels to neighbourhood microgrid stations.
                Book a time slot, drop off your spare energy or charge up - and check in with a QR code.
              </p>
              <div className="d-flex flex-wrap gap-2">
                <Link to={loginLink} className="btn btn-accent btn-lg">{loginText}</Link>
                <button type="button" className="btn btn-outline-light btn-lg" onClick={() => scrollToSection('how')}>
                  How it works
                </button>
              </div>
            </div>
            <p className="small mt-4 mb-0">
              <i className="bi bi-phone me-1" aria-hidden="true"></i>
              Prosumers use the SunShare Android app.
            </p>
          </div>
        </div>
      </>
    </>
  );
}

```

```

    <div className="col-lg-6">
      <div className="position-relative mx-auto" style={{ maxWidth: '26rem' }}>
        <div className="hero-sun position-absolute top-0 end-0" style={{ transform: 'translate(25%, -35%)' }} aria-hidden="true"></div>
        <div className="card hero-card p-4 position-relative">
          <div className="d-flex justify-content-between align-items-start gap-2">
            <div>
              <div className="small text-secondary">Your next booking</div>
              <div className="fw-bold fs-5 text-dark">Malabe Solar Hub</div>
            </div>
            <StatusBadge status="Approved" />
          </div>
          <div className="text-secondary small mt-1">Tue 11:00 - 13:00 · 12.5 kWh · Sell</div>
          <hr />
          <div className="d-flex align-items-center gap-3">
            <i className="bi bi-qrcode display-4 text-dark" aria-hidden="true"></i>
            <p class="small text-secondary mb-0">Show this QR code at the station. The operator scans it and finishes your energy transfer.</p>
          </div>
        </div>
        <div className="card hero-card p-3 mt-3 ms-auto position-relative" style={{ maxWidth: '17rem' }}>
          <div className="d-flex align-items-center gap-3">
            <span className="feature-icon"><i className="bi bi-geo-alt-fill" aria-hidden="true"></i></span>
            <div>
              <div className="fw-semibold text-dark">4 stations nearby</div>
              <div className="small text-secondary">Nearest: 0.4 km away</div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</header>

<main id="content" tabIndex={-1}>
  <section id="how" className="container py-5" aria-labelledby="how-title">
    <div className="text-center mb-5">
      <h2 id="how-title" className="display-6 fw-bold">How it works</h2>
      <p className="text-secondary">Three simple steps from rooftop to grid.</p>
    </div>
    <ol className="row g-4 list-unstyled">
      {STEPS.map((step, index) => (
        <li key={step.title} className="col-md-4">
          <div className="card h-100 p-4">
            <div className="d-flex align-items-center gap-3 mb-3">
              <span className="step-number">{index + 1}</span>
              <i className={`bi ${step.icon}` fs-3 text-primary` } aria-hidden="true"></i>
            </div>
            <h3 className="h5">{step.title}</h3>
            <p className="text-secondary mb-0">{step.text}</p>
          </div>
        </li>
      )))}
    </ol>
  </section>

  <section className="bg-white py-5" aria-labelledby="roles-title">
    <div className="container">
      <div className="text-center mb-5">
        <h2 id="roles-title" className="display-6 fw-bold">One system, three roles</h2>
        <p className="text-secondary">Everyone sees exactly the tools they need.</p>
      </div>
      <div className="row g-4">
        {ROLES.map((role) => (
          <div key={role.title} className="col-md-4">
            <div className="card h-100 p-4">
              <span className="feature-icon mb-3"><i className={`bi ${role.icon}` } aria-hidden="true"></i></span>
              <h3 className="h5 mb-1">{role.title}</h3>
              <span className="badge rounded-pill role-where align-self-start mb-3">{role.where}</span>
              <p className="text-secondary mb-0">{role.text}</p>
            </div>
          </div>
        )))}
      </div>
    </div>
  </section>

  <section className="home-cta py-5" aria-labelledby="cta-title">
    <div className="container d-flex flex-column flex-md-row align-items-md-center justify-content-between gap-3">
      <div>
        <h2 id="cta-title" className="h3 mb-1">Keep the microgrid running smoothly</h2>
        <p className="mb-0">Stations, slots, prosumers and bookings - all in one place for staff.</p>
      </div>

```

```

        <Link to={loginLink} className="btn btn-accent btn-lg">{loginText}</Link>
      </div>
    </section>
  </main>

  <footer className="home-footer py-5">
    <div className="container">
      <div className="row g-4">
        <div className="col-md-5">
          <div className="fs-5 fw-bold text-white mb-2">
            <i className="bi bi-sun-fill text-warning me-2" aria-hidden="true"></i>SunShare
          </div>
          <p className="mb-2">Smart Solar Microgrid Trading System.</p>
          <p className="small mb-0">SE4040 Enterprise Application Development · Assignment 1 · SLIIT 2026</p>
        </div>
        <div className="col-md-4">
          <h2 className="h6 text-white">Team</h2>
          <ul className="list-unstyled small mb-0">
            {TEAM.map((member) => (
              <li key={member.id}>{member.name} · {member.id}</li>
            ))}
          </ul>
        </div>
        <div className="col-md-3">
          <h2 className="h6 text-white">System status</h2>
          <p className="small mb-0" role="status">
            <i
              className={`bi bi-circle-fill me-2 ${serverStatus === 'online' ? 'text-success' : serverStatus ===
'offline' ? 'text-danger' : 'text-secondary'}}
              aria-hidden="true"
            ></i>
            {serverStatus === 'online' && 'Server and database online'}
            {serverStatus === 'offline' && 'Server offline'}
            {serverStatus === 'checking' && 'Checking server...'}
          </p>
        </div>
      </div>
    </div>
  </footer>
</>
);
}

```

web/src/pages/Login.jsx

```

/*
=====
* File      : Login.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : A - Accounts & Access
* Author    : Gimhan T P K (IT22266996)
* Created   : 2026-09-29
* Description : W2 Login - NIC + password. Backoffice and Grid Operators go
*              to their dashboard; Prosumers are told to use the mobile app.
*              Pending / Deactivated / wrong password messages come from the API.
* =====
*/
import { useState } from 'react';
import { Link, Navigate, useLocation } from 'react-router-dom';
import { login as loginRequest } from '../api/authApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import { useAuth } from '../context/AuthContext.jsx';

// The staff login page (public).
export default function Login() {
  const { user, login } = useAuth();
  const location = useLocation();
  const [nic, setNic] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [busy, setBusy] = useState(false);

  // Logged in (already, or just now after login() below)? Go to the page the user first tried to
  // open, or to the dashboard. This is the only redirect, so the two can't race each other.
  if (user) {
    return <Navigate to={location.state?.from ?? '/dashboard'} replace />;
  }

  // Sends the login request. The API checks the NIC, password and account status (R3, R4);
  // this page only makes sure both boxes are filled in.
  async function handleSubmit(event) {
    event.preventDefault();
    setError('');
  }

```

```

if (!nic.trim() || !password) {
  setError('Please enter your NIC and password.');
```

```

  return;
}

setBusy(true);
try {
  const session = await loginRequest(nic.trim(), password);
  // The web app is for staff only; prosumers have the Android app (docs/01-SPEC.md §1).
  if (session.role === 'Prosumer') {
    setError('Prosumers use the SunShare mobile app. Please log in on your phone.');
```

```

    return;
  }
  login(session);
} catch (err) {
  setError(err.message);
} finally {
  setBusy(false);
}
}

return (
  <div className="login-page min-vh-100 d-flex align-items-center justify-content-center p-3">
    <main id="content" className="w-100" style={{ maxWidth: '26rem' }}>
      <div className="text-center mb-4">
        <Link to="/" className="fs-3 fw-bold text-white text-decoration-none">
          <i className="bi bi-sun-fill text-warning me-2" aria-hidden="true"></i>SunShare
        </Link>
      </div>

      <div className="card p-4 hero-card">
        <h1 className="h4 mb-1">Staff login</h1>
        <p className="text-secondary mb-4">For Backoffice officers and Grid Operators.</p>

        <AlertMessage message={error} onClose={() => setError('')} />

        <form onSubmit={handleSubmit} noValidate>
          <div className="mb-3">
            <label htmlFor="nic" className="form-label">NIC</label>
            <input
              id="nic"
              className="form-control"
              autoComplete="username"
              placeholder="e.g. 200012345678"
              value={nic}
              onChange={(e) => setNic(e.target.value)}
              autoFocus
            />
          </div>
          <div className="mb-4">
            <label htmlFor="password" className="form-label">Password</label>
            <input
              id="password"
              type="password"
              className="form-control"
              autoComplete="current-password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
            />
          </div>
          <button type="submit" className="btn btn-primary w-100" disabled={busy}>
            {busy && <span className="spinner-border spinner-border-sm me-2" aria-hidden="true"></span>}
            {busy ? 'Logging in...' : 'Log in'}
          </button>
        </form>
      </div>

      <p className="text-center small mt-3 mb-0">
        <Link to="/" className="text-white">
          <i className="bi bi-arrow-left me-1" aria-hidden="true"></i>Back to home
        </Link>
      </p>
    </main>
  </div>
);
}

```

web/src/pages/NotFound.jsx

```

/*
=====
* File       : NotFound.jsx
* Project    : SunShare - Smart Solar Microgrid Trading System

```

```

* Module      : SE4040 Enterprise Application Development - Assignment 1
* Part       : B - Shared foundation (web shell)
* Author      : Ranathunga R A K N (IT22552860)
* Created     : 2026-09-29
* Description  : W15 - friendly "page not found" screen for unknown addresses.
* =====
*/
import { Link } from 'react-router-dom';

// Shown for any address that doesn't match a route.
export default function NotFound() {
  return (
    <main id="content" className="min-vh-100 d-flex align-items-center justify-content-center p-4 text-center">
      <div>
        <i className="bi bi-sun display-1 text-warning" aria-hidden="true"></i>
        <h1 className="display-5 fw-bold mt-3">Page not found</h1>
        <p className="text-secondary mb-4">The page you're looking for isn't here. It may have moved, or the address has a
typo.</p>
        <Link to="/" className="btn btn-primary">
          <i className="bi bi-house me-2" aria-hidden="true"></i>Back to home
        </Link>
      </div>
    </main>
  );
}

```

web/src/pages/PendingActivations.jsx

```

/*
* =====
* File       : PendingActivations.jsx
* Project    : SunShare - Smart Solar Microgrid Trading System
* Module     : SE4040 Enterprise Application Development - Assignment 1
* Part      : A - Accounts & Access
* Author     : Gimhan T P K (IT22266996)
* Created    : 2026-09-29
* Description : W8 Pending activations - prosumers waiting for Backoffice:
*              new sign-ups (Pending, R3) and deactivated accounts that need
*              reactivation (R4). One click activates them.
* =====
*/
import { useEffect, useState } from 'react';
import { activateUser, getPendingActivations, notifyActivationsChanged } from '../api/usersApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import StatusBadge from '../components/StatusBadge.jsx';
import { formatDateTime } from '../utils/format.js';

// The Backoffice "to do" list of accounts to activate. The API gives them longest-waiting first.
export default function PendingActivations() {
  const [prosumers, setProsumers] = useState([]);
  const [reloadKey, setReloadKey] = useState(0);
  const [loading, setLoading] = useState(true);
  const [busyNic, setBusyNic] = useState('');
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');

  // Loads the waiting prosumers (again after every activation). "cancelled" ignores late answers.
  useEffect(() => {
    let cancelled = false;
    getPendingActivations()
      .then((list) => {
        if (!cancelled) setProsumers(list);
      })
      .catch((err) => {
        if (!cancelled) setError(err.message);
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
    return () => {
      cancelled = true;
    };
  }, [reloadKey]);

  // Activates one prosumer (R3 / R4 - only Backoffice may do this; the API checks it).
  async function handleActivate(prosumer) {
    setBusyNic(prosumer.nic);
    setError('');
    try {
      await activateUser(prosumer.nic);
      setSuccess(`${prosumer.fullName} can now log in to the mobile app.`);
    }
  }

```

```

        notifyActivationsChanged();
        setReloadKey((key) => key + 1);
    } catch (err) {
        setSuccess('');
        setError(err.message);
    } finally {
        setBusyNic('');
    }
}

return (
    <>
        <PageHeader
            title="Pending activations"
            subtitle="New prosumer sign-ups and deactivated accounts waiting for a Backoffice officer."
        />

        <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
        <AlertMessage message={error} onClose={() => setError('')} />

        <div className="card">
            {loading && <div className="px-3"><LoadingSpinner text="Loading waiting accounts..." /></div>}
            {!loading && prosumers.length === 0 && (
                <EmptyState icon="bi-check2-circle" title="All caught up" text="No prosumers are waiting for activation." />
            )}
            {!loading && prosumers.length > 0 && (
                <div className="table-responsive">
                    <table className="table table-hover align-middle mb-0">
                        <caption className="visually-hidden">Prosumers waiting for activation</caption>
                        <thead>
                            <tr>
                                <th scope="col">NIC</th>
                                <th scope="col">Prosumer</th>
                                <th scope="col">Phone</th>
                                <th scope="col">Reason</th>
                                <th scope="col">Waiting since</th>
                                <th scope="col" className="text-end">Action</th>
                            </tr>
                        </thead>
                        <tbody>
                            {prosumers.map((prosumer) => (
                                <tr key={prosumer.nic}>
                                    <td className="font-monospace small">{prosumer.nic}</td>
                                    <td>
                                        <div className="fw-semibold">{prosumer.fullName}</div>
                                        <div className="small text-secondary">{prosumer.email}</div>
                                    </td>
                                    <td>{prosumer.phone}</td>
                                    <td>
                                        <StatusBadge status={prosumer.status} />
                                        <div className="small text-secondary mt-1">
                                            {prosumer.status === 'Pending' ? 'New sign-up' : 'Needs reactivation'}
                                        </div>
                                    </td>
                                    <td className="small">{formatDateTime(prosumer.updatedAt)}</td>
                                    <td className="text-end">
                                        <button
                                            type="button"
                                            className="btn btn-sm btn-success"
                                            disabled={busyNic === prosumer.nic}
                                            onClick={() => handleActivate(prosumer)}
                                        >
                                            <i className="bi bi-check-lg me-1" aria-hidden="true"></i>
                                            {prosumer.status === 'Pending' ? 'Activate' : 'Reactivate'}
                                            <span className="visually-hidden"> {prosumer.fullName}</span>
                                        </button>
                                    </td>
                                </tr>
                            ))}
                        </tbody>
                    </table>
                </div>
            )}
        </div>
    </>
);
}

```

web/src/pages/Prosumers.jsx

```

/*
 * =====
 * File      : Prosumers.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 */

```

```

* Part      : A - Accounts & Access
* Author    : Gimhan T P K (IT22266996)
* Created   : 2026-09-29
* Description : W6 Prosumers - list with search and status filter, Edit,
*             Deactivate / Reactivate, and Add prosumer.
* =====
*/
import { useEffect, useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { listUsers } from '../api/usersApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import UserTable from '../components/UserTable.jsx';

// The prosumer accounts page (Backoffice only).
export default function Prosumers() {
  const location = useLocation();
  const [prosumers, setProsumers] = useState([]);
  const [status, setStatus] = useState('');
  const [searchText, setSearchText] = useState('');
  const [search, setSearch] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(location.state?.message ?? '');

  const [reloadKey, setReloadKey] = useState(0);

  // Loads prosumers whenever the status / search changes (or reloadKey goes up after an action).
  // "cancelled" stops an older, slower answer from overwriting a newer one.
  useEffect(() => {
    let cancelled = false;
    listUsers({ role: 'Prosumer', status, search })
      .then((list) => {
        if (!cancelled) setProsumers(list);
      })
      .catch((err) => {
        if (!cancelled) setError(err.message);
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
    return () => {
      cancelled = true;
    };
  }, [status, search, reloadKey]);

  // Applies the search box when the filter form is submitted.
  function handleSearch(event) {
    event.preventDefault();
    setSearch(searchText.trim());
  }

  // After Deactivate / Reactivate: show the message and load the list again.
  function handleChanged(message) {
    setSuccess(message);
    setError('');
    setReloadKey((key) => key + 1);
  }

  // When the API refuses an action, show its message (and hide any older success message).
  function handleError(message) {
    setError(message);
    setSuccess('');
  }

  return (
    <>
      <PageHeader title="Prosumers" subtitle="Homes with solar panels that trade energy through SunShare.">
        <Link to="/prosumers/new" className="btn btn-primary">
          <i className="bi bi-person-plus me-2" aria-hidden="true"></i>Add prosumer
        </Link>
      </PageHeader>

      <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
      <AlertMessage message={error} onClose={() => setError('')} />

      <div className="card">
        <form className="row g-2 p-3 border-bottom" onSubmit={handleSearch}>
          <div className="col-sm-5 col-lg-3">
            <label htmlFor="status" className="form-label small text-secondary mb-1">Status</label>
            <select id="status" className="form-select" value={status} onChange={(e) => setStatus(e.target.value)}>

```



```

        <option value="">Any status</option>
        <option value="Active">Active</option>
        <option value="Pending">Pending</option>
        <option value="Deactivated">Deactivated</option>
      </select>
    </div>
    <div className="col-sm-7 col-lg-6">
      <label htmlFor="search" className="form-label small text-secondary mb-1">Search</label>
      <div className="input-group">
        <input
          id="search"
          className="form-control"
          placeholder="NIC, name or email"
          value={searchText}
          onChange={(e) => setSearchText(e.target.value)}
        />
        <button type="submit" className="btn btn-outline-primary">
          <i className="bi bi-search" aria-hidden="true"></i><span className="visually-hidden">Search</span>
        </button>
      </div>
    </div>
  </form>

  {loading && <div className="px-3"><LoadingSpinner text="Loading prosumers..." /></div>}
  {!loading && prosumers.length === 0 && (
    <EmptyState icon="bi-house-heart" title="No prosumers found" text="Try other filters, or add a prosumer." />
  )}
  {!loading && prosumers.length > 0 && (
    <UserTable
      caption="Prosumers"
      users={prosumers}
      showAddress
      editBase="/prosumers"
      onChange={handleChange}
      onError={handleError}
    />
  )}
</div>
</>
);
}

```

web/src/pages/ReservationForm.jsx

```

/*
 * =====
 * File      : ReservationForm.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author    : Malkith G W L (IT22630834)
 * Created   : 2026-09-29
 * Description : W14 Reservation form - staff book on behalf of a prosumer, or
 *              edit a booking: prosumer -> station -> bookable slot -> kWh ->
 *              Sell/Buy. The API checks every rule (R9-R13, R17) and its
 *              message (e.g. the 7-day or 12-hour rule) is shown here.
 * =====
 */
import { useEffect, useState } from 'react';
import { Link, useNavigate, useParams } from 'react-router-dom';
import { createReservation, getReservation, updateReservation } from '../api/reservationsApi.js';
import { listAvailableSlots } from '../api/slotsApi.js';
import { listStations } from '../api/stationsApi.js';
import { listUsers } from '../api/usersApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import StatusBadge from '../components/StatusBadge.jsx';
import { formatDateTime, formatTime } from '../utils/format.js';

// Text shown for one slot in the drop-down, e.g. "Wed, 30 Sep, 08:00 - 10:00 · 4 free".
function slotLabel(slot) {
  return `${formatDateTime(slot.startTime)} - ${formatTime(slot.endTime)} · ${slot.availableSlots} free`;
}

// New booking (/reservations/new) or edit (/reservations/:id/edit).
export default function ReservationForm() {
  const { id } = useParams();
  const isEdit = Boolean(id);
  const navigate = useNavigate();
  const [prosumers, setProsumers] = useState([]);
  const [stations, setStations] = useState([]);
  const [slots, setSlots] = useState([]);
  const [current, setCurrent] = useState(null);
  const [form, setForm] = useState({ prosumerNic: '', stationId: '', slotId: '', energyKwh: '', type: 'Sell' });

```

```

const [loading, setLoading] = useState(true);
const [saving, setSaving] = useState(false);
const [error, setError] = useState('');

// Once: load the active stations, plus either the active prosumers (new) or the booking (edit).
useEffect(() => {
  let cancelled = false;
  const second = isEdit ? getReservation(id) : listUsers({ role: 'Prosumer', status: 'Active' });
  Promise.all([listStations(true), second])
    .then(([stationList, data]) => {
      if (cancelled) return;
      setStations(stationList);
      if (isEdit) {
        setCurrent(data);
        setForm({ prosumerNic: data.prosumerNic, stationId: data.stationId, slotId: data.slotId, energyKwh:
String(data.energyKwh), type: data.type });
      } else {
        setProsumers(data);
      }
    })
    .catch((err) => {
      if (!cancelled) setError(err.message);
    })
    .finally(() => {
      if (!cancelled) setLoading(false);
    });
  return () => {
    cancelled = true;
  };
}, [isEdit, id]);

// Whenever the station changes: load its bookable slots (the API applies R9 and R11).
useEffect(() => {
  if (!form.stationId) {
    return undefined;
  }
  let cancelled = false;
  listAvailableSlots(form.stationId)
    .then((list) => {
      if (!cancelled) setSlots(list);
    })
    .catch((err) => {
      if (!cancelled) setError(err.message);
    });
  return () => {
    cancelled = true;
  };
}, [form.stationId]);

// Copies what the user picked/typed into the form. A new station clears the chosen slot.
function handleChange(event) {
  const { name, value } = event.target;
  if (name === 'stationId') {
    setSlots([]);
    setForm((previous) => ({ ...previous, stationId: value, slotId: '' }));
    return;
  }
  setForm((previous) => ({ ...previous, [name]: value }));
}

// Saves the booking. The browser only makes sure every box is filled in; the API checks the
// rules and the page shows its message if something is not allowed.
async function handleSubmit(event) {
  event.preventDefault();
  setError('');
  setSaving(true);
  const data = { slotId: form.slotId, energyKwh: Number(form.energyKwh), type: form.type };
  try {
    if (isEdit) {
      const saved = await updateReservation(id, data);
      const backToPending = current.status === 'Approved' && saved.status === 'Pending';
      navigate('/reservations', { state: { message: backToPending ? 'Booking updated. It went back to Pending and needs
approval again.' : 'Booking updated.' } });
    } else {
      const saved = await createReservation({ ...data, prosumerNic: form.prosumerNic });
      navigate('/reservations', { state: { message: `Booking created for ${saved.prosumerName} at ${saved.stationName}. It
is Pending until approved.` } });
    }
  } catch (err) {
    setError(err.message);
    setSaving(false);
  }
}

// When editing, the booking's own slot stays in the list even if it is now full.

```

```

const showCurrentSlot = isEdit && current && form.stationId === current.stationId && !slots.some((slot) => slot.id ===
current.slotId);

return (
  <>
    <PageHeader
      title={isEdit ? 'Edit reservation' : 'New reservation'}
      subtitle={isEdit ? 'Change the slot, energy amount or type.' : 'Book a slot on behalf of a prosumer. New bookings
start as Pending.'}
    >
      {current && <StatusBadge status={current.status} />}
    </PageHeader>

    <AlertMessage message={error} onClose={() => setError('')} />
    {current && !current.canModify && (
      <AlertMessage type="info" message="This booking can no longer be changed (it is finished, cancelled, or starts in
less than 12 hours)."/>
    )}

    {loading ? (
      <LoadingSpinner text="Loading..." />
    ) : (
      <form className="card p-4" style={{ maxWidth: '44rem' }} onSubmit={handleSubmit}>
        <div className="row g-3">
          <div className="col-12">
            <label htmlFor="prosumerNic" className="form-label">Prosumer</label>
            {isEdit ? (
              <input id="prosumerNic" className="form-control" value={current ? `${current.prosumerName}
`${current.prosumerNic}` : ''} readOnly />
            ) : (
              <select id="prosumerNic" name="prosumerNic" className="form-select" value={form.prosumerNic}
onChange={handleChange} required>
                <option value="">Choose an active prosumer...</option>
                {prosumers.map((prosumer) => (
                  <option key={prosumer.nic} value={prosumer.nic}>{prosumer.fullName} ({prosumer.nic})</option>
                ))}
              </select>
            )}
          </div>
          <div className="col-md-6">
            <label htmlFor="stationId" className="form-label">Station</label>
            <select id="stationId" name="stationId" className="form-select" value={form.stationId} onChange={handleChange}
required>
              <option value="">Choose a station...</option>
              {stations.map((station) => (
                <option key={station.id} value={station.id}>{station.name}</option>
              ))}
            </select>
          </div>
          <div className="col-md-6">
            <label htmlFor="slotId" className="form-label">Time slot</label>
            <select id="slotId" name="slotId" className="form-select" value={form.slotId} onChange={handleChange} required
disabled={!form.stationId}>
              <option value="">{form.stationId ? 'Choose a slot...' : 'Choose a station first'}</option>
              {showCurrentSlot && (
                <option value={current.slotId}>{formatDateTime(current.startTime)} - {formatTime(current.endTime)}
(current)</option>
              )}
              {slots.map((slot) => (
                <option key={slot.id} value={slot.id}>{slotLabel(slot)}</option>
              ))}
            </select>
            {form.stationId && slots.length === 0 && <div className="form-text">No bookable slots at this station in the
next 7 days.</div>
            </div>
          <div className="col-md-6">
            <label htmlFor="energyKwh" className="form-label">Energy (kWh)</label>
            <input id="energyKwh" name="energyKwh" type="number" step="any" className="form-control" value={form.energyKwh}
onChange={handleChange} required />
            <div className="form-text">More than 0 and up to 100 kWh.</div>
          </div>
          <fieldset className="col-md-6">
            <legend className="form-label fs-6 mb-2">Type</legend>
            <div className="form-check">
              <input id="typeSell" name="type" type="radio" value="Sell" className="form-check-input" checked={form.type
=== 'Sell'} onChange={handleChange} />
              <label htmlFor="typeSell" className="form-check-label">Sell - drop off spare energy</label>
            </div>
            <div className="form-check">
              <input id="typeBuy" name="type" type="radio" value="Buy" className="form-check-input" checked={form.type ===
'Buy'} onChange={handleChange} />
              <label htmlFor="typeBuy" className="form-check-label">Buy - charge from the station</label>
            </div>
          </fieldset>
        </div>
      </form>
    )}
  </>
)

```

```

        <div className="d-flex gap-2 mt-4">
          <button type="submit" className="btn btn-primary" disabled={saving}>
            {saving && <span className="spinner-border spinner-border-sm me-2" aria-hidden="true"></span>}
            {isEdit ? 'Save changes' : 'Create booking'}
          </button>
          <Link to="/reservations" className="btn btn-outline-secondary">Cancel</Link>
        </div>
      </form>
    )}
  </>
);
}

```

web/src/pages/Reservations.jsx

```

/*
 * =====
 * File       : Reservations.jsx
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : D - Dashboards & QR
 * Author     : Chamara R M L K (IT22076816)
 * Created    : 2026-09-29
 * Description: W13 Reservations - every booking with a filter bar: view
 *              (all / current / pending / history), status, station, date
 *              range and search. The API does the filtering (rule R16 and
 *              the list definitions in docs/01-SPEC.md §5).
 *              Row actions Edit / Cancel / Approve were added by Part C
 *              (Malkith G W L, IT22630834); they appear only when the API
 *              allows them (canModify, status Pending).
 * =====
 */
import { useEffect, useState } from 'react';
import { Link, useLocation, useSearchParams } from 'react-router-dom';
import { approveReservation, cancelReservation, listReservations } from '../api/reservationsApi.js';
import { listStations } from '../api/stationsApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import ConfirmButton from '../components/ConfirmButton.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import ReservationTable from '../components/ReservationTable.jsx';
import { fromDateInput } from '../utils/format.js';

const VIEWS = [
  { value: 'all', label: 'All' },
  { value: 'current', label: 'Current' },
  { value: 'pending', label: 'Pending' },
  { value: 'history', label: 'History' },
];

// Staff list of bookings. A "view" can come from the address, e.g. /reservations?view=pending (dashboard cards).
export default function Reservations() {
  const [searchParams] = useSearchParams();
  const location = useLocation();
  const [view, setView] = useState(searchParams.get('view') ?? 'all');
  const [status, setStatus] = useState('');
  const [stationId, setStationId] = useState('');
  const [fromDate, setFromDate] = useState('');
  const [toDate, setToDate] = useState('');
  const [searchText, setSearchText] = useState('');
  const [search, setSearch] = useState('');
  const [stations, setStations] = useState([]);
  const [reservations, setReservations] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(location.state?.message ?? '');
  const [reloadKey, setReloadKey] = useState(0);
  const [busyId, setBusyId] = useState('');

  // Once: load the stations for the station filter.
  useEffect(() => {
    listStations().then(setStations).catch((err) => setError(err.message));
  }, []);

  // Loads the bookings whenever a filter changes. The dates are local days turned into UTC
  // (from = start of that day, to = start of the day after, so the whole "to" day is included).
  useEffect(() => {
    let cancelled = false;
    const filters = {
      view,
      status,
      stationId,
      search,
    };

```

```

    from: fromDate ? fromDateInput(fromDate, 0) : '',
    to: toDate ? fromDateInput(toDate, 1) : '',
  });
  listReservations(filters)
    .then((list) => {
      if (!cancelled) {
        setReservations(list);
        setError('');
      }
    })
    .catch((err) => {
      if (!cancelled) setError(err.message);
    })
    .finally(() => {
      if (!cancelled) setLoading(false);
    });
  return () => {
    cancelled = true;
  };
}, [view, status, stationId, fromDate, toDate, search, reloadKey]);

// Runs one row action (cancel or approve) and shows the API's answer. The API checks the rules:
// R10 (12 hours' notice), R15 (finished bookings can't change), R13 (only Pending can be approved).
async function runAction(reservation, action, successText) {
  setBusyId(reservation.id);
  setError('');
  try {
    await action(reservation.id);
    setSuccess(successText);
    setReloadKey((key) => key + 1);
  } catch (err) {
    setSuccess('');
    setError(err.message);
  } finally {
    setBusyId('');
  }
}

// The buttons for one row. Edit / Cancel only if the API says canModify; Approve only for Pending.
function renderActions(reservation) {
  const busy = busyId === reservation.id;
  const who = `${reservation.prosumerName} at ${reservation.stationName}`;
  return (
    <>
    {reservation.status === 'Pending' && (
      <button type="button" className="btn btn-sm btn-primary me-1" disabled={busy}
        onClick={() => runAction(reservation, approveReservation, `Approved ${reservation.prosumerName}'s booking - the
QR code is ready in the app.`)}>
        Approve<span className="visually-hidden"> {who}</span>
      </button>
    )}
    {reservation.canModify && (
      <>
      <Link to={`/${reservations}/${reservation.id}/edit`} className="btn btn-sm btn-outline-primary me-1">
        Edit<span className="visually-hidden"> {who}</span>
      </Link>
      <ConfirmButton
        message={`Cancel ${reservation.prosumerName}'s booking at ${reservation.stationName}`}
        onConfirm={() => runAction(reservation, cancelReservation, 'The booking was cancelled and its place is free
again.')}
        className="btn btn-sm btn-outline-danger"
        disabled={busy}
      >
        Cancel<span className="visually-hidden"> {who}</span>
      </ConfirmButton>
    </>
    )}
  </>
  );
}

// Applies the search box when the filter form is submitted.
function handleSearch(event) {
  event.preventDefault();
  setSearch(searchText.trim());
}

// Clears every filter back to "all bookings".
function clearFilters() {
  setView('all');
  setStatus('');
  setStationId('');
  setFromDate('');
  setToDate('');
  setSearchText('');
  setSearch('');
}

```

```

}

return (
  <>
    <PageHeader title="Reservations" subtitle="Every energy booking. Current = approved and upcoming; History = completed,
cancelled or past.">
      <Link to="/reservations/new" className="btn btn-primary">
        <i className="bi bi-plus-lg me-2" aria-hidden="true"></i>New reservation
      </Link>
    </PageHeader>

    <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
    <AlertMessage message={error} onClose={() => setError('')} />

    <div className="card">
      <form className="p-3 border-bottom" onSubmit={handleSearch}>
        <div className="btn-group mb-3 flex-wrap" role="group" aria-label="View">
          {VIEWS.map((option) => (
            <button
              key={option.value}
              type="button"
              className={`btn btn-sm ${view === option.value ? 'btn-primary' : 'btn-outline-primary'}`}
              aria-pressed={view === option.value}
              onClick={() => setView(option.value)}
            >
              {option.label}
            </button>
          ))}
        </div>
        <div className="row g-2">
          <div className="col-sm-6 col-lg-2">
            <label htmlFor="status" className="form-label small text-secondary mb-1">Status</label>
            <select id="status" className="form-select" value={status} onChange={(e) => setStatus(e.target.value)}>
              <option value="">Any status</option>
              <option value="Pending">Pending</option>
              <option value="Approved">Approved</option>
              <option value="Completed">Completed</option>
              <option value="Cancelled">Cancelled</option>
            </select>
          </div>
          <div className="col-sm-6 col-lg-3">
            <label htmlFor="station" className="form-label small text-secondary mb-1">Station</label>
            <select id="station" className="form-select" value={stationId} onChange={(e) => setStationId(e.target.value)}>
              <option value="">All stations</option>
              {stations.map((station) => (
                <option key={station.id} value={station.id}>{station.name}</option>
              ))}
            </select>
          </div>
          <div className="col-6 col-lg-2">
            <label htmlFor="fromDate" className="form-label small text-secondary mb-1">From</label>
            <input id="fromDate" type="date" className="form-control" value={fromDate} onChange={(e) =>
setFromDate(e.target.value)} />
          </div>
          <div className="col-6 col-lg-2">
            <label htmlFor="toDate" className="form-label small text-secondary mb-1">To</label>
            <input id="toDate" type="date" className="form-control" value={toDate} onChange={(e) =>
setToDate(e.target.value)} />
          </div>
          <div className="col-lg-3">
            <label htmlFor="search" className="form-label small text-secondary mb-1">Search</label>
            <div className="input-group">
              <input
                id="search"
                className="form-control"
                placeholder="Station, prosumer, NIC or id"
                value={searchText}
                onChange={(e) => setSearchText(e.target.value)}
              />
              <button type="submit" className="btn btn-outline-primary">
                <i className="bi bi-search" aria-hidden="true"></i><span className="visually-hidden">Search</span>
              </button>
            </div>
          </div>
        </div>
        <div className="d-flex justify-content-between align-items-center mt-2">
          <span className="small text-secondary" role="status">{loading ? '' : `${reservations.length} booking(s)`}</span>
          <button type="button" className="btn btn-sm btn-link" onClick={clearFilters}>Clear filters</button>
        </div>
      </form>

      {loading && <div className="px-3"><LoadingSpinner text="Loading reservations..." /></div>}
      {!loading && reservations.length === 0 && (
        <EmptyState icon="bi-calendar-x" title="No bookings found" text="Try another view or clear the filters." />
      )}
      {!loading && reservations.length > 0 && (

```

```

        <ReservationTable caption="Reservations" reservations={reservations} renderActions={renderActions} />
      )}
    </div>
  </>
);
}

```

web/src/pages/SlotForm.jsx

```

/*
=====
* File      : SlotForm.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Nodes, Slots & Map
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : W12 Slot form - add a slot to a station (/slots/new?stationId=)
*              or edit one (/slots/:id/edit): start, end, number of places and
*              open/closed. Times are typed in local time and sent in UTC;
*              the API checks every rule (R8).
=====
*/
import { useEffect, useState } from 'react';
import { Link, useNavigate, useParams, useSearchParams } from 'react-router-dom';
import { createSlot, getSlot, updateSlot } from '../api/slotsApi.js';
import { getStation } from '../api/stationsApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import { fromDateTimeInput, toDateTimeInput } from '../utils/format.js';

// Suggested times for a new slot: tomorrow 08:00 - 10:00 local time.
function defaultTimes() {
  const start = new Date();
  start.setDate(start.getDate() + 1);
  start.setHours(8, 0, 0, 0);
  const end = new Date(start);
  end.setHours(10);
  return { startTime: toDateTimeInput(start), endTime: toDateTimeInput(end) };
}

// Add or edit one slot.
export default function SlotForm() {
  const { id } = useParams();
  const isEdit = Boolean(id);
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();
  const [stationId, setStationId] = useState(searchParams.get('stationId') ?? '');
  const [station, setStation] = useState(null);
  const [form, setForm] = useState({ ...defaultTimes(), totalSlots: '4', isActive: true });
  const [bookedSlots, setBookedSlots] = useState(0);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState('');

  // Loads what the form needs: for an edit the slot (then its station), for a new slot just the station.
  useEffect(() => {
    let cancelled = false;
    const loadSlot = isEdit
      ? getSlot(id).then((slot) => {
          if (!cancelled) {
            setStationId(slot.stationId);
            setBookedSlots(slot.bookedSlots);
            setForm({
              startTime: toDateTimeInput(slot.startTime),
              endTime: toDateTimeInput(slot.endTime),
              totalSlots: String(slot.totalSlots),
              isActive: slot.isActive,
            });
          }
        })
      : Promise.resolve(searchParams.get('stationId'));

    return slot.stationId;
  });

  loadSlot
    .then((theStationId) => getStation(theStationId))
    .then((stationData) => {
      if (!cancelled) setStation(stationData);
    })
    .catch((err) => {
      if (!cancelled) setError(err.message);
    })
    .finally(() => {

```

```

        if (!cancelled) setLoading(false);
    });
    return () => {
        cancelled = true;
    };
}, [isEdit, id, searchParams]);

// Copies what the user typed into the form state (checkbox uses "checked", others "value").
function handleChange(event) {
    const { name, value, type, checked } = event.target;
    setForm((previous) => ({ ...previous, [name]: type === 'checkbox' ? checked : value }));
}

// Saves the slot (times converted to UTC). The API checks R8: start in the future, end after start,
// 1..battery slots places, not below the booked places, and no time change once booked.
async function handleSubmit(event) {
    event.preventDefault();
    setError('');
    setSaving(true);
    const times = { startTime: fromDateTimeInput(form.startTime), endTime: fromDateTimeInput(form.endTime) };
    try {
        if (isEdit) {
            await updateSlot(id, { ...times, totalSlots: Number(form.totalSlots), isActive: form.isActive });
        } else {
            await createSlot({ ...times, stationId, totalSlots: Number(form.totalSlots) });
        }
        navigate(`/stations/${stationId}/slots`, { state: { message: isEdit ? 'The slot was saved.' : 'The slot was added.' } });
    } catch (err) {
        setError(err.message);
        setSaving(false);
    }
}

return (
    <>
        <PageHeader
            title={isEdit ? 'Edit slot' : 'New slot'}
            subtitle={station ? `${station.name} · ${station.batterySlots} battery slots · open ${station.openTime} -
            ${station.closeTime}` : ''}
        />

        <AlertMessage message={error} onClose={() => setError('')} />

        {loading ? (
            <LoadingSpinner text="Loading..." />
        ) : (
            <form className="card p-4" style={{ maxWidth: '40rem' }} onSubmit={handleSubmit}>
                <div className="row g-3">
                    <div className="col-sm-6">
                        <label htmlFor="startTime" className="form-label">Starts</label>
                        <input id="startTime" name="startTime" type="datetime-local" className="form-control" value={form.startTime}
                        onChange={handleChange} required />
                    </div>
                    <div className="col-sm-6">
                        <label htmlFor="endTime" className="form-label">Ends</label>
                        <input id="endTime" name="endTime" type="datetime-local" className="form-control" value={form.endTime}
                        onChange={handleChange} required />
                    </div>
                    <div className="col-sm-6">
                        <label htmlFor="totalSlots" className="form-label">Places in this slot</label>
                        <input id="totalSlots" name="totalSlots" type="number" step="1" className="form-control"
                        value={form.totalSlots} onChange={handleChange} required />
                        <div className="form-text">
                            {station && `Up to ${station.batterySlots} (the station's battery slots).`}
                            {isEdit && bookedSlots > 0 && ` ${bookedSlots} already booked - the time can't change and places can't go
                            below that.`}
                        </div>
                    </div>
                    <div>
                        {isEdit && (
                            <div className="col-sm-6 d-flex align-items-center">
                                <div className="form-check form-switch mt-sm-4">
                                    <input id="isActive" name="isActive" type="checkbox" role="switch" className="form-check-input"
                                    checked={form.isActive} onChange={handleChange} />
                                    <label htmlFor="isActive" className="form-check-label">Open for booking</label>
                                </div>
                            </div>
                        )}
                    </div>
                </div>

                <div className="d-flex gap-2 mt-4">
                    <button type="submit" className="btn btn-primary" disabled={saving || !stationId}>
                        {saving && <span className="spinner-border spinner-border-sm me-2" aria-hidden="true"></span>}
                        {isEdit ? 'Save changes' : 'Add slot'}
                    </button>
                </div>
            </form>
        )}
    </>
);

```



```

        <Link to={stationId ? `/${stationId}/slots` : `/${stationId}` } className="btn btn-outline-
secondary">Cancel</Link>
      </div>
    </form>
  )}
</>
);
}

```

web/src/pages/Slots.jsx

```

/*
 * =====
 * File       : Slots.jsx
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Nodes, Slots & Map
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-29
 * Description : W11 Slots - one station's booking windows for the next 14
 *               days: time, places (total / booked / free), open or closed,
 *               with Add, Edit and Delete for Backoffice and Grid Operators.
 * =====
 */
import { useEffect, useState } from 'react';
import { Link, useLocation, useParams } from 'react-router-dom';
import { deleteSlot, listStationSlots } from '../api/slotsApi.js';
import { getStation } from '../api/stationsApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import ConfirmButton from '../components/ConfirmButton.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import StatusBadge from '../components/StatusBadge.jsx';
import { formatDate, formatTime } from '../utils/format.js';

// Slots of the station in the address (/stations/:id/slots).
export default function Slots() {
  const { id } = useParams();
  const location = useLocation();
  const [station, setStation] = useState(null);
  const [slots, setSlots] = useState([]);
  const [reloadKey, setReloadKey] = useState(0);
  const [loading, setLoading] = useState(true);
  const [busyId, setBusyId] = useState('');
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(location.state?.message ?? '');

  // Loads the station and its slots together (again after a delete). "cancelled" ignores late answers.
  useEffect(() => {
    let cancelled = false;
    Promise.all([getStation(id), listStationSlots(id)])
      .then(([stationData, slotList]) => {
        if (!cancelled) {
          setStation(stationData);
          setSlots(slotList);
        }
      })
      .catch((err) => {
        if (!cancelled) setError(err.message);
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
    return () => {
      cancelled = true;
    };
  }, [id, reloadKey]);

  // Deletes one slot. The API refuses while it has active reservations (R8) and its message is shown.
  async function handleDelete(slot) {
    setBusyId(slot.id);
    setError('');
    try {
      await deleteSlot(slot.id);
      setSuccess('The slot was deleted.');
```

```

const title = station ? `Slots · ${station.name}` : 'Slots';
const subtitle = station
  ? `Booking windows for the next 14 days. This station has ${station.batterySlots} battery slots and opens
    ${station.openTime} - ${station.closeTime}.`
  : '';

return (
  <>
    <PageHeader title={title} subtitle={subtitle}>
      <Link to="/stations" className="btn btn-outline-secondary">
        <i className="bi bi-arrow-left me-2" aria-hidden="true"></i>Stations
      </Link>
      {station && (
        <Link to={`/slots/new?stationId=${id}`} className="btn btn-primary">
          <i className="bi bi-plus-lg me-2" aria-hidden="true"></i>Add slot
        </Link>
      )}
    </PageHeader>

    <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
    <AlertMessage message={error} onClose={() => setError('')} />
    {station && !station.isActive && (
      <AlertMessage type="info" message="This station is deactivated, so it can't get new slots and its slots can't be
        booked." />
    )}

    <div className="card">
      {loading && <div className="px-3"><LoadingSpinner text="Loading slots..." /></div>}
      {!loading && slots.length === 0 && (
        <EmptyState icon="bi-clock" title="No slots in the next 14 days" text="Add a time window so prosumers can book it."
      />
      )}
      {loading && slots.length > 0 && (
        <div className="table-responsive">
          <table className="table table-hover align-middle mb-0">
            <caption className="visually-hidden">Slots at {station?.name}</caption>
            <thead>
              <tr>
                <th scope="col">Date</th>
                <th scope="col">Time</th>
                <th scope="col">Total</th>
                <th scope="col">Booked</th>
                <th scope="col">Free</th>
                <th scope="col">Booking</th>
                <th scope="col" className="text-end">Actions</th>
              </tr>
            </thead>
            <tbody>
              {slots.map((slot) => (
                <tr key={slot.id}>
                  <td className="text-nowrap">{formatDate(slot.startTime)}</td>
                  <td className="text-nowrap">{formatTime(slot.startTime)} - {formatTime(slot.endTime)}</td>
                  <td>{slot.totalSlots}</td>
                  <td>{slot.bookedSlots}</td>
                  <td className={slot.availableSlots === 0 ? 'text-danger fw-semibold' : ''}>{slot.availableSlots}</td>
                  <td><StatusBadge status={slot.isActive ? 'Open' : 'Closed'} /></td>
                  <td className="text-end text-nowrap">
                    <Link to={`/slots/${slot.id}/edit`} className="btn btn-sm btn-outline-primary me-1">
                      Edit<span className="visually-hidden"> slot {formatDate(slot.startTime)}
                    {formatTime(slot.startTime)}</span>
                    </Link>
                    <ConfirmButton
                      message={`Delete the ${formatDate(slot.startTime)} ${formatTime(slot.startTime)} slot?`}
                      onConfirm={() => handleDelete(slot)}
                      className="btn btn-sm btn-outline-danger"
                      disabled={busyId === slot.id}
                    >
                      <i className="bi bi-trash" aria-hidden="true"></i>
                      <span className="visually-hidden">Delete slot {formatDate(slot.startTime)}
                    {formatTime(slot.startTime)}</span>
                    </ConfirmButton>
                  </td>
                </tr>
              )})}
            </tbody>
          </table>
        </div>
      )}
    </div>
  );
}

```

web/src/pages/StationForm.jsx

```

/*
=====
* File      : StationForm.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Nodes, Slots & Map
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description: W10 Station form - create or edit a station (name, address,
*              GPS position with an "Open in Google Maps" check, capacity,
*              battery slots, opening hours). The API checks every value.
=====
*/
import { useEffect, useState } from 'react';
import { Link, useNavigate, useParams } from 'react-router-dom';
import { createStation, getStation, updateStation } from '../api/stationsApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import { googleMapsUrl } from '../utils/format.js';

const EMPTY_FORM = { name: '', address: '', latitude: '', longitude: '', capacityKw: '', batterySlots: '', openTime: '06:00',
closeTime: '18:00' };

// Backoffice form for one station. With an :id in the address it edits, otherwise it creates.
export default function StationForm() {
  const { id } = useParams();
  const isEdit = Boolean(id);
  const navigate = useNavigate();
  const [form, setForm] = useState(EMPTY_FORM);
  const [loading, setLoading] = useState(isEdit);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState('');

  // When editing: load the station once and fill the form (numbers become text for the inputs).
  useEffect(() => {
    if (!isEdit) {
      return;
    }
    getStation(id)
      .then((station) => setForm({
        name: station.name,
        address: station.address,
        latitude: String(station.latitude),
        longitude: String(station.longitude),
        capacityKw: String(station.capacityKw),
        batterySlots: String(station.batterySlots),
        openTime: station.openTime,
        closeTime: station.closeTime,
      }))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, [isEdit, id]);

  // Copies what the user typed into the form state (one handler for every field, by its "name").
  function handleChange(event) {
    const { name, value } = event.target;
    setForm((previous) => ({ ...previous, [name]: value }));
  }

  // Saves the station: POST for new, PUT for edit. Number boxes are sent as numbers.
  // The API checks the ranges (GPS, capacity > 0, at least 1 battery slot, open before close).
  async function handleSubmit(event) {
    event.preventDefault();
    setError('');
    setSaving(true);
    const data = {
      ...form,
      latitude: Number(form.latitude),
      longitude: Number(form.longitude),
      capacityKw: Number(form.capacityKw),
      batterySlots: Number(form.batterySlots),
    };
    try {
      const saved = isEdit ? await updateStation(id, data) : await createStation(data);
      navigate('/stations', { state: { message: isEdit ? `${saved.name} was saved.` : `${saved.name} was added.` } });
    } catch (err) {
      setError(err.message);
      setSaving(false);
    }
  }

  const hasPosition = form.latitude !== '' && form.longitude !== '';

```

```

return (
  <>
    <PageHeader
      title={isEdit ? 'Edit station' : 'New station'}
      subtitle="Tip: find the place in Google Maps, right-click it and copy the two numbers (latitude, longitude)."/>
    />

    <AlertMessage message={error} onClose={() => setError('')} />

    {loading ? (
      <LoadingSpinner text="Loading station..." />
    ) : (
      <form className="card p-4" style={{ maxWidth: '48rem' }} onSubmit={handleSubmit}>
        <div className="row g-3">
          <div className="col-md-6">
            <label htmlFor="name" className="form-label">Station name</label>
            <input id="name" name="name" className="form-control" value={form.name} onChange={handleChange} required />
          </div>
          <div className="col-md-6">
            <label htmlFor="address" className="form-label">Address</label>
            <input id="address" name="address" className="form-control" value={form.address} onChange={handleChange}
required />
          </div>
          <div className="col-sm-6 col-md-4">
            <label htmlFor="latitude" className="form-label">Latitude</label>
            <input id="latitude" name="latitude" type="number" step="any" className="form-control" value={form.latitude}
onChange={handleChange} placeholder="6.9147" required />
          </div>
          <div className="col-sm-6 col-md-4">
            <label htmlFor="longitude" className="form-label">Longitude</label>
            <input id="longitude" name="longitude" type="number" step="any" className="form-control" value={form.longitude}
onChange={handleChange} placeholder="79.9730" required />
          </div>
          <div className="col-md-4 d-flex align-items-end">
            {hasPosition ? (
              <a className="btn btn-outline-primary w-100" href={googleMapsUrl(form.latitude, form.longitude)}
target="_blank" rel="noopener noreferrer">
                <i className="bi bi-map me-2" aria-hidden="true"></i>Open in Google Maps
                <span className="visually-hidden"> (opens in a new tab)</span>
              </a>
            ) : (
              <span className="form-text">Enter latitude and longitude to check the pin on Google Maps.</span>
            )}
          </div>
          <div className="col-sm-6 col-md-3">
            <label htmlFor="capacityKw" className="form-label">Capacity (kW)</label>
            <input id="capacityKw" name="capacityKw" type="number" step="any" className="form-control"
value={form.capacityKw} onChange={handleChange} required />
          </div>
          <div className="col-sm-6 col-md-3">
            <label htmlFor="batterySlots" className="form-label">Battery slots</label>
            <input id="batterySlots" name="batterySlots" type="number" step="1" className="form-control"
value={form.batterySlots} onChange={handleChange} required />
          </div>
          <div className="col-sm-6 col-md-3">
            <label htmlFor="openTime" className="form-label">Opens at</label>
            <input id="openTime" name="openTime" type="time" className="form-control" value={form.openTime}
onChange={handleChange} required />
          </div>
          <div className="col-sm-6 col-md-3">
            <label htmlFor="closeTime" className="form-label">Closes at</label>
            <input id="closeTime" name="closeTime" type="time" className="form-control" value={form.closeTime}
onChange={handleChange} required />
          </div>
        </div>

        <div className="d-flex gap-2 mt-4">
          <button type="submit" className="btn btn-primary" disabled={saving}>
            {saving && <span className="spinner-border spinner-border-sm me-2" aria-hidden="true"></span>}
            {isEdit ? 'Save changes' : 'Add station'}
          </button>
          <Link to="/stations" className="btn btn-outline-secondary">Cancel</Link>
        </div>
      </form>
    )}
  </>
);
}

```

web/src/pages/Stations.jsx

```

/*
 * =====

```

```

* File      : Stations.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Nodes, Slots & Map
* Author    : Ranathunga R A K N (IT22552860)
* Created   : 2026-09-29
* Description : W9 Stations - every solar station with its location, size,
*              schedule, status and active bookings. Backoffice can edit,
*              deactivate / activate and delete; everyone can open its slots.
* =====
*/
import { useEffect, useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { activateStation, deactivateStation, deleteStation, listStations } from '../api/stationsApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import ConfirmButton from '../components/ConfirmButton.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import StatusBadge from '../components/StatusBadge.jsx';
import { useAuth } from '../context/AuthContext.jsx';
import { googleMapsUrl } from '../utils/format.js';

// Stations page for Backoffice (full control) and Grid Operators (view + slots).
export default function Stations() {
  const { user } = useAuth();
  const isBackoffice = user.role === 'Backoffice';
  const location = useLocation();
  const [stations, setStations] = useState([]);
  const [reloadKey, setReloadKey] = useState(0);
  const [loading, setLoading] = useState(true);
  const [busyId, setBusyId] = useState('');
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(location.state?.message ?? '');

  // Loads the stations (again after every action). "cancelled" ignores late answers.
  useEffect(() => {
    let cancelled = false;
    listStations()
      .then((list) => {
        if (!cancelled) setStations(list);
      })
      .catch((err) => {
        if (!cancelled) setError(err.message);
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
    return () => {
      cancelled = true;
    };
  }, [reloadKey]);

  // Runs one station action (deactivate / activate / delete) and shows the API's answer.
  // The API decides if it is allowed: R6 (no deactivating with active bookings), R7 (no deleting with history).
  async function runAction(station, action, successText) {
    setBusyId(station.id);
    setError('');
    try {
      await action(station.id);
      setSuccess(successText);
      setReloadKey((key) => key + 1);
    } catch (err) {
      setSuccess('');
      setError(err.message);
    } finally {
      setBusyId('');
    }
  }

  return (
    <>
      <PageHeader title="Stations" subtitle="Solar microgrid stations, their battery places and opening hours.">
        {isBackoffice && (
          <Link to="/stations/new" className="btn btn-primary">
            <i className="bi bi-plus-lg me-2" aria-hidden="true"></i>Add station
          </Link>
        )}
      </PageHeader>

      <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
      <AlertMessage message={error} onClose={() => setError('')} />

      <div className="card">
        {loading && <div className="px-3"><LoadingSpinner text="Loading stations..." /></div>}
      </div>
    </>
  );
}

```

```

    {!loading && stations.length === 0 && (
      <EmptyState icon="bi-geo-alt" title="No stations yet" text={isBackoffice ? 'Add the first solar station.' :
'Backoffice has not added any stations yet.'} />
    )}
    {!loading && stations.length > 0 && (
      <div className="table-responsive">
        <table className="table table-hover align-middle mb-0">
          <caption className="visually-hidden">Solar stations</caption>
          <thead>
            <tr>
              <th scope="col">Station</th>
              <th scope="col">Location</th>
              <th scope="col">Capacity</th>
              <th scope="col">Battery slots</th>
              <th scope="col">Hours</th>
              <th scope="col">Active bookings</th>
              <th scope="col">Status</th>
              <th scope="col" className="text-end">Actions</th>
            </tr>
          </thead>
          <tbody>
            {stations.map((station) => (
              <tr key={station.id}>
                <td>
                  <div className="fw-semibold">{station.name}</div>
                  <div className="small text-secondary">{station.address}</div>
                </td>
                <td className="small text-nowrap">
                  {station.latitude}, {station.longitude}
                  <div>
                    <a href={googleMapsUrl(station.latitude, station.longitude)} target="_blank" rel="noopener
norereferrer">
                      <i className="bi bi-map me-1" aria-hidden="true"></i>Map<span className="visually-hidden"> of
{station.name} (opens in a new tab)</span>
                    </a>
                  </div>
                </td>
                <td className="text-nowrap">{station.capacityKw} kW</td>
                <td>{station.batterySlots}</td>
                <td className="text-nowrap">{station.openTime} - {station.closeTime}</td>
                <td>{station.activeReservationCount}</td>
                <td><StatusBadge status={station.isActive ? 'Active' : 'Deactivated'} /></td>
                <td className="text-end text-nowrap">
                  <Link to={`stations/${station.id}/slots`} className="btn btn-sm btn-outline-primary me-1">
                    <i className="bi bi-clock me-1" aria-hidden="true"></i>Slots<span className="visually-hidden"> of
{station.name}</span>
                  </Link>
                  {isBackoffice && (
                    <>
                      <Link to={`stations/${station.id}/edit`} className="btn btn-sm btn-outline-secondary me-1">
                        Edit<span className="visually-hidden"> {station.name}</span>
                      </Link>
                      {station.isActive ? (
                        <ConfirmButton
                          message={`Deactivate ${station.name}? It will disappear from the map and can't be booked.`}
                          onConfirm={() => runAction(station, deactivateStation, `${station.name} was deactivated.`)}
                          className="btn btn-sm btn-outline-warning me-1"
                          disabled={busyId === station.id}
                        >
                          Deactivate<span className="visually-hidden"> {station.name}</span>
                        </ConfirmButton>
                      ) : (
                        <button
                          type="button"
                          className="btn btn-sm btn-outline-success me-1"
                          disabled={busyId === station.id}
                          onClick={() => runAction(station, activateStation, `${station.name} is active again.`)}
                        >
                          Activate<span className="visually-hidden"> {station.name}</span>
                        </button>
                      )}
                    <ConfirmButton
                      message={`Delete ${station.name} and all its slots? This can't be undone.`}
                      onConfirm={() => runAction(station, deleteStation, `${station.name} was deleted.`)}
                      className="btn btn-sm btn-outline-danger"
                      disabled={busyId === station.id}
                    >
                      <i className="bi bi-trash" aria-hidden="true"></i><span className="visually-hidden">Delete
{station.name}</span>
                    </ConfirmButton>
                  </td>
                </tr>
              </td>
            </td>
          </tbody>
        </table>
      </div>
    )}
  )}

```

```

    })
  </div>
</>
);
}

```

web/src/pages/UserForm.jsx

```

/*
 * =====
 * File      : UserForm.jsx
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & Access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-29
 * Description : W5 User form and W7 Prosumer form (one component, two kinds):
 *              create an account or edit one. The NIC is read-only when
 *              editing. The API checks every value (R1: NIC format, unique).
 * =====
 */
import { useEffect, useState } from 'react';
import { Link, useNavigate, useParams } from 'react-router-dom';
import { createUser, getUser, updateUser } from '../api/usersApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import StatusBadge from '../components/StatusBadge.jsx';

const EMPTY_FORM = { nic: '', fullName: '', email: '', phone: '', address: '', role: 'GridOperator', password: '', status: '' };

// kind = "staff" (web users: Backoffice / Grid Operator) or "prosumer" (always role Prosumer, has an address).
export default function UserForm({ kind }) {
  const { nic } = useParams();
  const isEdit = Boolean(nic);
  const isProsumer = kind === 'prosumer';
  const listPath = isProsumer ? '/prosumers' : '/users';
  const navigate = useNavigate();
  const [form, setForm] = useState({ ...EMPTY_FORM, role: isProsumer ? 'Prosumer' : 'GridOperator' });
  const [loading, setLoading] = useState(isEdit);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState('');

  // When editing: load the account once and fill the form with it.
  useEffect(() => {
    if (!isEdit) {
      return;
    }
    getUser(nic)
      .then((user) => setForm({ ...EMPTY_FORM, ...user, address: user.address ?? '', password: '' }))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, [isEdit, nic]);

  // Copies what the user typed into the form state (one handler for every field, by its "name").
  function handleChange(event) {
    const { name, value } = event.target;
    setForm((previous) => ({ ...previous, [name]: value }));
  }

  // Saves: POST for a new account (starts Active), PUT for an edit. The browser only checks that
  // required boxes are filled; the API checks the rest and its message is shown if something is wrong.
  async function handleSubmit(event) {
    event.preventDefault();
    setError('');
    setSaving(true);
    const details = { fullName: form.fullName, email: form.email, phone: form.phone, address: form.address };
    try {
      if (isEdit) {
        await updateUser(nic, details);
      } else {
        await createUser({ ...details, nic: form.nic, role: form.role, password: form.password });
      }
      navigate(listPath, { state: { message: isEdit ? `${form.fullName}'s details were saved.` : `${form.fullName}'s account was created.` } });
    } catch (err) {
      setError(err.message);
      setSaving(false);
    }
  }

  const noun = isProsumer ? 'prosumer' : 'web user';

```

```

const title = isEdit ? `Edit ${noun}` : `New ${noun}`;

return (
  <>
    <PageHeader title={title} subtitle={isEdit ? 'The NIC is the account key and cannot be changed.' : 'The new account is
active straight away.'}>
      {isEdit && form.status && <StatusBadge status={form.status} />}
    </PageHeader>

    <AlertMessage message={error} onClose={() => setError('')} />

    {loading ? (
      <LoadingSpinner text="Loading account..." />
    ) : (
      <form className="card p-4" style={{ maxWidth: '48rem' }} onSubmit={handleSubmit}>
        <div className="row g-3">
          <div className="col-md-6">
            <label htmlFor="nic" className="form-label">NIC</label>
            <input id="nic" name="nic" className="form-control" value={form.nic} onChange={handleChange} readOnly={isEdit}
required />
            {!isEdit && <div className="form-text">Old: 9 digits + V/X (981234567V). New: 12 digits (199812345678).</div>}
          </div>
          <div className="col-md-6">
            <label htmlFor="role" className="form-label">Role</label>
            {isProsumer || isEdit ? (
              <input id="role" className="form-control" value={form.role === 'GridOperator' ? 'Grid Operator' : form.role}
readOnly />
            ) : (
              <select id="role" name="role" className="form-select" value={form.role} onChange={handleChange}>
                <option value="GridOperator">Grid Operator</option>
                <option value="Backoffice">Backoffice</option>
              </select>
            )}
          </div>
          <div className="col-md-6">
            <label htmlFor="fullName" className="form-label">Full name</label>
            <input id="fullName" name="fullName" className="form-control" value={form.fullName} onChange={handleChange}
autoComplete="name" required />
          </div>
          <div className="col-md-6">
            <label htmlFor="email" className="form-label">Email</label>
            <input id="email" name="email" className="form-control" inputMode="email" value={form.email}
onChange={handleChange} autoComplete="email" required />
          </div>
          <div className="col-md-6">
            <label htmlFor="phone" className="form-label">Phone</label>
            <input id="phone" name="phone" className="form-control" inputMode="tel" value={form.phone}
onChange={handleChange} autoComplete="tel" required />
          </div>
          {isProsumer && (
            <div className="col-md-6">
              <label htmlFor="address" className="form-label">Address <span className="text-
secondary">(optional)</span></label>
              <input id="address" name="address" className="form-control" value={form.address} onChange={handleChange}
autoComplete="street-address" />
            </div>
          )}
          {!isEdit && (
            <div className="col-md-6">
              <label htmlFor="password" className="form-label">Password</label>
              <input id="password" name="password" type="password" className="form-control" value={form.password}
onChange={handleChange} autoComplete="new-password" required />
              <div className="form-text">At least 6 characters.</div>
            </div>
          )}
        </div>

        <div className="d-flex gap-2 mt-4">
          <button type="submit" className="btn btn-primary" disabled={saving}>
            {saving && <span className="spinner-border spinner-border-sm me-2" aria-hidden="true"></span>}
            {isEdit ? 'Save changes' : 'Create account'}
          </button>
          <Link to={listPath} className="btn btn-outline-secondary">Cancel</Link>
        </div>
      </form>
    )}
  </>
);
}

```

web/src/pages/Users.jsx

```

/*
* =====

```



```

* File      : Users.jsx
* Project   : SunShare - Smart Solar Microgrid Trading System
* Module    : SE4040 Enterprise Application Development - Assignment 1
* Part      : A - Accounts & Access
* Author    : Gimhan T P K (IT22266996)
* Created   : 2026-09-29
* Description : W4 Web users - Backoffice and Grid Operator accounts with
*             role / status / search filters, Edit and Activate / Deactivate.
* =====
*/
import { useEffect, useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { listUsers } from '../api/usersApi.js';
import AlertMessage from '../components/AlertMessage.jsx';
import EmptyState from '../components/EmptyState.jsx';
import LoadingSpinner from '../components/LoadingSpinner.jsx';
import PageHeader from '../components/PageHeader.jsx';
import UserTable from '../components/UserTable.jsx';

// The staff accounts page (Backoffice only).
export default function Users() {
  const location = useLocation();
  const [users, setUsers] = useState([]);
  const [role, setRole] = useState('');
  const [status, setStatus] = useState('');
  const [searchText, setSearchText] = useState('');
  const [search, setSearch] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(location.state?.message ?? '');

  const [reloadKey, setReloadKey] = useState(0);

  // Loads the accounts whenever a filter changes (or reloadKey goes up after an action).
  // With no role chosen, both staff roles are shown (prosumers have their own page).
  // "cancelled" stops an older, slower answer from overwriting a newer one.
  useEffect(() => {
    let cancelled = false;
    listUsers({ role, status, search })
      .then((list) => {
        if (!cancelled) setUsers(role ? list : list.filter((u) => u.role !== 'Prosumer'));
      })
      .catch((err) => {
        if (!cancelled) setError(err.message);
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
    return () => {
      cancelled = true;
    };
  }, [role, status, search, reloadKey]);

  // Applies the search box when the filter form is submitted.
  function handleSearch(event) {
    event.preventDefault();
    setSearch(searchText.trim());
  }

  // After Activate / Deactivate: show the message and load the list again.
  function handleChanged(message) {
    setSuccess(message);
    setError('');
    setReloadKey((key) => key + 1);
  }

  // When the API refuses an action, show its message (and hide any older success message).
  function handleError(message) {
    setError(message);
    setSuccess('');
  }

  return (
    <>
      <PageHeader title="Web users" subtitle="Backoffice officers and Grid Operators who use the web app.">
        <Link to="/users/new" className="btn btn-primary">
          <i className="bi bi-person-plus me-2" aria-hidden="true"></i>Add user
        </Link>
      </PageHeader>

      <AlertMessage type="success" message={success} onClose={() => setSuccess('')} />
      <AlertMessage message={error} onClose={() => setError('')} />
    </>
  );
}

```

```

<div className="card">
  <form className="row g-2 p-3 border-bottom" onSubmit={handleSearch}>
    <div className="col-sm-6 col-lg-3">
      <label htmlFor="role" className="form-label small text-secondary mb-1">Role</label>
      <select id="role" className="form-select" value={role} onChange={(e) => setRole(e.target.value)}>
        <option value="">All staff</option>
        <option value="Backoffice">Backoffice</option>
        <option value="GridOperator">Grid Operator</option>
      </select>
    </div>
    <div className="col-sm-6 col-lg-3">
      <label htmlFor="status" className="form-label small text-secondary mb-1">Status</label>
      <select id="status" className="form-select" value={status} onChange={(e) => setStatus(e.target.value)}>
        <option value="">Any status</option>
        <option value="Active">Active</option>
        <option value="Deactivated">Deactivated</option>
      </select>
    </div>
    <div className="col-lg-6">
      <label htmlFor="search" className="form-label small text-secondary mb-1">Search</label>
      <div className="input-group">
        <input
          id="search"
          className="form-control"
          placeholder="NIC, name or email"
          value={searchText}
          onChange={(e) => setSearchText(e.target.value)}
        />
        <button type="submit" className="btn btn-outline-primary">
          <i className="bi bi-search" aria-hidden="true"></i><span className="visually-hidden">Search</span>
        </button>
      </div>
    </div>
  </form>

  {loading && <div className="px-3"><LoadingSpinner text="Loading users..." /></div>}
  {!loading && users.length === 0 && (
    <EmptyState icon="bi-person-badge" title="No users found" text="Try other filters, or add a new user." />
  )}
  {!loading && users.length > 0 && (
    <UserTable
      caption="Web users"
      users={users}
      showRole
      editBase="/users"
      onChange={handleChanged}
      onError={handleError}
    />
  )}
</div>
</>
</>
);
}

```

web/src/styles/theme.css

```

/*
 * =====
 * File       : theme.css
 * Project    : SunShare - Smart Solar Microgrid Trading System
 * Module     : SE4040 Enterprise Application Development - Assignment 1
 * Part       : B - Shared foundation (web shell)
 * Author     : Ranathunga R A K N (IT22552860)
 * Created    : 2026-09-29
 * Description : SunShare brand colours and small layout styles laid over
 *              Bootstrap 5 (design tokens from docs/05-SCREENS.md §1).
 * =====
 */

/* ----- Brand tokens ----- */
:root {
  --ss-primary: #0f766e;          /* Deep Teal - buttons, links, active menu, Approved */
  --ss-primary-dark: #0b5f58;
  --ss-primary-soft: #e6f4f2;
  --ss-accent: #f59e0b;          /* Solar Amber - sun logo, highlights, Pending (dark text on it) */
  --ss-accent-soft: #fef3c7;
  --ss-dark: #0b1f33;           /* Night Navy - sidebar, headings */
  --ss-bg: #f8fafc;             /* page background */
  --ss-success: #16a34a;
  --ss-danger: #dc2626;
  --ss-info: #0284c7;
  --ss-muted: #64748b;
  --ss-radius: 12px;
  --ss-shadow: 0 1px 3px rgba(15, 23, 42, 0.08), 0 1px 2px rgba(15, 23, 42, 0.04);
  --ss-shadow-lg: 0 12px 32px rgba(11, 31, 51, 0.12);
}

```

```

/* Tell Bootstrap to use our font, background and brand colour. */
--bs-body-font-family: 'Inter', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
--bs-body-bg: var(--ss-bg);
--bs-body-color: #1e293b;
--bs-primary: var(--ss-primary);
--bs-primary-rgb: 15, 118, 110;
--bs-link-color: var(--ss-primary);
--bs-link-color-rgb: 15, 118, 110;
--bs-link-hover-color: var(--ss-primary-dark);
--bs-link-hover-color-rgb: 11, 95, 88;
}

h1, h2, h3, h4, h5, h6,
.h1, .h2, .h3, .h4, .h5, .h6 {
  color: var(--ss-dark);
  font-weight: 700;
  letter-spacing: -0.01em;
}

/* ----- Bootstrap components in brand colours ----- */
.btn-primary {
  --bs-btn-bg: var(--ss-primary);
  --bs-btn-border-color: var(--ss-primary);
  --bs-btn-hover-bg: var(--ss-primary-dark);
  --bs-btn-hover-border-color: var(--ss-primary-dark);
  --bs-btn-active-bg: var(--ss-primary-dark);
  --bs-btn-active-border-color: var(--ss-primary-dark);
  --bs-btn-disabled-bg: var(--ss-primary);
  --bs-btn-disabled-border-color: var(--ss-primary);
  --bs-btn-focus-shadow-rgb: 15, 118, 110;
}

.btn-outline-primary {
  --bs-btn-color: var(--ss-primary);
  --bs-btn-border-color: var(--ss-primary);
  --bs-btn-hover-bg: var(--ss-primary);
  --bs-btn-hover-border-color: var(--ss-primary);
  --bs-btn-active-bg: var(--ss-primary-dark);
  --bs-btn-active-border-color: var(--ss-primary-dark);
  --bs-btn-focus-shadow-rgb: 15, 118, 110;
}

/* Amber button: dark text on amber for contrast (never white on amber). */
.btn-accent {
  --bs-btn-color: var(--ss-dark);
  --bs-btn-bg: var(--ss-accent);
  --bs-btn-border-color: var(--ss-accent);
  --bs-btn-hover-color: var(--ss-dark);
  --bs-btn-hover-bg: #fbbf24;
  --bs-btn-hover-border-color: #fbbf24;
  --bs-btn-active-bg: #d97706;
  --bs-btn-active-border-color: #d97706;
  --bs-btn-focus-shadow-rgb: 245, 158, 11;
  font-weight: 600;
}

.form-control:focus,
.form-select:focus {
  border-color: #5eb3ab;
  box-shadow: 0 0 0 0.25rem rgba(15, 118, 110, 0.2);
}

.form-check-input:checked {
  background-color: var(--ss-primary);
  border-color: var(--ss-primary);
}

.card {
  --bs-card-bg: #fff;
  --bs-card-border-radius: var(--ss-radius);
  --bs-card-inner-border-radius: var(--ss-radius);
  border: 0;
  box-shadow: var(--ss-shadow);
}

.table > :not(caption) > * > th {
  color: var(--ss-muted);
  font-size: 0.8rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.03em;
}

```

```

/* ----- Status badges (StatusBadge.jsx) ----- */
.status-badge { font-weight: 600; padding: 0.4em 0.7em; }
.status-pending { background: var(--ss-accent); color: var(--ss-dark); }
.status-approved { background: var(--ss-primary); color: #fff; }
.status-completed,
.status-active { background: var(--ss-success); color: #fff; }
.status-cancelled { background: #94a3b8; color: #fff; }
.status-deactivated { background: var(--ss-danger); color: #fff; }
.status-open { background: var(--ss-success); color: #fff; } /* slot open for booking */
.status-closed { background: #94a3b8; color: #fff; } /* slot closed by staff */

/* ----- Dashboard count cards (Dashboard.jsx) ----- */
.stat-icon {
  width: 3rem;
  height: 3rem;
  border-radius: 12px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  font-size: 1.4rem;
  flex-shrink: 0;
}
.stat-number { font-size: 1.9rem; font-weight: 800; line-height: 1.1; color: var(--ss-dark); }
.tone-amber { background: var(--ss-accent-soft); color: #b45309; }
.tone-teal { background: var(--ss-primary-soft); color: var(--ss-primary); }
.tone-blue { background: #e0f2fe; color: var(--ss-info); }
.tone-green { background: #dcfce7; color: var(--ss-success); }
.tone-navy { background: #e2e8f0; color: var(--ss-dark); }
.stat-card:hover { box-shadow: var(--ss-shadow-lg); }

/* ----- Keyboard focus and skip link (accessibility) ----- */
:where(a, button, .nav-link):focus-visible {
  outline: 3px solid var(--ss-accent);
  outline-offset: 2px;
}

.skip-link {
  position: fixed;
  top: 0.5rem;
  left: 0.5rem;
  z-index: 2000;
  padding: 0.5rem 1rem;
  background: var(--ss-dark);
  color: #fff;
  border-radius: 8px;
}

/* ----- App shell: sidebar + content (AppLayout.jsx, Sidebar.jsx) ----- */
.app-shell {
  display: flex;
  min-height: 100vh;
}

.app-sidebar {
  width: 260px;
  flex-shrink: 0;
  background: var(--ss-dark);
  color: #cbd5e1;
  position: sticky;
  top: 0;
  height: 100vh;
  overflow-y: auto;
  display: flex;
  flex-direction: column;
}

.app-sidebar .brand {
  color: #fff;
  font-weight: 800;
  font-size: 1.25rem;
  text-decoration: none;
}

.app-sidebar .brand .bi-sun-fill { color: var(--ss-accent); }

.app-sidebar .nav-link {
  color: #cbd5e1;
  border-radius: 8px;
  padding: 0.55rem 0.75rem;
  display: flex;
  align-items: center;
  gap: 0.6rem;
}

```

```

}

.app-sidebar .nav-link:hover { background: rgba(255, 255, 255, 0.08); color: #fff; }

.app-sidebar .nav-link.active { background: var(--ss-primary); color: #fff; }

.app-sidebar .sidebar-body {
  display: flex;
  flex-direction: column;
  flex-grow: 1;
}

.app-sidebar .user-box {
  border-top: 1px solid rgba(255, 255, 255, 0.1);
  font-size: 0.875rem;
}

.app-content {
  flex-grow: 1;
  min-width: 0;          /* lets wide tables scroll instead of stretching the page */
}

.app-content main:focus { outline: none; }

/* Small screens: the sidebar becomes a top bar and the links open with the menu button. */
@media (max-width: 991.98px) {
  .app-shell { flex-direction: column; }
  .app-sidebar { width: 100%; height: auto; position: sticky; z-index: 1020; }
  .app-sidebar .sidebar-body { display: none; }
  .app-sidebar.open .sidebar-body { display: flex; }
}

.page-subtitle { color: var(--ss-muted); }

/* ----- Login page (Login.jsx): same brand background as the Home hero ----- */
.login-page {
  background:
    radial-gradient(circle at 85% 20%, rgba(245, 158, 11, 0.35), transparent 40%),
    linear-gradient(135deg, var(--ss-dark) 0%, #123a55 55%, var(--ss-primary) 100%);
}

/* ----- Home page (Home.jsx) ----- */
.home-hero {
  background:
    radial-gradient(circle at 85% 20%, rgba(245, 158, 11, 0.35), transparent 40%),
    linear-gradient(135deg, var(--ss-dark) 0%, #123a55 55%, var(--ss-primary) 100%);
  color: #e2e8f0;
  overflow: hidden; /* the decorative sun may stick out of the edge */
}

.home-hero h1 { color: #fff; font-size: clamp(2.2rem, 5vw, 3.6rem); font-weight: 800; }

.home-hero .lead { color: #cbd5e1; }

.hero-sun {
  width: 7rem;
  height: 7rem;
  border-radius: 50%;
  background: radial-gradient(circle, #fde68a 0%, var(--ss-accent) 60%, #d97706 100%);
  box-shadow: 0 0 60px rgba(245, 158, 11, 0.6);
}

.hero-card { border-radius: 16px; box-shadow: var(--ss-shadow-lg); }

.step-number {
  width: 2.5rem;
  height: 2.5rem;
  border-radius: 50%;
  background: var(--ss-accent-soft);
  color: #92400e;
  font-weight: 800;
  display: inline-flex;
  align-items: center;
  justify-content: center;
}

.feature-icon {
  width: 3rem;
  height: 3rem;
}

```

```

border-radius: 12px;
background: var(--ss-primary-soft);
color: var(--ss-primary);
font-size: 1.5rem;
display: inline-flex;
align-items: center;
justify-content: center;
}

.role-where { background: var(--ss-primary-soft); color: var(--ss-primary-dark); font-weight: 600; }

.home-cta { background: var(--ss-primary); color: #fff; }
.home-cta h2 { color: #fff; }

.home-footer { background: var(--ss-dark); color: #94a3b8; }
.home-footer a { color: #e2e8f0; }

@media (prefers-reduced-motion: reduce) {
  * { scroll-behavior: auto !important; transition: none !important; }
}

```

web/src/utils/format.js

```

/*
 * =====
 * File      : format.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description: Display helpers. The API sends times in UTC; these show them
 *              in the browser's local time (Sri Lanka) - display only, no rules.
 * =====
 */

// "2026-09-29T05:30:00Z" -> "Tue, 29 Sep, 11:00" (local time). Empty text for no value.
export function formatDate(isoText) {
  if (!isoText) {
    return '';
  }
  return new Date(isoText).toLocaleString('en-GB', {
    weekday: 'short',
    day: '2-digit',
    month: 'short',
    hour: '2-digit',
    minute: '2-digit',
    hour12: false,
  });
}

// "2026-09-29T05:30:00Z" -> "11:00" (local time only).
export function formatTime(isoText) {
  if (!isoText) {
    return '';
  }
  return new Date(isoText).toLocaleTimeString('en-GB', { hour: '2-digit', minute: '2-digit', hour12: false });
}

// Link that opens Google Maps at a GPS point (used to check a station's pin).
// Reference: Google, "Maps URLs" https://developers.google.com/maps/documentation/urls/get-started
export function googleMapUrl(latitude, longitude) {
  return `https://www.google.com/maps/search/?api=1&query=${latitude},${longitude}`;
}

// "2026-09-29T05:30:00Z" -> "Tue, 29 Sep" (local date only).
export function formatDate(isoText) {
  if (!isoText) {
    return '';
  }
  return new Date(isoText).toLocaleDateString('en-GB', { weekday: 'short', day: '2-digit', month: 'short' });
}

// UTC time from the API -> the "YYYY-MM-DDTHH:mm" local text a <input type="datetime-local"> expects.
export function toDateTimeInput(isoText) {
  const date = new Date(isoText);
  // Two digits: 7 -> "07".
  const pad = (number) => String(number).padStart(2, '0');
  return `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())}T${pad(date.getHours())}:${pad(date.getMinutes())}`;
}

```

```
// The local text from a datetime-local input -> UTC ISO text for the API (the API works in UTC).
export function fromDateTimeInput(localText) {
  return new Date(localText).toISOString();
}

// A <input type="date"> value ("2026-09-30", a local day) -> UTC ISO text of that day's local midnight,
// moved by addDays (the "to" filter uses the next day, so the whole chosen day is included).
export function fromDateInput(dateText, addDays = 0) {
  const date = new Date(`${dateText}T00:00`);
  date.setDate(date.getDate() + addDays);
  return date.toISOString();
}
```

web/vite.config.js

```
/*
 * =====
 * File      : vite.config.js
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (web shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-27
 * Description: Vite settings: turn on the React plugin and run the dev
 *              server on port 5173 (the API's CORS list allows this port),
 *              unless a PORT environment variable asks for another one.
 * =====
 */
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// Reference: Vite, "Configuring Vite" https://vite.dev/config/
// Port: 5173 by default; a tool (e.g. a preview browser) can set PORT to pick a free one.
// strictPort = fail instead of silently moving to another port, which the API would block (CORS).
export default defineConfig({
  plugins: [react()],
  server: {
    port: Number(process.env.PORT) || 5173,
    strictPort: true,
  },
})
```

A.3 Android app - Java

android/app/src/main/java/com/sunshare/app/api/ApiClient.java

```
/*
 * =====
 * File      : ApiClient.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description: Builds the one Retrofit object the whole app uses to call
 *              the API. Before every request it adds the header
 *              "Authorization: Bearer <token>" from the SQLite session,
 *              and Gson turns the JSON answers into the model classes.
 * =====
 */
package com.sunshare.app.api;

import android.content.Context;

import com.sunshare.app.db.Session;
import com.sunshare.app.db.SunShareDbHelper;

import java.io.IOException;
import java.util.concurrent.TimeUnit;

import okhttp3.Interceptor;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.Response;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;
```

```

public final class ApiClient {

    // Built once and reused; thrown away by reset() when the server address changes.
    private static ApiService service;

    // Only static helpers - nobody needs to create an ApiClient object.
    private ApiClient() {
    }

    // Returns the API service, building it the first time (with the saved server address).
    public static synchronized ApiService get(Context context) {
        if (service == null) {
            Context app = context.getApplicationContext();
            service = build(app, ApiConfig.getBaseUrl(app));
        }
        return service;
    }

    // Forgets the built service so the next get() uses the new server address.
    public static synchronized void reset() {
        service = null;
    }

    // Creates OkHttp (the network engine, 15 s time limits, token added) and Retrofit on top of it.
    // Reference: Retrofit - A type-safe HTTP client for Android and Java https://square.github.io/retrofit/
    private static ApiService build(Context app, String baseUrl) {
        OkHttpClient http = new OkHttpClient.Builder()
            .connectTimeout(15, TimeUnit.SECONDS)
            .readTimeout(15, TimeUnit.SECONDS)
            .writeTimeout(15, TimeUnit.SECONDS)
            .addInterceptor(chain -> addToken(app, chain))
            .build();

        Retrofit retrofit = new Retrofit.Builder()
            .baseUrl(baseUrl)
            .client(http)
            .addConverterFactory(GsonConverterFactory.create())
            .build();

        return retrofit.create(ApiService.class);
    }

    // Runs before every request: if someone is logged in, adds their JWT token so the API knows who is calling (R2).
    // Reference: OkHttp Interceptors https://square.github.io/okhttp/features/interceptors/
    private static Response addToken(Context app, Interceptor.Chain chain) throws IOException {
        Request request = chain.request();
        Session session = SunShareDbHelper.get(app).getSession();
        if (session != null && session.token != null) {
            request = request.newBuilder()
                .header("Authorization", "Bearer " + session.token)
                .build();
        }
        return chain.proceed(request);
    }
}

```

android/app/src/main/java/com/sunshare/app/api/ApiConfig.java

```

/*
 * =====
 * File      : ApiConfig.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Where the SunShare API lives. Default is the emulator's
 *               address for the laptop (10.0.2.2) on the IIS port 8080; a
 *               real phone uses the laptop's Wi-Fi address, changed with the
 *               Server address setting and saved in SQLite (app_settings).
 * =====
 */
package com.sunshare.app.api;

import android.content.Context;

import com.sunshare.app.db.SunShareDbHelper;

import okhttp3.HttpUrl;

public final class ApiConfig {

```



```

public static final String DEFAULT_BASE_URL = "http://10.0.2.2:8080/";
private static final String SETTING_KEY = "api_base_url";

// Only static helpers - nobody needs to create an ApiConfig object.
private ApiConfig() {
}

// Returns the saved server address, or the emulator default when none was saved.
public static String getBaseUrl(Context context) {
    String saved = SunShareDbHelper.get(context).getSetting(SETTING_KEY);
    return (saved == null || saved.isEmpty()) ? DEFAULT_BASE_URL : saved;
}

// Saves a new server address and makes the next API call use it.
public static void setBaseUrl(Context context, String url) {
    SunShareDbHelper.get(context).saveSetting(SETTING_KEY, normalize(url));
    ApiClient.reset();
}

// True when the typed address can be used (e.g. "192.168.1.7:8080" or "http://192.168.1.7:8080/").
public static boolean isValidBaseUrl(String url) {
    return url != null && !url.trim().isEmpty() && HttpUrl.parse(normalize(url)) != null;
}

// Adds "http://" if missing and a "/" at the end (Retrofit needs the ending slash).
public static String normalize(String url) {
    String clean = url == null ? "" : url.trim();
    if (!clean.startsWith("http://") && !clean.startsWith("https://")) {
        clean = "http://" + clean;
    }
    if (!clean.endsWith("/")) {
        clean = clean + "/";
    }
    return clean;
}
}

```

android/app/src/main/java/com/sunshare/app/api/ApiErrorParser.java

```

/*
 * =====
 * File      : ApiErrorParser.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Turns a failed API call into a human-friendly message.
 *              The API always sends errors as { "message": "..."}
 *              (docs/02-ARCHITECTURE.md section 6), so the app just shows
 *              that text - it never makes up its own rule messages.
 * =====
 */
package com.sunshare.app.api;

import android.content.Context;

import com.google.gson.Gson;
import com.sunshare.app.api.models.MessageResponse;

import okhttp3.ResponseBody;
import retrofit2.Response;

public final class ApiErrorParser {

    // Only static helpers - nobody needs to create an ApiErrorParser object.
    private ApiErrorParser() {
    }

    // The API answered with an error (400, 401, 403, 404, 409, 500...): returns its "message".
    // If the body is not our JSON (e.g. an IIS error page), returns a general text with the HTTP code.
    public static String fromResponse(Response<?> response) {
        try (ResponseBody body = response.errorBody()) {
            if (body != null) {
                MessageResponse error = new Gson().fromJson(body.charStream(), MessageResponse.class);
                if (error != null && error.message != null && !error.message.isEmpty()) {
                    return error.message;
                }
            }
        } catch (Exception notJson) {
    }
}

```

```

        // Not our { "message" } JSON - fall through to the general text below.
    }
    return "The server answered with an error (HTTP " + response.code() + ").";
}

// The call never reached the API (server off, wrong address, phone not on the same Wi-Fi).
public static String fromFailure(Context context) {
    return "Can't reach the SunShare server at " + ApiConfig.getBaseUrl(context)
        + ". Check the server address (Ⓢ on the Login screen) and that the phone is on the same Wi-Fi as the
laptop.";
}
}

```

android/app/src/main/java/com/sunshare/app/api/ApiService.java

```

/*
 * =====
 * File      : ApiService.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Every API endpoint the Android screens (M1-M11 in
 *               docs/05-SCREENS.md) call, one line each. Retrofit writes
 *               the networking code from these annotations. Backoffice-only
 *               endpoints are left out: the web app is used for those.
 * =====
 */
package com.sunshare.app.api;

import com.sunshare.app.api.models.CreateReservationRequest;
import com.sunshare.app.api.models.HealthResponse;
import com.sunshare.app.api.models.LoginRequest;
import com.sunshare.app.api.models.LoginResponse;
import com.sunshare.app.api.models.MessageResponse;
import com.sunshare.app.api.models.ProsumerDashboardResponse;
import com.sunshare.app.api.models.RegisterRequest;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.api.models.SlotResponse;
import com.sunshare.app.api.models.StaffDashboardResponse;
import com.sunshare.app.api.models.StationResponse;
import com.sunshare.app.api.models.UpdateProfileRequest;
import com.sunshare.app.api.models.UpdateReservationRequest;
import com.sunshare.app.api.models.UserResponse;
import com.sunshare.app.api.models.VerifyQrRequest;

import java.util.List;

import retrofit2.Call;
import retrofit2.http.Body;
import retrofit2.http.GET;
import retrofit2.http.PATCH;
import retrofit2.http.POST;
import retrofit2.http.PUT;
import retrofit2.http.Path;
import retrofit2.http.Query;

// A Retrofit interface has no method bodies: each line maps one Java method to one HTTP call.
// Reference: Retrofit - A type-safe HTTP client for Android and Java https://square.github.io/retrofit/
public interface ApiService {

    // ----- Health (shared) -----

    // GET /api/health - is the API up and connected to MongoDB?
    @GET("api/health")
    Call<HealthResponse> health();

    // ----- Auth + profile (Part A) -----

    // POST /api/auth/login - NIC + password -> token and role (R3, R4).
    @POST("api/auth/login")
    Call<LoginResponse> login(@Body LoginRequest body);

    // POST /api/auth/register - a new prosumer account, status Pending (R1, R3).
    @POST("api/auth/register")
    Call<UserResponse> register(@Body RegisterRequest body);

    // GET /api/profile - the logged-in user's own details (R16).
    @GET("api/profile")

```

```

Call<UserResponse> getProfile();

// PUT /api/profile - save the prosumer's own details (R16).
@PUT("api/profile")
Call<UserResponse> updateProfile(@Body UpdateProfileRequest body);

// PATCH /api/profile/deactivate - the prosumer closes their own account (R5).
@PATCH("api/profile/deactivate")
Call<MessageResponse> deactivateProfile();

// ----- Stations + slots (Part B) -----

// GET /api/stations?activeOnly= - stations for the booking dropdown (null = leave the filter out).
@GET("api/stations")
Call<List<StationResponse>> getStations(@Query("activeOnly") Boolean activeOnly);

// GET /api/stations/nearby?lat=&lng=&radiusKm= - active stations near the phone, nearest first (R18).
@GET("api/stations/nearby")
Call<List<StationResponse>> getNearbyStations(@Query("lat") double lat, @Query("lng") double lng,
                                             @Query("radiusKm") Double radiusKm);

// GET /api/stations/{id} - one station.
@GET("api/stations/{id}")
Call<StationResponse> getStation(@Path("id") String id);

// GET /api/slots/available?stationId= - bookable slots in the next 7 days with a free place (R9, R11).
@GET("api/slots/available")
Call<List<SlotResponse>> getAvailableSlots(@Query("stationId") String stationId);

// GET /api/slots/{id} - one slot.
@GET("api/slots/{id}")
Call<SlotResponse> getSlot(@Path("id") String id);

// ----- Reservations: actions (Part C) -----

// POST /api/reservations - book a slot (the API checks R9, R11, R12, R13, R16, R17).
@POST("api/reservations")
Call<ReservationResponse> createReservation(@Body CreateReservationRequest body);

// PUT /api/reservations/{id} - change a booking (R10: at least 12 hours before).
@PUT("api/reservations/{id}")
Call<ReservationResponse> updateReservation(@Path("id") String id, @Body UpdateReservationRequest body);

// PATCH /api/reservations/{id}/cancel - cancel a booking (R10, R12, R15).
@PATCH("api/reservations/{id}/cancel")
Call<ReservationResponse> cancelReservation(@Path("id") String id);

// ----- Reservations: views, QR and dashboards (Part D) -----

// GET /api/reservations?view=&search= - the prosumer's bookings (view = current, pending or history).
@GET("api/reservations")
Call<List<ReservationResponse>> getReservations(@Query("view") String view, @Query("search") String search);

// GET /api/reservations/{id} - one booking, with qrData when Approved.
@GET("api/reservations/{id}")
Call<ReservationResponse> getReservation(@Path("id") String id);

// POST /api/reservations/verify-qr - the operator checks a scanned QR with the server (R14).
@POST("api/reservations/verify-qr")
Call<ReservationResponse> verifyQr(@Body VerifyQrRequest body);

// PATCH /api/reservations/{id}/complete - the operator finalizes the energy transfer (R14).
@PATCH("api/reservations/{id}/complete")
Call<ReservationResponse> completeReservation(@Path("id") String id, @Body VerifyQrRequest body);

// GET /api/dashboard/summary - Grid Operator home counts.
@GET("api/dashboard/summary")
Call<StaffDashboardResponse> getStaffDashboard();

// GET /api/dashboard/prosumer - Prosumer home counts and next booking.
@GET("api/dashboard/prosumer")
Call<ProsumerDashboardResponse> getProsumerDashboard();
}

```

android/app/src/main/java/com/sunshare/app/api/models/CreateReservationRequest.java

```

/*
 * =====
 * File      : CreateReservationRequest.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : JSON body for POST /api/reservations. A prosumer leaves
 *              prosumerNic empty; the API takes the NIC from the token (R16).
 * =====
 */
package com.sunshare.app.api.models;

// slotId + energy + type ("Sell" or "Buy").
public class CreateReservationRequest {
    public String prosumerNic;
    public String slotId;
    public double energyKwh;
    public String type;

    // Builds a booking request for the logged-in prosumer (no prosumerNic needed).
    public CreateReservationRequest(String slotId, double energyKwh, String type) {
        this.slotId = slotId;
        this.energyKwh = energyKwh;
        this.type = type;
    }
}

```

android/app/src/main/java/com/sunshare/app/api/models/HealthResponse.java

```

/*
 * =====
 * File      : HealthResponse.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Answer of GET /api/health: is the API running and
 *              connected to MongoDB?
 * =====
 */
package com.sunshare.app.api.models;

// Gson fills these fields from the JSON { "api": "ok", "database": "connected", "time": "..." }.
public class HealthResponse {
    public String api;
    public String database;
    public String time;
}

```

android/app/src/main/java/com/sunshare/app/api/models/LoginRequest.java

```

/*
 * =====
 * File      : LoginRequest.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : JSON body for POST /api/auth/login.
 * =====
 */
package com.sunshare.app.api.models;

// What the user typed on the Login screen.
public class LoginRequest {
    public String nic;
    public String password;

    // Builds the request from the two text boxes.
    public LoginRequest(String nic, String password) {
        this.nic = nic;
        this.password = password;
    }
}

```

android/app/src/main/java/com/sunshare/app/api/models/LoginResponse.java

```

/*
 * =====
 * File      : LoginResponse.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Answer of a successful login: the JWT token plus who
 *              the user is (saved in the SQLite session table).
 * =====
 */
package com.sunshare.app.api.models;

// { token, nic, fullName, role, expiresAt } - the role decides which home screen opens.
public class LoginResponse {
    public String token;
    public String nic;
    public String fullName;
    public String role;
    public String expiresAt;
}

```

android/app/src/main/java/com/sunshare/app/api/models/MessageResponse.java

```

/*
 * =====
 * File      : MessageResponse.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : A JSON answer that only has a message: every API error
 *              and simple answers like "Account deactivated.".
 * =====
 */
package com.sunshare.app.api.models;

// { "message": "human friendly text" } - the shape of every API error (docs/02-ARCHITECTURE.md section 6).
public class MessageResponse {
    public String message;
}

```

android/app/src/main/java/com/sunshare/app/api/models/ProsumerDashboardResponse.java

```

/*
 * =====
 * File      : ProsumerDashboardResponse.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Answer of GET /api/dashboard/prosumer (Prosumer home):
 *              the prosumer's own counts and next booking.
 * =====
 */
package com.sunshare.app.api.models;

// nextReservation is null when the prosumer has no upcoming booking.
public class ProsumerDashboardResponse {
    public long pendingCount;
    public long approvedFutureCount;
    public long completedCount;
    public ReservationResponse nextReservation;
}

```

android/app/src/main/java/com/sunshare/app/api/models/RegisterRequest.java

```

/*
 * =====
 * File      : RegisterRequest.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : JSON body for POST /api/auth/register (prosumer sign-up)

```

```

*           on the Register screen).
* =====
*/
package com.sunshare.app.api.models;

// The new prosumer's details. The API makes the account a Pending Prosumer (R3).
public class RegisterRequest {
    public String nic;
    public String fullName;
    public String email;
    public String phone;
    public String address;
    public String password;

    // Builds the request from the Register form.
    public RegisterRequest(String nic, String fullName, String email, String phone, String address, String password) {
        this.nic = nic;
        this.fullName = fullName;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.password = password;
    }
}

```

android/app/src/main/java/com/sunshare/app/api/models/ReservationResponse.java

```

/*
* =====
* File       : ReservationResponse.java
* Project    : SunShare - Smart Solar Microgrid Trading System
* Module     : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (Android shell)
* Author     : Ranathunga R A K N (IT22552860)
* Created    : 2026-09-29
* Description : A booking as the API sends it, including canModify (show
*               Edit/Cancel?) and qrData (QR text, only when Approved).
* =====
*/
package com.sunshare.app.api.models;

// Every booking screen reads this; the app never works out the rules itself (canModify comes from the API).
public class ReservationResponse {
    public String id;
    public String prosumerNic;
    public String prosumerName;
    public String stationId;
    public String stationName;
    public String slotId;
    public String startTime;
    public String endTime;
    public double energyKwh;
    public String type;
    public String status;
    public boolean canModify;
    public String qrData;
    public String approvedBy;
    public String approvedAt;
    public String completedBy;
    public String completedAt;
    public String cancelledBy;
    public String cancelledAt;
    public String createdAt;
    public String updatedAt;
}

```

android/app/src/main/java/com/sunshare/app/api/models/SlotResponse.java

```

/*
* =====
* File       : SlotResponse.java
* Project    : SunShare - Smart Solar Microgrid Trading System
* Module     : SE4040 Enterprise Application Development - Assignment 1
* Part      : B - Shared foundation (Android shell)
* Author     : Ranathunga R A K N (IT22552860)
* Created    : 2026-09-29
* Description : A bookable time window at a station (times in UTC).
* =====
*/
package com.sunshare.app.api.models;

// { id, stationId, stationName, startTime, endTime, totalSlots, availableSlots, bookedSlots, isActive }.

```

```

public class SlotResponse {
    public String id;
    public String stationId;
    public String stationName;
    public String startTime;
    public String endTime;
    public int totalSlots;
    public int availableSlots;
    public int bookedSlots;
    public boolean isActive;
}

```

android/app/src/main/java/com/sunshare/app/api/models/StaffDashboardResponse.java

```

/*
 * =====
 * File      : StaffDashboardResponse.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Answer of GET /api/dashboard/summary (Grid Operator home):
 *              counts worked out by the API.
 * =====
 */
package com.sunshare.app.api.models;

import java.util.List;

// Count cards + the next pending bookings (docs/01-SPEC.md section 5).
public class StaffDashboardResponse {
    public long pendingReservations;
    public long approvedFutureReservations;
    public long todayReservations;
    public long activeStations;
    public long pendingActivations;
    public List<ReservationResponse> pendingList;
}

```

android/app/src/main/java/com/sunshare/app/api/models/StationResponse.java

```

/*
 * =====
 * File      : StationResponse.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : A solar station as the API sends it; distanceKm is only
 *              filled by GET /api/stations/nearby (R18).
 * =====
 */
package com.sunshare.app.api.models;

// One station: position, size, hours, status, active bookings and (for /nearby) the distance in km.
public class StationResponse {
    public String id;
    public String name;
    public String address;
    public double latitude;
    public double longitude;
    public double capacityKw;
    public int batterySlots;
    public String openTime;
    public String closeTime;
    public boolean isActive;
    public long activeReservationCount;
    public Double distanceKm;
}

```

android/app/src/main/java/com/sunshare/app/api/models/UpdateProfileRequest.java

```

/*
 * =====
 * File      : UpdateProfileRequest.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)

```

```

* Created      : 2026-09-29
* Description  : JSON body for PUT /api/profile: the prosumer edits their
*                own details; newPassword is optional.
* =====
*/
package com.sunshare.app.api.models;

// newPassword = null keeps the current password.
public class UpdateProfileRequest {
    public String fullName;
    public String email;
    public String phone;
    public String address;
    public String newPassword;

    // Builds the request from the Profile form.
    public UpdateProfileRequest(String fullName, String email, String phone, String address, String newPassword) {
        this.fullName = fullName;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.newPassword = newPassword;
    }
}

```

android/app/src/main/java/com/sunshare/app/api/models/UpdateReservationRequest.java

```

/*
* =====
* File          : UpdateReservationRequest.java
* Project       : SunShare - Smart Solar Microgrid Trading System
* Module       : SE4040 Enterprise Application Development - Assignment 1
* Part        : B - Shared foundation (Android shell)
* Author       : Ranathunga R A K N (IT22552860)
* Created      : 2026-09-29
* Description   : JSON body for PUT /api/reservations/{id}: a new slot and/or
*                energy amount and type.
* =====
*/
package com.sunshare.app.api.models;

// The editable parts of a booking.
public class UpdateReservationRequest {
    public String slotId;
    public double energyKwh;
    public String type;

    // Builds the edit request from the Reservation form.
    public UpdateReservationRequest(String slotId, double energyKwh, String type) {
        this.slotId = slotId;
        this.energyKwh = energyKwh;
        this.type = type;
    }
}

```

android/app/src/main/java/com/sunshare/app/api/models/UserResponse.java

```

/*
* =====
* File          : UserResponse.java
* Project       : SunShare - Smart Solar Microgrid Trading System
* Module       : SE4040 Enterprise Application Development - Assignment 1
* Part        : B - Shared foundation (Android shell)
* Author       : Ranathunga R A K N (IT22552860)
* Created      : 2026-09-29
* Description   : An account as the API sends it (it never contains the
*                password hash).
* =====
*/
package com.sunshare.app.api.models;

// { nic, fullName, email, phone, address, role, status, createdAt, updatedAt }.
public class UserResponse {
    public String nic;
    public String fullName;
    public String email;
    public String phone;
    public String address;
    public String role;
    public String status;
    public String createdAt;
    public String updatedAt;
}

```



```
}
```

android/app/src/main/java/com/sunshare/app/api/models/VerifyQrRequest.java

```
/*
 * =====
 * File      : VerifyQrRequest.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : JSON body for POST /api/reservations/verify-qr and
 *              PATCH /api/reservations/{id}/complete: the scanned QR text.
 * =====
 */
package com.sunshare.app.api.models;

// { "qrData": "SUNSHARE|<reservationId>|<token>" } (R14).
public class VerifyQrRequest {
    public String qrData;

    // Wraps the text the camera read from the QR code.
    public VerifyQrRequest(String qrData) {
        this.qrData = qrData;
    }
}
```

android/app/src/main/java/com/sunshare/app/db/Session.java

```
/*
 * =====
 * File      : Session.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : The logged-in user as saved in the SQLite "session" table:
 *              NIC, name, role and the JWT token that is sent with every
 *              API call.
 * =====
 */
package com.sunshare.app.db;

import com.sunshare.app.api.models.LoginResponse;

import java.time.Instant;

public class Session {

    public static final String ROLE_PROSUMER = "Prosumer";
    public static final String ROLE_GRID_OPERATOR = "GridOperator";
    public static final String ROLE_BACKOFFICE = "Backoffice";

    public final String nic;
    public final String fullName;
    public final String role;
    public final String token;
    public final String loggedInAt;

    // Builds a session from the five columns of the session table.
    public Session(String nic, String fullName, String role, String token, String loggedInAt) {
        this.nic = nic;
        this.fullName = fullName;
        this.role = role;
        this.token = token;
        this.loggedInAt = loggedInAt;
    }

    // Builds a session from the API's login answer, stamped with the current time.
    public static Session fromLogin(LoginResponse login) {
        return new Session(login.nic, login.fullName, login.role, login.token, Instant.now().toString());
    }

    // True when the logged-in user is a Prosumer.
    public boolean isProsumer() {
        return ROLE_PROSUMER.equals(role);
    }
}
```

```

// True when the logged-in user is a Grid Operator.
public boolean isGridOperator() {
    return ROLE_GRID_OPERATOR.equals(role);
}
}

```

android/app/src/main/java/com/sunshare/app/db/SunShareDbHelper.java

```

/*
 * =====
 * File      : SunShareDbHelper.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : The phone's own small SQLite database "sunshare_local.db"
 *              (docs/02-ARCHITECTURE.md section 9). Three tables:
 *              session (who is logged in + their JWT token),
 *              stations_cache (stations for the map when offline) and
 *              app_settings (the server address). The real data lives in
 *              MongoDB behind the API; this is only a local helper store.
 * =====
 */
package com.sunshare.app.db;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

import com.sunshare.app.api.models.StationResponse;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

// Reference: Save data using SQLite (Android Developers) https://developer.android.com/training/data-storage/sqlite
public class SunShareDbHelper extends SQLiteOpenHelper {

    private static final String DB_NAME = "sunshare_local.db";
    private static final int DB_VERSION = 1;

    private static final String TABLE_SESSION = "session";
    private static final String TABLE_STATIONS = "stations_cache";
    private static final String TABLE_SETTINGS = "app_settings";

    // One shared helper for the whole app, so every screen reads the same database file.
    private static SunShareDbHelper instance;

    // Returns the shared helper, creating it the first time it is needed.
    public static synchronized SunShareDbHelper get(Context context) {
        if (instance == null) {
            instance = new SunShareDbHelper(context.getApplicationContext());
        }
        return instance;
    }

    // Private constructor: other classes use get(context) instead.
    private SunShareDbHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    // Runs once, the first time the app opens the database: creates the three tables.
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE_SESSION + " ("
            + "nic TEXT PRIMARY KEY, full_name TEXT, role TEXT, token TEXT, logged_in_at TEXT)");
        db.execSQL("CREATE TABLE " + TABLE_STATIONS + " ("
            + "id TEXT PRIMARY KEY, name TEXT, address TEXT, latitude REAL, longitude REAL, "
            + "capacity_kw REAL, battery_slots INTEGER, open_time TEXT, close_time TEXT, "
            + "distance_km REAL, cached_at TEXT)");
        db.execSQL("CREATE TABLE " + TABLE_SETTINGS + " (key TEXT PRIMARY KEY, value TEXT)");
    }

    // Runs when DB_VERSION goes up. Version 1 is the only version, so it simply rebuilds the tables.
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

```

```

        db.execSQL("DROP TABLE IF EXISTS " + TABLE_SESSION);
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_STATIONS);
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_SETTINGS);
        onCreate(db);
    }

    // ----- session (max 1 row) -----

    // Saves the logged-in user after a successful login. Old rows are removed first, so there is only ever one.
    public void saveSession(Session session) {
        SQLiteDatabase db = getWritableDatabase();
        db.delete(TABLE_SESSION, null, null);
        ContentValues values = new ContentValues();
        values.put("nic", session.nic);
        values.put("full_name", session.fullName);
        values.put("role", session.role);
        values.put("token", session.token);
        values.put("logged_in_at", session.loggedInAt);
        db.insert(TABLE_SESSION, null, values);
    }

    // Returns the saved session, or null when nobody is logged in.
    public Session getSession() {
        Cursor cursor = getReadableDatabase().query(TABLE_SESSION,
            new String[]{"nic", "full_name", "role", "token", "logged_in_at"},
            null, null, null, null, null, "1");
        try {
            if (!cursor.moveToFirst()) {
                return null;
            }
            return new Session(cursor.getString(0), cursor.getString(1), cursor.getString(2),
                cursor.getString(3), cursor.getString(4));
        } finally {
            cursor.close();
        }
    }

    // Forgets the logged-in user (logout, deactivated account, or the token expired - HTTP 401).
    public void clearSession() {
        getWritableDatabase().delete(TABLE_SESSION, null, null);
    }

    // ----- app_settings -----

    // Saves one setting (for example api_base_url). An existing value with the same key is replaced.
    public void saveSetting(String key, String value) {
        ContentValues values = new ContentValues();
        values.put("key", key);
        values.put("value", value);
        getWritableDatabase().insertWithOnConflict(TABLE_SETTINGS, null, values, SQLiteDatabase.CONFLICT_REPLACE);
    }

    // Returns one setting, or null if it was never saved.
    public String getSetting(String key) {
        Cursor cursor = getReadableDatabase().query(TABLE_SETTINGS, new String[]{"value"},
            "key = ?", new String[]{key}, null, null, null);
        try {
            return cursor.moveToFirst() ? cursor.getString(0) : null;
        } finally {
            cursor.close();
        }
    }

    // ----- stations_cache -----

    // Replaces the cached stations with the latest list from GET /api/stations/nearby.
    // A transaction makes the delete + inserts happen all together (or not at all).
    public void cacheStations(List<StationResponse> stations) {
        SQLiteDatabase db = getWritableDatabase();
        String now = Instant.now().toString();
        db.beginTransaction();
        try {
            db.delete(TABLE_STATIONS, null, null);
            for (StationResponse station : stations) {
                ContentValues values = new ContentValues();
                values.put("id", station.id);
                values.put("name", station.name);
                values.put("address", station.address);
                values.put("latitude", station.latitude);
                values.put("longitude", station.longitude);
                values.put("capacity_kw", station.capacityKw);
                values.put("battery_slots", station.batterySlots);
                values.put("open_time", station.openTime);
            }
        }
    }

```

```

        values.put("close_time", station.closeTime);
        values.put("distance_km", station.distanceKm);
        values.put("cached_at", now);
        db.insert(TABLE_STATIONS, null, values);
    }
    db.setTransactionSuccessful();
} finally {
    db.endTransaction();
}
}

// Returns the cached stations, nearest first (used by the map when the API can't be reached).
public List<StationResponse> getCachedStations() {
    List<StationResponse> stations = new ArrayList<>();
    Cursor cursor = getReadableDatabase().query(TABLE_STATIONS,
        new String[]{"id", "name", "address", "latitude", "longitude", "capacity_kw",
            "battery_slots", "open_time", "close_time", "distance_km"},
        null, null, null, null, "distance_km ASC");
    try {
        while (cursor.moveToNext()) {
            StationResponse station = new StationResponse();
            station.id = cursor.getString(0);
            station.name = cursor.getString(1);
            station.address = cursor.getString(2);
            station.latitude = cursor.getDouble(3);
            station.longitude = cursor.getDouble(4);
            station.capacityKw = cursor.getDouble(5);
            station.batterySlots = cursor.getInt(6);
            station.openTime = cursor.getString(7);
            station.closeTime = cursor.getString(8);
            station.distanceKm = cursor.isNull(9) ? null : cursor.getDouble(9);
            station.isActive = true; // only active stations are ever cached (R18)
            stations.add(station);
        }
    } finally {
        cursor.close();
    }
    return stations;
}

// Returns when the stations were cached (ISO UTC text), or null if the cache is empty.
public String getStationsCachedAt() {
    Cursor cursor = getReadableDatabase().query(TABLE_STATIONS, new String[]{"cached_at"},
        null, null, null, null, null, "1");
    try {
        return cursor.moveToFirst() ? cursor.getString(0) : null;
    } finally {
        cursor.close();
    }
}
}

```

android/app/src/main/java/com/sunshare/app/ui/auth/LoginActivity.java

```

/*
 * =====
 * File      : LoginActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-29
 * Description : M1 Login - the app's first screen. If a session is saved in
 *               SQLite the user goes straight to their home screen (stay
 *               logged in). Otherwise NIC + password -> POST /api/auth/login
 *               -> the session (token) is saved in SQLite -> Prosumer opens
 *               Prosumer Home (M3), Grid Operator opens Operator Home (M10),
 *               Backoffice is told to use the web app. The API decides
 *               wrong password / Pending (R3) / Deactivated (R4).
 *               The ☒ Server button changes the API address (saved in SQLite).
 * =====
 */
package com.sunshare.app.ui.auth;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.view.inputmethod.EditorInfo;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

```

```

import com.google.android.material.dialog.MaterialAlertDialogBuilder;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiConfig;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.HealthResponse;
import com.sunshare.app.api.models.LoginRequest;
import com.sunshare.app.api.models.LoginResponse;
import com.sunshare.app.db.Session;
import com.sunshare.app.db.SunShareDbHelper;
import com.sunshare.app.ui.operator.OperatorHomeActivity;
import com.sunshare.app.ui.prosumer.ProsumerHomeActivity;
import com.sunshare.app.util.UiUtils;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class LoginActivity extends AppCompatActivity {

    private EditText inputNic;
    private EditText inputPassword;
    private TextView textError;
    private TextView textServer;
    private Button buttonLogin;

    // Called when the app starts. A saved session skips the form (stay logged in);
    // otherwise the form, the Register link and the ☒ Server button are linked.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Session saved = SunShareDbHelper.get(this).getSession();
        if (saved != null) {
            openHome(saved);
            return;
        }

        setContentView(R.layout.activity_login);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), findViewById(R.id.header));
        inputNic = findViewById(R.id.inputNic);
        inputPassword = findViewById(R.id.inputPassword);
        textError = findViewById(R.id.textError);
        textServer = findViewById(R.id.textServer);
        buttonLogin = findViewById(R.id.buttonLogin);

        buttonLogin.setOnClickListener(v -> login());
        // Pressing "Done" on the keyboard in the password box also logs in.
        inputPassword.setOnEditorActionListener((v, actionId, event) -> {
            if (actionId == EditorInfo.IME_ACTION_DONE) {
                login();
                return true;
            }
            return false;
        });
        findViewById(R.id.buttonRegister).setOnClickListener(v -> startActivity(new Intent(this, RegisterActivity.class)));
        findViewById(R.id.buttonServer).setOnClickListener(v -> showServerDialog());
        showServerAddress();
    }

    // Checks both boxes are filled, then asks the API to log in.
    private void login() {
        UiUtils.hideMessage(textError);
        if (!UiUtils.requireFilled(inputNic, "Please enter your NIC.")
            || !UiUtils.requireFilled(inputPassword, "Please enter your password.")) {
            return;
        }
        LoginRequest body = new LoginRequest(UiUtils.textOf(inputNic), UiUtils.textOf(inputPassword));

        UiUtils.setLoading(buttonLogin, true);
        ApiClient.get(this).login(body).enqueue(new Callback<LoginResponse>() {
            // The API answered: 200 = logged in; 401 wrong NIC/password, 403 Pending (R3) or Deactivated (R4).
            @Override
            public void onResponse(Call<LoginResponse> call, Response<LoginResponse> response) {
                UiUtils.setLoading(buttonLogin, false);
                if (response.isSuccessful() && response.body() != null) {
                    onLoggedIn(response.body());
                } else {
                    UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
                }
            }
        })
    }
}

```

```

        // The call never reached the API - usually the server address (⊙) is wrong.
        @Override
        public void onFailure(Call<LoginResponse> call, Throwable error) {
            UiUtils.setLoading(buttonLogin, false);
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(LoginActivity.this));
        }
    });
}

// Saves the session in SQLite and opens the home screen for the user's role.
// Backoffice work is done on the web app, so a Backoffice session is not saved on the phone.
private void onLoggedIn(LoginResponse login) {
    if (Session.ROLE_BACKOFFICE.equals(login.role)) {
        UiUtils.showMessage(textError, "Backoffice accounts use the SunShare web app. Please log in on a computer.");
        return;
    }
    Session session = Session.fromLogin(login);
    SunShareDbHelper.get(this).saveSession(session);
    openHome(session);
}

// Opens Operator Home (M10) for Grid Operators, Prosumer Home (M3) for prosumers,
// and closes Login so the Back button doesn't return to it.
private void openHome(Session session) {
    Class<?> home = session.isGridOperator() ? OperatorHomeActivity.class : ProsumerHomeActivity.class;
    startActivity(new Intent(this, home));
    finish();
}

// Shows which server the app is talking to, under the form.
private void showServerAddress() {
    textServer.setText("Server: " + ApiConfig.getBaseUrl(this));
}

// ⊙ Server: a box to type the API address (emulator 10.0.2.2, or the laptop's Wi-Fi IP on a real phone).
private void showServerDialog() {
    View view = getLayoutInflater().inflate(R.layout.dialog_server_address, null);
    EditText inputServerUrl = view.findViewById(R.id.inputServerUrl);
    inputServerUrl.setText(ApiConfig.getBaseUrl(this));

    new MaterialAlertDialogBuilder(this)
        .setTitle("Server address")
        .setView(view)
        .setNegativeButton("Cancel", null)
        .setNeutralButton("Default", (dialog, which) -> saveServer(ApiConfig.DEFAULT_BASE_URL))
        .setPositiveButton("Save", (dialog, which) -> saveServer(UiUtils.textOf(inputServerUrl)))
        .show();
}

// Saves the new address in SQLite (if it looks like a web address), then tests it with GET /api/health.
private void saveServer(String url) {
    if (!ApiConfig.isValidBaseUrl(url)) {
        UiUtils.toast(this, "That doesn't look like a server address, e.g. http://192.168.1.7:8080/");
        return;
    }
    ApiConfig.setBaseUrl(this, url);
    showServerAddress();
    testServer();
}

// Calls GET /api/health on the new address and says whether it worked.
private void testServer() {
    ApiClient.get(this).health().enqueue(new Callback<HealthResponse>() {
        // The API answered: connected, or the API is up but MongoDB is not (503).
        @Override
        public void onResponse(Call<HealthResponse> call, Response<HealthResponse> response) {
            if (response.isSuccessful() && response.body() != null) {
                UiUtils.toast(LoginActivity.this, "Connected: API " + response.body().api + ", database " +
                response.body().database + ".");
            } else {
                UiUtils.toast(LoginActivity.this, "The server answered, but not OK (HTTP " + response.code() + ").");
            }
        }
    });

    // Nothing answered at that address (kept short: Android shows only 2 lines of a toast).
    @Override
    public void onFailure(Call<HealthResponse> call, Throwable error) {
        UiUtils.toast(LoginActivity.this, "Saved, but can't reach " + ApiConfig.getBaseUrl(LoginActivity.this));
    }
}
});
}
}

```

android/app/src/main/java/com/sunshare/app/ui/auth/RegisterActivity.java

```

/*
 * =====
 * File      : RegisterActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-29
 * Description : M2 Register - a new prosumer signs up with their NIC and
 *               details (POST /api/auth/register). The API checks the NIC
 *               format and that it is new (R1) and saves the account as
 *               Pending (R3), so they can log in only after a Backoffice
 *               officer activates it.
 * =====
 */
package com.sunshare.app.ui.auth;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.dialog.MaterialAlertDialogBuilder;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.RegisterRequest;
import com.sunshare.app.api.models.UserResponse;
import com.sunshare.app.util.UiUtils;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class RegisterActivity extends AppCompatActivity {

    private EditText inputNic;
    private EditText inputFullName;
    private EditText inputEmail;
    private EditText inputPhone;
    private EditText inputAddress;
    private EditText inputPassword;
    private EditText inputConfirmPassword;
    private TextView textError;
    private Button buttonRegister;

    // Called when the screen opens: links the form, the back arrow and the Create account button.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
        toolbar.setNavigationOnClickListener(v -> finish());

        inputNic = findViewById(R.id.inputNic);
        inputFullName = findViewById(R.id.inputFullName);
        inputEmail = findViewById(R.id.inputEmail);
        inputPhone = findViewById(R.id.inputPhone);
        inputAddress = findViewById(R.id.inputAddress);
        inputPassword = findViewById(R.id.inputPassword);
        inputConfirmPassword = findViewById(R.id.inputConfirmPassword);
        textError = findViewById(R.id.textError);
        buttonRegister = findViewById(R.id.buttonRegister);
        buttonRegister.setOnClickListener(v -> register());
    }

    // Checks the boxes are filled, then sends the sign-up to the API.
    // NIC format, duplicate NIC and password length are checked by the API (R1), which sends back the message to show.
    private void register() {
        UiUtils.hideMessage(textError);
        if (!UiUtils.requireFilled(inputNic, "Please enter your NIC.")
            || !UiUtils.requireFilled(inputFullName, "Please enter your full name.")
            || !UiUtils.requireFilled(inputEmail, "Please enter your email.")
            || !UiUtils.requireFilled(inputPhone, "Please enter your phone number.")
            || !UiUtils.requireFilled(inputPassword, "Please choose a password.")
            || !UiUtils.requireFilled(inputConfirmPassword, "Please type the password again.")) {
            return;
        }
    }

```

```

    }
    // Typing check only: both password boxes must be the same.
    if (!UiUtils.textOf(inputPassword).equals(UiUtils.textOf(inputConfirmPassword))) {
        UiUtils.showFieldError(inputConfirmPassword, "The two passwords don't match.");
        return;
    }

    String address = UiUtils.textOf(inputAddress);
    RegisterRequest body = new RegisterRequest(
        UiUtils.textOf(inputNic),
        UiUtils.textOf(inputFullName),
        UiUtils.textOf(inputEmail),
        UiUtils.textOf(inputPhone),
        address.isEmpty() ? null : address,
        UiUtils.textOf(inputPassword));

    UiUtils.setLoading(buttonRegister, true);
    ApiClient.get(this).register(body).enqueue(new Callback<UserResponse>() {
        // The API answered: 201 = registered as Pending; 400/409 = the reason (bad NIC, NIC already used...).
        @Override
        public void onResponse(Call<UserResponse> call, Response<UserResponse> response) {
            UiUtils.setLoading(buttonRegister, false);
            if (response.isSuccessful()) {
                showRegistered();
            } else {
                UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
            }
        }
    });

    // The call never reached the API (server off, wrong address, no Wi-Fi).
    @Override
    public void onFailure(Call<UserResponse> call, Throwable error) {
        UiUtils.setLoading(buttonRegister, false);
        UiUtils.showMessage(textError, ApiErrorParser.fromFailure(RegisterActivity.this));
    }
});
}

// The API saved the account as Pending (R3 is enforced there) - tell the user to wait for activation, then go back to
Login.
private void showRegistered() {
    new MaterialAlertDialogBuilder(this)
        .setTitle("Registered!")
        .setMessage("A Backoffice officer will activate your account. You can log in once it is active.")
        .setCancelable(false)
        .setPositiveButton("Back to login", (dialog, which) -> finish())
        .show();
}
}

```

android/app/src/main/java/com/sunshare/app/ui/map/StationMapActivity.java

```

/*
 * =====
 * File      : StationMapActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Stations, slots & map
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : M5 Nearby stations - for prosumers and Grid Operators.
 *             1) asks for the location permission and gets the phone's
 *                location (Fused Location); if it can't, uses SLIIT Malabe,
 *             2) GET /api/stations/nearby?lat=&lng=&radiusKm=25 - the API
 *                works out the distances and sorts nearest first (R18),
 *             3) saves the stations in the SQLite stations_cache table,
 *             4) shows them on a Google map: assets/map.html (Maps
 *                JavaScript API) inside a WebView, with the demo key and
 *                the data filled in (D53).
 *             If the API can't be reached, the saved stations are shown
 *             instead (offline fallback).
 * =====
 */
package com.sunshare.app.ui.map;

import android.Manifest;
import android.annotation.SuppressLint;
import android.content.ActivityNotFoundException;
import android.content.Intent;
import android.content.pm.ApplicationInfo;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.view.View;
import android.webkit.WebResourceRequest;

```



```

import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;
import android.widget.FrameLayout;
import android.widget.ProgressBar;
import android.widget.TextView;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.google.android.gms.location.CurrentLocationRequest;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.location.Priority;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.gson.Gson;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.StationResponse;
import com.sunshare.app.db.SunShareDbHelper;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.Map;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class StationMapActivity extends AppCompatActivity {

    // How far to look for stations, in km (the API's default too).
    private static final double RADIUS_KM = 25;
    // Where to search when the phone's location can't be used: the SLIIT Malabe campus.
    private static final double DEFAULT_LAT = 6.9147;
    private static final double DEFAULT_LNG = 79.9729;
    // The page is loaded "as if" from this address; Google needs a normal https origin for the map.
    private static final String PAGE_BASE_URL = "https://sunshare.local/";
    // Wait at most this long for the phone's position before using SLIIT Malabe instead.
    private static final long LOCATION_TIMEOUT_MS = 10_000;

    private FrameLayout mapContainer;
    private WebView webMap;
    private TextView textStatus;
    private ProgressBar progress;
    // Extra words for the status line when the default location is used (null = real location).
    private String locationNote;

    // Shows Android's "Allow SunShare to access this device's location?" box and receives the answer.
    private final ActivityResultLauncher<String[]> permissionLauncher = registerForActivityResult(
        new ActivityResultContracts.RequestMultiplePermissions(), this::onPermissionResult);

    // Called when the screen opens: makes sure someone is logged in, links the views
    // and starts with the location. (The WebView is only made in showMap - see there.)
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_station_map);
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
        if (SessionGuard.require(this) == null) {
            return;
        }
        toolbar.setNavigationOnClickListener(v -> finish());

        mapContainer = findViewById(R.id.mapContainer);
        textStatus = findViewById(R.id.textStatus);
        progress = findViewById(R.id.progress);
        askForLocation();
    }

    // Frees the WebView's memory when the screen closes.
    @Override
    protected void onDestroy() {

```

```

        if (webMap != null) {
            webMap.destroy();
        }
        super.onDestroy();
    }

    // The map page needs JavaScript. A tap on "Directions in Google Maps" opens the Google Maps app
    // instead of loading Google's website inside our small map view.
    @SuppressWarnings("SetJavaScriptEnabled")
    private void setupWebView() {
        WebSettings settings = webMap.getSettings();
        settings.setJavaScriptEnabled(true);
        settings.setDomStorageEnabled(true);
        webMap.setWebViewClient(new WebViewClient() {
            // Decides where a tapped link opens: Google Maps links go to the Google Maps app (or a browser).
            @Override
            public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest request) {
                String url = request.getUrl().toString();
                if (url.startsWith("https://www.google.com/maps")) {
                    try {
                        startActivity(new Intent(Intent.ACTION_VIEW, request.getUrl()));
                    } catch (ActivityNotFoundException noApp) {
                        UiUtils.toast(StationMapActivity.this, "No app on this phone can open Google Maps directions.");
                    }
                    return true;
                }
                return false;
            }
        });
    }

    // Uses the location straight away if allowed before; otherwise asks for it first.
    private void askForLocation() {
        boolean allowed = ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) ==
        PackageManager.PERMISSION_GRANTED
        || ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_COARSE_LOCATION) ==
        PackageManager.PERMISSION_GRANTED;
        if (allowed) {
            findLocation();
        } else {
            permissionLauncher.launch(new String[]{
                Manifest.permission.ACCESS_FINE_LOCATION, Manifest.permission.ACCESS_COARSE_LOCATION});
        }
    }

    // The user answered the permission box: precise or approximate location is fine; "Don't allow" -> SLIIT Malabe.
    private void onPermissionResult(Map<String, Boolean> answers) {
        if (answers.containsKey(true)) {
            findLocation();
        } else {
            useDefaultLocation("Location is off, so stations near SLIIT Malabe are shown.");
        }
    }

    // Asks Fused Location (Google Play services) for one position of the phone: GPS, Wi-Fi or mobile
    // network, a fix up to 1 minute old is fine, and after 10 seconds without one it gives up (null).
    // Reference: Google, "Get the last known location" / getCurrentLocation https://developer.android.com/develop/sensors-
    and-location/location/retrieve-current
    @SuppressWarnings("MissingPermission")
    private void findLocation() {
        setStatus(getString(R.string.map_finding_location), true);
        CurrentLocationRequest request = new CurrentLocationRequest.Builder()
            .setPriority(Priority.PRIORITY_HIGH_ACCURACY)
            .setMaxUpdateAgeMillis(60_000)
            .setDurationMillis(LOCATION_TIMEOUT_MS)
            .build();
        LocationServices.getFusedLocationProviderClient(this)
            .getCurrentLocation(request, null)
            .addOnSuccessListener(this, location -> {
                if (location == null) {
                    useDefaultLocation("Your location isn't available right now, so stations near SLIIT Malabe are
                    shown.");
                } else {
                    locationNote = null;
                    loadNearby(location.getLatitude(), location.getLongitude());
                }
            })
            .addOnFailureListener(this, error ->
                useDefaultLocation("Your location isn't available right now, so stations near SLIIT Malabe are
                shown."));
    }

    // Searches around the SLIIT Malabe campus instead, and says why on the status line.
    private void useDefaultLocation(String why) {
        locationNote = why;
        loadNearby(DEFAULT_LAT, DEFAULT_LNG);
    }

```

```

}

// GET /api/stations/nearby - the API keeps active stations within the radius, works out each distance
// and sorts nearest first (R18). The answer is saved in SQLite for offline use.
private void loadNearby(double lat, double lng) {
    setStatus(getString(R.string.map_loading_stations), true);
    ApiClient.get(this).getNearbyStations(lat, lng, RADIUS_KM).enqueue(new Callback<List<StationResponse>>() {
        // The API answered: save + draw the stations, or fall back to the saved ones.
        @Override
        public void onResponse(Call<List<StationResponse>> call, Response<List<StationResponse>> response) {
            if (response.isSuccessful() && response.body() != null) {
                List<StationResponse> stations = response.body();
                SunShareDbHelper.get(StationMapActivity.this).cacheStations(stations);
                showMap(lat, lng, stations);
                setStatus(withNote(countText(stations.size()), false));
            } else if (response.code() == 401) {
                SessionGuard.handleUnauthorized(StationMapActivity.this);
            } else {
                showSavedStations(lat, lng, ApiErrorParser.fromResponse(response));
            }
        }

        // The server can't be reached: use the stations saved last time.
        @Override
        public void onFailure(Call<List<StationResponse>> call, Throwable error) {
            showSavedStations(lat, lng, "Can't reach the SunShare server");
        }
    });
}

// Offline fallback: the stations from the SQLite stations_cache table, with the time they were saved.
private void showSavedStations(double lat, double lng, String reason) {
    SunShareDbHelper db = SunShareDbHelper.get(this);
    List<StationResponse> saved = db.getCachedStations();
    if (saved.isEmpty()) {
        setStatus(reason + ", and no stations have been saved on this phone yet.", false);
        return;
    }
    showMap(lat, lng, saved);
    setStatus("Offline: showing " + saved.size() + " saved stations (saved "
        + DateUtils.formatDateTime(db.getStationsCachedAt()) + ").", false);
}

// Fills the key and the data into assets/map.html and shows it. Gson turns the data into JSON,
// and escapes characters like < and > so the text can't break the page.
// The WebView is made here, not in the layout: the first WebView in an app loads Chrome's engine,
// which can take a few seconds on a slow phone, so the screen and the permission box come first.
private void showMap(double lat, double lng, List<StationResponse> stations) {
    if (webMap == null) {
        webMap = new WebView(this);
        setupWebView();
        mapContainer.addView(webMap, new FrameLayout.LayoutParams(
            FrameLayout.LayoutParams.MATCH_PARENT, FrameLayout.LayoutParams.MATCH_PARENT));
    }
    MapData data = new MapData();
    data.me = new MapData.Point(lat, lng);
    data.stations = stations;
    String html = readAsset("map.html")
        .replace("__MAPS_API_KEY__", mapsKey())
        .replace("__DATA__", new Gson().toJson(data));
    webMap.loadDataWithBaseURL(PAGE_BASE_URL, html, "text/html", "UTF-8", null);
}

// "3 stations within 25 km, nearest first." (or none).
private String countText(int count) {
    if (count == 0) {
        return "No active stations within 25 km.";
    }
    return count + (count == 1 ? " station" : " stations") + " within 25 km, nearest first. Tap a marker for details.";
}

// Adds the "location is off" note in front of the status text when the default location was used.
private String withNote(String text) {
    return locationNote == null ? text : locationNote + " " + text;
}

// Shows a line of text above the map, with the small spinner while something is loading.
private void setStatus(String text, boolean loading) {
    textStatus.setText(text);
    progress.setVisibility(loading ? View.VISIBLE : View.GONE);
}

// The Google Maps key: put into the manifest at build time from local.properties (never in Git).
private String mapsKey() {

```

```

    try {
        ApplicationInfo info = getPackageManager().getApplicationInfo(getPackageName(), PackageManager.GET_META_DATA);
        String key = info.metaData == null ? null : info.metaData.getString("com.sunshare.app.MAPS_API_KEY");
        return key == null ? "" : key;
    } catch (PackageManager.NameNotFoundException e) {
        return "";
    }
}

// Reads a file from the app's assets folder as text.
private String readAsset(String name) {
    try (InputStream in = getAssets().open(name)) {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        byte[] buffer = new byte[4096];
        int read;
        while ((read = in.read(buffer)) != -1) {
            out.write(buffer, 0, read);
        }
        return out.toString(StandardCharsets.UTF_8.name());
    } catch (IOException e) {
        return "<p>The map page could not be opened.</p>";
    }
}

// The data handed to the map page: my position and the stations (Gson turns it into JSON).
private static class MapData {
    Point me;
    List<StationResponse> stations;

    // A latitude / longitude pair.
    static class Point {
        final double lat;
        final double lng;

        // Makes a point from a latitude and longitude.
        Point(double lat, double lng) {
            this.lat = lat;
            this.lng = lng;
        }
    }
}
}

```

android/app/src/main/java/com/sunshare/app/ui/operator/OperatorHomeActivity.java

```

/*
 * =====
 * File      : OperatorHomeActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description: M10 Operator Home - the first screen after a Grid Operator
 *             logs in: greeting, today's counts (Today, Pending, Approved
 *             upcoming) worked out by the API (GET /api/dashboard/summary),
 *             refreshed every time the screen comes back, and a big
 *             Scan QR button that opens the ZXing camera scanner. The
 *             scanned text goes to the Scan Result screen (M11). (The Nearby
 *             stations button was added with M5, Part B.)
 * =====
 */
package com.sunshare.app.ui.operator;

import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;

import androidx.activity.result.ActivityResultLauncher;
import androidx.appcompat.app.AppCompatActivity;

import com.journeyapps.barcodescanner.ScanContract;
import com.journeyapps.barcodescanner.ScanIntentResult;
import com.journeyapps.barcodescanner.ScanOptions;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.StaffDashboardResponse;
import com.sunshare.app.db.Session;
import com.sunshare.app.ui.map.StationMapActivity;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

```

```

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class OperatorHomeActivity extends AppCompatActivity {

    private TextView textTodayCount;
    private TextView textPendingCount;
    private TextView textApprovedCount;
    private TextView textError;

    // Opens ZXing's camera scanner and receives what it read. It has to be registered when the screen
    // is created; ZXing asks for the camera permission itself the first time.
    // Reference: ZXing Android Embedded - Usage with ScanContract https://github.com/journeyapps/zxing-android-embedded
    private final ActivityResultLauncher<ScanOptions> scanLauncher =
        registerForActivityResult(new ScanContract(), this::onScanned);

    // Called when the screen opens: keeps it clear of the phone's bars, makes sure someone is
    // logged in (else back to Login), shows the greeting and links the buttons.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_operator_home);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), findViewById(R.id.header));
        Session session = SessionGuard.require(this);
        if (session == null) {
            return;
        }

        TextView textGreeting = findViewById(R.id.textGreeting);
        TextView textRole = findViewById(R.id.textRole);
        textGreeting.setText("Hello, " + firstName(session.fullName));
        textRole.setText("Grid Operator · NIC " + session.nic);
        textTodayCount = findViewById(R.id.textTodayCount);
        textPendingCount = findViewById(R.id.textPendingCount);
        textApprovedCount = findViewById(R.id.textApprovedCount);
        textError = findViewById(R.id.textError);

        findViewById(R.id.buttonScan).setOnClickListener(v -> startScan());
        findViewById(R.id.buttonMap).setOnClickListener(v -> startActivity(new Intent(this, StationMapActivity.class)));
        findViewById(R.id.buttonLogout).setOnClickListener(v -> SessionGuard.logout(this));
    }

    // Called every time the screen comes to the front (also after finishing a transfer): reloads the counts.
    @Override
    protected void onResume() {
        super.onResume();
        if (textTodayCount != null) {
            loadDashboard();
        }
    }

    // GET /api/dashboard/summary - the same numbers the web dashboard shows (docs/01-SPEC.md section 5).
    private void loadDashboard() {
        UiUtils.hideMessage(textError);
        ApiClient.get(this).getStaffDashboard().enqueue(new Callback<StaffDashboardResponse>() {
            // The API answered: show the numbers, or the reason it failed.
            @Override
            public void onResponse(Call<StaffDashboardResponse> call, Response<StaffDashboardResponse> response) {
                if (response.isSuccessful() && response.body() != null) {
                    StaffDashboardResponse dashboard = response.body();
                    textTodayCount.setText(String.valueOf(dashboard.todayReservations));
                    textPendingCount.setText(String.valueOf(dashboard.pendingReservations));
                    textApprovedCount.setText(String.valueOf(dashboard.approvedFutureReservations));
                } else if (response.code() == 401) {
                    SessionGuard.handleUnauthorized(OperatorHomeActivity.this);
                } else {
                    UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
                }
            }
        });

        // The call never reached the API.
        @Override
        public void onFailure(Call<StaffDashboardResponse> call, Throwable error) {
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(OperatorHomeActivity.this));
        }
    }
});

// Opens the camera scanner (our QrScanActivity = ZXing's screen kept clear of the phone's bars),
// set to read QR codes only, with a hint on screen.

```

```

private void startScan() {
    ScanOptions options = new ScanOptions();
    options.setDesiredBarcodeFormats(ScanOptions.QR_CODE);
    options.setPrompt("Point the camera at the prosumer's QR code");
    options.setBeepEnabled(false);
    options.setOrientationLocked(false);
    options.setCaptureActivity(QrScanActivity.class);
    scanLauncher.launch(options);
}

// The scanner closed: no text = the operator pressed Back; otherwise check the text with the server (M11).
private void onScanned(ScanIntentResult result) {
    if (result.getContents() == null) {
        UiUtils.toast(this, "Scan cancelled.");
        return;
    }
    Intent intent = new Intent(this, ScanResultActivity.class);
    intent.putExtra(ScanResultActivity.EXTRA_QR_DATA, result.getContents());
    startActivity(intent);
}

// "Kasun Silva" -> "Kasun" (a friendlier greeting).
private String firstName(String fullName) {
    if (fullName == null || fullName.trim().isEmpty()) {
        return "there";
    }
    return fullName.trim().split("\\s+")[0];
}
}

```

android/app/src/main/java/com/sunshare/app/ui/operator/QrScanActivity.java

```

/*
 * =====
 * File      : QrScanActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : ZXing's camera scanner screen, with one change: on Android
 *              15+ apps are drawn behind the status and navigation bars, so
 *              the scanner's hint text ended up hidden under the navigation
 *              bar. This screen pads the scanner by the size of those bars.
 *              Everything else (camera, permission, decoding) is ZXing's.
 * =====
 */
package com.sunshare.app.ui.operator;

import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.journeyapps.barcodescanner.CaptureActivity;
import com.journeyapps.barcodescanner.DecoratedBarcodeView;

public class QrScanActivity extends CaptureActivity {

    // ZXing calls this to build the scanner view; we keep its view and add padding for the phone's bars.
    @Override
    protected DecoratedBarcodeView initializeContent() {
        DecoratedBarcodeView scanner = super.initializeContent();
        ViewCompat.setOnApplyWindowInsetsListener(scanner, (view, insets) -> {
            Insets bars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            view.setPadding(bars.left, bars.top, bars.right, bars.bottom);
            return insets;
        });
        return scanner;
    }
}

```

android/app/src/main/java/com/sunshare/app/ui/operator/ScanResultActivity.java

```

/*
 * =====
 * File      : ScanResultActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 *

```

```

* Description : M11 Scan result - after the operator scans a QR code, the
*               text is checked with the server (POST
*               /api/reservations/verify-qr). Valid -> the booking's
*               prosumer, station, time and kWh with "Verified with server"
*               and a Finalize button (PATCH /api/reservations/{id}/complete,
*               which checks everything again and sets Completed). Invalid ->
*               a red card with the API's reason. The phone never decides
*               whether a QR is valid - the server does (R14).
* =====
*/
package com.sunshare.app.ui.operator;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.card.MaterialCardView;
import com.google.android.material.dialog.MaterialAlertDialogBuilder;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.api.models.VerifyQrRequest;
import com.sunshare.app.ui.prosumer.BookingAdapter;
import com.sunshare.app.ui.prosumer.ReservationSummaryActivity;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class ScanResultActivity extends AppCompatActivity {

    // The text the camera read from the QR code is passed in the Intent under this name.
    public static final String EXTRA_QR_DATA = "qrData";

    private String qrData;
    // The booking the server verified (null until then); Finalize completes this one.
    private ReservationResponse verified;

    private View checking;
    private MaterialCardView cardResult;
    private TextView textResultIcon;
    private TextView textResultTitle;
    private TextView textReason;
    private View bookingDetails;
    private Button buttonFinalize;

    // Called when the screen opens: makes sure someone is logged in, reads the scanned text,
    // links the views and asks the server to check the code.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_scan_result);
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
        if (SessionGuard.require(this) == null) {
            return;
        }
        toolbar.setNavigationOnClickListener(v -> finish());

        qrData = getIntent().getStringExtra(EXTRA_QR_DATA);
        checking = findViewById(R.id.checking);
        cardResult = findViewById(R.id.cardResult);
        textResultIcon = findViewById(R.id.textResultIcon);
        textResultTitle = findViewById(R.id.textResultTitle);
        textReason = findViewById(R.id.textReason);
        bookingDetails = findViewById(R.id.bookingDetails);
        buttonFinalize = findViewById(R.id.buttonFinalize);
        buttonFinalize.setOnClickListener(v -> confirmFinalize());
        findViewById(R.id.buttonBack).setOnClickListener(v -> finish());

        verify();
    }
}

```

```

// POST /api/reservations/verify-qr - the server checks the format, that the booking exists,
// is Approved and that the secret token matches (R14). Nothing is changed yet.
private void verify() {
    checking.setVisibility(View.VISIBLE);
    ApiClient.get(this).verifyQr(new VerifyQrRequest(qrData == null ? "" : qrData)).enqueue(new
Callback<ReservationResponse>() {
    // The API answered: valid -> show the booking; invalid -> show its reason.
    @Override
    public void onResponse(Call<ReservationResponse> call, Response<ReservationResponse> response) {
        checking.setVisibility(View.GONE);
        if (response.isSuccessful() && response.body() != null) {
            showVerified(response.body());
        } else if (response.code() == 401) {
            SessionGuard.handleUnauthorized(ScanResultActivity.this);
        } else {
            showInvalid(ApiErrorParser.fromResponse(response));
        }
    }

    // The call never reached the API.
    @Override
    public void onFailure(Call<ReservationResponse> call, Throwable error) {
        checking.setVisibility(View.GONE);
        showInvalid(ApiErrorParser.fromFailure(ScanResultActivity.this));
    }
});
}

// Green card: whose booking it is and what to transfer, plus the Finalize button.
private void showVerified(ReservationResponse booking) {
    verified = booking;
    showResult("✓", "Verified with server", R.color.ss_success);
    textReason.setVisibility(View.GONE);
    ((TextView) findViewById(R.id.textProsumer)).setText(booking.prosumerName + " (" + booking.prosumerNic + ")");
    ((TextView) findViewById(R.id.textStation)).setText(booking.stationName);
    UiUtils.showStatusBadge(findViewById(R.id.textStatus), booking.status);
    ((TextView) findViewById(R.id.textTime)).setText(DateUtils.formatRange(booking.startTime, booking.endTime));
    ((TextView) findViewById(R.id.textEnergy)).setText(BookingAdapter.energyText(booking));
    ((TextView) findViewById(R.id.textBookingId)).setText("Booking ID " + booking.id);
    bookingDetails.setVisibility(View.VISIBLE);
    buttonFinalize.setVisibility(View.VISIBLE);
}

// Red card with the server's reason (not a SunShare code, already completed, cancelled, not approved ...).
private void showInvalid(String reason) {
    verified = null;
    showResult("✗", "This QR code can't be used", R.color.ss_danger);
    textReason.setText(reason);
    textReason.setVisibility(View.VISIBLE);
    bookingDetails.setVisibility(View.GONE);
    buttonFinalize.setVisibility(View.GONE);
}

// Shows the result card with its icon, title and border colour.
private void showResult(String icon, String title, int colorRes) {
    textResultIcon.setText(icon);
    textResultIcon.setBackgroundTintList(ContextCompat.getColorStateList(this, colorRes));
    textResultTitle.setText(title);
    cardResult.setStrokeColor(ContextCompat.getColor(this, colorRes));
    cardResult.setVisibility(View.VISIBLE);
}

// Asks "are you sure?" first - a completed booking can't be changed any more (R15).
private void confirmFinalize() {
    if (verified == null) {
        return;
    }
    new MaterialAlertDialogBuilder(this)
        .setTitle("Finalize the energy transfer?")
        .setMessage(BookingAdapter.energyText(verified) + " for " + verified.prosumerName
            + " at " + verified.stationName + " will be marked Completed. This can't be undone.")
        .setNegativeButton("Not yet", null)
        .setPositiveButton("Finalize", (dialog, which) -> finalizeTransfer())
        .show();
}

// PATCH /api/reservations/{id}/complete with the same QR text - the server checks everything again
// (the booking could have changed since the scan), sets Completed + who/when, and the Summary opens.
private void finalizeTransfer() {
    UiUtils.setLoading(buttonFinalize, true);
    ApiClient.get(this).completeReservation(verified.id, new VerifyQrRequest(qrData)).enqueue(new
Callback<ReservationResponse>() {
    // The API answered: completed -> Summary (M7); otherwise its reason.
    @Override
    public void onResponse(Call<ReservationResponse> call, Response<ReservationResponse> response) {

```



```

        UiUtils.setLoading(buttonFinalize, false);
        if (response.isSuccessful() && response.body() != null) {
            ReservationSummaryActivity.open(ScanResultActivity.this, response.body(),
ReservationSummaryActivity.ACTION_COMPLETED);
            finish();
        } else if (response.code() == 401) {
            SessionGuard.handleUnauthorized(ScanResultActivity.this);
        } else {
            showInvalid(ApiErrorParser.fromResponse(response));
        }
    }

    // The call never reached the API.
    @Override
    public void onFailure(Call<ReservationResponse> call, Throwable error) {
        UiUtils.setLoading(buttonFinalize, false);
        UiUtils.toast(ScanResultActivity.this, ApiErrorParser.fromFailure(ScanResultActivity.this));
    }
}
}
}
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/BookingAdapter.java

```

/*
 * =====
 * File      : BookingAdapter.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : Turns a list of bookings into rows for the RecyclerView on
 *               the My bookings screen (M8). Each row shows the station,
 *               time, energy + type and a status badge; tapping a row tells
 *               the screen which booking was chosen.
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.sunshare.app.R;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.UiUtils;

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.List;

// A RecyclerView only draws the rows that fit on screen and re-uses them while scrolling;
// the adapter's job is to fill a row (ViewHolder) with the booking at a given position.
public class BookingAdapter extends RecyclerView.Adapter<BookingAdapter.BookingViewHolder> {

    // What the screen does when a row is tapped.
    public interface OnBookingClickListener {
        // Called with the booking whose row was tapped.
        void onBookingClick(ReservationResponse booking);
    }

    private final List<ReservationResponse> bookings = new ArrayList<>();
    private final OnBookingClickListener listener;

    // Creates an empty adapter; the screen passes what to do when a row is tapped.
    public BookingAdapter(OnBookingClickListener listener) {
        this.listener = listener;
    }

    // Replaces the rows with a new list from the API and redraws the list.
    public void setBookings(List<ReservationResponse> newBookings) {
        bookings.clear();
        if (newBookings != null) {
            bookings.addAll(newBookings);
        }
    }
}

```

```

        notifyDataSetChanged();
    }

    // Makes a new empty row from item_booking.xml (called only for the few rows visible at once).
    @NonNull
    @Override
    public BookingViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View row = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_booking, parent, false);
        return new BookingViewHolder(row);
    }

    // Fills a row with the booking at this position.
    @Override
    public void onBindViewHolder(@NonNull BookingViewHolder holder, int position) {
        holder.bind(bookings.get(position), listener);
    }

    // How many rows the list has.
    @Override
    public int getItemCount() {
        return bookings.size();
    }

    // "12.5 kWh · Sell" (whole numbers without ".0", e.g. "8 kWh · Buy"). Also used by the detail screen.
    public static String energyText(ReservationResponse booking) {
        return new DecimalFormat("0.##").format(booking.energyKwh) + " kWh · " + booking.type;
    }

    // Holds the views of one row so they are found once, not on every scroll.
    static class BookingViewHolder extends RecyclerView.ViewHolder {
        private final TextView textStation;
        private final TextView textStatus;
        private final TextView textTime;
        private final TextView textEnergy;

        // Finds the row's text views.
        BookingViewHolder(@NonNull View row) {
            super(row);
            textStation = row.findViewById(R.id.textStation);
            textStatus = row.findViewById(R.id.textStatus);
            textTime = row.findViewById(R.id.textTime);
            textEnergy = row.findViewById(R.id.textEnergy);
        }

        // Shows one booking in this row and makes the whole card tappable.
        void bind(ReservationResponse booking, OnBookingClickListener listener) {
            textStation.setText(booking.stationName);
            textStatus.showStatusBadge(textStatus, booking.status);
            textTime.setText(DateUtils.formatRange(booking.startTime, booking.endTime));
            textEnergy.setText(energyText(booking));
            itemView.setOnClickListener(v -> listener.onBookingClick(booking));
        }
    }
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/BookingsActivity.java

```

/*
 * =====
 * File      : BookingsActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : M8 My bookings - the prosumer's own bookings in three tabs:
 *               Current (Approved, still to come), Pending, and History
 *               (Completed, Cancelled or already started), plus a search
 *               box (station name or full booking id). The API decides what
 *               belongs in each tab (GET /api/reservations?view=&search=,
 *               docs/01-SPEC.md section 5); tapping a booking opens M9.
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.content.Intent;
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.view.View;
import android.view.inputmethod.EditorInfo;
import android.widget.EditText;

```

```

import android.widget.ProgressBar;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.tabs.TabLayout;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import java.util.List;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class BookingsActivity extends AppCompatActivity {

    // Which tab to open first; the value is one of VIEWS (e.g. "pending"). Used by the Home count cards.
    public static final String EXTRA_VIEW = "view";

    // Tab position -> the API's view name, the tab's title and the text shown when it is empty.
    private static final String[] VIEWS = {"current", "pending", "history"};
    private static final String[] TITLES = {"Current", "Pending", "History"};
    private static final String[] EMPTY_TEXTS = {
        "No upcoming approved bookings.",
        "No bookings waiting for approval.",
        "No completed or cancelled bookings yet."};

    private TabLayout tabs;
    private EditText inputSearch;
    private ProgressBar progress;
    private TextView textMessage;
    private final BookingAdapter adapter = new BookingAdapter(this::openDetail);
    // The list request that is running now, so a newer one can cancel it.
    private Call<List<ReservationResponse>> currentCall;

    // Called when the screen opens: makes sure someone is logged in, builds the tabs,
    // links the search box and the list. The first load happens in onResume.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_bookings);
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
        if (SessionGuard.require(this) == null) {
            return;
        }
        toolbar.setNavigationOnClickListener(v -> finish());

        tabs = findViewById(R.id.tabs);
        inputSearch = findViewById(R.id.inputSearch);
        progress = findViewById(R.id.progress);
        textMessage = findViewById(R.id.textMessage);
        RecyclerView list = findViewById(R.id.list);
        list.setLayoutManager(new LinearLayoutManager(this));
        list.setAdapter(adapter);

        setupTabs(getIntent().getStringExtra(EXTRA_VIEW));
        setupSearch();
    }

    // Called every time the screen comes to the front: reloads, so a booking changed elsewhere shows its new status.
    @Override
    protected void onResume() {
        super.onResume();
        if (tabs != null) {
            loadBookings();
        }
    }

    // Adds the three tabs, selects the one asked for (default Current) and reloads when another tab is chosen.
    private void setupTabs(String startView) {
        for (String title : TITLES) {

```

```

        tabs.addTab(tabs.newTab().setText(title));
    }
    for (int i = 0; i < VIEWS.length; i++) {
        if (VIEWS[i].equals(startView)) {
            tabs.selectTab(tabs.getTabAt(i));
        }
    }
    tabs.addTabOnTabSelectedListener(new TabLayout.OnTabSelectedListener() {
        // A different tab was chosen: load that list.
        @Override
        public void onTabSelected(TabLayout.Tab tab) {
            loadBookings();
        }

        // Nothing to do when a tab is left.
        @Override
        public void onTabUnselected(TabLayout.Tab tab) {
        }

        // Tapping the open tab again reloads it.
        @Override
        public void onTabReselected(TabLayout.Tab tab) {
            loadBookings();
        }
    });
}

// Searches when the keyboard's search key is pressed, and shows the full list again when the box is cleared.
private void setupSearch() {
    inputSearch.setOnEditorActionListener((v, actionId, event) -> {
        if (actionId == EditorInfo.IME_ACTION_SEARCH) {
            loadBookings();
            return true;
        }
        return false;
    });
    inputSearch.addTextChangedListener(new TextWatcher() {
        // Not needed.
        @Override
        public void beforeTextChanged(CharSequence s, int start, int count, int after) {
        }

        // Not needed.
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count) {
        }

        // The box became empty (typed away or the clear button): show the whole tab again.
        @Override
        public void afterTextChanged(Editable s) {
            if (s.length() == 0) {
                loadBookings();
            }
        }
    });
}

// GET /api/reservations?view=<tab>&search=<text>. A request still running for an older tab or search
// is cancelled first, so a slow old answer can never overwrite a newer one.
private void loadBookings() {
    if (currentCall != null) {
        currentCall.cancel();
    }
    int tab = Math.max(tabs.getSelectedTabPosition(), 0);
    String search = UiUtils.textOf(inputSearch);
    progress.setVisibility(View.VISIBLE);
    textMessage.setVisibility(View.GONE);

    currentCall = ApiClient.get(this).getReservations(VIEWS[tab], search.isEmpty() ? null : search);
    currentCall.enqueue(new Callback<List<ReservationResponse>>() {
        // The API answered: show the list, or the reason it failed.
        @Override
        public void onResponse(Call<List<ReservationResponse>> call, Response<List<ReservationResponse>> response) {
            progress.setVisibility(View.GONE);
            if (response.isSuccessful() && response.body() != null) {
                showBookings(response.body(), tab, search);
            } else if (response.code() == 401) {
                SessionGuard.handleUnauthorized(BookingsActivity.this);
            } else {
                showError(ApiErrorParser.fromResponse(response));
            }
        }
    });

    // No answer. A cancelled call also ends here - that one is simply ignored.

```

```

        @Override
        public void onFailure(Call<List<ReservationResponse>> call, Throwable error) {
            if (call.isCanceled()) {
                return;
            }
            progress.setVisibility(View.GONE);
            showError(ApiErrorParser.fromFailure(BookingsActivity.this));
        }
    });
}

// Shows the bookings, or a friendly "nothing here" line when the list is empty.
private void showBookings(List<ReservationResponse> bookings, int tab, String search) {
    adapter.setBookings(bookings);
    if (bookings.isEmpty()) {
        String message = search.isEmpty() ? EMPTY_TEXTS[tab] : "No bookings match \"" + search + "\".";
        showMessage(message, R.color.ss_muted);
    }
}

// Empties the list and shows the error in red.
private void showError(String message) {
    adapter.setBookings(null);
    showMessage(message, R.color.ss_danger);
}

// Shows a line of text in the middle of the list area, in the given colour.
private void showMessage(String message, int colorRes) {
    textMessage.setText(message);
    textMessage.setTextColor(ContextCompat.getColor(this, colorRes));
    textMessage.setVisibility(View.VISIBLE);
}

// A row was tapped: open its details (M9).
private void openDetail(ReservationResponse booking) {
    Intent intent = new Intent(this, ReservationDetailActivity.class);
    intent.putExtra(ReservationDetailActivity.EXTRA_RESERVATION_ID, booking.id);
    startActivity(intent);
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/ProfileActivity.java

```

/*
 * =====
 * File      : ProfileActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : A - Accounts & access
 * Author    : Gimhan T P K (IT22266996)
 * Created   : 2026-09-29
 * Description : M4 Profile - the prosumer sees and edits their own details
 *              and can set a new password (GET / PUT /api/profile). They
 *              can also deactivate their own account (PATCH
 *              /api/profile/deactivate, R5); the app then forgets the
 *              session and goes back to Login. The API takes the NIC from
 *              the login token, so nobody can edit someone else (R16).
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.dialog.MaterialAlertDialogBuilder;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.MessageResponse;
import com.sunshare.app.api.models.UpdateProfileRequest;
import com.sunshare.app.api.models.UserResponse;
import com.sunshare.app.db.Session;
import com.sunshare.app.db.SunShareDbHelper;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import retrofit2.Call;

```

```

import retrofit2.Callback;
import retrofit2.Response;

public class ProfileActivity extends AppCompatActivity {

    private TextView textNic;
    private TextView textStatus;
    private EditText inputFullName;
    private EditText inputEmail;
    private EditText inputPhone;
    private EditText inputAddress;
    private EditText inputNewPassword;
    private TextView textError;
    private Button buttonSave;
    private Button buttonDeactivate;

    // Called when the screen opens: makes sure someone is logged in, links the form and buttons,
    // then loads the current details from the API.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_profile);
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
        if (SessionGuard.require(this) == null) {
            return;
        }
        toolbar.setNavigationOnClickListener(v -> finish());

        textNic = findViewById(R.id.textNic);
        textStatus = findViewById(R.id.textStatus);
        inputFullName = findViewById(R.id.inputFullName);
        inputEmail = findViewById(R.id.inputEmail);
        inputPhone = findViewById(R.id.inputPhone);
        inputAddress = findViewById(R.id.inputAddress);
        inputNewPassword = findViewById(R.id.inputNewPassword);
        textError = findViewById(R.id.textError);
        buttonSave = findViewById(R.id.buttonSave);
        buttonDeactivate = findViewById(R.id.buttonDeactivate);
        buttonSave.setOnClickListener(v -> save());
        buttonDeactivate.setOnClickListener(v -> confirmDeactivate());

        loadProfile();
    }

    // GET /api/profile - fills the form with the details saved on the server.
    private void loadProfile() {
        UiUtils.setLoading(buttonSave, true);
        ApiClient.get(this).getProfile().enqueue(new Callback<UserResponse>() {
            // The API answered: show the details, or the reason it failed.
            @Override
            public void onResponse(Call<UserResponse> call, Response<UserResponse> response) {
                UiUtils.setLoading(buttonSave, false);
                if (response.isSuccessful() && response.body() != null) {
                    showUser(response.body());
                } else if (response.code() == 401) {
                    SessionGuard.handleUnauthorized(ProfileActivity.this);
                } else {
                    UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
                }
            }

            // The call never reached the API.
            @Override
            public void onFailure(Call<UserResponse> call, Throwable error) {
                UiUtils.setLoading(buttonSave, false);
                UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ProfileActivity.this));
            }
        });
    }

    // Puts the API's user details into the screen.
    private void showUser(UserResponse user) {
        textNic.setText("NIC " + user.nic);
        textStatus.setText(user.role + " " + user.status);
        inputFullName.setText(user.fullName);
        inputEmail.setText(user.email);
        inputPhone.setText(user.phone);
        inputAddress.setText(user.address);
        inputNewPassword.setText("");
    }

    // PUT /api/profile - saves the edited details. An empty new-password box keeps the old password;

```

```

// the API checks email, phone and password length and sends back the message to show.
private void save() {
    UiUtils.hideMessage(textError);
    if (!UiUtils.requireFilled(inputFullName, "Please enter your full name.")
        || !UiUtils.requireFilled(inputEmail, "Please enter your email.")
        || !UiUtils.requireFilled(inputPhone, "Please enter your phone number.")) {
        return;
    }
    String address = UiUtils.textOf(inputAddress);
    String newPassword = UiUtils.textOf(inputNewPassword);
    UpdateProfileRequest body = new UpdateProfileRequest(
        UiUtils.textOf(inputFullName),
        UiUtils.textOf(inputEmail),
        UiUtils.textOf(inputPhone),
        address.isEmpty() ? null : address,
        newPassword.isEmpty() ? null : newPassword);

    UiUtils.setLoading(buttonSave, true);
    ApiClient.get(this).updateProfile(body).enqueue(new Callback<UserResponse>() {
        // The API answered: saved, or the reason it was refused.
        @Override
        public void onResponse(Call<UserResponse> call, Response<UserResponse> response) {
            UiUtils.setLoading(buttonSave, false);
            if (response.isSuccessful() && response.body() != null) {
                showUser(response.body());
                rememberNewName(response.body().fullName);
                UiUtils.toast(ProfileActivity.this, "Profile saved.");
            } else if (response.code() == 401) {
                SessionGuard.handleUnauthorized(ProfileActivity.this);
            } else {
                UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
            }
        }
    });

    // The call never reached the API.
    @Override
    public void onFailure(Call<UserResponse> call, Throwable error) {
        UiUtils.setLoading(buttonSave, false);
        UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ProfileActivity.this));
    }
});
}

// Updates the name in the SQLite session so the Home greeting shows the new name.
private void rememberNewName(String fullName) {
    SunShareDbHelper db = SunShareDbHelper.get(this);
    Session old = db.getSession();
    if (old != null) {
        db.saveSession(new Session(old.nic, fullName, old.role, old.token, old.loggedInAt));
    }
}

// Asks "are you sure?" first - deactivating means the prosumer can't log in until Backoffice reactivates them.
private void confirmDeactivate() {
    new MaterialAlertDialogBuilder(this)
        .setTitle("Deactivate your account?")
        .setMessage("You will be logged out and can't log in again until a Backoffice officer reactivates your account.")
        .setNegativeButton("Cancel", null)
        .setPositiveButton("Deactivate", (dialog, which) -> deactivate())
        .show();
}

// PATCH /api/profile/deactivate (R5) - then forgets the session and returns to Login.
private void deactivate() {
    UiUtils.hideMessage(textError);
    UiUtils.setLoading(buttonDeactivate, true);
    ApiClient.get(this).deactivateProfile().enqueue(new Callback<MessageResponse>() {
        // The API answered: deactivated -> log out; otherwise show the reason.
        @Override
        public void onResponse(Call<MessageResponse> call, Response<MessageResponse> response) {
            UiUtils.setLoading(buttonDeactivate, false);
            if (response.isSuccessful()) {
                MessageResponse body = response.body();
                UiUtils.toast(ProfileActivity.this, body != null && body.message != null ? body.message : "Account deactivated.");
                SessionGuard.logout(ProfileActivity.this);
            } else if (response.code() == 401) {
                SessionGuard.handleUnauthorized(ProfileActivity.this);
            } else {
                UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
            }
        }
    });

    // The call never reached the API.

```

```

        @Override
        public void onFailure(Call<MessageResponse> call, Throwable error) {
            UiUtils.setLoading(buttonDeactivate, false);
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ProfileActivity.this));
        }
    });
}
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/ProsumerHomeActivity.java

```

/*
 * =====
 * File      : ProsumerHomeActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : M3 Prosumer Home - the first screen after a prosumer logs
 *              in: greeting, count cards (Pending, Approved upcoming,
 *              Completed) and the next booking, all worked out by the API
 *              (GET /api/dashboard/prosumer, docs/01-SPEC.md section 5),
 *              refreshed every time the screen comes back. Tapping a count
 *              opens that list (M8); the next booking opens its details
 *              (M9). (The My profile button was added with M4 Profile,
 *              Part A; the New booking button with M6, Part C; the Nearby
 *              stations button with M5, Part B.)
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.ProsumerDashboardResponse;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.db.Session;
import com.sunshare.app.db.SunShareDbHelper;
import com.sunshare.app.ui.map.StationMapActivity;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class ProsumerHomeActivity extends AppCompatActivity {

    private TextView textGreeting;
    private TextView textRole;
    private TextView textPendingCount;
    private TextView textApprovedCount;
    private TextView textCompletedCount;
    private View nextDetails;
    private TextView textNextStation;
    private TextView textNextStatus;
    private TextView textNextTime;
    private TextView textNextEnergy;
    private TextView textNoNext;
    private TextView textError;
    // The next booking from the last load (null if there is none), opened when its card is tapped.
    private ReservationResponse nextBooking;

    // Called when the screen opens: keeps it clear of the phone's bars, makes sure someone
    // is logged in (else back to Login), and links the cards and buttons.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_prosumer_home);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), findViewById(R.id.header));
        if (SessionGuard.require(this) == null) {
            return;
        }
    }
}

```



```

textGreeting = findViewById(R.id.textGreeting);
textRole = findViewById(R.id.textRole);
textPendingCount = findViewById(R.id.textPendingCount);
textApprovedCount = findViewById(R.id.textApprovedCount);
textCompletedCount = findViewById(R.id.textCompletedCount);
nextDetails = findViewById(R.id.nextDetails);
textNextStation = findViewById(R.id.textNextStation);
textNextStatus = findViewById(R.id.textNextStatus);
textNextTime = findViewById(R.id.textNextTime);
textNextEnergy = findViewById(R.id.textNextEnergy);
textNoNext = findViewById(R.id.textNoNext);
textError = findViewById(R.id.textError);

// Each count card opens the matching tab of My bookings.
findViewById(R.id.cardPending).setOnClickListener(v -> openBookings("pending"));
findViewById(R.id.cardApproved).setOnClickListener(v -> openBookings("current"));
findViewById(R.id.cardCompleted).setOnClickListener(v -> openBookings("history"));
findViewById(R.id.cardNext).setOnClickListener(v -> openNextBooking());
findViewById(R.id.buttonNewBooking).setOnClickListener(v -> startActivity(new Intent(this,
ReservationFormActivity.class)));
findViewById(R.id.buttonBookings).setOnClickListener(v -> openBookings("current"));
findViewById(R.id.buttonMap).setOnClickListener(v -> startActivity(new Intent(this, StationMapActivity.class)));
findViewById(R.id.buttonLogout).setOnClickListener(v -> SessionGuard.logout(this));
findViewById(R.id.buttonProfile).setOnClickListener(v -> startActivity(new Intent(this, ProfileActivity.class)));
}

// Called every time the screen comes to the front (also after returning from Profile or a booking):
// shows the greeting from the saved session and reloads the counts.
@Override
protected void onResume() {
    super.onResume();
    Session session = SunShareDbHelper.get(this).getSession();
    if (session == null || textGreeting == null) {
        return;
    }
    textGreeting.setText("Hello, " + firstName(session.fullName));
    textRole.setText("Prosumer · NIC " + session.nic);
    loadDashboard();
}

// GET /api/dashboard/prosumer - the API counts only this prosumer's bookings (NIC from the token, R16).
private void loadDashboard() {
    UiUtils.hideMessage(textError);
    ApiClient.get(this).getProsumerDashboard().enqueue(new Callback<ProsumerDashboardResponse>() {
        // The API answered: show the numbers, or the reason it failed.
        @Override
        public void onResponse(Call<ProsumerDashboardResponse> call, Response<ProsumerDashboardResponse> response) {
            if (response.isSuccessful() && response.body() != null) {
                showDashboard(response.body());
            } else if (response.code() == 401) {
                SessionGuard.handleUnauthorized(ProsumerHomeActivity.this);
            } else {
                UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
            }
        }

        // The call never reached the API.
        @Override
        public void onFailure(Call<ProsumerDashboardResponse> call, Throwable error) {
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ProsumerHomeActivity.this));
        }
    });
}

// Puts the three counts and the next booking on the screen.
private void showDashboard(ProsumerDashboardResponse dashboard) {
    textPendingCount.setText(String.valueOf(dashboard.pendingCount));
    textApprovedCount.setText(String.valueOf(dashboard.approvedFutureCount));
    textCompletedCount.setText(String.valueOf(dashboard.completedCount));

    nextBooking = dashboard.nextReservation;
    if (nextBooking == null) {
        nextDetails.setVisibility(View.GONE);
        textNoNext.setVisibility(View.VISIBLE);
        return;
    }
    textNoNext.setVisibility(View.GONE);
    nextDetails.setVisibility(View.VISIBLE);
    textNextStation.setText(nextBooking.stationName);
    UiUtils.showStatusBadge(textNextStatus, nextBooking.status);
    textNextTime.setText(DateUtils.formatRange(nextBooking.startTime, nextBooking.endTime));
    textNextEnergy.setText(BookingAdapter.energyText(nextBooking));
}

```

```

// Opens My bookings (M8) on the given tab: "current", "pending" or "history".
private void openBookings(String view) {
    Intent intent = new Intent(this, BookingsActivity.class);
    intent.putExtra(BookingsActivity.EXTRA_VIEW, view);
    startActivity(intent);
}

// Opens the next booking's details (M9), if there is one.
private void openNextBooking() {
    if (nextBooking == null) {
        return;
    }
    Intent intent = new Intent(this, ReservationDetailActivity.class);
    intent.putExtra(ReservationDetailActivity.EXTRA_RESERVATION_ID, nextBooking.id);
    startActivity(intent);
}

// "Nimal Perera" -> "Nimal" (a friendlier greeting).
private String firstName(String fullName) {
    if (fullName == null || fullName.trim().isEmpty()) {
        return "there";
    }
    return fullName.trim().split("\\s+")[0];
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/ReservationActions.java

```

/*
 * =====
 * File      : ReservationActions.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author    : Malkith G W L (IT22630834)
 * Created   : 2026-09-29
 * Description : The Edit and Cancel buttons on the Booking details screen
 *              (M9). They are shown only when the API says canModify
 *              (Pending/Approved and at least 12 hours away - R10, R15).
 *              Edit opens the booking form (M6); Cancel asks first, then
 *              PATCH /api/reservations/{id}/cancel (the API gives the slot
 *              place back, R12) and opens the Summary (M7).
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.app.Activity;
import android.content.Intent;
import android.view.View;
import android.widget.Button;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public final class ReservationActions {

    // Only static helpers - nobody needs to create a ReservationActions object.
    private ReservationActions() {
    }

    // Shows Edit and Cancel only when the API allows changes (canModify), and links what they do.
    public static void show(Activity activity, ReservationResponse booking, Button buttonEdit, Button buttonCancel) {
        int visibility = booking.canModify ? View.VISIBLE : View.GONE;
        buttonEdit.setVisibility(visibility);
        buttonCancel.setVisibility(visibility);
        buttonEdit.setOnClickListener(v -> openEdit(activity, booking));
        buttonCancel.setOnClickListener(v -> confirmCancel(activity, booking, buttonCancel));
    }

    // Opens the booking form (M6) in edit mode for this booking.
    private static void openEdit(Activity activity, ReservationResponse booking) {
        Intent intent = new Intent(activity, ReservationFormActivity.class);
    }
}

```

```

        intent.putExtra(ReservationFormActivity.EXTRA_RESERVATION_ID, booking.id);
        activity.startActivity(intent);
    }

    // Asks "are you sure?" first - a cancelled booking can't be brought back (R15).
    private static void confirmCancel(Activity activity, ReservationResponse booking, Button buttonCancel) {
        new MaterialAlertDialogBuilder(activity)
            .setTitle("Cancel this booking?")
            .setMessage("Your place at " + booking.stationName + " will be given back. This can't be undone.")
            .setNegativeButton("Keep it", null)
            .setPositiveButton("Cancel booking", (dialog, which) -> cancel(activity, booking, buttonCancel))
            .show();
    }

    // PATCH /api/reservations/{id}/cancel - the API checks the 12-hour rule (R10) and gives the place back (R12).
    private static void cancel(Activity activity, ReservationResponse booking, Button buttonCancel) {
        UiUtils.setLoading(buttonCancel, true);
        ApiClient.get(activity).cancelReservation(booking.id).enqueue(new Callback<ReservationResponse>() {
            // The API answered: cancelled -> Summary; otherwise its reason (e.g. less than 12 hours away).
            @Override
            public void onResponse(Call<ReservationResponse> call, Response<ReservationResponse> response) {
                UiUtils.setLoading(buttonCancel, false);
                if (response.isSuccessful() && response.body() != null) {
                    ReservationSummaryActivity.open(activity, response.body(), ReservationSummaryActivity.ACTION_CANCELLED);
                } else if (response.code() == 401) {
                    SessionGuard.handleUnauthorized(activity);
                } else {
                    UiUtils.toast(activity, ApiErrorParser.fromResponse(response));
                }
            }
        });

        // The call never reached the API.
        @Override
        public void onFailure(Call<ReservationResponse> call, Throwable error) {
            UiUtils.setLoading(buttonCancel, false);
            UiUtils.toast(activity, ApiErrorParser.fromFailure(error));
        }
    }
}
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/ReservationDetailActivity.java

```

/*
 * =====
 * File      : ReservationDetailActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : D - Dashboards & QR
 * Author    : Chamara R M L K (IT22076816)
 * Created   : 2026-09-29
 * Description : M9 Booking details - one of the prosumer's bookings
 *              (GET /api/reservations/{id}; the API refuses other people's,
 *              R16). When the booking is Approved the API sends qrData
 *              ("SUNSHARE|<id>|<token>", R14) and this screen draws it as a
 *              QR code with ZXing for the Grid Operator to scan.
 *              (Edit / Cancel buttons: Part C, see ReservationActions.)
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.graphics.Bitmap;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.ProgressBar;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.appbar.MaterialToolbar;
import com.google.zxing.BarcodeFormat;
import com.google.zxing.WriterException;
import com.journeyapps.barcodescanner.BarcodeEncoder;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

```

```

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class ReservationDetailActivity extends AppCompatActivity {

    // The booking to show is passed in the Intent under this name.
    public static final String EXTRA_RESERVATION_ID = "reservationId";

    private String reservationId;
    private ProgressBar progress;
    private TextView textError;
    private View content;
    private TextView textStation;
    private TextView textStatus;
    private TextView textTime;
    private TextView textEnergy;
    private TextView textBookingId;
    private TextView textHistory;
    private View cardQr;
    private ImageView imageQr;
    private TextView textNote;
    private Button buttonEdit;
    private Button buttonCancel;

    // Called when the screen opens: makes sure someone is logged in, reads which booking to show
    // and links the views. The booking itself is loaded in onResume.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_reservation_detail);
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
        if (SessionGuard.require(this) == null) {
            return;
        }
        toolbar.setNavigationOnClickListener(v -> finish());
        reservationId = getIntent().getStringExtra(EXTRA_RESERVATION_ID);

        progress = findViewById(R.id.progress);
        textError = findViewById(R.id.textError);
        content = findViewById(R.id.content);
        textStation = findViewById(R.id.textStation);
        textStatus = findViewById(R.id.textStatus);
        textTime = findViewById(R.id.textTime);
        textEnergy = findViewById(R.id.textEnergy);
        textBookingId = findViewById(R.id.textBookingId);
        textHistory = findViewById(R.id.textHistory);
        cardQr = findViewById(R.id.cardQr);
        imageQr = findViewById(R.id.imageQr);
        textNote = findViewById(R.id.textNote);
        buttonEdit = findViewById(R.id.buttonEdit);
        buttonCancel = findViewById(R.id.buttonCancel);
    }

    // Called every time the screen comes to the front, so the details are fresh
    // (for example after the booking was edited or cancelled).
    @Override
    protected void onResume() {
        super.onResume();
        if (reservationId != null && progress != null) {
            loadReservation();
        }
    }

    // GET /api/reservations/{id} - the API only returns the caller's own booking (R16).
    private void loadReservation() {
        progress.setVisibility(View.VISIBLE);
        UiUtils.hideMessage(textError);
        ApiClient.get(this).getReservation(reservationId).enqueue(new Callback<ReservationResponse>() {
            // The API answered: show the booking, or the reason (403 not yours, 404 not found).
            @Override
            public void onResponse(Call<ReservationResponse> call, Response<ReservationResponse> response) {
                progress.setVisibility(View.GONE);
                if (response.isSuccessful() && response.body() != null) {
                    showReservation(response.body());
                } else if (response.code() == 401) {
                    SessionGuard.handleUnauthorized(ReservationDetailActivity.this);
                } else {
                    UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
                }
            }
        })
    }
}

```

```

        // The call never reached the API.
        @Override
        public void onFailure(Call<ReservationResponse> call, Throwable error) {
            progress.setVisibility(View.GONE);
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ReservationDetailActivity.this));
        }
    });
}

// Puts the booking on the screen: details, status badge, QR code (Approved only), a "what next" note
// and the Edit / Cancel buttons.
private void showReservation(ReservationResponse booking) {
    content.setVisibility(View.VISIBLE);
    textStation.setText(booking.stationName);
    UiUtils.showStatusBadge(textStatus, booking.status);
    textTime.setText(DateUtils.formatRange(booking.startTime, booking.endTime));
    textEnergy.setText(BookingAdapter.energyText(booking));
    textBookingId.setText("Booking ID " + booking.id);
    textHistory.setText(historyText(booking));
    showQr(booking);
    showNote(booking);
    // Edit / Cancel (Part C): shown only when the API says canModify.
    ReservationActions.show(this, booking, buttonEdit, buttonCancel);
}

// "Booked Mon, 28 Sep, 10:00 · Approved Tue, 29 Sep, 09:00" - when each step happened.
private String historyText(ReservationResponse booking) {
    String text = "Booked " + DateUtils.formatDateTime(booking.createdAt);
    if (booking.approvedAt != null) {
        text += " · Approved " + DateUtils.formatDateTime(booking.approvedAt);
    }
    if (booking.completedAt != null) {
        text += " · Completed " + DateUtils.formatDateTime(booking.completedAt);
    }
    if (booking.cancelledAt != null) {
        text += " · Cancelled " + DateUtils.formatDateTime(booking.cancelledAt);
    }
    return text;
}

// Shows the QR card only when the API sent qrData (it does so only for Approved bookings).
private void showQr(ReservationResponse booking) {
    Bitmap qr = booking.qrData == null || booking.qrData.isEmpty() ? null : makeQrCode(booking.qrData);
    if (qr == null) {
        cardQr.setVisibility(View.GONE);
        return;
    }
    imageQr.setImageBitmap(qr);
    cardQr.setVisibility(View.VISIBLE);
}

// Draws the text "SUNSHARE|<id>|<token>" as a QR code picture (600 x 600 pixels) with ZXing.
// Reference: ZXing Android Embedded - Generate a barcode https://github.com/journeyapps/zxing-android-embedded
private Bitmap makeQrCode(String qrData) {
    try {
        return new BarcodeEncoder().encodeBitmap(qrData, BarcodeFormat.QR_CODE, 600, 600);
    } catch (WriterException e) {
        UiUtils.toast(this, "Could not draw the QR code.");
        return null;
    }
}

// A short "what happens next" line for bookings that have no QR code.
private void showNote(ReservationResponse booking) {
    String note;
    switch (booking.status == null ? "" : booking.status) {
        case "Pending":
            note = "Waiting for approval. Your QR code will appear here once a Backoffice officer or Grid Operator approves the booking.";
            break;
        case "Completed":
            note = "Energy transfer completed. Thank you for trading with SunShare!";
            break;
        case "Cancelled":
            note = "This booking was cancelled. Its place in the slot was given back.";
            break;
        default:
            note = null;
            break;
    }
    if (note == null) {
        textNote.setVisibility(View.GONE);
    } else {
        textNote.setText(note);
    }
}

```

```

        textNote.setVisibility(View.VISIBLE);
    }
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/ReservationFormActivity.java

```

/*
 * =====
 * File      : ReservationFormActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author    : Malkith G W L (IT22630834)
 * Created   : 2026-09-29
 * Description : M6 Booking form - a prosumer books a slot, or edits one:
 *               station (GET /api/stations?activeOnly=true) -> bookable slot
 *               at that station (GET /api/slots/available) -> kWh -> Sell or
 *               Buy -> POST /api/reservations (new) or PUT
 *               /api/reservations/{id} (edit). The API checks every rule
 *               (R9 7 days, R10 12 hours, R11 free place, R12 counter, R13
 *               back to Pending, R17 kWh) and its message is shown here.
 *               On success the Summary screen (M7) opens.
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.os.Bundle;
import android.widget.AdapterView;
import android.widget.AutoCompleteTextView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.RadioButton;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.textfield.TextInputLayout;
import com.sunshare.app.R;
import com.sunshare.app.api.ApiClient;
import com.sunshare.app.api.ApiErrorParser;
import com.sunshare.app.api.models.CreateReservationRequest;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.api.models.SlotResponse;
import com.sunshare.app.api.models.StationResponse;
import com.sunshare.app.api.models.UpdateReservationRequest;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.List;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class ReservationFormActivity extends AppCompatActivity {

    // Present = edit this booking; missing = make a new booking.
    public static final String EXTRA_RESERVATION_ID = "reservationId";

    private String reservationId;
    private ReservationResponse current;
    private final List<StationResponse> stations = new ArrayList<>();
    private final List<SlotResponse> slots = new ArrayList<>();
    private String chosenStationId;
    private String chosenSlotId;
    // The slot list request that is running now, so choosing another station can cancel it.
    private Call<List<SlotResponse>> slotsCall;

    private AutoCompleteTextView inputStation;
    private TextInputLayout layoutSlot;
    private AutoCompleteTextView inputSlot;
    private EditText inputEnergy;
    private RadioButton radioSell;
    private RadioButton radioBuy;
    private TextView textError;
    private Button buttonSave;

```

```

// Called when the screen opens: makes sure someone is logged in, switches the texts to
// "edit" mode when a booking id was passed, links the lists and loads the stations.
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_reservation_form);
    MaterialToolbar toolbar = findViewById(R.id.toolbar);
    UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), toolbar);
    if (SessionGuard.require(this) == null) {
        return;
    }
    toolbar.setNavigationOnClickListener(v -> finish());

    inputStation = findViewById(R.id.inputStation);
    layoutSlot = findViewById(R.id.layoutSlot);
    inputSlot = findViewById(R.id.inputSlot);
    inputEnergy = findViewById(R.id.inputEnergy);
    radioSell = findViewById(R.id.radioSell);
    radioBuy = findViewById(R.id.radioBuy);
    textError = findViewById(R.id.textError);
    buttonSave = findViewById(R.id.buttonSave);

    reservationId = getIntent().getStringExtra(EXTRA_RESERVATION_ID);
    if (reservationId != null) {
        toolbar.setTitle(R.string.form_title_edit);
        ((TextView) findViewById(R.id.textIntro)).setText(R.string.form_intro_edit);
        buttonSave.setText(R.string.save_changes);
    }

    inputStation.setOnItemClickListener((parent, view, position, id) -> onStationChosen(stations.get(position)));
    inputSlot.setOnItemClickListener((parent, view, position, id) -> {
        chosenSlotId = slots.get(position).id;
        UiUtils.clearFieldError(inputSlot);
    });
    buttonSave.setOnClickListener(v -> save());
    loadStations();
}

// GET /api/stations?activeOnly=true - fills the station list; in edit mode the booking is loaded next.
private void loadStations() {
    UiUtils.setLoading(buttonSave, true);
    ApiClient.get(this).getStations(true).enqueue(new Callback<List<StationResponse>>() {
        // The API answered: show the stations, or the reason it failed.
        @Override
        public void onResponse(Call<List<StationResponse>> call, Response<List<StationResponse>> response) {
            if (response.isSuccessful() && response.body() != null) {
                showStations(response.body());
                if (reservationId != null) {
                    loadBooking();
                } else {
                    UiUtils.setLoading(buttonSave, false);
                }
            } else {
                UiUtils.setLoading(buttonSave, false);
                handleError(response);
            }
        }
    });

    // The call never reached the API.
    @Override
    public void onFailure(Call<List<StationResponse>> call, Throwable error) {
        UiUtils.setLoading(buttonSave, false);
        UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ReservationFormActivity.this));
    }
}

// Puts the station names into the station drop-down.
private void showStations(List<StationResponse> list) {
    stations.clear();
    stations.addAll(list);
    List<String> names = new ArrayList<>();
    for (StationResponse station : stations) {
        names.add(station.name);
    }
    inputStation.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, names));
}

// Edit mode: GET /api/reservations/{id}, then fills the form with the booking's current choices.
private void loadBooking() {
    ApiClient.get(this).getReservation(reservationId).enqueue(new Callback<ReservationResponse>() {
        // The API answered: fill the form, or show why the booking can't be loaded.
        @Override

```

```

        public void onResponse(Call<ReservationResponse> call, Response<ReservationResponse> response) {
            UiUtils.setLoading(buttonSave, false);
            if (response.isSuccessful() && response.body() != null) {
                fillForm(response.body());
            } else {
                handleError(response);
            }
        }

        // The call never reached the API.
        @Override
        public void onFailure(Call<ReservationResponse> call, Throwable error) {
            UiUtils.setLoading(buttonSave, false);
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ReservationFormActivity.this));
        }
    });
}

// Shows the booking's station, slot, kWh and type. If the API says it can't be changed any more
// (canModify = false: finished, cancelled or less than 12 hours away), saving is switched off.
private void fillForm(ReservationResponse booking) {
    current = booking;
    inputEnergy.setText(new DecimalFormat("0.##").format(booking.energyKwh));
    radioBuy.setChecked("Buy".equals(booking.type));
    radioSell.setChecked(!"Buy".equals(booking.type));
    for (StationResponse station : stations) {
        if (station.id.equals(booking.stationId)) {
            inputStation.setText(station.name, false);
            chosenStationId = station.id;
            loadSlots(station.id, booking.slotId);
        }
    }
    if (!booking.canModify) {
        UiUtils.showMessage(textError, "This booking can no longer be changed (it is finished, cancelled, or starts in less than 12 hours).");
        buttonSave.setEnabled(false);
    }
}

// A station was picked: forget the old slot and load this station's bookable slots.
private void onStationChosen(StationResponse station) {
    UiUtils.clearFieldError(inputStation);
    if (station.id.equals(chosenStationId)) {
        return;
    }
    chosenStationId = station.id;
    chosenSlotId = null;
    inputSlot.setText("", false);
    loadSlots(station.id, null);
}

// GET /api/slots/available?stationId= - only slots the API says can be booked (active, a free place,
// in the future and within 7 days - R9, R11). keepSlotId (edit mode) is selected again afterwards.
private void loadSlots(String stationId, String keepSlotId) {
    if (slotsCall != null) {
        slotsCall.cancel();
    }
    layoutSlot.setEnabled(false);
    layoutSlot.setHelperText("Loading slots...");
    slotsCall = ApiClient.get(this).getAvailableSlots(stationId);
    slotsCall.enqueue(new Callback<List<SlotResponse>>() {
        // The API answered: show the slots, or the reason it failed.
        @Override
        public void onResponse(Call<List<SlotResponse>> call, Response<List<SlotResponse>> response) {
            if (response.isSuccessful() && response.body() != null) {
                showSlots(stationId, response.body(), keepSlotId);
            } else {
                layoutSlot.setHelperText(null);
                handleError(response);
            }
        }
    });

    // No answer. A cancelled call (another station was picked) is simply ignored.
    @Override
    public void onFailure(Call<List<SlotResponse>> call, Throwable error) {
        if (call.isCanceled()) {
            return;
        }
        layoutSlot.setHelperText(null);
        UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ReservationFormActivity.this));
    }
}

});
}

// Puts the slots into the slot drop-down. When editing, the booking's own slot stays in the list

```



```

// even if it is now full (its place is held by this booking) - same as the web form.
private void showSlots(String stationId, List<SlotResponse> list, String keepSlotId) {
    slots.clear();
    if (current != null && stationId.equals(current.stationId) && !containsSlot(list, current.slotId)) {
        slots.add(currentSlotOf(current));
    }
    slots.addAll(list);

    List<String> labels = new ArrayList<>();
    for (SlotResponse slot : slots) {
        labels.add(slotLabel(slot));
    }
    inputSlot.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, labels));
    layoutSlot.setEnabled(true);
    layoutSlot.setHelperText(slots.isEmpty()
        ? "No bookable slots at this station in the next 7 days."
        : "Only slots in the next 7 days with a free place are shown.");

    for (int i = 0; i < slots.size(); i++) {
        if (slots.get(i).id.equals(keepSlotId)) {
            inputSlot.setText(labels.get(i), false);
            chosenSlotId = keepSlotId;
        }
    }
}

// True when the list already has a slot with this id.
private boolean containsSlot(List<SlotResponse> list, String slotId) {
    for (SlotResponse slot : list) {
        if (slot.id.equals(slotId)) {
            return true;
        }
    }
    return false;
}

// A list entry for the booking's own slot, built from the booking (availableSlots -1 = "current").
private SlotResponse currentSlotOf(ReservationResponse booking) {
    SlotResponse slot = new SlotResponse();
    slot.id = booking.slotId;
    slot.stationId = booking.stationId;
    slot.startTime = booking.startTime;
    slot.endTime = booking.endTime;
    slot.availableSlots = -1;
    return slot;
}

// "Thu, 1 Oct, 08:00 - 10:00 · 3 free", or "... (your current slot)".
private String slotLabel(SlotResponse slot) {
    String time = DateUtils.formatRange(slot.startTime, slot.endTime);
    return slot.availableSlots < 0 ? time + " (your current slot)" : time + " · " + slot.availableSlots + " free";
}

// Checks a station, a slot and a kWh number were given, then sends the booking to the API.
// Whether the booking is allowed (7 days, 12 hours, free place, 0-100 kWh) is decided by the API.
private void save() {
    UiUtils.hideMessage(textError);
    if (chosenStationId == null) {
        UiUtils.showFieldError(inputStation, "Please choose a station.");
        return;
    }
    if (chosenSlotId == null) {
        UiUtils.showFieldError(inputSlot, "Please choose a time slot.");
        return;
    }
    if (!UiUtils.requireFilled(inputEnergy, "Please enter the energy in kWh. ")) {
        return;
    }
    double energyKwh;
    try {
        energyKwh = Double.parseDouble(UiUtils.textOf(inputEnergy));
    } catch (NumberFormatException notANumber) {
        UiUtils.showFieldError(inputEnergy, "Please enter a number, e.g. 12.5");
        return;
    }
    String type = radioBuy.isChecked() ? "Buy" : "Sell";

    UiUtils.setLoading(buttonSave, true);
    if (reservationId == null) {
        ApiClient.get(this).createReservation(new CreateReservationRequest(chosenSlotId, energyKwh, type))
            .enqueue(saveCallback(ReservationSummaryActivity.ACTION_CREATED));
    } else {
        ApiClient.get(this).updateReservation(reservationId, new UpdateReservationRequest(chosenSlotId, energyKwh, type))
            .enqueue(saveCallback(ReservationSummaryActivity.ACTION_UPDATED));
    }
}

```

```

    }
}

// What to do with the API's answer to a create or update: success opens the Summary (M7) and closes
// this form (so Back can't submit it twice); otherwise the API's reason is shown.
private Callback<ReservationResponse> saveCallback(String action) {
    return new Callback<ReservationResponse>() {
        // The API answered: saved, or refused with a reason (e.g. the 7-day or 12-hour rule, slot full).
        @Override
        public void onResponse(Call<ReservationResponse> call, Response<ReservationResponse> response) {
            UiUtils.setLoading(buttonSave, false);
            if (response.isSuccessful() && response.body() != null) {
                ReservationSummaryActivity.open(ReservationFormActivity.this, response.body(), action);
                finish();
            } else {
                handleError(response);
            }
        }

        // The call never reached the API.
        @Override
        public void onFailure(Call<ReservationResponse> call, Throwable error) {
            UiUtils.setLoading(buttonSave, false);
            UiUtils.showMessage(textError, ApiErrorParser.fromFailure(ReservationFormActivity.this));
        }
    };
}

// An API error: 401 = the login ended (back to Login); anything else shows the API's message in red.
private void handleError(Response<?> response) {
    if (response.code() == 401) {
        SessionGuard.handleUnauthorized(this);
    } else {
        UiUtils.showMessage(textError, ApiErrorParser.fromResponse(response));
    }
}
}
}

```

android/app/src/main/java/com/sunshare/app/ui/prosumer/ReservationSummaryActivity.java

```

/*
 * =====
 * File      : ReservationSummaryActivity.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : C - Reservations
 * Author    : Malkith G W L (IT22630834)
 * Created   : 2026-09-29
 * Description : M7 Summary - shown after every booking action (Created,
 *                  Updated, Cancelled, and Completed by a Grid Operator): a big
 *                  icon, what happened, the booking as the API saved it, what
 *                  happens next, and buttons to My bookings / Home. The screen
 *                  makes no API call: the booking the API just returned is
 *                  handed over as JSON text in the Intent.
 * =====
 */
package com.sunshare.app.ui.prosumer;

import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.google.gson.Gson;
import com.sunshare.app.R;
import com.sunshare.app.api.models.ReservationResponse;
import com.sunshare.app.db.Session;
import com.sunshare.app.ui.operator.OperatorHomeActivity;
import com.sunshare.app.util.DateUtils;
import com.sunshare.app.util.SessionGuard;
import com.sunshare.app.util.UiUtils;

public class ReservationSummaryActivity extends AppCompatActivity {

    // Names of the values passed in the Intent.
    private static final String EXTRA_RESERVATION_JSON = "reservationJson";
    private static final String EXTRA_ACTION = "action";

```

```

// What just happened to the booking.
public static final String ACTION_CREATED = "created";
public static final String ACTION_UPDATED = "updated";
public static final String ACTION_CANCELLED = "cancelled";
public static final String ACTION_COMPLETED = "completed";

private Session session;

// Opens the summary for the booking the API just returned. Gson turns the booking into JSON text
// so it can travel inside the Intent to the next screen.
public static void open(Activity from, ReservationResponse booking, String action) {
    Intent intent = new Intent(from, ReservationSummaryActivity.class);
    intent.putExtra(EXTRA_RESERVATION_JSON, new Gson().toJson(booking));
    intent.putExtra(EXTRA_ACTION, action);
    from.startActivity(intent);
}

// Called when the screen opens: makes sure someone is logged in, turns the JSON back into
// a booking and shows the summary.
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_reservation_summary);
    UiUtils.setupEdgeToEdge(this, findViewById(R.id.main), findViewById(R.id.toolbar));
    session = SessionGuard.require(this);
    if (session == null) {
        return;
    }

    String json = getIntent().getStringExtra(EXTRA_RESERVATION_JSON);
    ReservationResponse booking = new Gson().fromJson(json, ReservationResponse.class);
    if (booking == null) {
        finish();
        return;
    }
    String action = getIntent().getStringExtra(EXTRA_ACTION);
    showHeadline(booking, action);
    showBooking(booking);

    View buttonBookings = findViewById(R.id.buttonBookings);
    buttonBookings.setVisibility(session.isProsumer() ? View.VISIBLE : View.GONE);
    buttonBookings.setOnClickListener(v -> openBookings(booking));
    findViewById(R.id.buttonHome).setOnClickListener(v -> goHome());
}

// The big icon, the title and the "what happens next" line for this action.
private void showHeadline(ReservationResponse booking, String action) {
    String icon = "✓";
    int color = R.color.ss_primary;
    String title;
    String next;
    if (ACTION_CANCELLED.equals(action)) {
        icon = "✗";
        color = R.color.ss_grey;
        title = "Booking cancelled";
        next = "Its place in the slot was given back. You can make a new booking any time.";
    } else if (ACTION_COMPLETED.equals(action)) {
        color = R.color.ss_success;
        title = "Energy transfer completed";
        next = "The booking is now Completed and can't be changed.";
    } else if (ACTION_UPDATED.equals(action)) {
        title = "Booking updated";
        next = "Pending".equals(booking.status)
            ? "It is Pending until a Backoffice officer or Grid Operator approves it again. An old QR code no longer
works." : "Nothing needed changing, so your booking stays as it was.";
    } else {
        title = "Booking created";
        next = "It is Pending. Once a Backoffice officer or Grid Operator approves it, your QR code appears in My
bookings.";
    }

    TextView textIcon = findViewById(R.id.textIcon);
    textIcon.setText(icon);
    textIcon.setBackgroundTintList(ContextCompat.getColorStateList(this, color));
    ((TextView) findViewById(R.id.textTitle)).setText(title);
    ((TextView) findViewById(R.id.textNext)).setText(next);
}

// The booking as the API saved it. Grid Operators also see whose booking it is.
private void showBooking(ReservationResponse booking) {
    ((TextView) findViewById(R.id.textStation)).setText(booking.stationName);
}

```

```

        UiUtils.showStatusBadge(findViewById(R.id.textStatus), booking.status);
        ((TextView) findViewById(R.id.textTime)).setText(DateUtils.formatRange(booking.startTime, booking.endTime));
        ((TextView) findViewById(R.id.textEnergy)).setText(BookingAdapter.energyText(booking));
        ((TextView) findViewById(R.id.textBookingId)).setText("Booking ID " + booking.id);
        if (session.isGridOperator()) {
            TextView textProsumer = findViewById(R.id.textProsumer);
            textProsumer.setText(booking.prosumerName + " (" + booking.prosumerNic + ")");
            textProsumer.setVisibility(View.VISIBLE);
        }
    }

// Home screen for this user's role, with every screen above it closed (so Back won't return to a form).
private Intent homeIntent() {
    Class<?> home = session.isGridOperator() ? OperatorHomeActivity.class : ProsumerHomeActivity.class;
    Intent intent = new Intent(this, home);
    intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_SINGLE_TOP);
    return intent;
}

// Back to Home.
private void goHome() {
    startActivity(homeIntent());
    finish();
}

// Home, then My bookings on top of it, opened on the tab this booking now belongs to.
private void openBookings(ReservationResponse booking) {
    Intent bookings = new Intent(this, BookingsActivity.class);
    bookings.putExtra(BookingsActivity.EXTRA_VIEW, tabFor(booking.status));
    startActivity(new Intent[]{homeIntent(), bookings});
    finish();
}

// Pending -> the Pending tab, Approved -> Current, Completed / Cancelled -> History.
private String tabFor(String status) {
    if ("Pending".equals(status)) {
        return "pending";
    }
    if ("Approved".equals(status)) {
        return "current";
    }
    return "history";
}
}

```

android/app/src/main/java/com/sunshare/app/util/DateUtils.java

```

/*
 * =====
 * File      : DateUtils.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Turns the API's UTC time text (e.g. "2026-09-30T05:30:00Z")
 *              into the phone's local time for display only, e.g.
 *              "Wed, 30 Sep, 11:00". All time rules (7 days, 12 hours) are
 *              worked out by the API, never here (docs/02-ARCHITECTURE.md
 *              section 8).
 * =====
 */
package com.sunshare.app.util;

import java.time.Instant;
import java.time.LocalDateTime;
import java.time.OffsetDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.Locale;

public final class DateUtils {

    private static final DateTimeFormatter DATE_TIME = DateTimeFormatter.ofPattern("EEE, d MMM, HH:mm", Locale.ENGLISH);
    private static final DateTimeFormatter DATE = DateTimeFormatter.ofPattern("EEE, d MMM yyyy", Locale.ENGLISH);
    private static final DateTimeFormatter TIME = DateTimeFormatter.ofPattern("HH:mm", Locale.ENGLISH);

    // Only static helpers - nobody needs to create a DateUtils object.
    private DateUtils() {

```

```

}

// Reads an ISO time from the API and moves it to the phone's time zone. Returns null if the text is empty or unreadable.
// "...Z" and "+05:30" times are read as they are; a time with no zone at all is taken as UTC (same as the API).
public static ZonedDateTime toLocal(String isoText) {
    if (isoText == null || isoText.isEmpty()) {
        return null;
    }
    ZoneId phoneZone = ZoneId.systemDefault();
    try {
        return Instant.parse(isoText).atZone(phoneZone);
    } catch (DateTimeParseException notZ) {
        try {
            return OffsetDateTime.parse(isoText).atZoneSameInstant(phoneZone);
        } catch (DateTimeParseException notOffset) {
            try {
                return LocalDateTime.parse(isoText).atOffset(ZoneOffset.UTC).atZoneSameInstant(phoneZone);
            } catch (DateTimeParseException unreadable) {
                return null;
            }
        }
    }
}

// "Wed, 30 Sep, 11:00" - for booking cards and details.
public static String formatDateTime(String isoText) {
    ZonedDateTime local = toLocal(isoText);
    return local == null ? "-" : DATE_TIME.format(local);
}

// "Wed, 30 Sep 2026" - dates only.
public static String formatDate(String isoText) {
    ZonedDateTime local = toLocal(isoText);
    return local == null ? "-" : DATE.format(local);
}

// "11:00" - times only.
public static String formatTime(String isoText) {
    ZonedDateTime local = toLocal(isoText);
    return local == null ? "-" : TIME.format(local);
}

// "Wed, 30 Sep, 11:00 - 12:00" - a slot or booking window (start date + both times).
public static String formatRange(String startIso, String endIso) {
    return formatDateTime(startIso) + " - " + formatTime(endIso);
}
}

```

android/app/src/main/java/com/sunshare/app/util/SessionGuard.java

```

/*
 * =====
 * File      : SessionGuard.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Keeps logged-out users out of logged-in screens. A screen
 *              calls require() when it opens; logout and an expired token
 *              (HTTP 401) clear the SQLite session and go back to the
 *              app's first screen (Login), so Back can't return.
 * =====
 */
package com.sunshare.app.util;

import android.app.Activity;
import android.content.Intent;

import com.sunshare.app.db.Session;
import com.sunshare.app.db.SunShareDbHelper;

public final class SessionGuard {

    // Only static helpers - nobody needs to create a SessionGuard object.
    private SessionGuard() {
    }

    // Returns the logged-in user. If nobody is logged in, opens the first screen and returns null,
    // so the calling screen should stop: "Session s = SessionGuard.require(this); if (s == null) return;"
    public static Session require(Activity activity) {

```

```

        Session session = SunShareDbHelper.get(activity).getSession();
        if (session == null) {
            goToStart(activity);
        }
        return session;
    }

    // Logout button: forgets the session and goes back to the first screen.
    public static void logout(Activity activity) {
        SunShareDbHelper.get(activity).clearSession();
        goToStart(activity);
    }

    // The API answered 401 on a logged-in screen: the token expired (8 hours) or is no longer valid.
    public static void handleUnauthorized(Activity activity) {
        SunShareDbHelper.get(activity).clearSession();
        UiUtils.toast(activity, "Your session has ended. Please log in again.");
        goToStart(activity);
    }

    // Opens the app's launcher screen as a fresh start (old screens are removed, so Back can't reopen them).
    private static void goToStart(Activity activity) {
        Intent intent = activity.getPackageManager().getLaunchIntentForPackage(activity.getPackageName());
        if (intent != null) {
            intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
            activity.startActivity(intent);
        }
        activity.finish();
    }
}

```

android/app/src/main/java/com/sunshare/app/util/UiUtils.java

```

/*
 * =====
 * File      : UiUtils.java
 * Project   : SunShare - Smart Solar Microgrid Trading System
 * Module    : SE4040 Enterprise Application Development - Assignment 1
 * Part      : B - Shared foundation (Android shell)
 * Author    : Ranathunga R A K N (IT22552860)
 * Created   : 2026-09-29
 * Description : Small screen helpers every Activity uses: show a message,
 *               read a text box, check a box is not empty (the only check
 *               the app does itself - real rules are in the API), put a
 *               button into a "Please wait..." state during an API call,
 *               colour a booking status label, and keep the screen clear of
 *               the phone's status/navigation bars and keyboard
 *               (edge-to-edge, required from Android 15).
 * =====
 */
package com.sunshare.app.util;

import android.content.Context;
import android.graphics.Color;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.ComponentActivity;
import androidx.activity.EdgeToEdge;
import androidx.activity.SystemBarStyle;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.android.material.textfield.TextInputLayout;
import com.sunshare.app.R;

public final class UiUtils {

    // Only static helpers - nobody needs to create a UiUtils object.
    private UiUtils() {
    }

    // Shows a short message at the bottom of the screen (used for API messages and errors).
    public static void toast(Context context, String message) {
        Toast.makeText(context, message, Toast.LENGTH_LONG).show();
    }
}

```

```

}

// Returns what the user typed, without spaces at the start or end.
public static String textOf(EditText field) {
    return field.getText() == null ? "" : field.getText().toString().trim();
}

// "Field not empty" check: shows the message on the box and returns false when nothing was typed.
public static boolean requireFilled(EditText field, String message) {
    if (textOf(field).isEmpty()) {
        showFieldError(field, message);
        return false;
    }
    clearFieldError(field);
    return true;
}

// Shows an error on one box: in red under the box when it sits in a Material TextInputLayout
// (so it never covers the box's icons), otherwise as Android's small popup.
public static void showFieldError(EditText field, String message) {
    TextInputLayout layout = findInputLayout(field);
    if (layout != null) {
        layout.setError(message);
    } else {
        field.setError(message);
    }
    field.requestFocus();
}

// Removes the error from a box (called when the box is filled in on the next try).
public static void clearFieldError(EditText field) {
    TextInputLayout layout = findInputLayout(field);
    if (layout != null) {
        layout.setError(null);
    } else {
        field.setError(null);
    }
}

// Walks up from the text box to the TextInputLayout around it (null if there is none).
private static TextInputLayout findInputLayout(View view) {
    ViewParent parent = view.getParent();
    while (parent instanceof View) {
        if (parent instanceof TextInputLayout) {
            return (TextInputLayout) parent;
        }
        parent = parent.getParent();
    }
    return null;
}

// Shows a message in a TextView on the screen (e.g. the red error line under a form).
public static void showMessage(TextView view, String message) {
    view.setText(message);
    view.setVisibility(View.VISIBLE);
}

// Hides that message again (called when the user tries again).
public static void hideMessage(TextView view) {
    view.setText("");
    view.setVisibility(View.GONE);
}

// Colours a status label like the web badges (docs/05-SCREENS.md §1): Pending amber with dark text,
// Approved teal, Completed green, Cancelled grey. The label uses style Widget.SunShare.StatusBadge.
public static void showStatusBadge(TextView badge, String status) {
    int background;
    int text = R.color.white;
    switch (status == null ? "" : status) {
        case "Pending":
            background = R.color.ss_accent;
            text = R.color.ss_dark;
            break;
        case "Approved":
            background = R.color.ss_primary;
            break;
        case "Completed":
            background = R.color.ss_success;
            break;
        default:
            background = R.color.ss_grey;
            break;
    }
    badge.setText(status);
}

```

```

        badge.setBackgroundTintList(ContextCompat.getColorStateList(badge.getContext(), background));
        badge.setTextColor(ContextCompat.getColor(badge.getContext(), text));
    }

    // Android 15+ always draws apps behind the status bar (top) and navigation bar (bottom).
    // This keeps content clear of them: the dark top bar (toolbar or header) grows by the status bar's
    // height so the bar area is navy with white icons, and the root layout gets bottom padding for the
    // navigation bar - or for the keyboard while it is open, so the focused box is never hidden.
    // (Spacing inside the root must be margins or an inner layout: this replaces the root's padding - C8.)
    public static void setupEdgeToEdge(ComponentActivity activity, View root, View topBar) {
        EdgeToEdge.enable(activity, SystemBarStyle.dark(Color.TRANSPARENT));
        int topBarPaddingTop = topBar.getPaddingTop();
        ViewCompat.setOnApplyWindowInsetsListener(root, (v, insets) -> {
            Insets bars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            Insets keyboard = insets.getInsets(WindowInsetsCompat.Type.ime());
            topBar.setPadding(topBar.getPaddingLeft(), topBarPaddingTop + bars.top,
                topBar.getPaddingRight(), topBar.getPaddingBottom());
            v.setPadding(bars.left, 0, bars.right, Math.max(bars.bottom, keyboard.bottom));
            return insets;
        });
    }

    // Disables the button and shows "Please wait..." while a call runs, then puts the old text back.
    // The old text is kept in the button's tag so no extra variable is needed in the screen.
    public static void setLoading(Button button, boolean loading) {
        if (loading) {
            if (button.getTag() == null) {
                button.setTag(button.getText());
            }
            button.setText("Please wait...");
            button.setEnabled(false);
        } else {
            if (button.getTag() != null) {
                button.setText((CharSequence) button.getTag());
                button.setTag(null);
            }
            button.setEnabled(true);
        }
    }
}

```

A.4 Android app - manifest, layouts, resources and map page

android/app/build.gradle.kts

// File: app/build.gradle.kts | Part B (Android shell) | Build settings and libraries of the SunShare Android app.
import java.util.Properties

```

plugins {
    alias(libs.plugins.android.application)
}

// Read local.properties (never committed to git) so the Google Maps key stays private.
val localProperties = Properties().apply {
    val file = rootProject.file("local.properties")
    if (file.exists()) {
        file.inputStream().use { load(it) }
    }
}

android {
    namespace = "com.sunshare.app"
    compileSdk {
        version = release(37)
    }

    defaultConfig {
        applicationId = "com.sunshare.app"
        minSdk = 26
        targetSdk = 37
        versionCode = 1
        versionName = "1.0"

        // ${MAPS_API_KEY} in AndroidManifest.xml is replaced by the key from local.properties (empty if missing).
        manifestPlaceholders["MAPS_API_KEY"] = localProperties.getProperty("MAPS_API_KEY", "")
    }

    buildTypes {
        release {
            optimization {

```



```

        enable = false
    }
}
}
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_11
    targetCompatibility = JavaVersion.VERSION_11
}
}

dependencies {
    // Template defaults
    implementation(libs.activity.ktx)
    implementation(libs.appcompat)
    implementation(libs.constraintlayout)
    implementation(libs.material)
    // Lists (bookings)
    implementation(libs.recyclerview)
    // Calling the SunShare API: Retrofit + Gson (JSON <-> Java objects)
    implementation(libs.retrofit)
    implementation(libs.retrofit.converter.gson)
    // QR codes: make (prosumer) and scan (operator)
    implementation(libs.zxing.android.embedded)
    // The phone's location for the nearby stations map (the map itself is Google's JavaScript API in a WebView, D53)
    implementation(libs.play.services.location)
}

```

android/app/src/main/AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: AndroidManifest.xml | Part B (Android shell) | App name, icon, permissions, screens and the Maps key. -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- Call the SunShare API over the network. -->
    <uses-permission android:name="android.permission.INTERNET" />
    <!-- Grid Operators scan QR codes with the camera. -->
    <uses-permission android:name="android.permission.CAMERA" />
    <!-- Find the phone's location for "nearby stations". -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <!-- The camera is used but not required (the app still installs on devices without one). -->
    <uses-feature android:name="android.hardware.camera" android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:networkSecurityConfig="@xml/network_security_config"
        android:theme="@style/Theme.SunShare">

        <!-- Google Maps (JavaScript API) key, filled from local.properties at build time (see app/build.gradle.kts).
        StationMapActivity reads it and puts it into the map page (D53). -->
        <meta-data
            android:name="com.sunshare.app.MAPS_API_KEY"
            android:value="${MAPS_API_KEY}" />

        <!-- M1 Login (Part A) - the app's first screen -->
        <activity
            android:name=".ui.auth.LoginActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <!-- M2 Register (Part A) -->
        <activity
            android:name=".ui.auth.RegisterActivity"
            android:exported="false"
            android:windowSoftInputMode="adjustResize" />
        <!-- M4 Profile (Part A) -->
        <activity
            android:name=".ui.prosumer.ProfileActivity"
            android:exported="false"

```

```

        android:windowSoftInputMode="adjustResize" />

<!-- M3 Prosumer Home (Part D) -->
<activity
    android:name=".ui.prosumer.ProsumerHomeActivity"
    android:exported="false" />
<!-- M8 My bookings (Part D) -->
<activity
    android:name=".ui.prosumer.BookingsActivity"
    android:exported="false"
    android:windowSoftInputMode="adjustResize" />
<!-- M9 Booking details (Part D) -->
<activity
    android:name=".ui.prosumer.ReservationDetailActivity"
    android:exported="false" />
<!-- M6 Booking form (Part C) -->
<activity
    android:name=".ui.prosumer.ReservationFormActivity"
    android:exported="false"
    android:windowSoftInputMode="adjustResize" />
<!-- M7 Summary (Part C) -->
<activity
    android:name=".ui.prosumer.ReservationSummaryActivity"
    android:exported="false" />
<!-- M5 Nearby stations map (Part B) -->
<activity
    android:name=".ui.map.StationMapActivity"
    android:exported="false" />
<!-- M10 Operator Home (Part D) -->
<activity
    android:name=".ui.operator.OperatorHomeActivity"
    android:exported="false" />
<!-- M11 Scan result (Part D) -->
<activity
    android:name=".ui.operator.ScanResultActivity"
    android:exported="false" />
<!-- QR scanner (Part D): ZXing's scanner screen, kept clear of the phone's bars; turns with the phone -->
<activity
    android:name=".ui.operator.QrScanActivity"
    android:exported="false"
    android:screenOrientation="fullSensor"
    android:stateNotNeeded="true"
    android:theme="@style/zxing_CaptureTheme"
    android:windowSoftInputMode="stateAlwaysHidden" />
</application>

</manifest>

```

android/app/src/main/assets/map.html

```

<!DOCTYPE html>
<!--
=====
File      : map.html
Project   : SunShare - Smart Solar Microgrid Trading System
Module    : SE4040 Enterprise Application Development - Assignment 1
Part      : B - Stations, slots & map
Author    : Ranathunga R A K N (IT22552860)
Created   : 2026-09-29
Description : The page shown inside the Nearby stations screen (M5) by a
WebView. StationMapActivity (Java) replaces __MAPS_API_KEY__
and __DATA__ (my location + the nearby stations from the API
or the SQLite cache) before loading it. This page only DRAWS:
a Google map (Maps JavaScript API, D53), a blue "you are
here" dot and one numbered marker per station (1 = nearest);
tapping a marker shows the station's details. If the Google
map can't load (no internet, key refused) it shows a plain
list of the same stations instead.
=====
-->
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Nearby stations</title>
    <style>
        html, body, #map { height: 100%; margin: 0; }
        body { font-family: sans-serif; color: #1E293B; background: #F8FAFC; }
        .info { max-width: 240px; font-size: 14px; line-height: 1.4; }
        .info b { font-size: 16px; font-weight: bold; color: #0B1F33; }
        .info .muted { color: #64748B; }
        .info a { display: inline-block; margin-top: 6px; color: #0F766E; font-weight: bold; }
        #fallback { padding: 16px; }
        #fallback h2 { margin: 0 0 4px; color: #0B1F33; }
        #fallback li { margin-bottom: 12px; }
    </style>

```

```

</style>
</head>
<body>
<div id="map"></div>
<div id="fallback" hidden>
  <h2>Nearby stations</h2>
  <p id="fallbackNote" class="muted"></p>
  <ol id="list"></ol>
</div>
<script>
  // Filled in by StationMapActivity before the page loads:
  // { "me": { "lat": 6.91, "lng": 79.97 }, "stations": [ { "name": ..., "latitude": ..., ... }, ... ] }
  var DATA = __DATA__;

  var map = null;
  var infoWindow = null;

  // Called by the Maps JavaScript API once it has loaded: draws the map, my dot and the station markers,
  // then zooms so that all of them fit on the screen.
  function initMap() {
    var me = { lat: DATA.me.lat, lng: DATA.me.lng };
    map = new google.maps.Map(document.getElementById('map'), {
      center: me, zoom: 12, streetViewControl: false, mapTypeControl: false, fullscreenControl: false
    });
    infoWindow = new google.maps.InfoWindow();
    var bounds = new google.maps.LatLngBounds();
    bounds.extend(me);
    addMyLocation(me);
    DATA.stations.forEach(function (station, index) {
      addStationMarker(station, index + 1);
      bounds.extend({ lat: station.latitude, lng: station.longitude });
    });
    if (DATA.stations.length > 0) {
      map.fitBounds(bounds, 60);
    }
  }

  // The blue "you are here" dot.
  function addMyLocation(me) {
    new google.maps.Marker({
      position: me, map: map, title: 'You are here', zIndex: 1,
      icon: { path: google.maps.SymbolPath.CIRCLE, scale: 9, fillColor: '#1A73E8', fillOpacity: 1, strokeColor:
'FFFFFFFF', strokeWeight: 3 }
    });
  }

  // One marker per station, labelled 1, 2, 3 ... (nearest first). Tapping it opens the details window (R18).
  function addStationMarker(station, number) {
    var marker = new google.maps.Marker({
      position: { lat: station.latitude, lng: station.longitude }, map: map, title: station.name,
      label: { text: String(number), color: 'FFFFFFFF', fontWeight: 'bold' }, zIndex: 2
    });
    marker.addListener('click', function () {
      infoWindow.setContent(detailsFor(station));
      infoWindow.open({ anchor: marker, map: map });
    });
  }

  // Builds the details box: name, address, capacity, battery slots, opening hours, distance and a
  // Directions link. Text is added with textContent, so a name can never be read as HTML.
  function detailsFor(station) {
    var box = document.createElement('div');
    box.className = 'info';
    addLine(box, 'b', station.name);
    addLine(box, 'div', station.address, 'muted');
    addLine(box, 'div', station.capacityKw + ' kW · ' + station.batterySlots + ' battery slots');
    addLine(box, 'div', 'Open ' + station.openTime + ' - ' + station.closeTime);
    if (station.distanceKm !== null && station.distanceKm !== undefined) {
      addLine(box, 'div', station.distanceKm + ' km away');
    }
    var link = document.createElement('a');
    link.href = directionsUrl(station);
    link.textContent = 'Directions in Google Maps';
    box.appendChild(link);
    return box;
  }

  // Adds one line of text (a new element of the given tag) to a box.
  function addLine(box, tag, text, className) {
    var line = document.createElement(tag);
    line.textContent = text;
    if (className) {
      line.className = className;
    }
    box.appendChild(line);
  }

```

```

    if (tag === 'b') {
        box.appendChild(document.createElement('br'));
    }
}

// A Google Maps URL (no key needed); Java opens it in the Google Maps app.
// Reference: Google, "Get started with Maps URLs" https://developers.google.com/maps/documentation/urls/get-started
function directionsUrl(station) {
    return 'https://www.google.com/maps/dir/?api=1&destination=' + station.latitude + ',' + station.longitude;
}

// When the Google map can't be shown: hide it and list the same stations as plain text.
function showList(note) {
    document.getElementById('map').hidden = true;
    document.getElementById('fallback').hidden = false;
    document.getElementById('fallbackNote').textContent = note;
    var list = document.getElementById('list');
    list.innerHTML = '';
    DATA.stations.forEach(function (station) {
        var item = document.createElement('li');
        item.appendChild(detailsFor(station));
        list.appendChild(item);
    });
}

// Google calls this if it refuses the key (for example the demo key's daily limit was reached).
window.gm_authFailure = function () {
    showList('The Google map is not available right now, so the stations are listed instead.');
```

```

};
</script>
<!-- Reference: Google, "Maps JavaScript API - Load the Maps JavaScript API"
https://developers.google.com/maps/documentation/javascript/load-maps-js-api -->
<script async
    src="https://maps.googleapis.com/maps/api/js?key=__MAPS_API_KEY__&loading=async&callback=initMap"
    onerror="showList('No internet connection, so the Google map can't load. The stations are listed
instead.')"></script>
</body>
</html>

```

android/app/src/main/res/layout/activity_bookings.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_bookings.xml | Part D | M8 My bookings: Current / Pending / History tabs, search box, list. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.prosumer.BookingsActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/bookings_title" />

    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tabs"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@color/ss_dark"
        app:tabIndicatorColor="@color/ss_accent"
        app:tabSelectedTextColor="@color/white"
        app:tabTextColor="@color/ss_grey" />

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginHorizontal="16dp"
        android:layout_marginTop="12dp"
        android:hint="@string/search_hint"
        app:endIconMode="clear_text"
        app:startIconContentDescription="@string/search_hint"
        app:startIconDrawable="@android:drawable/ic_menu_search">

        <com.google.android.material.textfield.TextInputEditText

```

```

        android:id="@+id/inputSearch"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:imeOptions="actionSearch"
        android:inputType="text" />
    </com.google.android.material.textfield.TextInputLayout>

    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/list"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:clipToPadding="false"
            android:paddingHorizontal="16dp"
            android:paddingTop="12dp"
            android:paddingBottom="16dp" />

        <ProgressBar
            android:id="@+id/progress"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:layout_marginTop="48dp"
            android:visibility="gone" />

        <TextView
            android:id="@+id/textMessage"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginHorizontal="24dp"
            android:layout_marginTop="48dp"
            android:gravity="center"
            android:textColor="@color/ss_muted"
            android:textSize="15sp"
            android:visibility="gone"
            tools:text="No bookings waiting for approval." />
    </FrameLayout>
</LinearLayout>

```

android/app/src/main/res/layout/activity_login.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_login.xml | Part A | M1 Login: brand header, NIC + password form, server address setting. -->
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:fillViewport="true"
    tools:context=".ui.auth.LoginActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Navy brand header with the server setting (Ⓞ) in the corner -->
        <LinearLayout
            android:id="@+id/header"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@color/ss_dark"
            android:gravity="center_horizontal"
            android:orientation="vertical"
            android:paddingBottom="32dp">

            <com.google.android.material.button.MaterialButton
                android:id="@+id/buttonServer"
                style="@style/Widget.Material3.Button.TextButton"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="end"
                android:layout_marginEnd="8dp"
                android:text="@string/server_button"
                android:textColor="@color/ss_grey" />

```

```

<ImageView
    android:layout_width="88dp"
    android:layout_height="88dp"
    android:contentDescription="@string/app_name"
    android:src="@mipmap/ic_launcher" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="@string/app_name"
    android:textColor="@color/white"
    android:textSize="30sp"
    android:textStyle="bold" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/tagline"
    android:textColor="@color/ss_accent"
    android:textSize="16sp" />
</LinearLayout>

<!-- Login form -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    app:cardBackgroundColor="@color/white"
    app:cardCornerRadius="16dp"
    app:strokeColor="@color/ss_primary_soft">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/login_title"
            android:textColor="@color/ss_dark"
            android:textSize="22sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            android:text="@string/login_subtitle"
            android:textColor="@color/ss_muted"
            android:textSize="14sp" />

        <com.google.android.material.textfield.TextInputLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="@string/nic">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/inputNic"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="textCapCharacters|textNoSuggestions" />
        </com.google.android.material.textfield.TextInputLayout>

        <com.google.android.material.textfield.TextInputLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:hint="@string/password"
            app:endIconMode="password_toggle">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/inputPassword"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:imeOptions="actionDone"
                android:inputType="textPassword" />
        </com.google.android.material.textfield.TextInputLayout>

    <TextView

```

```

        android:id="@+id/textError"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:textAppearance="@style/TextAppearance.SunShare.Error"
        android:visibility="gone" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonLogin"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/login_button" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonRegister"
            style="@style/Widget.Material3.Button.TextButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/register_link" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<TextView
    android:id="@+id/textServer"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="20dp"
    android:layout_marginBottom="20dp"
    android:gravity="center"
    android:textColor="@color/ss_muted"
    android:textSize="13sp"
    tools:text="Server: http://10.0.2.2:8080/" />
</LinearLayout>
</ScrollView>

```

android/app/src/main/res/layout/activity_operator_home.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_operator_home.xml | Part D | M10 Operator Home: greeting, today's counts and the Scan QR button. -->
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:fillViewport="true"
    tools:context=".ui.operator.OperatorHomeActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Navy header: greeting, who is logged in, Log out -->
        <LinearLayout
            android:id="@+id/header"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@color/ss_dark"
            android:orientation="vertical"
            android:paddingHorizontal="20dp"
            android:paddingTop="16dp"
            android:paddingBottom="28dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <TextView
                    android:id="@+id/textGreeting"
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:textColor="@color/white"
                    android:textSize="24sp"
                    android:textStyle="bold"
                    tools:text="Hello, Kasun" />

```

```

        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonLogout"
            style="@style/Widget.Material3.Button.TextButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/log_out"
            android:textColor="@color/ss_accent" />
    </LinearLayout>

    <TextView
        android:id="@+id/textRole"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="@color/ss_grey"
        android:textSize="14sp"
        tools:text="Grid Operator · NIC 199845678912" />
</LinearLayout>

<!-- Count cards (from GET /api/dashboard/summary) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="20dp"
    android:layout_marginTop="20dp"
    android:orientation="horizontal">

    <com.google.android.material.card.MaterialCardView
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        app:cardBackgroundColor="@color/white"
        app:cardCornerRadius="14dp"
        app:strokeColor="@color/ss_primary_soft">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="14dp">

            <TextView
                android:id="@+id/textTodayCount"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/count_loading"
                android:textColor="@color/ss_dark"
                android:textSize="28sp"
                android:textStyle="bold" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/count_today"
                android:textColor="@color/ss_dark"
                android:textSize="13sp" />
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_marginHorizontal="6dp"
        android:layout_weight="1"
        app:cardBackgroundColor="@color/ss_accent_soft"
        app:cardCornerRadius="14dp"
        app:strokeWidth="0dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="14dp">

            <TextView
                android:id="@+id/textPendingCount"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/count_loading"
                android:textColor="@color/ss_dark"
                android:textSize="28sp"
                android:textStyle="bold" />

```



```

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/count_pending"
            android:textColor="@color/ss_dark"
            android:textSize="13sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_marginStart="6dp"
    android:layout_weight="1"
    app:cardBackgroundColor="@color/ss_primary_soft"
    app:cardCornerRadius="14dp"
    app:strokeWidth="0dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="14dp">

        <TextView
            android:id="@+id/textApprovedCount"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/count_loading"
            android:textColor="@color/ss_primary_dark"
            android:textSize="28sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/count_approved"
            android:textColor="@color/ss_primary_dark"
            android:textSize="13sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<TextView
    android:id="@+id/textError"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="20dp"
    android:layout_marginTop="12dp"
    android:textAppearance="@style/TextAppearance.SunShare.Error"
    android:visibility="gone" />

<!-- Actions -->
<LinearLayout
    android:id="@+id/actions"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/operator_ready"
        android:textColor="@color/ss_dark"
        android:textSize="18sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="@string/scan_help"
        android:textColor="@color/ss_muted"
        android:textSize="14sp" />

<!-- Big Scan QR button (ZXing scanner) -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonScan"
    android:layout_width="match_parent"
    android:layout_height="72dp"
    android:layout_marginTop="16dp"
    android:text="@string/scan_qr"

```

```

        android:textSize="18sp"
        app:cornerRadius="16dp" />

        <!-- M5 Nearby stations (Part B) -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonMap"
            style="@style/Widget.Material3.Button.OutlinedButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:text="@string/nearby_stations" />
    </LinearLayout>
</LinearLayout>
</ScrollView>

```

android/app/src/main/res/layout/activity_profile.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_profile.xml | Part A | M4 Profile: edit own details, change password, deactivate account. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.prosumer.ProfileActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/profile_title" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:fillViewport="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <!-- NIC and status can't be edited here -->
            <com.google.android.material.card.MaterialCardView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginBottom="16dp"
                app:cardBackgroundColor="@color/ss_primary_soft"
                app:cardCornerRadius="12dp"
                app:strokeWidth="0dp">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:padding="16dp">

                    <TextView
                        android:id="@+id/textNic"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:textColor="@color/ss_dark"
                        android:textSize="16sp"
                        android:textStyle="bold"
                        tools:text="NIC 199712345678" />

                    <TextView
                        android:id="@+id/textStatus"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:textColor="@color/ss_primary_dark"
                        android:textSize="14sp"
                        tools:text="Prosumer · Active" />
                </LinearLayout>
            </com.google.android.material.card.MaterialCardView>
        </LinearLayout>
    </ScrollView>

```

```

</com.google.android.material.card.MaterialCardView>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/full_name">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputFullName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPersonName|textCapWords" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:hint="@string/email">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:hint="@string/phone">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputPhone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="phone" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:hint="@string/address_optional">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputAddress"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPostalAddress|textCapWords" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:hint="@string/new_password_optional"
    app:endIconMode="password_toggle"
    app:helperText="@string/new_password_helper">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputNewPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:imeOptions="actionDone"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout>

<TextView
    android:id="@+id/textError"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textAppearance="@style/TextAppearance.SunShare.Error"
    android:visibility="gone" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonSave"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"

```

```

        android:text="@string/save_changes" />

<!-- Close the account (R5) -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:text="@string/close_account_title"
    android:textColor="@color/ss_dark"
    android:textSize="16sp"
    android:textStyle="bold" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="@string/close_account_text"
    android:textColor="@color/ss_muted"
    android:textSize="14sp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonDeactivate"
    style="@style/Widget.Material3.Button.OutlinedButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="@string/deactivate_account"
    android:textColor="@color/ss_danger"
    app:strokeColor="@color/ss_danger" />
</LinearLayout>
</ScrollView>
</LinearLayout>

```

android/app/src/main/res/layout/activity_prosumer_home.xml

```

<?xml version="1.0" encoding="utf-8">
<!-- File: activity_prosumer_home.xml | Part D | M3 Prosumer Home: greeting, count cards, next booking and actions. -->
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:fillViewport="true"
    tools:context=".ui.prosumer.ProsumerHomeActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Navy header: greeting, who is logged in, Log out -->
        <LinearLayout
            android:id="@+id/header"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@color/ss_dark"
            android:orientation="vertical"
            android:paddingHorizontal="20dp"
            android:paddingTop="16dp"
            android:paddingBottom="28dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <TextView
                    android:id="@+id/textGreeting"
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:textColor="@color/white"
                    android:textSize="24sp"
                    android:textStyle="bold"
                    tools:text="Hello, Nimal" />

                <com.google.android.material.button.MaterialButton
                    android:id="@+id/buttonLogout"
                    style="@style/Widget.Material3.Button.TextButton"

```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/log_out"
        android:textColor="@color/ss_accent" />
    </LinearLayout>

    <TextView
        android:id="@+id/textRole"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="@color/ss_grey"
        android:textSize="14sp"
        tools:text="Prosumer · NIC 199712345678" />
</LinearLayout>

<!-- Count cards: tap one to open that list -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="20dp"
    android:layout_marginTop="20dp"
    android:orientation="horizontal">

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/cardPending"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:clickable="true"
        android:focusable="true"
        app:cardBackgroundColor="@color/ss_accent_soft"
        app:cardCornerRadius="14dp"
        app:strokeWidth="0dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="14dp">

            <TextView
                android:id="@+id/textPendingCount"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/count_loading"
                android:textColor="@color/ss_dark"
                android:textSize="28sp"
                android:textStyle="bold" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/count_pending"
                android:textColor="@color/ss_dark"
                android:textSize="13sp" />
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/cardApproved"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_marginHorizontal="6dp"
        android:layout_weight="1"
        android:clickable="true"
        android:focusable="true"
        app:cardBackgroundColor="@color/ss_primary_soft"
        app:cardCornerRadius="14dp"
        app:strokeWidth="0dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="14dp">

            <TextView
                android:id="@+id/textApprovedCount"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="@string/count_loading"
                android:textColor="@color/ss_primary_dark"

```

```

        android:textSize="28sp"
        android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/count_approved"
            android:textColor="@color/ss_primary_dark"
            android:textSize="13sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:id="@+id/cardCompleted"
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_marginStart="6dp"
    android:layout_weight="1"
    android:clickable="true"
    android:focusable="true"
    app:cardBackgroundColor="@color/ss_success_soft"
    app:cardCornerRadius="14dp"
    app:strokeWidth="0dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="14dp">

        <TextView
            android:id="@+id/textCompletedCount"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/count_loading"
            android:textColor="@color/ss_success"
            android:textSize="28sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/count_completed"
            android:textColor="@color/ss_success"
            android:textSize="13sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- Next booking -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/cardNext"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="20dp"
    android:layout_marginTop="16dp"
    android:clickable="true"
    android:focusable="true"
    app:cardBackgroundColor="@color/white"
    app:cardCornerRadius="14dp"
    app:strokeColor="@color/ss_primary_soft">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/next_booking"
            android:textColor="@color/ss_muted"
            android:textSize="13sp"
            android:textStyle="bold" />

        <LinearLayout
            android:id="@+id/nextDetails"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:orientation="vertical"
            android:visibility="gone">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
        android:id="@+id/textNextStation"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textColor="@color/ss_dark"
        android:textSize="17sp"
        android:textStyle="bold"
        tools:text="Malabe Solar Hub" />

    <TextView
        android:id="@+id/textNextStatus"
        style="@style/Widget.SunShare.StatusBadge"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        tools:text="Approved" />
</LinearLayout>

<TextView
    android:id="@+id/textNextTime"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:textColor="@color/ss_text"
    android:textSize="15sp"
    tools:text="Wed, 30 Sep, 11:00 - 12:00" />

<TextView
    android:id="@+id/textNextEnergy"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="@color/ss_muted"
    android:textSize="14sp"
    tools:text="12.5 kWh · Sell" />
</LinearLayout>

<TextView
    android:id="@+id/textNoNext"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:text="@string/no_next_booking"
    android:textColor="@color/ss_text"
    android:textSize="15sp"
    android:visibility="gone" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<TextView
    android:id="@+id/textError"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="20dp"
    android:layout_marginTop="12dp"
    android:textAppearance="@style/TextAppearance.SunShare.Error"
    android:visibility="gone" />

<!-- Actions -->
<LinearLayout
    android:id="@+id/actions"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="20dp"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:text="@string/home_prompt"
        android:textColor="@color/ss_dark"
        android:textSize="18sp"
        android:textStyle="bold" />

<!-- M6 New booking (Part C) -->

```

```

        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonNewBooking"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/new_booking" />

        <!-- M8 My bookings -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonBookings"
            style="@style/Widget.Material3.Button.OutlinedButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/my_bookings" />

        <!-- M5 Nearby stations (Part B) -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonMap"
            style="@style/Widget.Material3.Button.OutlinedButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/nearby_stations" />

        <!-- M4 Profile (Part A) -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/buttonProfile"
            style="@style/Widget.Material3.Button.OutlinedButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/my_profile" />
    </LinearLayout>
</LinearLayout>
</ScrollView>

```

android/app/src/main/res/layout/activity_register.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_register.xml | Part A | M2 Register: prosumer sign-up form. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.auth.RegisterActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/register_title" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:fillViewport="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginBottom="12dp"
                android:text="@string/register_intro"
                android:textColor="@color/ss_muted"
                android:textSize="15sp" />

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="@string/nic"
                app:helperText="@string/nic_helper">

```



```

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputNic"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textCapCharacters|textNoSuggestions" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="@string/full_name">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputFullName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPersonName|textCapWords" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="@string/email">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textEmailAddress" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="@string/phone">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputPhone"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="phone" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="@string/address_optional">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputAddress"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPostalAddress|textCapWords" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="@string/password"
        app:enablePasswordToggle="password_toggle"
        app:helperText="@string/password_helper">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPassword" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="@string/confirm_password"
        app:enablePasswordToggle="password_toggle">

```

```

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputConfirmPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:imeOptions="actionDone"
            android:inputType="textPassword" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:id="@+id/textError"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:textAppearance="@style/TextAppearance.SunShare.Error"
        android:visibility="gone" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonRegister"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/register_button" />
</LinearLayout>
</ScrollView>
</LinearLayout>

```

android/app/src/main/res/layout/activity_reservation_detail.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_reservation_detail.xml | Part D | M9 Booking details: status, details and the QR code when Approved. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.prosumer.ReservationDetailActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/detail_title" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:fillViewport="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <ProgressBar
                android:id="@+id/progress"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="center_horizontal"
                android:layout_marginTop="32dp" />

            <TextView
                android:id="@+id/textError"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:textAppearance="@style/TextAppearance.SunShare.Error"
                android:visibility="gone" />

            <!-- Everything below is shown once the booking has loaded -->
            <LinearLayout
                android:id="@+id/content"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"

```

```

        android:orientation="vertical"
        android:visibility="gone">

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardBackgroundColor="@color/white"
    app:cardCornerRadius="16dp"
    app:strokeColor="@color/ss_primary_soft">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <TextView
                android:id="@+id/textStation"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:textColor="@color/ss_dark"
                android:textSize="20sp"
                android:textStyle="bold"
                tools:text="Malabe Solar Hub" />

            <TextView
                android:id="@+id/textStatus"
                style="@style/Widget.SunShare.StatusBadge"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginStart="8dp"
                tools:text="Approved" />
        </LinearLayout>

        <TextView
            android:id="@+id/textTime"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:textColor="@color/ss_text"
            android:textSize="16sp"
            tools:text="Wed, 30 Sep, 11:00 - 12:00" />

        <TextView
            android:id="@+id/textEnergy"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:textColor="@color/ss_text"
            android:textSize="16sp"
            tools:text="12.5 kWh · Sell" />

        <TextView
            android:id="@+id/textBookingId"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:textColor="@color/ss_muted"
            android:textIsSelectable="true"
            android:textSize="13sp"
            tools:text="Booking ID 6abb6457eb5429ade68d6395" />

        <TextView
            android:id="@+id/textHistory"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="2dp"
            android:textColor="@color/ss_muted"
            android:textSize="13sp"
            tools:text="Booked Mon, 28 Sep, 10:00 · Approved Tue, 29 Sep, 09:00" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- QR code, only for Approved bookings -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/cardQr"

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:visibility="gone"
        app:cardBackgroundColor="@color/white"
        app:cardCornerRadius="16dp"
        app:strokeColor="@color/ss_primary_soft">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/qr_title"
        android:textColor="@color/ss_dark"
        android:textSize="18sp"
        android:textStyle="bold" />

    <ImageView
        android:id="@+id/imageQr"
        android:layout_width="240dp"
        android:layout_height="240dp"
        android:layout_marginTop="12dp"
        android:background="@color/white"
        android:contentDescription="@string/qr_description" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:gravity="center"
        android:text="@string/qr_help"
        android:textColor="@color/ss_muted"
        android:textSize="14sp" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- What happens next, for Pending / Completed / Cancelled -->
<TextView
    android:id="@+id/textNote"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:textColor="@color/ss_muted"
    android:textSize="15sp"
    android:visibility="gone"
    tools:text="Waiting for approval." />

<!-- Edit / Cancel buttons (Part C, ReservationActions): only when the API says canModify -->
<LinearLayout
    android:id="@+id/actions"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:orientation="vertical">

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonEdit"
        style="@style/Widget.Material3.Button.OutlinedButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/edit_booking"
        android:visibility="gone" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonCancel"
        style="@style/Widget.Material3.Button.OutlinedButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/cancel_booking"
        android:textColor="@color/ss_danger"
        android:visibility="gone"
        app:strokeColor="@color/ss_danger" />
</LinearLayout>
</LinearLayout>
</LinearLayout>
</ScrollView>
</LinearLayout>

```

android/app/src/main/res/layout/activity_reservation_form.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_reservation_form.xml | Part C | M6 Booking form (new + edit): station, slot, kWh, Sell/Buy. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.prosumer.ReservationFormActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/form_title_new" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:fillViewport="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <TextView
                android:id="@+id/textIntro"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginBottom="12dp"
                android:text="@string/form_intro_new"
                android:textColor="@color/ss_muted"
                android:textSize="15sp" />

            <!-- Station (drop-down list, no typing) -->
            <com.google.android.material.textfield.TextInputLayout
                android:id="@+id/layoutStation"
                style="@style/Widget.Material3.TextInputLayout.OutlinedBox.ExposedDropdownMenu"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="@string/station">

                <com.google.android.material.textfield.MaterialAutoCompleteTextView
                    android:id="@+id/inputStation"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="none" />
            </com.google.android.material.textfield.TextInputLayout>

            <!-- Slot at that station (only bookable ones, from the API) -->
            <com.google.android.material.textfield.TextInputLayout
                android:id="@+id/layoutSlot"
                style="@style/Widget.Material3.TextInputLayout.OutlinedBox.ExposedDropdownMenu"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="8dp"
                android:enabled="false"
                android:hint="@string/time_slot"
                app:helperText="@string/slot_helper_choose_station">

                <com.google.android.material.textfield.MaterialAutoCompleteTextView
                    android:id="@+id/inputSlot"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="none" />
            </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"

```

```

        android:layout_marginTop="8dp"
        android:hint="@string/energy_kwh"
        app:helperText="@string/energy_helper">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputEnergy"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:imeOptions="actionDone"
            android:inputType="numberDecimal" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/type"
        android:textColor="@color/ss_dark"
        android:textSize="15sp"
        android:textStyle="bold" />

    <RadioGroup
        android:id="@+id/groupType"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:checkedButton="@+id/radioSell">

        <RadioButton
            android:id="@+id/radioSell"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/type_sell" />

        <RadioButton
            android:id="@+id/radioBuy"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/type_buy" />
    </RadioGroup>

    <TextView
        android:id="@+id/textError"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:textAppearance="@style/TextAppearance.SunShare.Error"
        android:visibility="gone" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonSave"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/create_booking" />
</LinearLayout>
</ScrollView>
</LinearLayout>

```

android/app/src/main/res/layout/activity_reservation_summary.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_reservation_summary.xml | Part C | M7 Summary shown after every booking action. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.prosumer.ReservationSummaryActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:title="@string/summary_title" />

    <ScrollView
        android:layout_width="match_parent"

```

```

android:layout_height="0dp"
android:layout_weight="1"
android:fillViewport="true">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    android:padding="24dp">

    <!-- Big round icon: ✓ or ✕ in the action's colour -->
    <TextView
        android:id="@+id/textIcon"
        android:layout_width="88dp"
        android:layout_height="88dp"
        android:layout_marginTop="16dp"
        android:background="@drawable/bg_status_badge"
        android:gravity="center"
        android:importantForAccessibility="no"
        android:textColor="@color/white"
        android:textSize="44sp"
        android:textStyle="bold"
        tools:text="✓" />

    <TextView
        android:id="@+id/textTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:gravity="center"
        android:textColor="@color/ss_dark"
        android:textSize="24sp"
        android:textStyle="bold"
        tools:text="Booking created" />

    <TextView
        android:id="@+id/textNext"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:gravity="center"
        android:textColor="@color/ss_muted"
        android:textSize="15sp"
        tools:text="Your booking is Pending until it is approved." />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        app:cardBackgroundColor="@color/white"
        app:cardCornerRadius="16dp"
        app:strokeColor="@color/ss_primary_soft">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <TextView
                    android:id="@+id/textStation"
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:textColor="@color/ss_dark"
                    android:textSize="18sp"
                    android:textStyle="bold"
                    tools:text="Malabe Solar Hub" />

                <TextView
                    android:id="@+id/textStatus"
                    style="@style/Widget.SunShare.StatusBadge"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginStart="8dp"
                    tools:text="Pending" />

```

```

</LinearLayout>

<TextView
    android:id="@+id/textProsumer"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:textColor="@color/ss_text"
    android:textSize="15sp"
    android:visibility="gone"
    tools:text="Nimal Perera (199712345678)" />

<TextView
    android:id="@+id/textTime"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:textColor="@color/ss_text"
    android:textSize="15sp"
    tools:text="Thu, 1 Oct, 08:00 - 10:00" />

<TextView
    android:id="@+id/textEnergy"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="2dp"
    android:textColor="@color/ss_text"
    android:textSize="15sp"
    tools:text="10 kWh · Sell" />

<TextView
    android:id="@+id/textBookingId"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/ss_muted"
    android:textIsSelectable="true"
    android:textSize="13sp"
    tools:text="Booking ID 6abb6457eb5429ade68d6395" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonBookings"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:text="@string/view_my_bookings" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonHome"
    style="@style/Widget.Material3.Button.OutlinedButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/back_to_home" />
</LinearLayout>
</ScrollView>
</LinearLayout>

```

android/app/src/main/res/layout/activity_scan_result.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_scan_result.xml | Part D | M11 Scan result: verified booking + Finalize, or the reason it can't be used. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.operator.ScanResultActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/scan_result_title" />

```



```

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <!-- While the server checks the QR code -->
        <LinearLayout
            android:id="@+id/checking"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="32dp"
            android:gravity="center_horizontal"
            android:orientation="vertical">

            <ProgressBar
                android:layout_width="wrap_content"
                android:layout_height="wrap_content" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:text="@string/checking_qr"
                android:textColor="@color/ss_muted"
                android:textSize="15sp" />
        </LinearLayout>

        <!-- The result: ✓ verified or ✗ not valid -->
        <com.google.android.material.card.MaterialCardView
            android:id="@+id/cardResult"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:visibility="gone"
            app:cardBackgroundColor="@color/white"
            app:cardCornerRadius="16dp"
            app:strokeWidth="2dp"
            tools:strokeColor="@color/ss_success"
            tools:visibility="visible">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="20dp">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:gravity="center_vertical"
                    android:orientation="horizontal">

                    <TextView
                        android:id="@+id/textResultIcon"
                        android:layout_width="44dp"
                        android:layout_height="44dp"
                        android:background="@drawable/bg_status_badge"
                        android:gravity="center"
                        android:importantForAccessibility="no"
                        android:textColor="@color/white"
                        android:textSize="24sp"
                        android:textStyle="bold"
                        tools:backgroundTint="@color/ss_success"
                        tools:text="✓" />

                    <TextView
                        android:id="@+id/textResultTitle"
                        android:layout_width="0dp"
                        android:layout_height="wrap_content"
                        android:layout_marginStart="12dp"
                        android:layout_weight="1"
                        android:textColor="@color/ss_dark"
                        android:textSize="20sp"
                        android:textStyle="bold"
                        tools:text="Verified with server" />

```

```

</LinearLayout>

<TextView
    android:id="@+id/textReason"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/ss_danger"
    android:textSize="16sp"
    android:visibility="gone"
    tools:text="This booking is already completed - a QR code can only be used once." />

<!-- Booking details, only when verified -->
<LinearLayout
    android:id="@+id/bookingDetails"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:orientation="vertical"
    android:visibility="gone"
    tools:visibility="visible">

    <TextView
        android:id="@+id/textProsumer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/ss_dark"
        android:textSize="18sp"
        android:textStyle="bold"
        tools:text="Nimal Perera (199712345678)" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:gravity="center_vertical"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/textStation"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:textColor="@color/ss_text"
            android:textSize="16sp"
            tools:text="Malabe Solar Hub" />

        <TextView
            android:id="@+id/textStatus"
            style="@style/Widget.SunShare.StatusBadge"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            tools:text="Approved" />
    </LinearLayout>

    <TextView
        android:id="@+id/textTime"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:textColor="@color/ss_text"
        android:textSize="16sp"
        tools:text="Wed, 30 Sep, 11:00 - 13:00" />

    <TextView
        android:id="@+id/textEnergy"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:textColor="@color/ss_text"
        android:textSize="16sp"
        tools:text="12.5 kWh · Sell" />

    <TextView
        android:id="@+id/textBookingId"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:textColor="@color/ss_muted"
        android:textSize="13sp"
        tools:text="Booking ID 6abb6457eb5429ade68d6395" />
</LinearLayout>

```

```

        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonFinalize"
        android:layout_width="match_parent"
        android:layout_height="64dp"
        android:layout_marginTop="24dp"
        android:text="@string/finalize_transfer"
        android:textSize="17sp"
        android:visibility="gone"
        app:cornerRadius="16dp" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonBack"
        style="@style/Widget.Material3.Button.OutlinedButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="@string/back_to_home" />
</LinearLayout>
</ScrollView>
</LinearLayout>

```

android/app/src/main/res/layout/activity_station_map.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: activity_station_map.xml | Part B | M5 Nearby stations: status line + the Google map page in a WebView. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ss_bg"
    android:orientation="vertical"
    tools:context=".ui.map.StationMapActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        style="@style/Widget.SunShare.Toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:navigationContentDescription="@string/back"
        app:navigationIcon="?attr/homeAsUpIndicator"
        app:title="@string/map_title" />

    <!-- One line saying what the map shows (or why it can't) -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@color/ss_primary_soft"
        android:gravity="center_vertical"
        android:orientation="horizontal"
        android:paddingHorizontal="16dp"
        android:paddingVertical="10dp">

        <ProgressBar
            android:id="@+id/progress"
            android:layout_width="20dp"
            android:layout_height="20dp"
            android:layout_marginEnd="10dp" />

        <TextView
            android:id="@+id/textStatus"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="@string/map_finding_location"
            android:textColor="@color/ss_primary_dark"
            android:textSize="14sp" />
    </LinearLayout>

    <!-- The WebView with the Google map is added here by the code once the stations are ready -->
    <FrameLayout
        android:id="@+id/mapContainer"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>

```

android/app/src/main/res/layout/dialog_server_address.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: dialog_server_address.xml | Part A (+ shared server setting) | The ☉ Server address box on the Login screen. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingHorizontal="24dp"
    android:paddingTop="8dp">

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/server_address"
        app:helperText="@string/server_address_helper">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputServerUrl"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textUri" />
    </com.google.android.material.textfield.TextInputLayout>
</FrameLayout>

```

android/app/src/main/res/layout/item_booking.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: item_booking.xml | Part D | One row (card) in the My bookings list (M8). -->
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="12dp"
    android:clickable="true"
    android:focusable="true"
    app:cardBackgroundColor="@color/white"
    app:cardCornerRadius="14dp"
    app:strokeColor="@color/ss_primary_soft">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <TextView
                android:id="@+id/textStation"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:textColor="@color/ss_dark"
                android:textSize="17sp"
                android:textStyle="bold"
                tools:text="Malabe Solar Hub" />

            <TextView
                android:id="@+id/textStatus"
                style="@style/Widget.SunShare.StatusBadge"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginStart="8dp"
                tools:text="Approved" />
        </LinearLayout>

        <TextView
            android:id="@+id/textTime"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:textColor="@color/ss_text"
            android:textSize="15sp"
            tools:text="Wed, 30 Sep, 11:00 - 12:00" />

        <TextView

```

```

        android:id="@+id/textEnergy"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="2dp"
        android:textColor="@color/ss_muted"
        android:textSize="14sp"
        tools:text="12.5 kWh · Sell" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

android/app/src/main/res/values/colors.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: colors.xml | Part B (Android shell) | SunShare brand colours (same as the web, docs/05-SCREENS.md §1). -->
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFF</color>

    <!-- Brand -->
    <color name="ss_primary">#FF0F766E</color>          <!-- Deep Teal: buttons, links, Approved -->
    <color name="ss_primary_dark">#FF0B5F58</color>
    <color name="ss_primary_soft">#FFE6F4F2</color>
    <color name="ss_accent">#FFF59E0B</color>          <!-- Solar Amber: sun, highlights, Pending (dark text on it) -->
    <color name="ss_accent_soft">#FFFEF3C7</color>
    <color name="ss_dark">#FF0B1F33</color>          <!-- Night Navy: toolbar, headings -->
    <color name="ss_bg">#FFF8FAC</color>          <!-- page background -->
    <color name="ss_text">#FF1E293B</color>
    <color name="ss_muted">#FF64748B</color>

    <!-- Status colours (badges / chips) -->
    <color name="ss_success">#FF16A34A</color>          <!-- Completed, Active -->
    <color name="ss_success_soft">#FFDCFCF7</color>
    <color name="ss_danger">#FFDC2626</color>          <!-- Cancelled errors, Deactivated -->
    <color name="ss_info">#FF0284C7</color>
    <color name="ss_grey">#FF94A3B8</color>          <!-- Cancelled -->
</resources>

```

android/app/src/main/res/values/strings.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: strings.xml | Part B (Android shell) | Texts used by the app. -->
<resources>
    <string name="app_name">SunShare</string>
    <string name="tagline">Trade your sunshine</string>
    <string name="back">Back</string>

    <!-- Form fields (shared by Register and Profile) -->
    <string name="nic">NIC</string>
    <string name="nic_helper">9 digits + V/X, or 12 digits</string>
    <string name="full_name">Full name</string>
    <string name="email">Email</string>
    <string name="phone">Phone</string>
    <string name="address_optional">Address (optional)</string>
    <string name="password">Password</string>
    <string name="password_helper">At least 6 characters</string>
    <string name="confirm_password">Confirm password</string>

    <!-- M1 Login (Part A) -->
    <string name="login_title">Log in</string>
    <string name="login_subtitle">Use your NIC and password.</string>
    <string name="login_button">Log in</string>
    <string name="register_link">New prosumer? Create an account</string>
    <string name="server_button">☉ Server</string>
    <string name="server_address">Server address</string>
    <string name="server_address_helper">Emulator: http://10.0.2.2:8080/ · Phone: http://&lt;laptop Wi-Fi
IP&gt;;8080/</string>

    <!-- M2 Register (Part A) -->
    <string name="register_title">Create account</string>
    <string name="register_intro">Sign up as a prosumer. A Backoffice officer activates new accounts before you can log
in.</string>
    <string name="register_button">Create account</string>

    <!-- M4 Profile (Part A) -->
    <string name="profile_title">My profile</string>
    <string name="new_password_optional">New password (optional)</string>
    <string name="new_password_helper">Leave empty to keep your current password</string>
    <string name="save_changes">Save changes</string>
    <string name="close_account_title">Close your account</string>
    <string name="close_account_text">You won't be able to log in until a Backoffice officer reactivates your
account.</string>

```

```

<string name="deactivate_account">Deactivate my account</string>

<!-- Home screens -->
<string name="log_out">Log out</string>
<string name="home_prompt">What would you like to do?</string>
<string name="my_profile">My profile</string>
<string name="operator_ready">Ready for today\'s energy transfers.</string>

<!-- M3 Prosumer Home counts (Part D) -->
<string name="count_loading"></string>
<string name="count_pending">Pending</string>
<string name="count_approved">Approved upcoming</string>
<string name="count_completed">Completed</string>
<string name="next_booking">NEXT BOOKING</string>
<string name="no_next_booking">No upcoming bookings.</string>
<string name="my_bookings">My bookings</string>

<!-- M8 My bookings (Part D) -->
<string name="bookings_title">My bookings</string>
<string name="search_hint">Search station or booking ID</string>

<!-- M9 Booking details (Part D) -->
<string name="detail_title">Booking details</string>
<string name="qr_title">Your QR code</string>
<string name="qr_description">QR code for this booking</string>
<string name="qr_help">Show this at the station. The Grid Operator scans it to finish the energy transfer.</string>

<!-- M6 Booking form (Part C) -->
<string name="new_booking">New booking</string>
<string name="form_title_new">New booking</string>
<string name="form_title_edit">Edit booking</string>
<string name="form_intro_new">Pick a station and one of its free slots. New bookings are Pending until they are
approved.</string>
<string name="form_intro_edit">Change the slot, energy or type. An approved booking goes back to Pending and needs
approval again.</string>
<string name="station">Station</string>
<string name="time_slot">Time slot</string>
<string name="slot_helper_choose_station">Choose a station first</string>
<string name="energy_kwh">Energy (kWh)</string>
<string name="energy_helper">More than 0 and up to 100 kWh</string>
<string name="type">Type</string>
<string name="type_sell">Sell - drop off spare energy</string>
<string name="type_buy">Buy - charge from the station</string>
<string name="create_booking">Create booking</string>

<!-- M9 Edit / Cancel (Part C) -->
<string name="edit_booking">Edit booking</string>
<string name="cancel_booking">Cancel booking</string>

<!-- M7 Summary (Part C) -->
<string name="summary_title">Summary</string>
<string name="view_my_bookings">View my bookings</string>
<string name="back_to_home">Back to home</string>

<!-- M10 Operator Home + M11 Scan result (Part D) -->
<string name="count_today">Today</string>
<string name="scan_help">Scan the prosumer\'s QR code at the station. The server checks it, then you finalize the energy
transfer.</string>
<string name="scan_qr"> Scan QR code</string>
<string name="scan_result_title">Scan result</string>
<string name="checking_qr">Checking the QR code with the server.</string>
<string name="finalize_transfer">Finalize energy transfer</string>

<!-- M5 Nearby stations map (Part B) -->
<string name="nearby_stations">Nearby stations</string>
<string name="map_title">Nearby stations</string>
<string name="map_finding_location">Finding your location.</string>
<string name="map_loading_stations">Loading nearby stations.</string>
</resources>

```

android/app/src/main/res/values/themes.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: themes.xml | Part B (Android shell) | Material 3 light theme in the SunShare colours. -->
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Base.Theme.SunShare" parent="Theme.Material3.Light.NoActionBar">
        <item name="colorPrimary">@color/ss_primary</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorPrimaryContainer">@color/ss_primary_soft</item>
        <item name="colorOnPrimaryContainer">@color/ss_primary_dark</item>
        <item name="colorSecondary">@color/ss_accent</item>
    </style>
</resources>

```

```

        <item name="colorOnSecondary">@color/ss_dark</item>
        <item name="colorSecondaryContainer">@color/ss_accent_soft</item>
        <item name="colorOnSecondaryContainer">@color/ss_dark</item>
        <item name="colorError">@color/ss_danger</item>
        <item name="android:colorBackground">@color/ss_bg</item>
        <item name="android:windowBackground">@color/ss_bg</item>
        <item name="colorOnBackground">@color/ss_text</item>
        <item name="colorOnSurface">@color/ss_text</item>
        <item name="android:windowLightStatusBar">false</item>
    </style>

    <style name="Theme.SunShare" parent="Base.Theme.SunShare" />

    <!-- Navy top bar used on inner screens: white title, grey subtitle, white back arrow. -->
    <style name="Widget.SunShare.Toolbar" parent="Widget.Material3.Toolbar">
        <item name="android:background">@color/ss_dark</item>
        <item name="titleTextColor">@color/white</item>
        <item name="subtitleTextColor">@color/ss_grey</item>
        <item name="navigationIconTint">@color/white</item>
    </style>

    <!-- Small coloured pill for a booking status (Pending / Approved / Completed / Cancelled). -->
    <style name="Widget.SunShare.StatusBadge" parent="">
        <item name="android:background">@drawable/bg_status_badge</item>
        <item name="android:paddingStart">10dp</item>
        <item name="android:paddingEnd">10dp</item>
        <item name="android:paddingTop">3dp</item>
        <item name="android:paddingBottom">3dp</item>
        <item name="android:textSize">12sp</item>
        <item name="android:textStyle">bold</item>
    </style>

    <!-- Red line under a form that shows the API's error message (hidden until needed). -->
    <style name="TextAppearance.SunShare.Error" parent="TextAppearance.Material3.BodyMedium">
        <item name="android:textColor">@color/ss_danger</item>
    </style>
</resources>

```

android/app/src/main/res/xml/backup_rules.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: backup_rules.xml | Part B (Android shell) | Backup rules for Android 11 and older
    (Android 12+ uses data_extraction_rules.xml): the SQLite file with the login token is
    never included in a backup. -->
<full-backup-content>
    <exclude domain="database" path="sunshare_local.db" />
</full-backup-content>

```

android/app/src/main/res/xml/data_extraction_rules.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: data_extraction_rules.xml | Part B (Android shell) | Backup rules for Android 12+:
    the phone's SQLite file (login token, cached stations, server address) is never copied to a
    cloud backup or to a new phone - after a restore the user simply logs in again. -->
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="database" path="sunshare_local.db" />
    </cloud-backup>
    <device-transfer>
        <exclude domain="database" path="sunshare_local.db" />
    </device-transfer>
</data-extraction-rules>

```

android/app/src/main/res/xml/network_security_config.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- File: network_security_config.xml | Part B (Android shell) | Allows plain HTTP to the SunShare API.
    The demo API runs on IIS over HTTP on the local network (10.0.2.2 for the emulator, the laptop's
    Wi-Fi IP for a phone), and that IP changes between networks, so HTTP is allowed for all hosts.
    Production would use HTTPS instead (decision D15). -->
<network-security-config>
    <base-config cleartextTrafficPermitted="true" />
</network-security-config>

```

A.5 Deployment scripts (PowerShell)

scripts/deploy-api.ps1

```
<#
=====
File       : deploy-api.ps1
Project    : SunShare - Smart Solar Microgrid Trading System
Module     : SE4040 Enterprise Application Development - Assignment 1
Part       : B - Hosting
Author     : Ranathunga R A K N (IT22552860)
Created    : 2026-09-28
Description : Publishes SunShare.Api and hosts it on IIS as the site
              "SunShareApi" on port 8080 (docs/08-SETUP-AND-HOSTING.md section 3).
              Safe to run again after every code change: it only creates
              the app pool, site and firewall rule when they are missing.
How to run  : in an ADMINISTRATOR PowerShell, from the repo folder:
              powershell -ExecutionPolicy Bypass -File .\scripts\deploy-api.ps1
Reference   : Microsoft Learn, "Host ASP.NET Core on Windows with IIS"
              https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/
=====
#>

$SiteName = 'SunShareApi'
$Port     = 8080
$PublishDir = 'C:\inetpub\sunshare\api'
$Project   = Join-Path $PSScriptRoot '..\api\SunShare.Api'
$AppCmd    = "$env:windir\system32\inetsrv\appcmd.exe" # IIS's own command-line tool

# Stops the script with a clear message unless it runs as Administrator
# (creating IIS sites and writing to C:\inetpub need admin rights).
function Assert-Admin
{
    $identity = [Security.Principal.WindowsIdentity]::GetCurrent()
    $isAdmin =
    ([Security.Principal.WindowsPrincipal]$identity).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
    if (-not $isAdmin)
    {
        Write-Host 'Please run this in an ADMINISTRATOR PowerShell (right-click PowerShell > Run as administrator).' -
        ForegroundColor Red
        exit 1
    }
    if (-not (Test-Path $AppCmd))
    {
        Write-Host 'IIS is not installed (appcmd.exe not found). See docs/08-SETUP-AND-HOSTING.md section 3.1.' -
        ForegroundColor Red
        exit 1
    }
}

# Finds a dotnet.exe that has an SDK (needed for "publish"). The Hosting Bundle adds a runtime-only
# dotnet in Program Files to the PATH, which can hide the SDK, so the user's own SDK folder is tried too.
function Get-DotnetWithSdk
{
    $candidates = @("$env:ProgramFiles\dotnet\dotnet.exe", "$env:USERPROFILE\.dotnet\dotnet.exe")
    foreach ($candidate in $candidates)
    {
        if (Test-Path $candidate)
        {
            {
                $sdks = & $candidate --list-sdks
                if ($sdks) { return $candidate }
            }
        }
    }
    Write-Host 'No .NET SDK found. Install the .NET 10 SDK (docs/08-SETUP-AND-HOSTING.md section 1).' -ForegroundColor Red
    exit 1
}

# Creates the application pool once. "No Managed Code" = IIS must not load the old .NET Framework;
# ASP.NET Core brings its own runtime and IIS only forwards requests to it.
function Initialize-AppPool
{
    & $AppCmd list apppool $SiteName | Out-Null
    if ($LASTEXITCODE -ne 0)
    {
        Write-Host "Creating application pool '$SiteName' (No Managed Code)..."
        & $AppCmd add apppool "/name:$SiteName" '/managedRuntimeVersion:' | Out-Host
    }
    else
    {
        Write-Host "Application pool '$SiteName' already exists."
    }
}

# Creates the IIS site once: port 8080, pointing at the published folder, using our app pool.
```



```

function Initialize-Site
{
    & $AppCmd list site $SiteName | Out-Null
    if ($LASTEXITCODE -ne 0)
    {
        Write-Host "Creating IIS site '$SiteName' on port $Port..."
        & $AppCmd add site "/name:$SiteName" "/bindings:http:*:$Port}" "/physicalPath:$PublishDir" | Out-Host
        & $AppCmd set app "$SiteName/" "/applicationPool:$SiteName" | Out-Host
    }
    else
    {
        Write-Host "IIS site '$SiteName' already exists."
    }
}

# Lets other devices on the network (the phone) reach port 8080 through Windows Firewall. Created once.
function Initialize-FirewallRule
{
    $ruleName = "SunShare API $Port"
    if (-not (Get-NetFirewallRule -DisplayName $ruleName -ErrorAction SilentlyContinue))
    {
        Write-Host "Adding firewall rule '$ruleName'..."
        New-NetFirewallRule -DisplayName $ruleName -Direction Inbound -Protocol TCP -LocalPort $Port -Action Allow | Out-Null
    }
    else
    {
        Write-Host "Firewall rule '$ruleName' already exists."
    }
}

# Calls GET /api/health on IIS (the first call also starts the app) and shows the result,
# plus the address the phone should use.
function Show-Health
{
    try
    {
        $health = Invoke-RestMethod "http://localhost:$Port/api/health" -TimeoutSec 60
        Write-Host "IIS API is up: api=$(($health.api)), database=$(($health.database))" -ForegroundColor Green
    }
    catch
    {
        Write-Host "Health check failed: $($_.Exception.Message)" -ForegroundColor Red
        Write-Host 'Tips: is MongoDB running (Get-Service MongoDB)? See the troubleshooting table in docs/08.' -
        ForegroundColor Yellow
    }
    $wifi = Get-NetIPAddress -AddressFamily IPv4 -ErrorAction SilentlyContinue |
    Where-Object { $_.InterfaceAlias -match 'Wi-Fi|Wireless|Ethernet' -and $_.IPAddress -notlike '169.*' } |
    Select-Object -First 1
    Write-Host "Swagger: http://localhost:$Port/swagger"
    if ($wifi) { Write-Host "Phone: http://$(($wifi.IPAddress)):$Port/api/health (same Wi-Fi)" }
}

# ----- Main steps -----
Assert-Admin
$dotnet = Get-DotnetWithSdk
Initialize-AppPool

# The running app locks its files, so stop the app pool before copying new ones in.
& $AppCmd stop apppool "/apppool.name:$SiteName" 2>&1 | Out-Null
Start-Sleep -Seconds 2

Write-Host "Publishing the API to $PublishDir ..."
& $dotnet publish $Project -c Release -o $PublishDir
if ($LASTEXITCODE -ne 0)
{
    Write-Host 'Publish failed - see the errors above.' -ForegroundColor Red
    exit 1
}

Initialize-Site
& $AppCmd start apppool "/apppool.name:$SiteName" 2>&1 | Out-Null
& $AppCmd start site "/site.name:$SiteName" 2>&1 | Out-Null
Initialize-FirewallRule
Show-Health

```

scripts/deploy-web.ps1

```

<#
=====
File       : deploy-web.ps1
Project    : SunShare - Smart Solar Microgrid Trading System
Module     : SE4040 Enterprise Application Development - Assignment 1
Part       : B - Hosting

```

```

Author       : Ranathunga R A K N (IT22552860)
Created      : 2026-09-29
Description   : Builds the React web app and hosts it on IIS as the site
                "SunShareWeb" on port 8081 (docs/08-SETUP-AND-HOSTING.md
                section 4). The built app calls the IIS API on port 8080
                (web/.env.production). Safe to run again after every change.
How to run   : in an ADMINISTRATOR PowerShell, from the repo folder:
                powershell -ExecutionPolicy Bypass -File .\scripts\deploy-web.ps1
=====
#>

$SiteName = 'SunShareWeb'
$Port     = 8081
$SiteDir  = 'C:\inetpub\sunshare\web'
$WebFolder = Join-Path $PSScriptRoot '..\web'
$AppCmd    = "$env:windir\system32\inetsrv\appcmd.exe" # IIS's own command-line tool

# Stops the script with a clear message unless it runs as Administrator (IIS and C:\inetpub need it).
function Assert-Admin
{
    $identity = [Security.Principal.WindowsIdentity]::GetCurrent()
    $isAdmin =
    ([Security.Principal.WindowsPrincipal]$identity).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
    if (-not $isAdmin)
    {
        Write-Host 'Please run this in an ADMINISTRATOR PowerShell (right-click PowerShell > Run as administrator).' -
        ForegroundColor Red
        exit 1
    }
    if (-not (Test-Path $AppCmd))
    {
        Write-Host 'IIS is not installed (appcmd.exe not found). See docs/08-SETUP-AND-HOSTING.md section 3.1.' -
        ForegroundColor Red
        exit 1
    }
}

# Builds the web app with "npm run build" (Vite uses .env.production -> API at http://localhost:8080).
function Invoke-WebBuild
{
    Write-Host 'Building the web app (npm run build) ...'
    Push-Location $WebFolder
    npm run build
    $buildOk = ($LASTEXITCODE -eq 0)
    Pop-Location
    if (-not $buildOk)
    {
        Write-Host 'The web build failed - see the errors above.' -ForegroundColor Red
        exit 1
    }
}

# Copies web\dist into the IIS folder. /MIR makes the folder an exact mirror (old files are removed).
# robocopy exit codes 0-7 mean success; 8 or more means something could not be copied.
function Copy-WebFiles
{
    Write-Host "Copying the built files to $SiteDir ..."
    robocopy (Join-Path $WebFolder 'dist') $SiteDir /MIR /NFL /NDL /NJH /NJS /NP | Out-Null
    if ($LASTEXITCODE -ge 8)
    {
        Write-Host "Copying failed (robocopy exit code $LASTEXITCODE)." -ForegroundColor Red
        exit 1
    }
}

# Creates the application pool once. The site is only static files, so no .NET code runs in it.
function Initialize-AppPool
{
    & $AppCmd list apppool $SiteName | Out-Null
    if ($LASTEXITCODE -ne 0)
    {
        Write-Host "Creating application pool '$SiteName' (No Managed Code)..."
        & $AppCmd add apppool "/name:$SiteName" '/managedRuntimeVersion:' | Out-Host
    }
    else
    {
        Write-Host "Application pool '$SiteName' already exists."
    }
}

# Creates the IIS site once: port 8081, pointing at the copied files, using our app pool.
function Initialize-Site
{
    & $AppCmd list site $SiteName | Out-Null
    if ($LASTEXITCODE -ne 0)

```

```

{
    Write-Host "Creating IIS site '$SiteName' on port $Port..."
    & $AppCmd add site "/name:$SiteName" "/bindings:http/*:$Port/:" "/physicalPath:$SiteDir" | Out-Host
    & $AppCmd set app "$SiteName/" "/applicationPool:$SiteName" | Out-Host
}
else
{
    Write-Host "IIS site '$SiteName' already exists."
}
}

# Opens the home page and the icon font on IIS to prove the site works.
function Show-Result
{
    try
    {
        $page = Invoke-WebRequest "http://localhost:$Port/" -UseBasicParsing -TimeoutSec 30
        $cssFile = ([regex]::Match($page.Content, 'assets/[^"]+\.css')).Value
        Write-Host "IIS web app is up: http://localhost:$Port/ (HTTP $($page.StatusCode))" -ForegroundColor Green
        if ($cssFile) { Write-Host "Styles file: $cssFile" }
    }
    catch
    {
        Write-Host "Could not open http://localhost:$Port/ : $($_.Exception.Message)" -ForegroundColor Red
    }
    Write-Host "The web app calls the IIS API at http://localhost:8080 - check it with scripts\deploy-api.ps1 if login fails."
}

# ----- Main steps -----
Assert-Admin
Invoke-WebBuild
Initialize-AppPool
Copy-WebFiles
Initialize-Site
& $AppCmd start apppool "/apppool.name:$SiteName" 2>&1 | Out-Null
& $AppCmd start site "/site.name:$SiteName" 2>&1 | Out-Null
Show-Result

```